UF2406: El ciclo de vida del desarrollo de aplicaciones

Presentación

Identificación de la Unidad Formativa

Bienvenido a la Unidad Formativa **UF2406: El ciclo de vida del desarrollo de aplicaciones**. Esta Unidad Formativa pertenece al **Módulo Formativo MF0227_3: Programación orientada a objetos** que forma parte del Certificado de Profesionalidad **IFCD0112: Programación de lenguajes orientados a objetos y bases de datos relacionales**, de la familia de **Informática y comunicaciones**.

Presentación de los contenidos

La finalidad de esta Unidad Formativa es enseñar al alumno a implementar los componentes software encomendados, manipular bases de datos a través de interfaces para integrar el lenguaje de programación con el lenguaje de acceso a datos, probar los componentes software desarrollados, así como utilizar los componentes orientados a objeto y elaborar la documentación del código desarrollado según los estándares de la organización.

Para ello, se desarrollará el proceso de ingeniería del software, planificación y seguimiento, se realizará el diagramado, el desarrollo de la GUI, y por último, se analizará la calidad en el desarrollo del software, pruebas, excepciones y documentación.

Objetivos de la Unidad Formativa

Al finalizar esta Unidad Formativa aprenderás a:

– Manejar las herramientas de ingeniería de software.

– Verificar la corrección de las clases desarrolladas mediante la realización de pruebas.

– Elaborar la documentación completa relativa a las clases desarrolladas y pruebas realizadas.

– Realizar modificaciones de clases existentes por cambios en las especificaciones.

– Desarrollar interfaces de usuario en lenguajes de programación orientados a objeto, a partir del diseño detallado.

Índice

UD1. Proceso de ingeniería del software

1.1. Distinción de las fases del proceso de ingeniería software: especificación, diseño, construcción y pruebas unitarias, validación, implantación y mantenimiento 11

1.2. Análisis de los modelos del proceso de ingeniería: modelo en cascada, desarrollo evolutivo, desarrollos formarles, etc ... 13

1.3. Identificación de requisitos: concepto, evolución y trazabilidad ..20

1.4. Análisis de metodologías de desarrollo orientadas a objeto 27

1.5. Resolución de un caso práctico de metodologías de desarrollo que utilizan UML 40

1.6. Definición del concepto de Herramienta CASE 53

1.6.1. Herramientas de Ingeniería del Software..... 60

1.6.2. Entornos de desarrollo 63

1.6.3. Herramientas de prueba 70

1.6.4. Herramientas de gestión de configuración..... 79

1.6.5. Herramientas para métricas 86

UD2. Planificación y seguimiento

2.1. Realización de estimaciones 99

2.2. Planificaciones. Modelos de diagramado. Diagramas de Gantt 124

2.3. Análisis del proceso de seguimiento. Reuniones e informes. 147

UD3. Diagramado

3.1. Identificación de los principios básicos de UML 167

3.2. Empleo de diagramas de uso 177

UD4. Desarrollo de la GUI

4.1. Análisis del modelo de componentes y objetos 263

4.2. Identificación de los elementos de la GUI 273

4.3. Presentación del diseño orientado al usuario. Nociones de usabilidad 298

4.4. Empleo de herramientas de Interfaz Gráfica 308

UD5. Calidad en el desarrollo del software

5.1. Enumeración de los criterios de calidad 319

5.2. Análisis de métricas y estándares de calidad 340

UD6. Identificación de los tipos de pruebas

6.1. Identificación de los tipos de pruebas 357

6.2. Análisis de pruebas de defectos. Pruebas de caja negra. Pruebas estructurales. Pruebas de trayectoria. Pruebas de integración. Pruebas de interfaces 362

6.2.1. Preparación de los casos de prueba 371

6.2.2. Casos de prueba 375

6.2.3. Codificar las pruebas 380

6.2.4. Definir procesos de prueba..... 387

6.2.5. Ejecución de pruebas 396

6.2.6. Generación de informes de pruebas 400

UD7. Excepciones

7.1. Definición. Fuente de excepciones. Tratamiento de excepciones. Prevención de fallos. Excepciones definidas y lanzadas por el programador..... 417

7.2. Uso de las excepciones tratadas como objetos 431

UD8. Documentación

8.1. Cómo producir un documento..... 457

8.2. Estructura de un documento 470

8.3. Generación automática de documentación 475

Glosario 493

Soluciones 497

Área: informática y comunicaciones

UD1

Proceso de ingeniería del software

1.1. Distinción de las fases del proceso de ingeniería software: especificación, diseño, construcción y pruebas unitarias, validación, implantación y mantenimiento

1.2. Análisis de los modelos del proceso de ingeniería: modelo en cascada, desarrollo evolutivo, desarrollos formarles, etc

1.3. Identificación de requisitos: concepto, evolución y trazabilidad

1.4. Análisis de metodologías de desarrollo orientadas a objeto

1.5. Resolución de un caso práctico de metodologías de desarrollo que utilizan UML

1.6. Definición del concepto de herramienta CASE

- 1.6.1. Herramientas de Ingeniería del Software
- 1.6.2. Entornos de desarrollo
- 1.6.3. Herramientas de prueba
- 1.6.4. Herramientas de gestión de configuración
- 1.6.5. Herramientas para métricas

1.1. Distinción de las fases del proceso de ingeniería software: especificación, diseño, construcción y pruebas unitarias, validación, implantación y mantenimiento

Desde el momento en el que se introdujeron computadores con capacidad para soportar aplicaciones de tamaño considerable en los años sesenta, se descubrió que las técnicas de desarrollo para los hasta entonces pequeños sistemas dejaron progresivamente de ser válidas.

Estas primitivas técnicas consistían básicamente en codificar y corregir, es decir, no existe necesariamente una especificación del producto final, en su lugar se tienen algunas anotaciones sobre las características generales del programa.

Inmediatamente al comienzo de un proyecto se empieza la codificación y simultáneamente se va depurando el programa resultante. Cuando el programa cumple con las especificaciones y parece que no tiene errores se entrega.

Las ventajas de esta forma de trabajar son que no se gasta tiempo en planificación, gestión de los recursos, documentación, etc.

En el caso de que el proyecto es de un tamaño muy pequeño y lo realiza una sola persona no es un mal sistema para producir un resultado pronto, aunque este enfoque no es muy adecuado cuando se trata de desarrollar un trabajo en equipo, como ocurren en el desarrollo de la mayoría de sistemas software

Hoy en día es un método de desarrollo que se usa cuando hay plazos muy breves para entregar el producto final y no existe una exigencia explícita por parte de la organización de usar alguna metodología de ingeniería del software. Puede dar resultado en algunas ocasiones pero la calidad es imprevisible.

Las consecuencias de este enfoque, que desembocaron en lo que se denomino la crisis del software, fueron:

– El costo de los proyectos era mucho mayor de lo originalmente previsto.

– El tiempo de desarrollo también excedía lo proyectado.

– La calidad del código producido era imprevisible y en promedio baja.

– Era prácticamente imposible mantener las aplicaciones así desarrolladas.



Sabías que

La Ingeniería del software surgió en aquella época como disciplina con el objetivo de idear métodos y técnicas que solucionaran estos problemas y proporcionaran un marco de trabajo técnico adecuado para llevar acabo la construcción del software.

La Ingeniería del Software se puede definir como aquella rama de las ciencias de la computación que trata del establecimiento de los principios y métodos de la ingeniería, orientados a obtener software económico, que sea fiable y funcione de manera eficiente sobre máquinas reales.

El software requiere de un tiempo y esfuerzo considerable para ser desarrollado, y durante aún más tiempo debe de estar en uso antes de ser retirado o substituido.

Durante todo este período de tiempo se identifican una serie de etapas que en su conjunto se denominan “ciclo de vida del software”.

Las etapas principales de cualquier ciclo de vida son las siguientes:

– Análisis: se identifican los requisitos que debe de cumplir el software y se construye un modelo de dichos requisitos.

– Diseño: A partir del modelo de análisis se identifican los procesos y las estructuras de datos en las que se descomponen el sistema, y además se construye un modelo del sistema a desarrollar.

– Codificación: se construye el sistema en sí mismo.

– Prueba: se comprueba que el sistema construido es correcto y cumple con el modelo de requisitos.

– Mantenimiento: esta fase tiene lugar tras la entrega del producto acabado y en ella se trata de asegurar que el sistema siga funcionando y adaptándose a nuevos requisitos.

Desde cualquiera de ellas se puede volver a la anterior si el desarrollo posterior detecta algún error cometido en las fases anteriores.

Dependiendo de la manera en que se estructuren estas etapas surgen los diversos ciclos de vida del software los cuales se pueden clasificar en tres tipos genéricos, ciclos de vida en cascada, ciclos de vida en espiral o incrementales y ciclos de vida Orientados a Objetos

1.2. Análisis de los modelos del proceso de ingeniería: modelo en cascada, desarrollo evolutivo, desarrollos formarles, etc

El ciclo de vida en cascada, inicialmente propuesto por Royce en 1970, fue el primer ciclo de vida que se propuso y es, actualmente el más ampliamente seguido por una multitud de organizaciones y empresas de desarrollo.

Este modelo tiene la posibilidad de hacer iteraciones o repeticiones, es decir, que si durante las modificaciones y cambios que se hacen en la fase de mantenimiento se puede detectar la necesidad de cambiar algo en el diseño, por ejemplo, lo cual significa que se van a hacer los cambios que sean necesarios en la codificación y se tendrán que realizar de nuevo las pruebas.

Sin embargo, si se tiene que volver a una de las fase anteriores al mantenimiento hay que realizar de nuevo el resto de las etapas hasta llegar al final.

Después de cada etapa se hace una revisión para chequear si se puede pasar a la siguiente etapa. En el se trabaja en base a documentos, es decir, la entrada y la salida de cada etapa es un tipo de documento específico.

Este ciclo de vida conlleva una serie de ventajas:

– La planificación del proyectoes sencilla y fácil de hacer.

– La calidad del producto si se aplica correctamente es alta.

– Permite trabajar con empleados con menor cualificación.

Sin embargo también presenta una serie de inconvenientes bastante graves que hacen que no se suela implementar “tal cual” en la realidad:

– Su mayor inconveniente es la necesidad de detallar todos los requisitos al comienzo del proyecto. Lo normal es que el cliente no tenga perfectamente claras las especificaciones del software que desea, o puede ser que surjan otras necesidades no previstas durante el proyecto.

– Si se cometen errores en una fase y no se detectan a tiempo es difícil volver atrás, ya que una vez que se ha finalizado una fase y se ha generado la documentación correspondiente, un paso atrás representa repetir la fase completamente

– No se desarrolla el producto hasta el final, esto quiere decir que si se tiene un fallo la fase de análisis este probablemente no será descubierto hasta la entrega, con el lo que conlleva un gasto inútil de recursos. Debido a esto el cliente no ve resultados hasta el final, con lo que puede impacientarse.

– No se tienen indicadores fiables del progreso del trabajo, lo cual puede llevar al síndrome del 90%, es decir, las tareas se indican como realizadas en un 90% pero el restante 10% va a necesitar de un esfuerzo considerablemente mayor que el resto.

Sin embargo fue el primer modelo de desarrollo de software que se planteó por lo que ha influenciado numerosos ciclos ce vida que se han propuesto posteriormente.

El ciclo de vida en cascada ha inspirado numeroso modelos de ciclos de vida, como el ciclo de vida en V, el modelo sashimi, o el ciclo de vida en espiral.

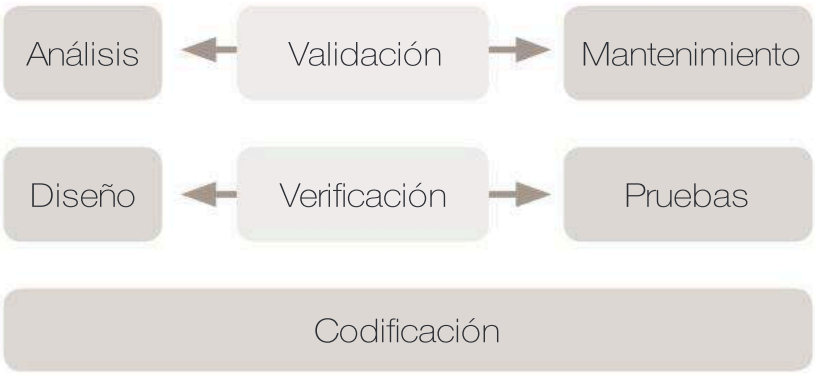
El ciclo de vida en V fue propuesto por Alan Davis, y tiene las mismas fases que el ciclo de vida en cascada pero teniendo en consideración en consideración el nivel de abstracción de cada una.

Se considera que la fase con mayor nivel de abstracción es la fase de análisis, para posteriormente pasar a trabajar a menos nivel en el diseño.

En la codificación se trabaja al mínimo nivel de abstracción. Posteriormente durante las distintas fases de prueba se va subiendo de nivel de abstracción

fase además de utilizarse como entrada para la siguiente, sirve para validar o verificar otras fases posteriores . La estructura de las tareas es la que se muestra en el esquema.

Una fase además de utilizarse como entrada para la siguiente, sirve para validar o verificar otras fases posteriores.



De esta forma la tarea de validación consiste en comprobar los resultados del análisis, es decir, si el software cumple con los requisitos que se le exigieron al principio del desarrollo.

Esto se hace durante la fase de mantenimiento en la que el usuario final durante su trabajo del día a día informa al desarrollador de aquellos aspectos que no cumplen con lo especificado y determinado durante el análisis.

De la misma forma surge el concepto de Verificación, en el que se comprueba que el software funciona correctamente acorde al diseño que se ha realizado.

Estos dos conceptos, verificación y validación son los mayores aportes de este modelo de ciclo de vida, y se han extendido a toda la ingeniería del software.

Definición

Podemos definir **verificación** como el proceso para determinar que un sistema software está libre de errores, y la validación como el proceso para determinar que un determinado sistema cumple con los requisitos esperados.

El modelo Sashimi es otro modelo de ciclos de vida. Si seguimos el modelo en cascada como fue definido, una fase sólo puede empezar cuando ha terminado la anterior.

En el caso de este ciclo de vida, sin embargo, se permite un solapamiento entre fases. Por ejemplo, sin tener terminado del todo el diseño se puede comenzar a implementar.



El nombre "sashimi" deriva del estilo de presentación en rodajas de pescado crudo en Japón.

Una ventaja de este modelo es que no necesita generar tanta documentación como el ciclo de vida en cascada puro debido a que se continúa con el mismo grupo de trabajo durante las distintas fases y por lo tanto conocen el proyecto en profundidad.

Los problemas que plantea este modelo de ciclo de vida son básicamente los mismos que el modelo de ciclo de vida en cascada, pero agravados, y son los siguientes:

– Es más difícil que en ciclo de vida clásico el controlar el progreso del proyecto, debido a la falta de puntos de referencia. Debido a que las fases se solapan constantemente, los finales de fase ya no son un punto de referencia específico.

– Al realizar las fases en paralelo, pueden ocurrir a menudo problemas de comunicación entre los miembros del equipo, de los que pueden surgir inconsistencias que necesiten de cambios y modificaciones alterando la planificación.

La fase de "concepto" se añade en este modelo de ciclo de vida y en ella se trata de definir los objetivos del proyecto, beneficios, tipo de tecnología y tipo de ciclo de vida.

La fase de diseño se divide a su vez en dos fases diferentes, el diseño arquitectónico y el diseño detallado o de componentes.

En la fase de diseño arquitectónico es diseño de alto nivel de abstracción, el detallado es de bajo nivel de abstracción, cuando se especifican en detalle cada uno de los componentes del software.

Al terminar una iteración se comprueba que lo que se ha realizado cumple con los requisitos que se establecieron al principio. También se verifica que funciona correctamente y es el propio cliente quien evalúa el producto para ver si es satisfactorio para resolver su necesidad.

En el ciclo de vida Sahimi no existe una diferencia muy clara entre cuándo termina el proyecto y cuándo empieza la fase de mantenimiento ya que cuando hay que hacer un cambio, éste puede consistir en un nuevo ciclo.

Presenta numerosas ventajas en cuanto a su utilización:

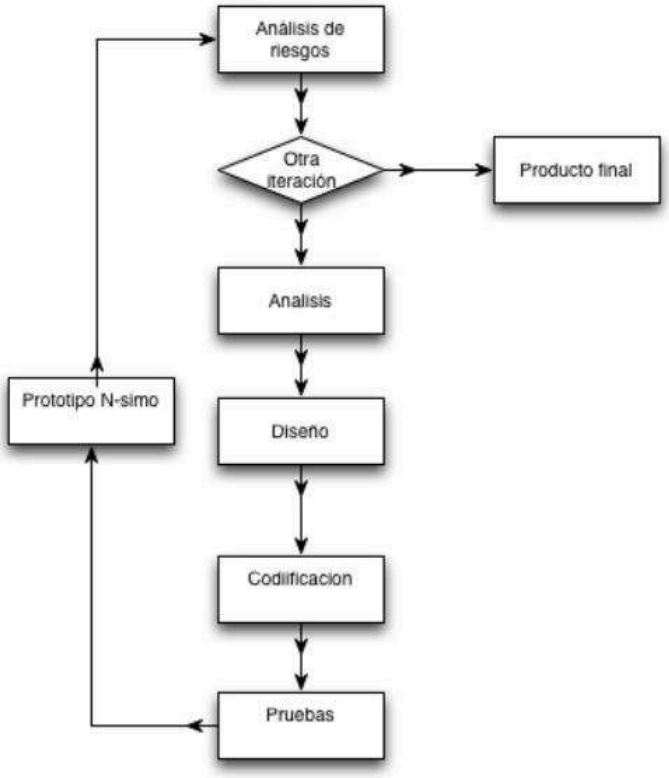
- No necesita una definición detallada de los requisitos para empezar a funcionar.
- Al entregar productos desde el final de la primera iteración es más fácil validar los requisitos frente a la solicitud del usuario.
- El riesgo en general es menor porque, si todo se hace mal, sólo se pierde el tiempo y recursos invertidos en una iteración (las anteriores iteraciones están bien por definición).
- El riesgo de sufrir retrasos es menor, ya que se identifican los problemas en etapas tempranas cuando aun hay tiempo de subsanarlos.

Sin embargo también presenta algunos inconvenientes que conviene tener en cuenta si se decide optar por el a la hora de realizar un proyecto:

- Es difícil llevar a cabo una evaluación correcta de los riesgos. Por el propio concepto de riesgo, este lleva implicado una dosis de incertidumbre que limita el hecho de poder realizar una estimación adecuada.
- Necesita de la participación continua de la parte cliente, esfuerzo al que algunos clientes pueden no estar de acuerdo en realzar, por lo que convienen aclarar cual va a ser su participación antes de comenzar.



Los tipos de ciclos de vida que se han visto hasta ahora se refieren al análisis y diseño estructurados, pero hay que tener en cuenta que el desarrollo de sistemas orientados a objetos tiene la particularidad de estar basados en un diseñado de componentes que se relacionan entre sí a través de una serie de interfaces, o lo que es lo mismo, son más modulares y por lo tanto el trabajo se puede particionar en un conjunto de pequeños proyectos o miniproyectos.



Esquema de ciclo de vida incremental

Además, hoy en día se trata de tender a reducir los riesgos y, en este sentido, el ciclo de vida en cascada no proporciona muchas ventajas. Debido a todo esto, el ciclo de vida típico en una metodología de diseño orientado a objetos alguna variación del ciclo de vida en espiral.

Un ejemplo de ciclo de vida Orientado a Objetos es elllamado “modelo fuente”, que fue desarrollado por Henderson-Sellers y Edwards en 1990.Es un tipo de ciclo de vida pensado para ser aplicado siguiendo el paradigma de la orientación a objetos y posiblemente el más seguido con la ventaja de que permite un desarrollo solapado e iterativo.

Un proyecto en modelo fuente se divide en las siguientes fases:

– Planificación del negocio.

– Construcción: Es la más importante y se subvidide a su vez en otras tantas actividades: Planificación, Investigación, Especificación, Implementación y Revisión.

– Entrega o ”liberación”.

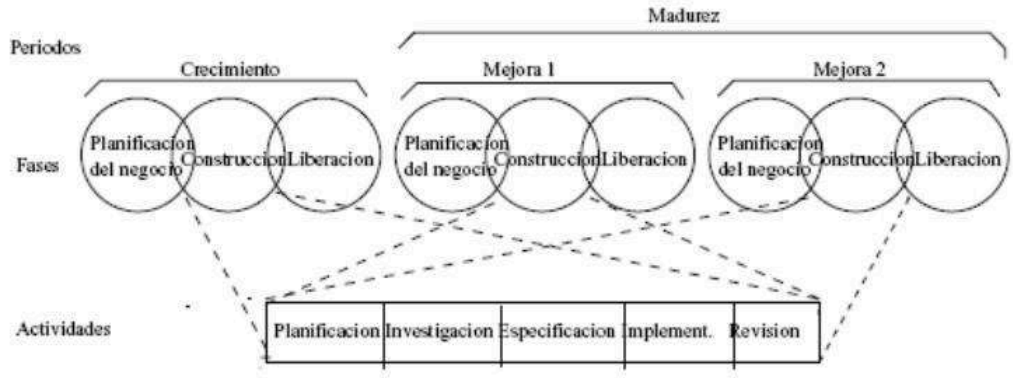
La primera y la tercera fase son independientes de la metodología de desarrollo orientado a objetos. Que se utilice Además de las tres fases, existen dos periodos:

– Crecimiento: Es el tiempo durante el cual se esta construyendo el sistema.

– Madurez: Es el periodo de mantenimiento del producto. En el cada mejora se planifica igual que el periodo anterior, o sea, llevando a cabo las fases de Planificación del negocio, Construcción y Entrega.

Cada una de las clases de la aplicación desarrollada puede tener un ciclo de vida propio debido a que cada clase puede estar en una fase diferente en un momento cualquiera.

Un esquema de su estructura es el siguiente:



Esquema de ciclo de vida orientado a objetos

La fase de análisis de requisitos comienza tras el análisis del sistema donde se va a encuadrar el producto que se va desarrollar y tiene como objetivo crear un modelo de los requisitos que debe de cumplir el software. Este modelo se plasmará en un documento, la "especificación de requisitos del software" que será el producto final de esta fase y el punto de comienzo de la siguiente fase, el diseño.

El análisis de requisitos podemos subdividirlo en tres partes, la búsqueda de requisitos y el modelado de requisitos.

La búsqueda de requisitos tiene como objetivo descubrir las verdaderas necesidades del cliente que ha encargado el desarrollo del sistema software.

En la mayoría de las ocasiones el cliente que desea un desarrollo no tiene al comienzo muy claro que producto necesita. En otras ocasiones puede tener claro que es lo que quiere pero la labor del ingeniero del software es determinar que es lo que necesita.

Para ello se pueden utilizar varias técnicas, como las que se definen a continuación.

1.3. Identificación de requisitos: concepto, evolución y trazabilidad



Entrevistas: reuniones entre el cliente y el equipo desarrollador, en la que se determinan los requisitos del sistema.

Estas siempre la tendremos, al menos al inicio del proyecto, aunque conviene que se hagan con cierta frecuencia para que se expliciten los requisitos y estos se refinen. Conviene que las entrevistas no solamente sean con la alta dirección o la gerencia, sino que en ella se involucren otros actores, preferiblemente los usuarios que van a trabajar con la aplicación final y que conocerán mucho menos los rusitos de esta.



Desarrollo conjunto de aplicaciones (JAD): Es un tipo de entrevista con muchos participantes desarrollada por IBM que se apoya en la dinámica de grupos. LaPlanificación conjunta de requisitos (JRP) es un subconjunto de las sesiones JAD, dirigidas a la alta dirección y los productos que resultan de ellas son los requisitos de alto nivel o estratégicos.

Las entrevistas de JAD son unas entrevistas muy estructuradas en las que hay participantes de todos los niveles de la organización que van a estar involucrados en el proyecto de software. Siguiendo un guión previamente marcado se van explicitando uno a uno los requisitos de la aplicación.

En el caso del JRP se trata de detectar los requisitos estratégicos de la organización, que sirvan como base de otros desarrollos. Por esta razón en ella se involucran sobre todo elementos de la alta dirección de la empresa cliente y de la empresa desarrolladora.



Brainstorming: es otro tipo de entrevista de grupo. A diferencia del JAD que está fuertemente estructurada, se caracteriza precisamente por lo contrario, ser de una naturaleza muy libre.

La principal característica del brainstorming es que los participantes han de sentirse libres de expresar su opinión o inquietudes en un entrono sin críticas que coarten la expresión. Posteriormente las ideas que se recopilen se refinan y se irán detectando sus ventajas e inconvenientes.



Prototipos: un prototipo es una versión reducida de la aplicación final. Se puede construir para obtener y validar requisitos.

Casos de uso: son una forma de especificar los requisitos de un sistema. Un caso de uso consiste en una interacción de algo externo al sistema y el sistema.

Aunque es una técnica definida dentro del ambiente del análisis orientado a objetos no tiene que ver con objetos, se podría utilizar perfectamente dentro del análisis estructurado.



Los casos de uso fueron introducidos por Jacobson en 1992 para ser utilizado con la metodología de análisis orientada a objetos que había desarrollado.

Un caso de uso describe la interacción entre un actor externo al sistema y el sistema con texto en lenguaje natural y representa los requisitos funcionales desde el punto de vista del usuario y por lo tanto produce un resultado observable por éste.

De esta manera el sistema que se va a desarrollar se presenta como una serie de interacciones entre los usuarios y el sistema.

Especificando así el sistema, se empiezan a determinar los límites del sistema y que es lo que se espera que este haga desde el punto de vista del usuario

El modelado de requisitos tiene como objetivo crear un modelo del problema que se pretende solventar y de los requisitos que debe de cumplir el sistema a implementar.De esta forma sirva para comprobar la coherencia de los requisitorios y servir como punto de partida del diseño, que consistirá en crear otro modelo, pero esta vez de la solución propuesta.

Dentro del modelado de los requisitos en la actualidad conviven dos tendencias:

– Análisis orientado a objetos: representa el problema en términos de clases, propiedades de estas clases y relaciones entre ellas.

– Análisis estructurado: crea un modelo con tres vertientes, modela los datos, los procesos y el comportamiento haciendo uso de varias herramientas.

La trazabilidad de requisitos es una subdisciplina de la gestión de requisitos en el desarrollo de software y la ingeniería de sistemas .

La trazabilidad de requisitos se refiere a la documentación generada durante la vida de un requisito y proporcionar información sobre los enlaces que tengan entre sí varios requisitos asociados.

La trazabilidad lo que permite es que los usuarios puedan encontrar **fácilmente** el origen de cada requisito y el seguimiento de cada cambio realizado a este requisito. Para este fin, es necesario documentar cada cambio realizado al requisito.



La trazabilidad se puede definir de manera genérica como la «capacidad de interrelacionar cronológicamente las entidades singularmente identificables de una manera significativa.

Lo que importa en la gestión de requisitos no es una evolución temporal tanto como una evolución estructural: es decir, dejar un rastro de que los requisitos que se derivan de, cómo se cumplen, la forma en que se ponen a prueba, y qué impacto producirá si se cambian.

Los requisitos provienen de diferentes fuentes, como el directivo que encarga el producto, el gerente de marketing y el usuario. Estas personas tienen diferentes visiones y requisitos sobre el producto.

Con el uso de requisitos de trazabilidad, una característica implementada se puede buscar de nuevo a la persona o grupo que lo quiso durante la búsqueda y elicitación de requisitos.

Esto puede ser utilizado durante el proceso de desarrollo para priorizar el requisito, la determinación de cómo de valioso es el requisito para un determinado usuario.

También se puede utilizar después de la implementación, en la fase de mantenimiento, cuando los estudios muestran que una característica que no se usa, para ver por qué se requirió originalmente.

La trazabilidad de requisitos a la documentación de las relaciones entre los requisitos y otros elementos de desarrollo.

Su finalidad es la de facilitar la calidad global del producto o productos en fase de desarrollo, la comprensión del producto en fase de desarrollo y su elemento final, el producto terminado así como el mantenimiento adaptativo, es decir la capacidad de gestionar un cambio solicitado por el cliente

No sólo los propios requisitos deben ser rastreados sino también la relación con los requisitos de todos los elementos asociados a el, como los modelos, resultados de análisis, casos de prueba, procedimientos de prueba, resultados de pruebas y documentación de todo tipo.

Incluso las personas y grupos de usuarios asociados con los requisitos deben ser trazables.



Definición

Una definición muy común de **trazabilidad de los requisitos:** se refiere a la capacidad de describir y seguir la vida de un requisito, tanto hacia delante como hacia atrás(es decir, desde sus orígenes, a través de su desarrollo y especificación, para su posterior distribución y uso, y a través de todos los períodos del ciclo de vida así como de iteraciones en cualquiera de sus fases.

Si bien esta definición enfatiza el seguimiento de la vida de un requisito a través de todas las fases de desarrollo, no señala explícitamente que la trazabilidad puede documentar las relaciones entre muchos tipos de elementos de desarrollo, como los requisitos, declaraciones de especificaciones, diseños, pruebas, modelos y componentes desarrollados.

Por las razones expuestas la definición anterior se puede completar formulándose de la siguiente manera:

Trazabilidad de requisitos se refiere la capacidad de definir, capturar y seguir las huellas dejadas por las necesidades capturadas en otros elementos del entorno de desarrollo de software y la huella dejada por dichos elementos sobre los requisitos.

Esta definición enfatiza el uso de la trazabilidad para documentar la transformación de un requisito en el diseño y desarrollo de elementos.



Importante

En el campo de la ingeniería de requisitos, la trazabilidad es la comprensión de cómo los requisitos de alto nivel - los objetivos, metas, objetivos, aspiraciones, expectativas, necesidades - se transforman en requisitos de bajo nivel y por lo tanto, se refiere principalmente a las relaciones entre las capas de información y niveles de abstracción.

La relación principal se hace referencia en la trazabilidad de requisitos se puede caracterizar como satisfacción. Es decir se trata de responder a la pregunta: ¿cómo es un requisito satisfecha por elementos del software desarrollado? Es decir, indicar qué elemento satisface un requisito.

Otras relaciones indican la manera en que puede un requisito ser rastreado desde el software en producción hasta una necesidad.

En este caso se habla de una relación de «verificación», o sea se trata de responder a ¿cómo es un requisito verificado por los elementos de la prueba?

Uno de los elementos fundamentales a la hora de poder determinar la traza de unos requisitos es la matriz de trazabilidad.

Una **matriz de trazabilidad** es un documento, por lo general en forma de una tabla, que se correlaciona dos documentos que mantienen una relación de muchos a muchos para determinar la integridad de la relación.

Se utiliza con frecuencia con requisitos de alto nivel (estos a menudo consisten en requisitos de comercialización) y los requisitos detallados del producto con los elementos que los satisfacen de requisitos de diseño de alto nivel, diseño detallado, plan de pruebas y casos de prueba .

Una matriz de trazabilidad de requisitos se puede utilizar para comprobar si se están cumpliendo los requisitos actuales del proyecto, y para ayudar en la creación de una solicitud de proyecto, de la especificación de requisitos de software, las tareas de diversos documentos de entrega, y el plan del proyecto.

La forma más común de utilizarla es tomar el identificador para cada uno de los elementos de un documento y colocarlos en la columna de la izquierda. Los identificadores para el otro documento se colocan en la fila superior.

Cuando un elemento de la columna de la izquierda se relaciona con un elemento en la parte superior, se coloca una marca en la celda de intersección. Se debe determinar si una relación debe existir para cada elemento de la matriz.

El número de relaciones se suman para cada fila y cada columna. Este valor indica el grado de interrelación de los dos elementos. Los valores de cero indican que no existe ninguna relación. Los valores elevados implican que la relación es muy compleja y debe simplificarse.

Para facilitar la creación de matrices de trazabilidad, es recomendable añadir las relaciones con los documentos fuente, tanto para la trazabilidad hacia atrás y trazabilidad hacia delante.

De esta manera, cuando un elemento se cambia en un documento línea base, es fácil ver lo que necesita ser cambiado en la otra.

Ejemplo de una matriz de requisitos es la siguiente:

Identificador Requisito	Req Probad	Req1 UC 1.1	Req1 UC 1.2	Req1 UC 1.3	Req1 UC 2.1	Req1 UC 2.2	Req1 UC 2.3.1	Req1 UC 2.3.2	Req1 UC 2.3.3
Casos de Prueba	321	3	2	3	1	1	1	1	1
Probado implícitamente	77								
1.1.1	1	x							
1.1.2	2		x	x					
1.1.3	2	x							
1.1.4	1			x					
1.1.5	2	x							
1.1.6	1		x						
1.1.7	1			x					
1.2.1	2				x		x		
1.2.2	2					x		x	
1.2.3	2								x
1.3.1	1								
1.3.2	1								
1.3.3	1								
1.3.4	1								
1.3.5	1								

1.4. Análisis de metodologías de desarrollo orientadas a objeto

El proceso de construcción del software requiere, como cualquier otra ingeniería, identificar las tareas que se han de realizar sobre el software y aplicar esas tareas de una forma ordenada y efectiva.

Además el desarrollo del software se debe realizar por un conjunto coordinado de personas simultáneamente, y por lo tanto sus esfuerzos deben estar dirigidos por un mismo plan de trabajo o metodología que permita estructurar las diferentes fases del desarrollo.

En la literatura sobre ingeniería del software existen muchas definiciones sobre lo que es una metodología. Más o menos todas ellas coinciden en que debería tener al menos las siguientes características:

– Define como se divide un proyecto en fases y las tareas a realizar en cada una.

– Para cada una de las fases está especificado cuáles son las entradas que recibe y las salidas que produce.

– Tienen alguna forma de gestionar el proyecto.



Teniendo esto en cuenta estas características se puede dar una definición, escueta, pero no por eso menos completa de **metodología** como un modo sistemático de producir software.

El objetivo de una metodología será crear un producto software que cumpla con las siguientes características:

– Adecuación: el sistema satisface las expectativas del cliente o usuario.

– Mantenibilidad: facilidad para realizar cambios una vez que el sistema está funcionando

– Usabilidad: es la facilidad en aprender a manejar el sistema por parte de un usuario final.

– Fiabilidad: es la capacidad de un sistema de funcionar correctamente durante un tiempo dado. Se diferencia de la corrección en que lo que mide es el tiempo, es decir, no se trata del número absoluto de defectos en el sistema sino de los que se manifiestan en un intervalo de tiempo; está muy relacionada con la siguiente característica.

– Disponibilidad: probabilidad de que el sistema esté funcionando en un instante dado.

– Corrección: densidad de defectos mínima.

– Eficiencia: el sistema es capaz de realizar su tarea con el mínimo consumo de recursos necesario

Existen dos grupos de metodologías en función de la idea con la que se aborda el problema: metodologías estructuradas y metodología orientada a objetos.

– Metodología estructurada: estas metodologías consisten en la primera aproximación a los problemas planteados por la Ingeniería del Software. Estas metodologías están en general orientada a procesos, es decir, se centra en especificar y descomponer la funcionalidad del sistema.

  Es decir, de lo que se trata es de ir especificando sé es lo que va a hacer el sistema y como lo va a a hacer desde el punto de vista de la máquina.

  La mentalidad que subyace al diseño estructurado es tratar de dividir el sistema en partes más pequeñas que puedan ser resueltas por algoritmos sencillos y qué información se intercambian.De esta manera se buscó en primer lugar resolver los problemas que se habían detectado por la creación de software de manera artesanal.

  Este tipo de metodologías fueron desarrollándose durante los años 70 y siguen siendo muy utilizadas, sin embargo se han desarrollado muchos enfoques posteriores que permiten abordar el problema de la complejidad del software de formas mas sofisticadas.

– Metodologías orientadas a objetos: Es una aproximación posterior y más avanzado. Se basa en as anteriores, pero busca unificar los procesos y la información en las mismas entidades, lo que se denominan los tipos abstractos de datos.

En el diseño orientado a objetos la idea es identificar los tipos de datos que hay que utilizar, qué características tienen y cómo se relacionan.

Actualmente cuenta con mayor número de seguidores y prácticamente en los últimos años esta sustituyendo a la anterior.



Las metodologías orientadas a objeto cuentan con una serie de ventajas importantes como el hecho de estar basadas en componentes, lo que significa que es más fácil reutilizar código hecho por terceras personas.

Otra ventaja importante es que las aplicaciones son más fáciles de mantener debido a que los cambios están más localizados y es más sencillo añadir nuevos requisitos o modificar los existente.



Durante muchos años el término orientado a objetos se usó para referirse a un enfoque de desarrollo de software que usaba uno de los lenguajes orientados a objetos (C++,Eiffel, Smalltalk, Java). Hoy en día el paradigma OO encierra una completa visión de la ingeniería del software.

Ejemplo

Para entender la visión orientada a objetos, consideremos un ejemplo de un objeto del mundo real, por ejemplo una silla.

Silla es un miembro (o instancia) de una clase mucho más grande de objetos que llamaremos Mobiliario. Un conjunto de atributos genéricos puede asociarse con cada objeto, en la clase Mobiliario. Por ejemplo, todo mueble tiene un costo, dimensiones, peso, localización y color, entre otros muchos posibles atributos.

Estos son aplicables a cualquier elemento sobre el que se hable, una mesa o silla, un sofá o un armario. Como Silla es un miembro de la clase Mobiliario, hereda todos los atributos definidos para dicha clase.

Todo objeto de una clase puede manipularse de varias maneras. Por ejemplo el mobiliario puede comprarse y venderse, modificarse físicamente o moverse de un lugar a otro.

Cada una de las operaciones (otros términos son servicios o métodos) modificará uno o más atributos del objeto.

Dicho de otra manera, una clase es una descripción generalizada (por ejemplo, una plantilla, un patrón o un prototipo) que describe una colección de objetos similares. Por definición, todos los objetos que existen dentro de una clase heredan sus atributos y las operaciones disponibles para la manipulación de los atributos.



Definición

Una **superclase** es una colección de clases y una **subclase** es una instancia de una clase.

Estas definiciones implican la existencia de una jerarquía de clases en la cual los atributos y operaciones de la superclase son heredados por subclases que pueden añadir, cada una de ellas, atributos y métodos.

Un objeto encapsula datos (representados como una colección de atributos) y los algoritmos que procesan estos datos. Estos algoritmos son llamados operaciones, métodos o servicio y pueden ser vistos como módulos en un sentido convencional.

Cada una de las operaciones encapsuladas por un objeto proporciona una representación de uno de los comportamientos del objeto. Cada vez que un objeto recibe un estímulo, éste inicia un cierto comportamiento, tan simple o tan complejo como se necesite.

Los mensajes son el medio a través del cual interactúan los objetos. Usando la terminología orientada a objetos, un mensaje estimula la ocurrencia de cierto comportamiento en el objeto receptor. El comportamiento se realiza cuando se ejecuta una operación.

Hay tres características de los sistemas orientados a objetos que los hacen muy versátiles: la encapsulación, la herencia y el polimorfismo.

Como ya hemos observado, las clases y los objetos derivados de ella encapsulan los datos y las operaciones que trabajan sobre estos en un único paquete.

Esto proporciona un número importante de beneficios:

- Los detalles de implementación interna de datos y procedimientos están ocultos al mundo exterior (ocultación de la información). Esto reduce la propagación de efectos colaterales cuando ocurren cambios.

- Las estructuras de datos y las operaciones que las manipulan están mezcladas en una entidad sencilla: la clase. Esto facilita la reutilización de componentes.

- Las interfaces entre objetos encapsulados están simplificadas.

- Un objeto que envía un mensaje no tiene que preocuparse de los detalles de las estructuras de datos internas en el objeto receptor, lo que simplifica la interacción y hace que el acoplamiento del sistema tienda a reducirse.



La herencia es una de las diferencias clave entre sistemas convencionales y sistemas OO. Una subclase hereda todos los atributos y operaciones asociadas con su superclase.

Esto significa que todas las estructuras de datos y algoritmos originalmente diseñados e implementados para la clase base están inmediatamente disponibles para la clase heredada.

La reutilización se realiza directamente. Cuando se hereda de una clase en ese mismo momento se disponen de todas las operaciones y datos que disponía la clase padre.

Cualquier cambio en los datos u operaciones que se realice en el interior de una superclase es heredado inmediatamente por todas las subclases que se derivan de la superclase.

Debido a esto, la jerarquía de clases se convierte en un mecanismo a través del cual los cambios (a altos niveles) pueden propagarse inmediatamente a través de todo el sistema.

Esto conlleva múltiples ventajas peso también en ocasiones si no se realiza un buen diseño, puede conllevar cambios y efectos no deseados cuando se implementa una modificación en alguna parte del código .

Es importante destacar que en cada nivel de la jerarquía de clases, pueden añadirse nuevos atributos y operaciones a aquellos que han sido heredados de niveles superiores de la jerarquía.

De hecho, cada vez que se debe crear una nueva clase, el ingeniero del software tiene varias opciones:

– La clase puede diseñarse y construirse de la nada: Esto es, no se usa la herencia.

– La jerarquía de clases puede ser rastreada para determinar si una clase ascendiente contiene la mayoría de los atributos y operaciones requeridas.

– De esta forma, La nueva clase hereda de su clase ascendiente, y pueden añadirse otros elementos si hacen falta.

– La jerarquía de clases puede reestructurarse de tal manera que los atributos y operaciones requeridos puedan ser heredados por la nueva clase.

– Las características de una clase existente pueden sobrescribirse y se pueden implementar versiones privadas de atributos u operaciones para la nueva clase.

El polimorfismo es una característica que reduce en gran medida el esfuerzo necesario para extender un sistema Orientado a Objetos.



Definición

El **polimorfismo** consiste en que dependiendo de varios factores, en el momento en que recibe un mensaje, dependiendo de los parámetros, o del tipo de objeto que recibe este mensaje, el objeto exhibirá un comportamiento u otro.

Ejemplo

Por ejemplo, imaginemos una aplicación que debe dibujar cuatro tipos diferentes de gráficos: gráficos de líneas, gráficos de tarta, histogramas y diagramas de Kiviat.

Una vez que se han recogido los datos necesarios para un tipo particular de gráfico, el gráfico debe dibujarse él mismo.

Para realizar esto en una aplicación convencional sería necesario desarrollar módulos de dibujo para cada tipo de gráfico.

Añadir nuevos tipos de gráficos puede ser complicado, pues hay que crear un nuevo módulo de dibujo para cada tipo de gráfico y actualizar la lógica de control de la aplicación para cada gráfic

Para resolver este problema, cada uno de los gráficos mencionados anteriormente se convierte en una subclase de una clase general llamada Gráfico. Usando el concepto de polimorfismo, cada subclase define una operación llamada dibujar.

Un objeto puede enviar un mensaje dibujar a cualquiera de los objetos instanciados de cualquiera de las subclases. El objeto que recibe el mensaje invocará su propia operación dibujar para crear el gráfico apropiado.

A continuación se describen brevemente las principales metodologías que se han propuesto para el análisis y diseño de sistemas siguiendo el paradigma orientado a objetos:

El método de Booch abarca un “microproceso de desarrollo” y un “macroproceso de desarrollo”. El nivel micro define un conjunto de tareas de análisis que se reaplican en cada etapa en el macro proceso. Por tanto sigue un enfoque evolutivo.

El micro proceso de desarrollo se realizan tres tareas:

– En primer lugar se identifica clases y objetos, es decir los ítems de datos del problema que se va a tratar de resolver.

– Posteriormente se busca la semántica de dichas clases y objetos, es decir se definen las relaciones entre clases y objetos o lo que es lo mismo, como va a ser el comportamiento externo de los objetos de datos.

– Por último realiza una serie de refinamientos para elaborar el modelo del análisis, es decir, se aplica la misma metodología al comportamiento de cada ítem en particular con el fin de continuar detallando el sistema.

Posteriormente en el diseño se pasan a definir las características a bajo nivel de las clases.

El método de Rumbaugh toma el nombre del autor que desarrolló la Técnica de Modelado de Objetos (OMT) para el análisis, diseño del sistema y diseño a nivel de objetos.

En OMT la actividad de análisis crea tres modelos: el modelo de objetos, el modelo dinámico y el modelo funcional.

– El modelo de objetos consisten en una representación de los objetos, clases, jerarquías y relaciones presentes en el sistema.

– El modelo dinámico es una representación del comportamiento del sistema y los objetos que lo forman, así como de la interacción de estos objetos.

– El modelo funcional trata de ser una representación a alto nivel del flujo de información a través del sistema, utilizando para ello lo que se denominan los Diagramas de Flujo de datos, o DFD.

El método de Jacobson, También llamado OOSE (en español Ingeniería del Software Orientada a Objetos), es una versión simplificada del método denominado Objector, el cual es un método también desarrollado por Jacobson.

Este método se diferencia de los otros por la importancia que da al caso de uso, una descripción o escenario que describe cómo el usuario interactúa con el producto o sistema



Sabías que

El método de Coad y Yourdon se considera, con frecuencia, como uno de los métodos del Análisis orientado a Objetos más sencillos de aprender.

La notación del modelado de Coad y Yourdon es relativamente simple y las reglas para desarrollar el modelo de análisis son sencillas.

Una descripción simple del proceso es:

– Identificar objetos, definir una estructura de generalización-especificación.

– Definir una estructura de todo-parte.

– Identificar temas (representaciones de componentes de subsistemas).

– Definir atributos y definir servicios.

El método de Wirfs-Brock: EN este no se hace una distinción clara entre las tareas de análisis y diseño, sino que estas están interrelacionadas.

En lugar del análisis y diseño clásico, lo que propone este método es un proceso continuo que comienza con la valoración de unaespecificación del cliente y termina con el diseño. Básicamente la tareas a realizar son:

– Evaluar la especificación del cliente.

– Usar un análisis gramatical para extraer clases candidatas de la especifi-cación.

– Agrupar las clases en un intento de determinar superclases.

– Definir responsabilidades para cada clase.

– Asignar responsabilidades a cada clase.

– Identificar relaciones entre clases.

– Definir colaboraciones entre clases basándose en sus responsabilidades.

– Construir representaciones jerárquicas de clases para mostrar relaciones de herencia.

– Construir un gafo de colaboraciones para el sistema.

– Diseñar a bajo nivel las clases y objetos.



La metodología métrica es la metodología oficial de desarrollo de aplicaciones de la administración española.

El objetivo de la metodología métricas es sistematizar todas las actividades del ciclo de vida y las que no son parte de este, pero que influyen en el. Las principales características de la última versión (Métrica V3) son:

– Cubre los dos tipos de desarrollos, estructurados y Orientados a Objetos.

– Incluye procesos que no forman parte del ciclo de vida.

– Tiene en cuenta las tecnologías para el diseño de sistemas distribuidos, como Cliente-Servidor y el desarrollo de Interfaces Gráficas de usuario.

Divide las fases del desarrollo en dos partes, procesos principales e interfaces.



Procesos principales son los procesos relativos a la planificación, desarrollo y mantenimiento de la aplicación. Interfaces son procesos asociados al desarrollo que deben de producirse durante todas las fases de éste, por ejemplo Gestión de la Configuración del Software, gestión de proyectos, aseguramiento de la calidad…

Cada proceso se divide en una serie de actividades y cada actividad tiene asociada una descripción y una tabla de tareas a desarrollar.

Cada tarea está descrita y define los productos que necesita como entradas, los que produce como salida, las acciones que hay que llevar a cabo y los participantes en dicha tarea.

Además tiene una estructura flexible, no es necesario seguir todos los procesos y actividades sino que éstas pueden adaptarse en función del proyecto.

Brevemente un esquema de los procesos e interfaces es el siguiente:

Métrica V3:

– Procesos:

  · Planificación (PSI).

  · Desarrollo:

- › Estudio de Viabilidad (EVS).
- › Análisis del Sistema de Información (ASI).
- › Diseño del Sistema de Información (DSI).
- › Construcción del Sistema de información (CSI).
- › Implantación y aceptación del S.I. (IAS).

· Mantenimiento (MSI).

– Interfaces:

· Gestión de la configuración (GC).

· Gestión del proyecto (GP).

– Aseguramiento de la calidad (CAL).

– Seguridad (SEG).



Merisse es la metodología de desarrollo de sistemas de información adoptada por al administración francesa.

El desarrollo de un proceso en Merisse se divide en las siguientes etapas:

– Esquema director: en ella se fijan las orientaciones generales para el desarrollo del sistema de información. Se trata de la primera fase del proceso de ingeniería de procesos de negocio.

– Estudio preliminar: se definen los límites del sistema y la interacción con el resto de la organización. En esta fase se detallan como se va interrelacionar el sistema con otros desarrollos ya hecho

– Análisis detallado: se especifican y modelan los requisitos de la aplicación a desarrollar.

– Análisis técnico. Diseño arquitectónico y a bajo nivel de la aplicación.

– Realización: implementación del sistema, pruebas e implantación.

– Mantenimiento: tareas a llevar a cabo una vez el sistema está en funcionamiento para corregir fallos detectado y adaptarlo a los nuevos requisitos.



Sabías que

Structured System And Design Method (SSADM) es la metodología oficial de las administraciones de Reino Unido.

Las fases de SSADM, (denominadas módulos) en las que se divide el desarrollo son las siguientes:

– Feasibility Study: se lleva a cabo un análisis de alto nivel sobre el sistema a implementar. Realmente de lo que se trata es de determinar la viabilidad del sistema qye se desarrollará

– Requirement Analysis: se buscan los requisitos del sistema. Corresponde a la fase clásica del extracción y elicitación de requisitos.

– Requirement Specification: en esta fase se modelan los requisitos del sistema a desarrollar. Realmente corresponde a lo que hemos denominado modelado de requisitos, ya que se hace un modelo del sistema a desarrollar.

– Logical System Specification: se define el sistema a desarrollar, en este módulo se llevan a cabo las tareas de diseño. Se hace el diseño del sistema tanto arquitectónico como a bajo nivel.

– Physical Design: implementación del sistema. Incluye la fase de pruebas y de implantación

El Proceso Unificado del Rational es una de las metodologías más extendidas y conocidas entre las empresas de desarrollo de software. Está basada en el Lenguaje Unificado de Modelado (UML) propuesto por Jacobson, Booch y Rumbaugh.



A raíz de que propusieran esta metodología, a Jacobson, Booch y Rumbaugh se les conoce como “los tres amigos” en el mundo de la ingeniería del software.

Sus características principales son:

– Está basada en componentes, que son básicamente clases y paquetes reutilizables, que se interconectan entre sí mediante interfaces.

– Su notación tanto para el análisis como para el diseño es UML. Para ello se usan unas serie de esquemas estandarizados que representan cada uno de los puntos de vista del sistema que se quiere desarrollar y sirven de pilar básico a las tareas de ingeniería.

– Es dirigida por casos de Uso, los casos de uso son un tipo de herramienta de análisis que permitan representar una serie de requisitos funcionales desde el punto de vista del usuario de la aplicación.

– Los casos de uso guían en esta metodología todo el proceso de análisis, diseño, implementación y pruebas. Se trata que el sistema se diseñe teniendo en consideración el punto de vista del usuario que va a interactuar con el y el modo en el que se desarrolla esta interacción.

– Utiliza un ciclo de vida iterativo e incremental, en el cual el proyecto se divide en subproyectos, cada uno de los cuales se realiza en una iteración.

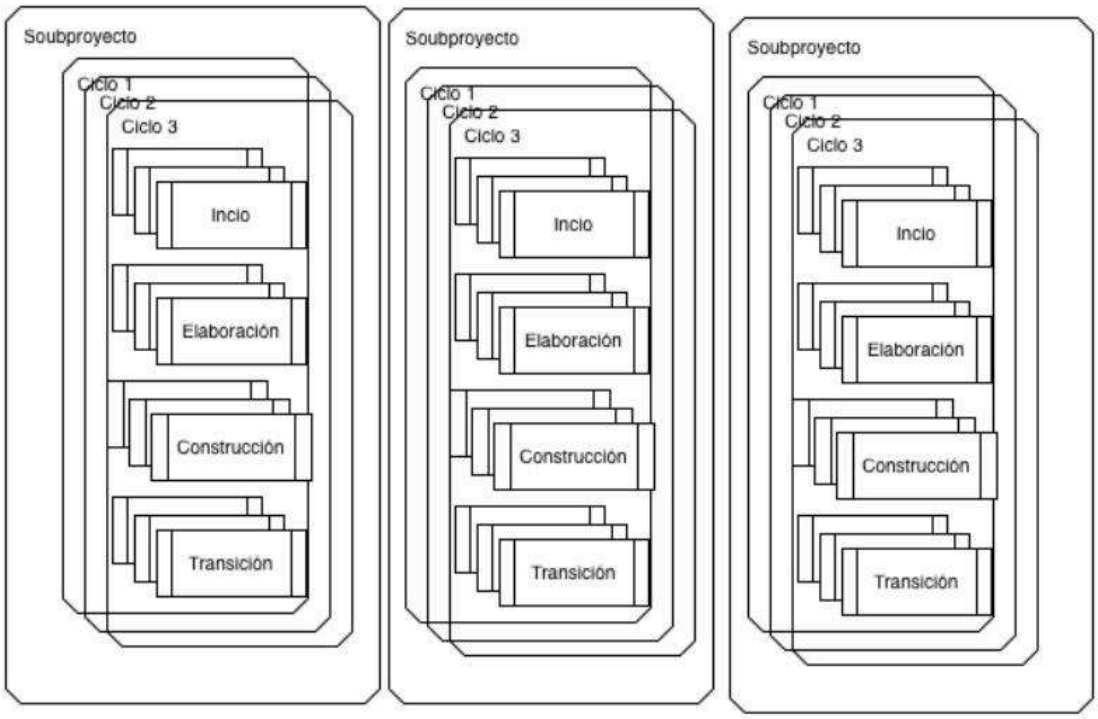
– En cada iteración se trabaja con una serie de casos de uso y se tratan de solucionar una serie de riesgos.

– Después de cada ciclo se tiene una visión del producto. Cada ciclo se divide en las siguientes fases: Inicio, elaboración, construcción y transición.

La fase de inicio consiste en un análisis de riesgos y de las características que debe tener la iteración y si esta se va desarrollar o no. En la elaboración se realizan lo que son propiamente al características de análisis y diseño del sistema que se quiere construir.

Posteriormente pasamos a la fase de construcción en la a que se realiza la implementación del sistema, así como las pruebas unitarias y de integración de la nueva versión que se ha desarrollado en esta fase.

Por último en la fase de transición consiste en el proceso de migración desde la anterior versión a la nueva que se ha desarrollado durante este ciclo de ingeniería. Una vez acabada, se pasaría de nuevo a hacer un análisis de riesgos en la fase de inicio.



Esquema de metodología del proceso unificado del Rational

1.5. Resolución de un caso práctico de metodologías de desarrollo que utilizan UML

Como ejemplo de cómo utilizar UML vamos a resolver un problema que consistirá en diseñar una aplicación de gestión para un videoclub que tiene una serie de socios y cuyo negocio consiste en el alquiler de películas.

El problema a resolver puede formularse de la siguiente manera:

Ejemplo

Se desea desarrollar un programa Orientado a Objetos que permita gestionar un Videoclub.

El videoclub es una empresa cuyo negocio consiste en ofrecer a a sus clientes una serie de películas durante un período de tiempo determinado para que estos las disfruten.

Para ello el videoclub tiene en sualmacén una serie de películas. Por tanto la aplicación que se desea desarrollar debe permitir dar de alta y consultar información sobre estas películas.

Además el videoclub cuenta con una serie de clientes. De estos clientes se de vital importancia para que la organización consigue sus objetivos el guardar información personal sobre estos, y ° por tanto de nuevo será necesario poder dar de alta y modificar estos datos.

En tercer lugar el programa deberá poder automatizar el proceso de alquiler, en el que un cliente se lleva una película durante un período de tiempo determinado, no estando por tanto disponible para otros clientes.

La devolución también deberá ser gestionada, y será el proceso por el cual el cliente trae de nuevo la película y ésta pasa a estar disponible para el resto.

A partir de este enunciado vamos a seguir los siguientes pasos para realizar el análisis y diseño de una aplicación orientada a objetos:

- En primer lugar como los requisitos del sistema están claros (son los que aparecen en el enunciado) se van a identificar los casos de uso.

- Los casos de uso anteriormente identificados se van a documentar en detalle, y posteriormente se va a construir el diagrama de casos de uso.

- Con el diagrama de casos de uso, y la descripción de los requisitos del sistema se van a hallar las clases presentes en el sistema, y se va a plantear un primer diagrama de clases.

- Posteriormente se van a desarrollar los diagramas de secuencias o escenarios para cada caso de uso, De esta manera se irán identificando los posibles mensajes que se pasen los objetos.

- Ya que están identificados los mensajes, cada uno de ellos se corresponderá con un método, y por tanto se podrá plantear el diagrama de clases definitivo.

Para construir el diagrama de casos de uso en primer lugar se identifican los actores.

En este problema solamente vamos a tener un actor, el usuario de la aplicación que será aquella persona que se encarga de dar de alta y de baja películas, dar de alta y de baja a clientes y llevar a cabo los procesos de alquiler y devolución.

Una vez identificado al actor, pasamos a considerar los posibles casos de uso de nuestro sistema. Del análisis del enunciado del problema planteado podemos obtener una primera lista de casos de uso:

– Dar de alta una película.

– Dar de alta un cliente.

– Realizar un alquiler.

– Realizar una devolución.

A partir de esta primera lista podemos construir un primer esbozo del diagrama de casos de uso:

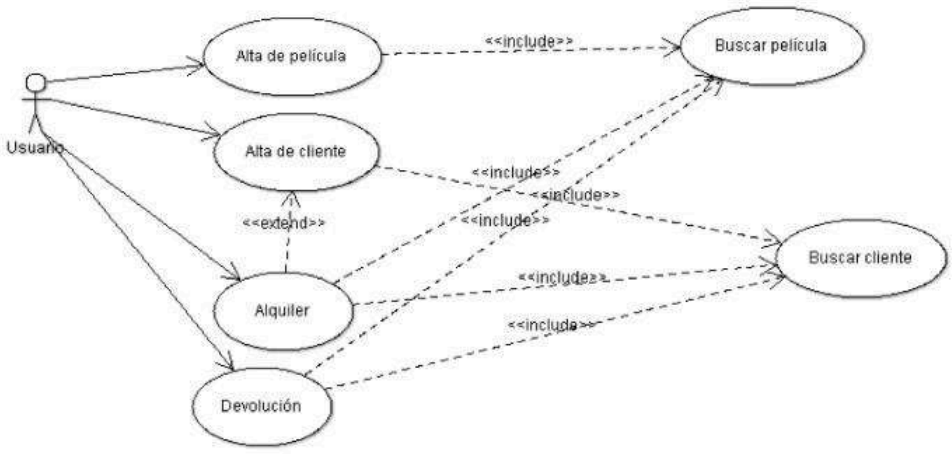


Diagrama de casos de uso

Se puede observar en la descarga anterior casos de uso anteriormente comentados. Los cuales representan cada una de las funcionalidades básicas

que va a llevar a cabo el sistema que se está desarrollando desde el punto de vista de un usuario externo que está interactuando con el .

Los casos de uso de dar de alta una película, realizar un alquiler, alta de cliente y devolución son llamados directamente por el usuario de la aplicación

Este interactuará con el interfaz del sistema que estamos desarrollando y realizará sobre este las acciones necesarias para llevar a cabo cada una de las acciones correspondientes.

Por ejemplo al dar de alta una película introducirá los datos de la película, se cerciorará de su corrección, etc.

De a misma manera actuará cuando quiera realizar otra acción sobre el interfaz. De esta manera los casos de uso se constituyen en un escenario de trabajo del usuario sobre la aplicación que hay que modelar en el sistema, y es la primera fase para identificar los requisitos que debe de cumplir nuestro sistema y que al finalizarlo habrá que verificar.

A los casos de uso anteriores se han añadido en el diagrama otros dos, pues parece lógico que la aplicación deberá contar con un método para buscar películas y clientes tanto a la hora de dar de alta a alguno de ellos como cuando se gestione un alquiler o una devolución.

Estos dos casos de uso no son utilizados directamente por el actor sino que son usados por otros casos de uso cuando lo requieran.

Como siguiente paso vamos a ir definiendo cada caso de uso por separado:

Caso de uso: Alta de película. Actor: Usuario	
Curso normal	Alternativas
El usuario introduce los datos de la película El sistema comprueba que la película no esté introducida (llama al caso de uso buscar película) El sistema almacena los datos de la película	SI la película está ya previamente introducida, muestra el error.

Caso de uso: Alta de cliente. Actor: Usuario	
Curso normal	Alternativas
El usuario introduce los datos del cliente. El sistema comprueba que el cliente no esté introducido (llama al caso de uso buscar cliente). El sistema almacena los datos del cliente.	Si el cliente está ya previamente introducido, muestra el error.

Caso de uso: devolución Actor: Usuario	
Curso normal	Alternativas
El usuario introduce los datos del cliente El sistema muestra el importe el sistema almacena los datos de la devolución	SI no hay alquiler muestra mensaje de error.

Caso de uso: Alquiler Actor: Usuario	
Curso normal	Alternativas
El usuario introduce los datos de la película. El sistema comprueba que la película existe. El sistema comprueba que la película no esta alquilada. El usuario introduce los datos del cliente. El sistema almacena los datos del alquiler.	SI la película no existe muestra el mensaje de error. Si la película esta alquilada muestra mensaje de error. Si el cliente no existe llama al caso de uso “alta de cliente”.

Caso de uso: Buscar cliente	
Curso normal	Alternativas
Se recibe campo de búsqueda Se buscan los clientes que cumplen este campo Se almacena en una lista todos los clientes que cumplan el criterio Se devuelve la lista de clientes	Si no hay clientes que cumplan con el campo se devuelve un error

Caso de uso: Buscar película	
Curso normal	Alternativas
Se recibe campo de búsqueda Se buscan las películas que cumplen este campo Se almacena en una lista todas las películas que cumplan el criterio Se devuelve la lista de películas	Si no hay películas que cumplan con el campo se devuelve un error

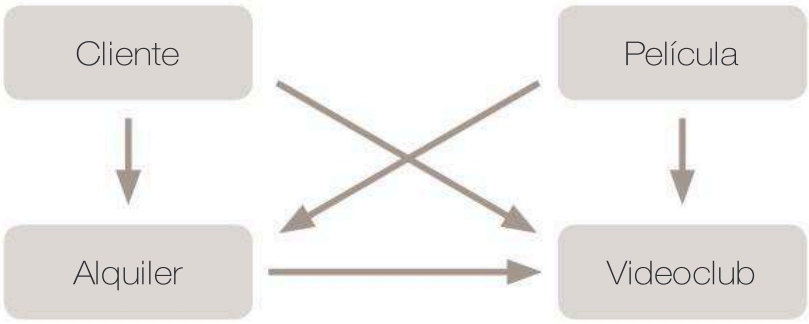
Una vez obtenidos y documentados los casos de uso de nuestra aplicación, hemos finalizado la fase de captura de requisitos, y pasamos a realizar un modelado del análisis del problema.

Examinando los casos de uso, en primer lugar vamos a buscar las clases que formarán parte de nuestro problema:

- Clase película: almacena los datos relativos a las películas almacenadas en el videoclub
- Clase cliente: almacena datos sobre clientes del videoclub.
- Clase Alquiler: almacena los datos del cliente, la película y la fecha en la que se produce un alquiler.

– Clase videoclub: sirve como almacén de datos de toda la aplicación. Guarda una serie de clientes, de películas y de alquileres, y proporcionará la funcionalidad necesaria para almacenar estos datos en un soporte no volátil (Ficheros, bases de datos,etc.)

Una vez realizado este primer análisis podemos crear un diagrama de clases con las clases identificadas y las relaciones que existen entre ellas:



Se puede observar que un alquiler mantienen una relación 1 a 1 con los clientes, ya que un cliente realiza un alquiler, y un alquiler es realizado por un solo cliente.

De la misma manera se observa que un alquiler solamente puede ser hecho para un asola película, si un cliente desea alquilar varias a la vez se considerarán por el sistema alquileres distintos

Todas las clases mantienen relaciones 1 a n con la clase videoclub ya que como se ha mencionado es la encargada de mantener la persistencia, esto es de que se puedan almacenar de manera continua en el tiempo los datos de la aplicación.

Tras realizar este primer esbozo del diagrama de clases pasamos a buscar los atributos y métodos de calda una de las clases. Para eso podemos plantear en primer lugar los diagramas de secuencias para dada uno de los casos de uso:

Realizaremos un diagrama de secuencias para cada caso de uso, lo que se denomina el escenario del caso de uso.

En estos diagramas aparece una nueva clase, denominada GUI, que representa al interfaz de usuario, y por tanto es desde la que se inicia toda la interacción del escenario.

Los escenarios documentados son los siguientes:

Alta de películas:

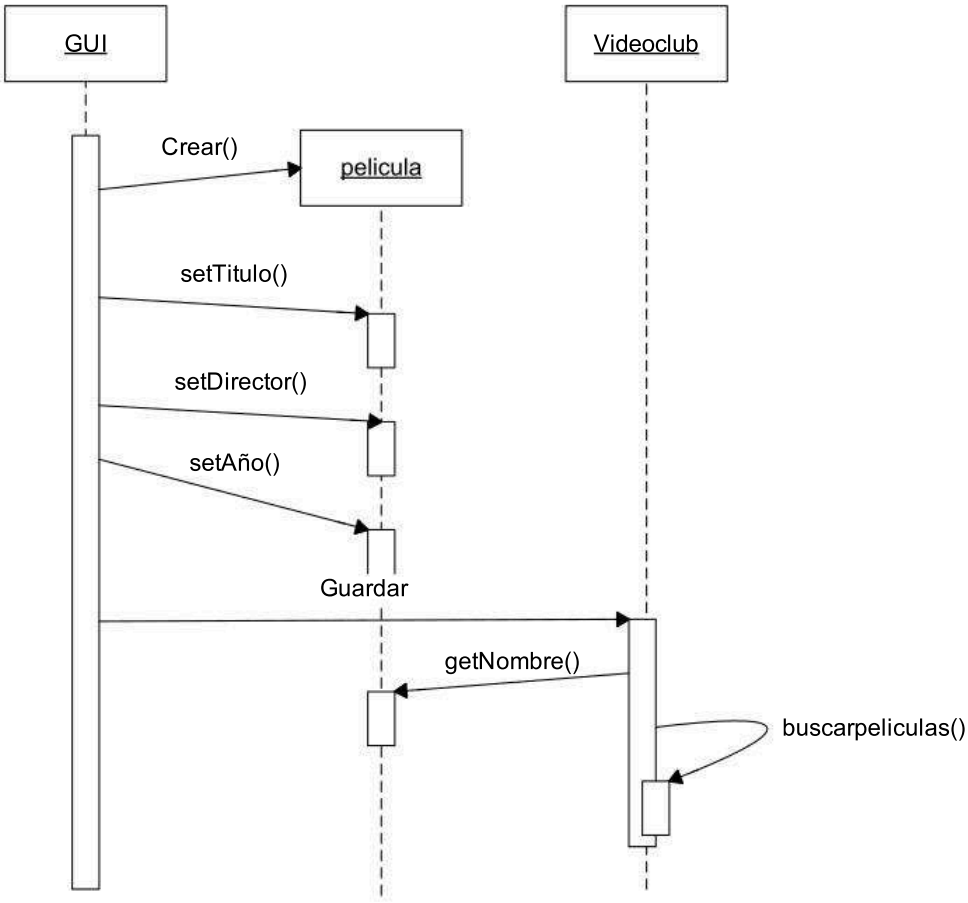


Diagrama de secuencias

En el se puede observar como cuando el usuario da de alta una película lo primero que hace el sistema es crear un objeto película.

Posteriormente llama a un método que el indica el título. Consecutivamente llama a los métodos que le indican el director y el año. Para terminar el proceso, se llama al método guardar de la clase videoclub para que se almacene la película.

Este método llama a buscarPelicula para comprobar si la película esta ya almacenada o no en el sistema. Dicho método hace búsquedas por nombre y por tanto ha de conocer el nombre de la película. Por tanto se llama el método getNombre para que nos devuelva l nombre del objeto película.

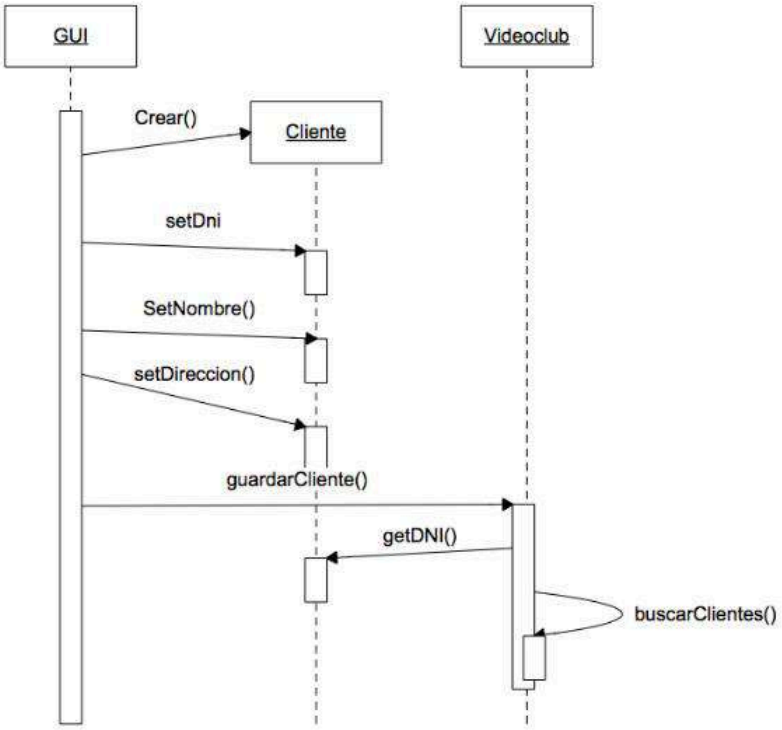


Diagrama de secuencias

El caso en el que se da de alta un cliente es muy similar al de alta de una película Primero se crea el objeto correspondiente por parte del interfaz gráfico.

Posteriormente se llama a los métodos setDNI, setNombre y setDirección para indicar el DNI, nombre y dirección del usuario a añadir, respectivamente.

El ultimo paso es el almacenamiento persistente del objeto. Para ello se llama al métodoguardarCliente de la clase Videoclub. Este método será el que realice la funcionalidad necesario para poder completar el proceso de alquiler, para lo cual tienen que hacer determinadas funciones:

En primer lugar dicho método ha de comprobar ha de comprobar si ya existe un cliente con el mismo nombre. Por esta razón ha de llamar al método buscarCliente.

Para poder realizarse una búsqueda por DNI, el métodobuscar necesita saber el DNI del cliente. Esta información esta almacenada en el objeto de cliente y

y por esta razón, dicho método vuelve a mandar un mensaje al objeto cliente para que le conteste con este dato.

A la hora de hacer un alquiler el proceso es el siguiente:

En primer lugar desde el interfaz gráfico se inicia el proceso de alquilar

Para ello en primer lugar se llama al método buscar cliente con la intención de buscar el cliente sobre el que se va a hacer el alquiler.Este método nos devuelve un objeto de tipo cliente que se conservará para su posterior uso.

De la misma manera, se llama al método buscar película, esta devolverá el objeto película que se va a alquilar en el alquiler.

Para terminar con el proceso, se crea un nuevo objeto alquiler. A este objeto se le envían mensajes con el objeto cliente y el objeto película que se van a alquilar.

Para finalizar con el caso de uso, se envía un nuevo mensaje a la clase videoclub para que se almacene el objeto alquiler que se había creado anteriormente.

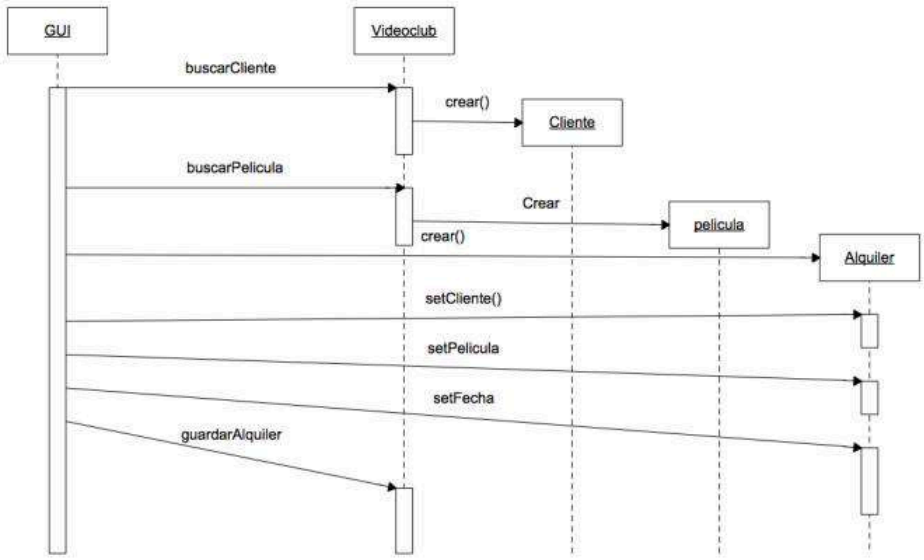


Diagrama de secuencias para el alta de alquileres

Cuando se desarrolla una devolución el proceso es el siguiente, en primer lugar desde el interfaz hay que localizar el objeto alquiler correspondiente.

Posteriormente a este objeto alquiler se le envía un mensaje en el que se le indica que el estado del alquilar para a ser devuelto.De esta manera internamente se marcará el alquilar como devuelto en este objeto.

Por ultimo hay que guardar en la clase videoclub el nuevo objeto alquilar con los cambios realizados, para que estos cambios se almacenen de forma persistente.

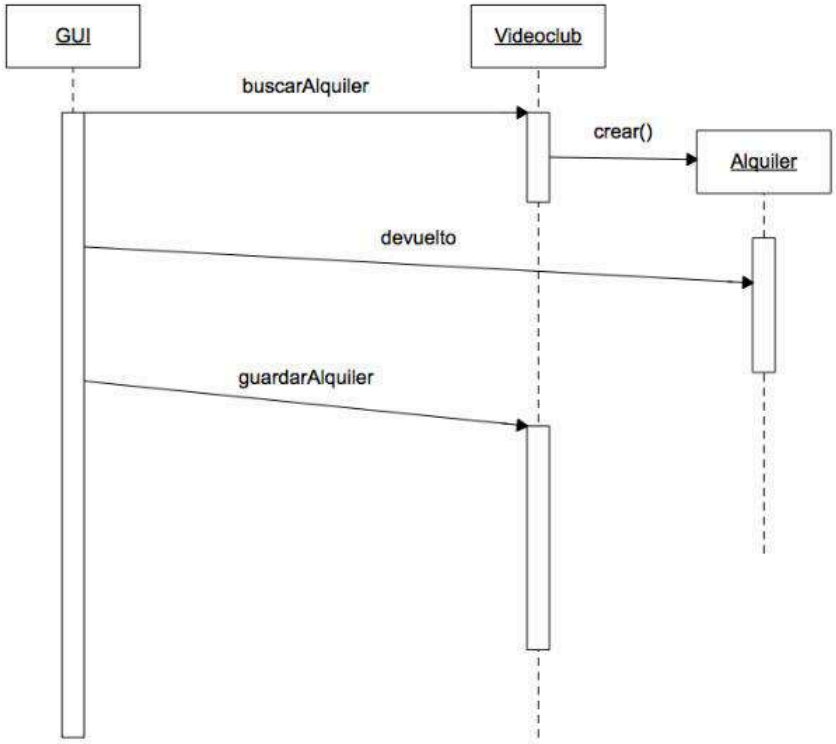


Diagrama de secuencias para el proceso de devolución

Una vez realizados los diagramas de secuencias anteriores podemos comenzar con el diseño arquitectónico, volviendo a trabajar sobre el diagrama de clases, pero esta vez definimos ya los métodos (Obtenidos de los escenarios de los casos de uso) y los posibles atributos que pueda tener cada clase:

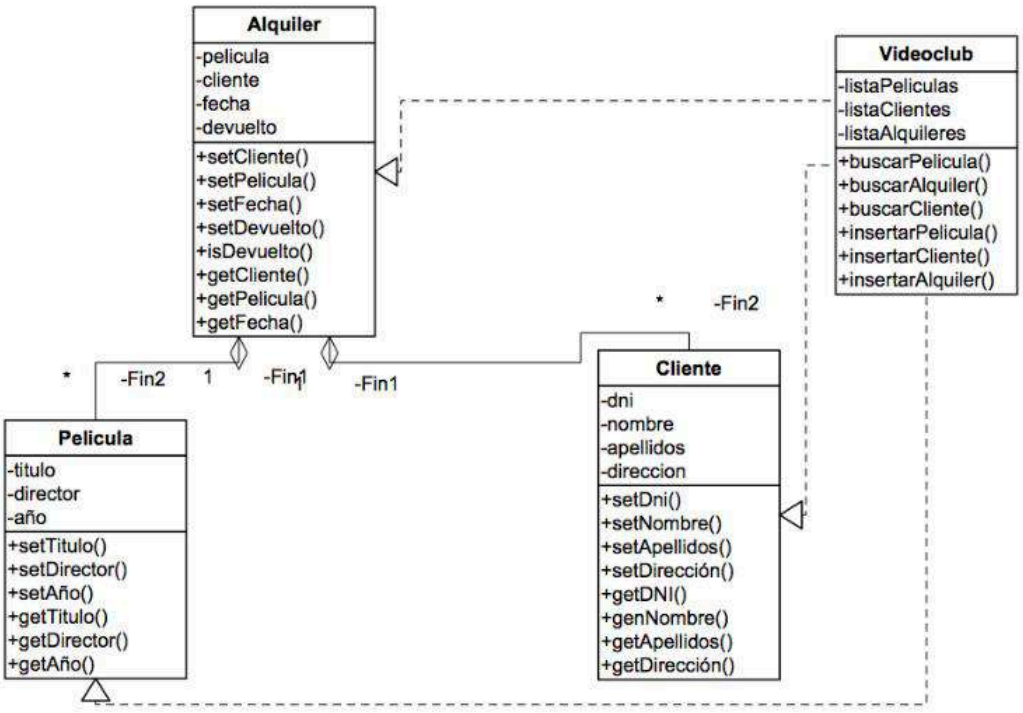


Diagrama de clases definitivo

Con esto queda claramente detallada la arquitectura de nuestro sistema. En el diagrama de clases definitivo se incluyen todos los elementos que se han identificado en los diagrama de secuencias.

Los mensajes que pasan a ser métodos de cada una de las clases, se han asociado a la clase correspondiente. Asimismo los atributos que se ha considerado necesario para poder realizar las acciones del diagrama de secuencias también se han asociado con cada una de las clases del sistema

En la siguiente fase, diseño detallado, pasamos a examinar cada clase separadamente.

En nuestro caso al ser un problema muy sencillo, simplemente vamos a detallar los parámetros que recibe y devuelve cada método, así como los tipos de datos de cada atributo.

También debería especificarse cada método, o bien con una narrativa de proceso (utilizando lenguaje natural) o bien en pseudocódigo, aunque en este caso debido a la simplicidad del problema a resolver, no se ha considerado necesario.

Nuestro diagrama de clases detallado quedaría como se indica en la imagen:

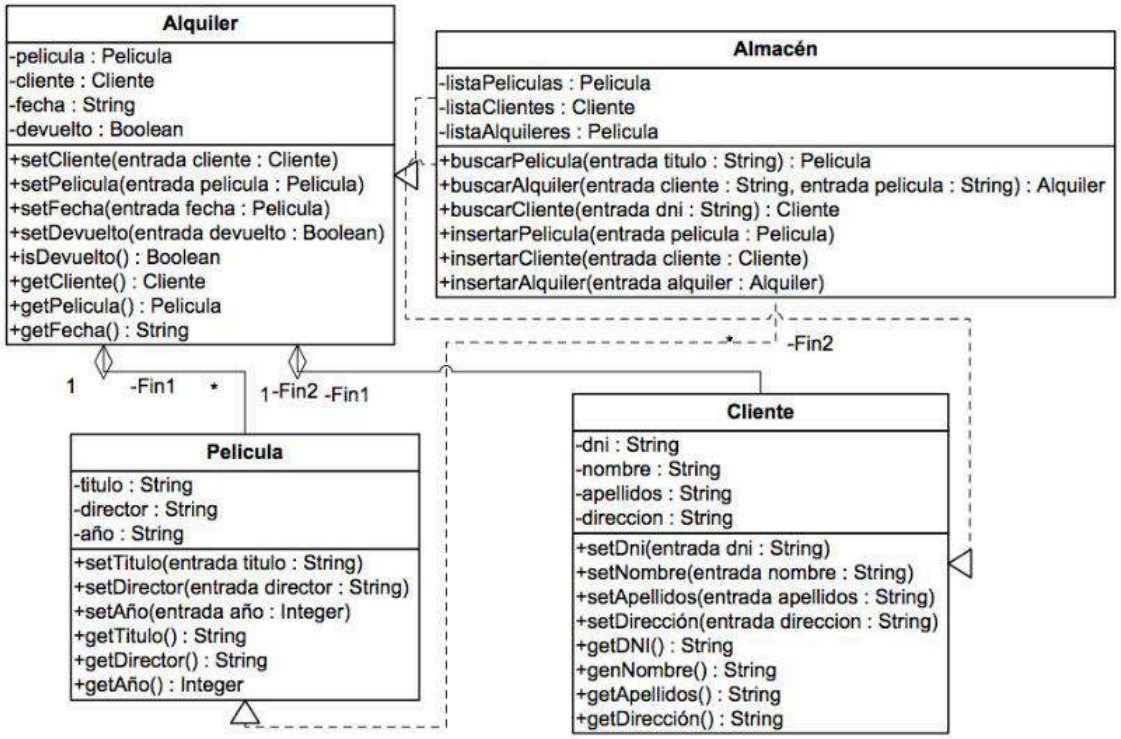


Diagrama de clases detallado

En él se pueden observar cada una de las cuatro clases en las que se ha dividido el sistema. La clase película tiene tres atributos de tipo cadena de caracteres.

En dichos atributos se almacenan el título, el director de la película y al año de estreno. Además tiene los métodos observadores y mutadores necesarios, es decir los métodos para poder observar el valor de cada uno de los tres atributos, y para poder cambiarlo.

La clase cliente es conceptualmente bastante similar a la de película. Tiene también tres atributos para el DNI, nombre y apellidos. Además contiene sus correspondientes métodos de observación y modificación.

La clase alquiler contienen como atributos enlaces a un objeto cliente, y a un objeto película. De esta forma se mantienen las relaciones uno a uno de esta clase.

Además contiene un atributo booleano, es decir que almacena valores verdadero o falso, que indican si se ha producido o no la devolución correspondiente al alquiler.

Asimismo contiene los métodos correspondientes a los atributos que posee para que estos puedan ser observados y mutados.

Por último la clase almacén, que es el nombre que se le ha dado a la clase para almacenar la persistencia, mantiene una lista de objetos para cada una de las tres clase anteriores, una lista de películas, una lista de clientes y una lista de alquileres. Además posee métodos que nos permitan buscar e insertar en estas tres listas.



CASE, Acrónimo de Computer Assisted Software Engineering se puede definir como el conjunto de herramientas y metodologías que prestan soporte a la ingeniería del desarrollo de software, en alguna o en todas las fases del ciclo de vida del software.

1.6. Definición del concepto de Herramienta CASE

En definitiva, las Herramientas CASE son un conjunto de programas y ayudas que proporcionan asistencia a los analistas, ingenieros de software y desarrolladores, durante todos los pasos del Ciclo de Vida de desarrollo de un Software.

La tecnología CASE se ha denominado la “informatización de la informática” ya que su objetivo es la automatización del desarrollo del software

El término CASE no solamente se utiliza para describir las herramientas de asistencia, sino que se usa en varios contextos con significados similares, pero con matices:

CASE se refiere al conjunto de métodos, utilidades y técnicas que facilitan la automatización del ciclo de vida del desarrollo de sistemas de información,

completamente o en alguna de sus fases. También se puede referir a una serie de programas y un paradigma de desarrollo de software que ayuda a automatizar el ciclo de vida de desarrollo de los sistemas.

Además el concepto de CASE hace referencia a la conjunción herramientas automatizadas de software y las metodologías de desarrollo de software formales.

Los objetivos de las herramientas CASE son los siguientes:

– Permitir la aplicación práctica de metodologías estructuradas.

– Facilitar la realización de prototipos y el desarrollo conjunto de aplicaciones.

– Simplificar el mantenimiento de los programas.

– Mejorar y estandarizar la documentación.

– Aumentar la portabilidad de las aplicaciones.

– Facilitar la reutilización de componentes del software.

– Permitir un desarrollo y un refinamiento visual de las aplicaciones.

En resumen CASE proporciona al ingeniero la posibilidad de automatizar actividades manuales y de mejorar su visión general de la ingeniería.

Al igual que las herramientas de ingeniería y de diseño asistidos por computadora que utilizan los ingenieros de otras disciplinas, las herramientas CASE ayudan a garantizar que la calidad se diseñe antes de llegar a construir el producto.

Las herramientas CASE se pueden clasificar por su función, por su papel como instrumentos para administradores o personal técnico, por su utilización en los distintos pasos del proceso de ingeniería del software, por la arquitectura del entorno (hardware y software) que les presta su apoyo, o incluso por su origen o coste.

Una primera clasificación puede ser en función de la fase del ciclo de vida que cubre.

En este caso las herramientas se clasifican dependiendo de la fase del ciclo de vida en la que se utilizan y del nivel de abstracción a la que trabaja el usuario de estas.

Desde este punto de vista tenemos que distinguir entre tres tipos de herramientas CASE:

- Upper CASE, herramientas que ayudan en las fases de planificación, análisis de requisitos y estrategia del desarrollo.

- Middle CASE, herramientas para automatizar tareas en el análisis y diseño de la aplicación.

- Lower CASE, herramientas que semiautomatizan la generación de código, crean programas de detección de errores, soportan la depuración de programas y pruebas. Además automatizan la documentación completa de la aplicación.



Aunque se pueden obtener muchos beneficios utilizando individualmente las herramientas CASE que abarcan actividades de ingeniería del software por separado, la verdadera potencia de CASE solamente se puede lograr mediante la integración de varios procesos en una sola herramienta.

Los beneficios que conllevan las herramientas CASE integradas, también llamado I-CASE, son los siguientes:

- Una transferencia fluida de información (modelos, programas, documentos, datos) entre una herramienta y otra, y entre un paso del ciclo de vida y el siguiente.

- Una reducción del esfuerzo necesario para efectuar actividades globales tales como la gestión de configuración de software, el control de calidad y la producción de documentos.

- Un aumento del control del proyecto que se logra mediante una mejor planificación, monitorización y comunicación.

- Una mejor coordinación entre los miembros del personal que estén trabajando en grandes proyectos de software.

La integración de herramientas facilita la transferencia de información entre unas y otras; ara lograr esto, deberán existir una serie de componentes de arquitectura que faciliten esta labor, el denominado marco de referencia CASE

El marco de referencia CASE consta de los siguientes elementos:

– Una base de datos para almacenar toda la información que manejará de la herramienta integrada (repositorio CASE).

– Un sistema de gestión de objetos (objetos de la configuración del software) para gestionar los cambios efectuados en la información.

– Un mecanismo de control de herramientas para coordinar la utilización varias herramientas CASE.

– Una interfaz de usuario que proporcione un medio de comunicación entre las acciones efectuadas por el usuario y las herramientas que están dentro del entorno.



Sabías que

La mayoría de los modelos propuestos del marco de referencia de integración de herramientas representan a estos componentes como si fueran capas. Se distinguen la capa de interfaz, la capa de presentación, la capa de herramientas, la capa de gestión de objetos y el repositorio compartido.

La capa de interfaz del usuario incorpora un conjunto de herramientas de interfaz estandarizado, con un protocolo de presentación común. El protocolo de presentación es el conjunto de líneas generales que proporciona un mismo aspecto a todas las herramientas CASE.

El kit de herramientas de interfaz contiene software para la gestión de la interfaz hombre-máquina, y una biblioteca de objetos de visualización. Ambos proporcionan un mecanismo consecuente para la comunicación entre la interfaz ylas herramientas CASE individuales.

Las convenciones del diseño de pantalla, nombres y organización del menú, iconos, nombres de los objetos, utilización del teclado y del ratón, y el mecanismo para acceder a las herramientas se definen todos ellos como parte del protocolo de presentación.

La capa de herramientas incorpora un conjunto de servicios de gestión de herramientas con las herramientas CASE en sí. Los servicios de gestión de herramientas controlan el comportamiento de las herramientas dentro del entorno.

Si durante la ejecución de una o más herramientas se emplea la multitarea este servicio efectúa la sincronización y comunicación multitarea, coordina el flujo de información desde el repositorio y el sistema de gestión de objetos a las herramientas, realiza las funciones de seguridad y auditoría yrecoge métricas acerca de la utilización de herramientas.

La capa de gestión de objetos lleva a cabo las funciones de gestión de configuración del software En esencia, el software de esta capa de la arquitectura de marco de referencia proporciona el mecanismo para la integración de herramientas. Cada herramienta CASE se conecta en la capa de gestión de objetos.

En conjunto con el repositorio CASE, la capa de gestión de objetos proporciona los servicios de integración, es decir, un conjunto de módulos estándar que acoplan las herramientas con el repositorio.

Además, la capa de gestión proporciona los servicios de gestión de configuración haciendo posible la identificación de todos los objetos de configuración, llevando a cabo el control de versiones y proporcionando apoyo para el control de cambios, auditorias y contabilidad de estados.

La capa de repositorio compartido es la base de datos CASE y las funciones de control de acceso que hacen posible que la capa de gestión de objetos interactúe con la base de datos. El repositorio es la pieza fundamental que hace posible la integración de todo el proceso software dentro de un mismo entorno. En el siguiente apartado se detalla su composición y funcionamiento.



Definición

El repositorio CASE es una base de datos que actúa como centro tanto para la acumulación como para el almacenamiento de información de ingeniería del software. En un entorno CASE, el papel del ingeniero del software es interactuar con el repositorio empleando herramientas CASE integradas con el.

Se define también el repositorio de un entorno I-CASE como el conjunto de mecanismos y de estructuras de datos que consiguen la integración entre datos y herramientas, y entre datos provenientes de varias herramientas diferentes.

El repositorio proporciona las funciones de un sistema de gestión de bases de datos, pero, además, el repositorio lleva a cabo las funciones siguientes:

– Integridad de datos: incluye funciones para validar las entradas efectuadas en el repositorio, para asegurar la consistencia entre objetos relacionados, y para efectuar automáticamente modificaciones en cascada cuando un cambio efectuado en un objeto exige algún cambio en otros objetos relacionados con él.

– Información compartida: proporciona un mecanismo para compartir información entre múltiples desarrolladores y entre múltiples herramientas. Gestiona y controla el acceso multiusuario a los datos, y bloquea/desbloquea objetos para que los cambios no se superpongan inadvertidamente.

– Integración datos-herramientas: establece un modelo de datos al que pueden acceder todas las herramientas del entorno I-CASE, controlando el acceso a los datos, y lleva a cabo las funciones de gestión de configuración adecuadas.

Integración datos-datos: este concepto se refiere al sistema de gestión de bases de datos relaciona los objetos de datos de tal manera que se puedan alcanzar las demás funciones.

La utilización de una herramienta CASE también lleva consigo la imposición de la metodología ya que el modelo de datos almacenado en el repositorio puede implicar un paradigma específico de ingeniería del software. Como mínimo, las relaciones y los objetos definen un conjunto de pasos que se llevará a cabo para construir el contenido del repositorio.

Por otro lado el simple hecho de su utilización conlleva la estandarización de documentos al producirse la definición de objetos de la base de datos, esto da lugar directamente a un enfoque estándar para la creación de documentos de ingeniería del software.

Para conseguir que el repositorio CASE pueda cumplir con sus funciones, se define el repositorio en función de un metamodelo.

Se podría decir que el metamodelo es el modelo conceptual de la base de datos que almacena el repositorio.



El metamodelo determina la forma en que se almacena la información en el repositorio, la forma en que las herramientas pueden acceder a los datos y estos datos pueden ser visualizados por los ingenieros de software, el grado hasta el cual se puede mantener la seguridad e integridad de los datos, y la facilidad con que se puede ampliar el modelo ya existente para admitir nuevas necesidades.

Ejemplo

Un ejemplo de la información que suele almacenarse en un repositorio es el siguiente:

Información de la empresa: Estructura organizativa, Áreas de negocios, funciones de negocios, modelos de procesos, arquitectura del sistema de información.

- Diseño de aplicaciones: Reglas de metodología, representaciones gráficas diagramas de sistema, estándares de denominación, reglas de integridad referenciales, estructuras de datos, definiciones de proceso, definiciones de clases, árboles de menú, criterios de rendimiento, restricciones temporales, definiciones de pantalla, definiciones de informes, lógicas de comportamiento, algoritmos, reglas de transformación.

- Construcción: código fuente, código objeto, instrucciones de construcción del sistema, imágenes binarias, dependencias de configuración, información de cambios.

- Validación y verificación: Plan de comprobación, casos de prueba, análisis estadísticos, métricas de calidad de software.

- Información de gestión del proyecto: Planes de proyecto, Estructura de desglose de tareas, estimaciones, planificaciones, recursos; informe de problemas, cambios; informes de estado, información de auditoría.

- Documentación del sistema: Documentos de requisitos, diseños, manuales de usuario.

1.6.1. Herramientas de Ingeniería del Software

Las herramientas de análisis y diseño hacen posible que el ingeniero del software cree modelos del sistema que vaya a construir.

Los modelos contienen una representación de los datos, función y comportamiento (en el nivel de análisis), así como caracterizaciones del diseño de datos, de arquitectura, a nivel de componentes e interfaz.

Al efectuar una comprobación de consistencia y validez de los modelos, las herramientas de análisis y diseño proporcionan al ingeniero del software un cierto grado de visión global, y le ayudan a eliminar errores antes de que se propaguen al diseño o a la implementación



Las herramientas de ingeniería del software son aquellas herramientas que actúan fundamentalmente a alto nivel, centrándose sobre todo en las actividades de análisis, diseño y validación del software. Fundamentalmente se utilizan en el modelado de procesos de negocio.

Cuando se modelan los requisitos de información estratégica de una organización, las herramientas de ingeniería de procesos de negocios proporcionan un metamodelo del cual se derivan sistemas de información específicos.

El objetivo primordial de las herramientas de esta categoría consiste en representar objetos de datos de negocio, sus relaciones y la forma en que fluyen estos objetos de datos entre distintas zonas de negocio en el seno de unas compañía.



El modelado de procesos de negocios (BPM) en la ingeniería de sistemas es la actividad de representación de los procesos de una empresa, con el fin de que el proceso tal y como se lleva a cabo puede ser analizado y mejorado.

El modelado de procesos de negocio se realiza típicamente por los analistas de negocios o gerentes que buscan mejorar la eficiencia de los procesos, la calidad y reducir los costes. Las mejores pueden requiero o no de tecnologías de la información.

Las Tecnologías de la Información y Comunicación han desempeñado históricamente un papel importante en el concepto de reingeniería de procesos de negocio.

Normalmente la automatización de los procesos es un pilar clave en el concepto de reingeniería. Por ello el trabajo del ingeniero del software se considera primordial en la decisión de estos procesos automatizados.



Si una organización trabaja por mejorar un proceso de negocio (o de software) lo primero que debe hacer es entenderlo. Las herramientas de modelado de procesos (llamadas también herramientas de tecnología de procesos) se utilizan para representar los elementos clave del proceso de manera que sea posible entenderlo mejor.

Las herramientas de esta categoría se centran en dos áreas primordiales: estimación de costes y de esfuerzos del proyecto de software y planificación de proyectos.

Dentro de este tipo de herramientas destacan por su importancia las herramientas ERP que, aunque no se apliquen en exclusiva al desarrollo del software sino a cualquier proceso de negocio, merecen un tratamiento aparte:



El software de Planificación de recursos empresariales o ERP es un software de gestión empresarial, constituido por lo general por un conjunto de aplicaciones integradas, que una empresa puede utilizar para recopilar, almacenar, gestionar e interpretar datos de muchas de las actividades de su negocio.

Los sistemas ERP (Enterprise Resource Planning) suelen incluir las siguientes características:

– Un sistema integrado que opera en tiempo real, sin depender en general de actualizaciones periódicas.

– Una base de datos común que de soporte todas las aplicaciones.

– Un aspecto coherente en todos sus módulos.

– La instalación de sistemas integrados dentro de la aplicación.

Un sistema ERP cubre Una serie de áreas funcionales comunes, las cuales se denominan módulos en muchos sistemas ERP.

Las áreas comunes que cubre un ERP suelen ser las que se describen a continuación, aunque dependiendo del sistema concreto, estas son más o menos.

– Contabilidad financiera : contabilidad general, activos fijos, cuentas de pago, cuentas de efectivo, gestión de tesorería y consolidación financiera

– La contabilidad de gestión: presupuesto, cálculo de costos, la gestión de costes, costeo basado en actividades

– Recursos humanos: reclutamiento, entrenamiento, elaboración de turnos, de nómina, beneficios, gestión de la diversidad, gestión de acción social, etc.

– Fabricación: ingeniería, lista de materiales, órdenes de trabajo, programación, capacidad, gestión de flujo de trabajo, control de calidad, proceso de fabricación, proyectos de fabricación, el flujo de fabricación, gestión del ciclo de vida del producto.

– Procesamiento de órdenes: orden de dinero en efectivo, la entrada de pedidos, control de crédito, fijación de precios, disponible para prometer, inventario, envío, análisis de ventas y presentación de informes, la puesta en venta.

– Gestión de la cadena de proveedores: la planificación de la cadena de suministro, planificación de proveedores, configurador de productos, a fin de dinero en efectivo, compras, inventario, procesamiento de reclamaciones, supermercado (recepción, putaway, picking y embalaje).

– Gestión de proyectos: planificación de proyectos, planificación de recursos, costos de proyectos, la estructura de desglose del trabajo, facturación, tiempo y gastos, las unidades de rendimiento, gestión de la actividad.

– Gestión de relaciones con clientes: ventas y marketing, comisiones, servicios, contacto con el cliente, centro de atención telefónica de apoyo - Los sistemas de CRM no siempre se consideran parte de los sistemas ERP, sino más bien los sistemas de soporte de negocios o los denominados BSS).

1.6.2. Entornos de desarrollo



Definición

Entorno de desarrollo integrado o IDE, también llamado entorno de desarrollo interactivo es una aplicación de software que proporciona servicios integrales para los programadores de ordenador para el desarrollo de software.

Un IDE normalmente consiste en un editor de código fuente, herramientas de compilación y enlazadoautomáticas, y un depurador . Además suele contar con otras herramientas que faciliten el trabajo del programador mientras está llevando a cabo la fase de diseño.

La mayoría de los IDEs modernos ofrecen además características inteligentes de composición de código, así como ayudas “en línea” que permiten que el programador pueda acceder a información sobre la sintaxis y funcionamiento del código que está escribiendo en este mismo momento.

Algunos IDEs también contienen un compilador, intérprete, o ambas cosas, como NetBeans o Eclipse, otros no lo hacen, comoSharpDevelop y por tanto deben utilizarse con un compilador aparte del propio entorno de desarrollo

No está bien definido el límite entre un entorno de desarrollo integrado y otras partes del entorno de desarrollo de software en general. A veces, un sistema de control de versión y varias herramientas están integrados para simplificar la construcción de una interfaz gráfica de usuario .

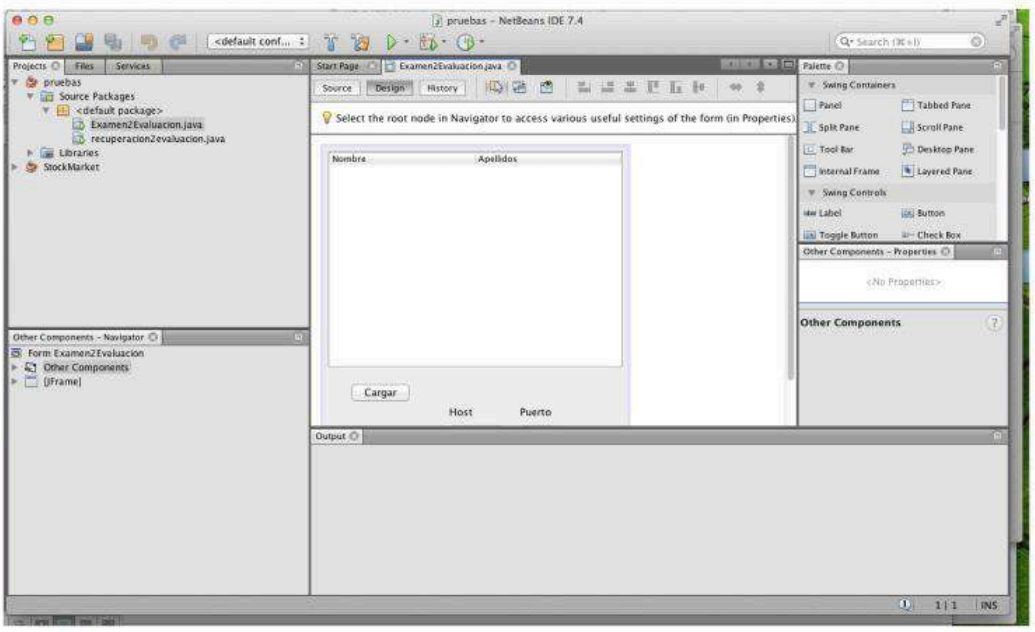
La mayoría de los IDEs modernos tienen un navegador de clases, un explorador de objetos, y muestran dinámicamente un diagrama de jerarquía de clases, para su uso durenate el desarrollo de software orientado a objetos.

Además suelen disponer de herramientas de documentación en línea que permiten al programador ir documentando el código que produce al mismo tiempo que lo está desarrollando, facilitándose de esta manera dicha labor.

Los entornos de desarrollo integrados están diseñados para maximizar la productividad del programador, proporcionando componentes específicos, con similares interfaces de usuario.De esta manera el programador siempre está trabajando en un mismo entorno familiar si tener que estar constantemente cambiando entre herramientas de distinto tipo.

Importante

Muchos IDEs presentan un solo programa en el que todo el desarrollo se lleva a cabo. Este programa normalmente proporciona todas las características necesarias para la creación, modificación, compilación, implementación y depuración desoftware. Esto contrasta con el desarrollo de software cono se hacía hace unos años, utilizando herramientas separadas, como por ejemplo vi, GCC o make .



Entorno de desarrollo Integrado NetBeans

Un objetivo de la IDE es reducir el esfuerzo para la instalación y configuración que es necesario hacer a la hora de instalar y desplegar múltiples utilidades de desarrollo. En lugar de tener que hacer esta tarea, el IDE proporciona el mismo conjunto de capacidades pero como una unidad cohesiva.

Reducir el tiempo de configuración aumenta la productividad de los desarrolladores, ya que generalmente es más rápido aprender a usar el IDE que tener que familiarizarse con todas las herramientas individuales que se utilizan en el caso del desarrollo de forma manual.

La mayor integración de todas las tareas de desarrollo tiene además el potencial para mejorar la productividad en general aparte de ayudar con las tareas de configuración. Por ejemplo, el código se puede analizar de forma continua mientras se está editando, proporcionando información instantánea cuando se introducen errores de sintaxis.

Eso puede acelerar el aprendizaje de un nuevo lenguaje de programación y sus librerías asociadas, ya que el programador puede ir resolviendo sus dudas en línea, en lugar que tener que estar constantemente consultando manuales o otras fuentes de información.

Algunos IDEs están dedicados a un determinado lenguaje de programación, lo que permite un conjunto de características que más se acerque a los paradigmas de programación del lenguaje.

Otros sin embargo son multi-lenguaje, lo cual también presenta sus ventajas, ya que el programador que tenga que desarrollar código en varios lenguajes para un mismo proyecto, o similar, puede hacerlo en el mismo entorno familiar, sin tener que estar cambiando de herramienta. Incluso algunos disponen de elementos para combinar código de varios lenguajes.

Ejemplo

Ejemplos de dichos entornos son Xcode, Xojo y Delphi, los cuales están dedicados a un lenguaje cerrado o conjunto restringido de lenguajes de programación.

Sin embargo, hay muchos IDEs en varios idiomas que son multi-lenguaje, en el sentido que pueden ser utilizados para programar un una multitud de lenguajes de programación. Ejemplos de estos entornos podrían ser, Eclipse, ActiveState Komodo, Intelli IDEA, Oracle JDeveloper, NetBeans, Codenvy y Microsoft Visual Studio .



Aunque la mayoría de los IDEs modernos son gráficos, entornos de desarrollo basados en texto, como Turbo Pascal eran de popular antes de la amplia disponibilidad de sistemas de ventanas tales como Microsoft Windows y el sistema X Window (X11).Actualmente se suelen encontrar en algunos entornos más restringidos. En ellos Se suelen utilizar las teclas de función o teclas de acceso directo para ejecutar comandos o macros de uso frecuente.

Generalmente se requiere de la utilización de un IDE cuando se utiliza el paradigma de programación visual, lo que normalmente se hace a la hora de desarrollar la parte del interfaz gráfico de una aplicación de escritorio.

Los IDEs visuales permiten a los usuarios crear nuevas aplicaciones moviendo líneas de programación, bloques de construcción, o nodos de código para crear diagramas de flujo o diagramas de estructura que luego son compilados o interpretados. Estos diagramas de flujo a menudo se basan en el Lenguaje Unificado de Modelado o UML.

El modelo de trabajo FlowCode fue popularizado con el entorno Lego Mindstorms, y está siendo utilizado activamente por un número de empresas que desean aprovechar el poder de los navegadores personalizados como Mozilla.

KTechlab apoya Flowcode es un IDE de código abierto y además un simulador para el desarrollo de software para microcontroladores.

La programación Visual también es responsable de desarrollar la potencia de la programación distribuida, como por ejemplo en los entornosLabVIEW y EICASLAB de desarrollo de software específicamente distribuido.



Uno de los primeros sistema de programación visual fue Max, y fue diseñado par el modelado y mezclado de música digital, tras realizar un muestreo analógico.Se ha utilizado como base para desarrollar software musical en tiempo real desde los años 80.

Otro ejemplo de los primeros sistemas de desarrollo integrado es el entorno Prograph, el cual se utilizaba para realizar un flujos de datos dentro de un sistema y que fue originalmente desarrollado para el Macintosh .

Este enfoque se utiliza también en software especializado, tales como Openlab, donde los usuarios finales desean la flexibilidad de un lenguaje de programación completo, sin la curva de aprendizaje tradicional asociada con una aplicación clásica.

NetBeans es un entorno de desarrollo integrado (IDE) para el desarrollo principalmente con Java, pero también con otros lenguajes, en particular, PHP, C / C ++ y HTML5 . También es una plataforma de aplicaciones de ventana para las aplicaciones de escritorio Java y otros.

Este IDE está escrito en Java y por ello, al ser completamente multi-plataforma, puede ejecutarse en nuemrosos sistemas operativos como Windows, OS X, Linux, Solaris y otras plataformas que soportan una JVM .

La plataforma NetBeans permite que las aplicaciones que se desarrollan a partir de un conjunto de componentes de software llamados módulos. Las aplicaciones basadas en la plataforma NetBeans (incluyendo el propio IDE NetBeans) se puede extender por desarrolladores de terceros que implementen módulos para una labor determinada.



Las **Herramientas PRO/SIM** o las herramientas de construcción de prototipos y simulación proporcionan al ingeniero del software la capacidad de predecir el comportamiento de un sistema en tiempo real antes de llegar a construirlo.

Además, también le capacitan para desarrollar simulaciones del sistema de tiempo real, lo que permitirá obtener ideas acerca de su funcionamiento, comportamiento y respuesta antes de la verdadera implementación.

El propósito de un prototipo es permitir que los usuarios del software puedan evaluar las propuestas de los desarrolladores para el diseño del producto final y experimentar con dichos diseños, en lugar de tener que interpretar y evaluar el diseño basado en las descripciones que le hagan los ingenieros.

Los prototipos también puede ser utilizado por los usuarios finales para describir y demostrar los requisitos que no han sido debidamente considerados, y por tanto pueden ser un factor clave en la relación comercial entre los desarrolladores y sus clientes.

El diseño de interacción, en particular, debe de hacer un uso intensivo de los prototipos. Los Prototipos también puede evitar el enorme gasto y la dificultad de cambiar de un producto de software final.El uso eficiente de prototipos requiere que una organización tenga las herramientas adecuadas y un personal capacitado para utilizar estas herramientas.

Las herramientas utilizadas en la creación de prototipos pueden variar desde el uso de herramientas individuales, del estilo de lenguajes de programación de 4 ª generación, utilizados para la creación rápida de prototipos, a complejos entornos integrados con herramientas CASE.



Sabías que

Los lenguajes de cuarta generación, los que utilizan paradigmas visuales de programación como Visual Basic y ColdFusion se utilizan con frecuencia, ya que son baratos, muy conocido y relativamente fácil y rápido de usar.

También se están desarrollando herramientas orientadas a objetos como Lymb. En ella Los usuarios pueden crear prototipos de elementos de una aplicación por sí mismos en una hoja de cálculo .

Dado que las aplicaciones web van a continuar creciendo en popularidad, va a ser necesario contar con herramientas para la creación de prototipos de este tipo de aplicaciones.

Entornos de trabajo como Bootstrap, Foundation y AngularJS también proporcionan las herramientas necesarias para desarrollar rápidamente un concepto de prueba . Estos entornos de trabajo suelen consistir en un conjunto de controles, las interacciones y las directrices de diseño que permiten a los desarrolladores crear prototipos rápidamente de aplicaciones web.



También se utilizan generalmente los programas de generación de pantallas que permiten el prototipado para mostrar el sistema que se va a desarrollar a los usuarios,como un sistema no funcional, pero cuyo detalle de pantallas es similar.

El desarrollo de Interfaces Hombre-Máquina muchas veces puede ser una parte fundamental de los esfuerzos de desarrollo realizados en un proyecto, ya que para la usuarios la interfaz es esencialmente la imagen que tienen del sistema.

Las herramientas de 4ª generación permiten generar código mediante la combinación de componentes modulares que son fragmentos de un programa que están preparador para su utilización directa. Esto hace que dichas herramientas sean ideales para aplicaciones de creación de prototipos, ya que este enfoque puede entregar rápidamente los programas con el comportamiento deseado, con una mínima cantidad de codificación manual.

Un nuevo tipo de software llamado software de definición de aplicaciones o software de simulación permite a los usuarios crear rápidamente simulaciones de otro programa, sin necesidad de escribir código.

De esta manera, el software de simulación y definición de aplicaciones permite a los usuarios técnicos y no técnicos experimentar, probar, colaborar y validar el programa simulado, y proporciona informes tales como anotaciones, la captura de pantalla y esquemas.

Como técnica de especificación de la solución, la aplicación de simulaciones y prototipos se encuentra entre las de más bajo riesgo que, por ejemplo, la simulación mediante descripciones de texto de dibujo a base de maquetas. Esta técnica a veces se le ha llamado documentación basada en el prototipado.

Se utiliza en proyectos de alto riesgo que requieren mucho tiempo de prototipado permitiendo que los profesionales del softwarepuedan validar los requisitos y opciones de diseño desde el principio, ante s de que comience el desarrollo. Al hacerlo de esta manera, los riesgos y los costos asociados con las implementaciones de software se puede reducir drásticamente.

Para simular las aplicaciones también se puede utilizar el software que simulan los programas de software del mundo real para entrenamiento mediante ordenador, servicios de demostración y de atención al cliente, tales como software de screencasting, así como aquellas áreas que están estrechamente relacionadas.

En las pruebas de software, el concepto de automatización de pruebas hace referencia al uso de software especial(aparte del software que se está probado) para controlar la ejecución de las pruebas y la comparación de los resultados reales con los resultados previstos.

La automatización de pruebas permite automatizar tareas repetitivas, pero necesarias en un proceso de pruebas ya existente que ha sido formalizado, o bien agregar pruebas adicionales que serían difíciles de realizar de forma manual ya que algunas tareas de pruebas de software, pueden ser laboriosas y requiere mucho tiempo para llevarlas a cabo manualmente.

Además, el enfoque manual no siempre podría ser eficaz en la búsqueda de ciertas clases de defectos. La automatización de pruebas ofrece una posibilidad para llevar a cabo estos tipos de pruebas maximizando la eficacia.

Una vez que se han desarrollado pruebas automatizadas, estas se pueden ejecutar de forma rápida y repetidamente. Muchas veces, esto puede ser un método rentable para las pruebas de integración de los productos de software que tienen una larga vida de mantenimiento, ya que en cada ciclo de mantenimiento hay que volver a ejecutar las pruebas de software,

1.6.3. Herramientas de prueba

Incluso los parches menores pueden causar errores en las características existentes desde hace mucho tiempo, durante la vida útil de la aplicación por lo que es necesario ejecutar todas las pruebas cada vez que se hace un cambio, aunque este sea menor.

Hay muchos enfoques para realizar la automatización del proceso de pruebas, a continuación se mencionan los métodos generales más ampliamente utilizados:

– Motor de Pruebas de Código. Las interfaces públicas de las clases, módulos o bibliotecas se ponen a prueba con una variedad de argumentos de entrada para validar que los resultados que se devuelven son correctas.

– Interfaz gráfica de usuario de prueba. Un marco de pruebas genera eventos de interfaz de usuario, tales como pulsaciones de teclado y los clics del ratón, y observa los cambios que resultan en la interfaz de usuario, para validar que el comportamiento observable del programa es correcto.

– API generador de pruebas. Un marco de pruebas que utiliza Una interfaz de programación de aplicación para validar el comportamiento bajo prueba. Típicamente las pruebas mediante API generadora pasa por alto la interfaz de usuario de aplicación por completo.

El uso de herramientas de automatización de pruebas pueden ser costoso, y por lo general se emplean en combinación con las pruebas manuales. La automatización de pruebas se puede hacer rentable en el largo plazo, especialmente cuando se utiliza en varias ocasiones en las pruebas de integración



En contexto de pruebas automatizadas el ingeniero de pruebas o de Calidad de Software debe tener la capacidad de modificar el código de software, ya que los casos de prueba se escriben en forma de código fuente que, cuando se ejecutan, producen una salida de acuerdo con las aserciones que esta ha introducido, y que son parte de esta codificación.

Una forma de generar casos de prueba automáticamente es a través del uso de un modelo de sistema para la generación de casos de prueba, pero aun no se ha establecido la el mejor enfoque para hacerlo, la investigación continúa en una variedad de metodologías alternativas para hacerlo.

En algunos casos, este enfoque permitirá a los usuarios no técnicos crear casos de prueba automatizados para un modelo de negocio, de tal forma que no es necesario hacer modificaciones de ningún tipo para configurarlos en múltiples sistemas operativos, navegadores y dispositivos de hardware.

Qué hay que automatizar, cuando automatizar las pruebas, o incluso si uno realmente necesita la automatización son algunas de las decisiones cruciales que el equipo de pruebas o de desarrollo tienen que tomar.



Una elección adecuada de los aspectos a probar del producto sobre el que se va a aplicar una técnica para la automatización de pruebas determina en gran medida el éxito de la automatización. La automatización de pruebas de características inestables o características que están experimentando cambios debe ser cuidadosa o directamente evitada.

Una tendencia creciente en el desarrollo de software es el uso de entornos de pruebas tales como las framework xUnit (por ejemplo, JUnit y NUnit) que permiten la ejecución de las pruebas unitarias para determinar si varias secciones del código están actuando como se espera en diversas circunstancias. Los casos de prueba describen las pruebas que requierenal ser ejecutado el programa para verificar que el programa se comporta como se esperaba.

Las pruebas automatizadas realizadas mediante código es una de las características clave en el desarrollo de software ágil. También se le conoce como desarrollo basado en pruebas (TDD). En ellas, las pruebas unitarias son desarrolladas para definir la funcionalidad de un programa antes de escribir el código.

Sin embargo, estas pruebas unitarias evolucionan y se amplían a medida que avanza la codificación, los problemas que se detectan y el código se somete a reestructuración y refractorización. Sólo cuando se pasan todas las pruebas para todas las características exigidas entonces el código se considera completo.

Sus defensores sostienen que produce el software que es tanto más fiable y menos costoso que el código que se prueba por exploración manual. Se considera más fiable porque la cobertura contra errores en el código es mejor, y porque el proceso e pruebas es realizado constantemente durante el desarrollo en lugar de una vez al final de un ciclo de desarrollo.

El desarrollador descubre defectos inmediatamente después de realizar un cambio, cuando es menos costosos de arreglar.

Por último, la refactorización de código es más segura, ya que esta transformar el código de un modo más sencillo con menos duplicación de código, pero de comportamiento equivalente, siendo por tanto menos probable que el desarrollador pueda introducir nuevos defectos.



Muchas de las herramientas de automatización de pruebas proporcionan funciones de grabación y reproducción que permiten a los usuarios grabar de forma interactiva las acciones del usuario y reproducirlas de nuevo las veces que haga falta, para comparar los resultados reales a los esperados.

La ventaja de este enfoque es que requiere poco o ningún desarrollo de software . Este enfoque se puede aplicar a cualquier aplicación que tenga una interfaz gráfica de usuario . Sin embargo, la dependencia que tiene de las características del interfaz plantea grandes problemas de fiabilidad y mantenibilidad.

Ejemplo

Por ejemplo reetiquetar un botón o moverlo a otra parte de la ventana puede exigir la prueba que se regrabada. Grabación y reproducción también se añade a menudo actividades irrelevantes o puede que registre algunas actividades incorrectamente.

Una variación de las herramientas de prueba automatizada mediante grabación es el tipo que se para la prueba de los sitios web. Aquí, la interfaz de usuario es la página web. Este tipo de herramientas apenas requiere poco o ningún desarrollo de software.

Sin embargo, dicho entorno utiliza técnicas completamente diferentes a las que se usan en interfaces gráficos clásicos, ya que la herramienta de pruebas procede a la lectura del código HTML en vez de observar eventos de interfaz.

Las pruebas generadas por la API también son ampliamente utilizado por los testeadores de software web, ya que cada vez es más difícil de crear y mantener las pruebas de automatización basada en GUI Los programadores o los diseñadores de pruebas en este caso han de escribir scripts usando un lenguaje de programación que llamará a la interfaz mostrada por la aplicación que se prueba.

Los scripts de prueba creados se ejecutan y se comparan los resultados de las pruebas con el comportamiento esperado de la aplicación.



Las herramientas de prueba puede ayudar a automatizar tareas como la instalación del producto, la creación de los datos de prueba, la interacción GUI, la detección de problemas, el registro de defectos, etc., por lo tanto no es necesario considerar la automatización de prueba como un fin en sí mismo.

Hay que considerar, cuando se va a implantar una política de automatización de pruebas, cuales van a ser las necesidades más comunes que deben de contemplarse por la aplicación de automatización. Los aspectos que hay que tener en cuenta de manera más general se enumeran a continuación:

– La independencia de la plataforma y del sistema operativo.

– Posibilidad de especificar con detalle la correspondencia de los de datos con respecto a las salidas observadas con respecto a las entradas suministradas, considerando los metadatos también.

– Los informes personalizables que se generen por la aplicación.

– La facilidad de depuración y de acceso a los datos de la aplicación mientras se ejecutan las pruebas.

– Un control de versiones adecuado, y que el conjunto de archivos suministrado sea mínimo, es decir, que no se incluya "basura" en el código fuente.

– Extensibilidad y personalización, si es necesario que el sistema tenga APIs para ser capaz de integrarse con otras herramientas.

– "Hilo conductor común" (por ejemplo, en el ecosistema de desarrollo de Java). De esta manera esta característica permitirá que las pruebas de integración se puedan compaginar con los flujos de trabajo de los desarrolladores sin necesidad de duplicidades. Así se optimizan los flujos de trabajo dentro de la organización de desarrollo

– Soporte para prueba desatendida: de tal forma que la prueba se pueda ejecutar sin supervisión directa o en modo por lotes en caso de que haya que realizar un gran número de pruebas de manera simultanea .

– Soporte entorno de ejecución distribuida de tal modo que las pruebas puedan ejecutarse de manera desatendida en diversos servidores simultáneamente.



Un marco de trabajo de automatización de pruebas es un sistema integrado que establece las reglas de la automatización para un producto específico.

Este sistema integra las bibliotecas de funciones, fuentes de datos de prueba, los detalles de los objetos a probar y diversos módulos reutilizables.

Estos componentes actúan como pequeños bloques de construcción que tienen que ser montados para representar un proceso de negocio. El marco proporciona la base de la automatización de pruebas y simplifica el esfuerzo de automatización.



La principal ventaja de un marco de pruebas es que proporcionan apoyo a las pruebas de software automatizado es el bajo costo de mantenimiento . Si posteriormente hay un cambio en cualquier caso de prueba sólo el archivo de caso de prueba tiene que ser actualizado.

El resto del conjunto de pruebas y script de inicio seguirá siendo el mismo. Idealmente, no hay necesidad de actualizar las secuencias de comandos en caso de cambios sucesivos en la aplicación, sino solamente cada caso de prueba individualmente. La elección de la técnica de marco y script adecuada ayuda a mantener bajos los costes de prueba.

Los costes asociados con secuencias de comandos de prueba se deben a los esfuerzos de desarrollo y mantenimiento, por tanto el enfoque de programación utilizado en la automatización de pruebas tiene efecto en los costes de cada prueba. A mayor numero de pruebas que se prevea que se van a realizar, menor coste unitario por prueba.

Se utilizan diversas técnicas de script, como por ejemplo las siguientes:

- Lineal: código de procedimiento, generalmente generados por herramientas como las que utilizan las técnicas la grabación y reproducción.
- Estructurado: utiliza las estructuras de control - usualmente, ‹switch› ‹if-else›, ‹para›, ‹mientras que› condiciones / declaraciones).
- Controlada por datos :los datos se almacenan en un entorno externo al de las pruebas en una base de datos, hoja de cálculo, u otro sistema.
- Generado por Palabra clave.
- Híbrido, en el cual se utilizandos o más de los patrones anteriores.

El marco de pruebas es responsable de:

– Definir el formato en el que expresar los resultados esperados.

– La creación de un mecanismo para enlazary ejecutar la aplicación bajo prueba.

– La ejecución de las pruebas en sí mismas.

– Generar informes de resultados.



Definición

Las **Interfaces de automatización** de pruebas son las plataformas que proporcionan un único espacio de trabajo para la gestión de múltiples herramientas de prueba y los marcos de las pruebas del sistema / Integración de aplicación bajo prueba.

El objetivo de la interfaz automatización de prueba es simplificar el proceso de pruebas desde mapeo a nivel criterios de negocio hasta la codificación codificación debe venir especificado en forma de proceso. Se supone que la interfaz de automatización de pruebas permite mejorar la eficiencia y la flexibilidad del mantenimiento y actualización de los scripts de prueba.

Los motores de la interfaz constituyen la parte superior de la jerarquía del entorno de las interfaces de prueba. El motor de interfaz consiste en un analizador y un módulo de ejecución de pruebas.

El analizador sirve para analizar los archivos de objetos procedentes del repositorio de objetos en el lenguaje de programación específico de prueba. El módulo de ejecución ejecuta los scripts de prueba utilizando un instrumento de prueba.

Los repositorios de objetos son una recopilación de datos de objetos de interfaz de usuario o aplicación almacenados por la herramienta de prueba, mientras que se somete a examen la exploración de la solicitudes e interacciones de cada objeto.

Las herramientas están diseñadas específicamente para trabajar sobre algún entorno de prueba en particular, como Windows, o Linux, o con herramientas de automatización web. Todas estas herramientas pueden servir como agente para conducir un proceso de automatización.

Sin embargo, un marco completo de automatización no es una herramienta para llevar a cabo una tarea específica, sino más bien una infraestructura que proporciona la solución en la que las diferentes herramientas pueden hacer su trabajo de una manera unificada. Esto proporciona una plataforma común para el equipo de trabajo de automatización de pruebas en un proyecto software determinado.

Como resumen se puede establecer la siguiente clasificación de las herramientas que se utilizan en la automatización de pruebas de software, teniendo en cuenta que estas se debe integrar para su utilización mas efectiva en un marco común de pruebas:

- Herramientas de integración y pruebas. Hay diferentes tipos de herramientas de pruebas aunque muchas de las herramientas de prueba poseen características que abarcan dos categorías o más

    - De adquisición de datos: herramientas que adquieren los datos que se utilizarán durante la prueba.

    - De medida estática: herramientas que analizan el código fuente sin ejecutar casos de prueba.

    - De medida dinámica: herramientas que analizan el código fuente durante la ejecución.

    - De simulación: herramientas que simulan las funciones del hardware o de otros elementos externos.

- Herramientas de gestión de pruebas: herramientas que prestan su asistencia en la planificación, desarrollo y control de las pruebas.

- Herramientas de pruebas cliente/servidor. El entorno C/S exige unas herramientas de pruebas especializadas que ejerciten la interfaz gráfica de usuario y los requisitos de comunicaciones en red para el cliente y el servidor.

- Herramientas de reingeniería. Las herramientas para el software heredado abarca un conjunto de actividades de mantenimiento que actualmente absorben un porcentaje importante de todo el esfuerzo relacionado con el software. La categoría de herramientas de reingeniería se puede subdividir en las funciones siguientes:

    - Herramientas de ingeniería inversa para producir especificaciones: se toma el código fuente como entrada y se generan modelos gráficos de análisis y diseño estructurados, listas de utilización y más información sobre el diseño.

- Herramientas de reestructuración y análisis de código: se analiza la sintaxis del programa, se genera una gráfica de control de flujo y se genera automáticamente un programa estructurado.

- Herramientas de reingeniería para sistemas de bases de datos: se utilizan para modificar sistemas de bases de datos con modelos lógicos antiguos (por ejemplo SGBD en red o jerárquicos) y convertirlos en SGBD.



Las herramientas de gestión de la configuración buscan automatizar las tareas que conllevan los procesos de gestión de la configuración y seguimiento y trazabilidad de los requisitos de desarrollo del software.

Para ello permiten que a lo largo se todas las fases del ciclo de vida se pueda trazar y gestionar cada elemento susceptible de configuración, encargándose por ejemplo de la gestión del versionado de los elementos de configuración, y de mantener los enlaces entre los distintos elementos de configuración.



Herramientas de seguimiento de requisitos: el objetivo de las herramientas de seguimiento es proporcionar un enfoque sistemático para el aislamiento de los requisitos, comenzando por lo indicado por el cliente o por la especificación.

Las herramientas de producción de documentos y de autoedición prestan su apoyo a casi todos los aspectos de la ingeniería del software. La mayoría de las organizaciones dedicadas al desarrollo de software invierten una cantidad de tiempo considerable en el desarrollo de documentos, y en muchos casos el proceso de documentación en sí resulta bastante deficiente.



Es frecuente que una organización de desarrollo de software invierta en la documentación hasta un 20 o un 30% de su esfuerzo global de desarrollo de software. Por esta razón, las herramientas de documentación y gestión de la configuración suponen una oportunidad importante para mejorar la productividad.

1.6.4. Herramientas de gestión de configuración

Típicamente CASE es una tecnología de estaciones de trabajo, aunque actualmente el entorno CASE se está adaptando a los sistema en red, al correo electrónico, a los tablones de anuncios electrónicos y a otras posibilidades de comunicarse entre todos los participantes en el proceso de desarrollo del software.



Herramientas de control de calidad: la mayor parte de las herramientas CASE de control de calidad son en realidad herramientas de métricas que hacen una auditoría del código fuente para determinar si se ajusta o no a ciertos estándares del lenguaje. Otras herramientas extraen métricas técnicas para tratar de determinar la calidad del software que se está construyendo.

A continuación se hace un repaso de las principales herramientas de gestión de la configuración que actualmente se aparecen en el mercado:

– Ansible: esta herramienta combina el despliegue de múltiples nodos, la ejecución de tareas ad-hoc, y gestión de la configuración en un solo paquete. Gestiona los nodos a través de SSH, y no requiere ningún software adicional remoto (excepto python 2.4 o posterior) para ser instalado. Módulos de trabajo sobre JSON y salida estándar y se pueden escribir en cualquier lenguaje. Utiliza YAML para expresar descripciones reutilizables de sistemas.

– bcfg2: software para gestionar la configuración de un gran número de ordenadores que utilizan un modelo de configuración central y el paradigma cliente-servidor. El sistema permite la conciliación entre el estado de los clientes y la especificación de la configuración central. Los informes detallados proporcionan una manera de identificar la configuración no administrado en los hosts. Generadores permiten código o la generación de plantilla basada en los archivos de configuración de un repositorio de datos central.

– cfengine: sistema de agente ligero el cual administra la configuración de un gran número de ordenadores que utilizan el paradigma cliente-servidor o independiente. Cualquier estado de cliente que es diferente de la descripción de la política se revierte al estado deseado. Se especifica el estado de configuración a través de un lenguaje declarativo.

– cdist: Es un sistema de gestión de la configuración de dependencia cero: Se requiere sólo ssh en el host de destino, que se suele activar en todas las máquinas Unix. Sólo el host de administración necesita tener instalado Python 3.2.

– Chef: Chef es una herramienta de gestión de la configuración escrito en Ruby, y utiliza una conexión DSL rubí puro para la escritura de elementos de configuración llamados “recetas”. Estas recetas contienen recursos que deben ser puestas en el estado declarado. Chef se puede utilizar como una herramienta de cliente-servidor, o utilizar en modo único.

– ISconf: Herramienta para ejecutar comandos y reproducir archivos en todos los nodos. Los nodos no necesitan ser de hasta; los comandos se ejecutarán cuando arrancan. El sistema no tiene ningún servidor central lo que los comandos se puede iniciar desde cualquier nodo y se replicarán en todos los nodos.

– Juju: Esta herramienta se centra en la noción de servicio, haciendo abstracción del concepto de máquina o servidor, y define las relaciones entre los servicios que se actualizan automáticamente cuando dos servicios vinculados al observar una modificación notable entre ellos.

– Sistema de configuración local (LCFG): LCFG administra la configuración con un lenguaje de descripción de central en XML, que especifica los recursos, los aspectos y perfiles. La configuración se implementa utilizando el paradigma cliente-servidor. scriptsadecuados para que se ejecuten sobre clientes, llamados componentes, transcriben los recursos en los archivos de configuración y reiniciar los servicios según sea necesario.

– Integración del servidor de PC abierta (OPSI): La integración del servidor PC abierta (OPSI) es un software de administración de escritorio para los

clientes de Windows basadas en servidores Linux. Proporciona la implementación de software automática (distribución), instalación desatendida del sistema operativo, administración de parches, inventario de hardware y software, gestión de activos y gestión de licencias de software, y las tareas administrativas para la gestión de la configuración.

– PIKT: PIKT es un sistema de monitoreo que también hace la gestión de configuración. PIKT consiste en un sofisticado preprocesador de funciones de archivos, un innovador lenguaje de programación con características de ahorro de trabajo únicas, una solución flexible, planificador de procesos dirigidos centralmente, un instalador de archivos de personalización, una colección de potentes extensiones de línea de comandos y otras herramientas útiles .

– PUPPET: PUPPET consiste en un idioma personalizado declarativa para describir la configuración del sistema, distribuida utilizando el paradigma cliente-servidory una biblioteca para realizar la configuración.

  La capa de abstracción de recursos permite a los administradores para describir la configuración en términos de alto nivel, como los usuarios, los servicios y paquetes. PUPPET entonces se encarga de garantizar que el estado del servidor coincide con la descripción.

– Quattor: El modelo de información Quattor se basa en la distinción entre el estado deseado y la situación real. El estado deseado se ha registrado en una base de datos de configuración de todo el sistema. Se caracteriza por el uso de un lenguaje de configuración especialmente diseñada llamada Pan para expresar y validar configuraciones, compuesto de bloques de construcción reutilizables jerárquicos llamados plantillas. Las configuraciones se propagan hacia arriba y se almacenan en caché en los nodos administrados.

– Radmind: Radmind administra la configuración de hosts a nivel del sistema de archivos. Este puede detectar los cambios externos en la configuración administrado, y opcionalmente puede revertir los cambios. Radmind no tiene elemento de configuración de nivel superior .

– Rex es un sistema de ejecución remota con capacidades de implementación de gestión de configuración integrada y software.

  El administrador proporciona instrucciones de configuración a través de los llamados Rexfiles. Están escritos en un pequeño lenguaje, pero también pueden contener Perl arbitrario

– Rundeck: Rundeck es un programador de software de trabajo de código abierto y el sistema Run Book Automation para la automatización de los procesos de rutina a través de entornos de desarrollo y producción.

  Combina la programación de tareas, ejecución de comandos de varios nodos, la orquestación de flujo de trabajo y registro de todo lo que ocurre. Implementa una política de control de acceso que gobierna ejecuta acciones en los nodos a través de la configuración de nodo ejecutor y no requiere ningún software de control remoto adicional.

– SALT: SALT comenzó como una herramienta para la administración remota de servidores. Como su uso ha crecido, se ha ganado una serie de características ampliadas, incluyendo un mecanismo más amplio para la configuración de hosts.

  Esta es una característica relativamente nueva que se produce a través de la utilización de componentes de SATL Unidos.

– SmartFrog: Herramienta basada en Java para desplegar y configurar aplicaciones distribuidas a través de múltiples máquinas. En ella no hay un servidor central sino que se puede implementar un archivo de configuración de SF en cualquier nodo y ester será distribuido a igual nodos de acuerdo a la información de la distribución que figura dentro del descriptor de despliegue en sí mismo.

– SPACEWALK: Es una solución de gestión de sistemas Linux y Solaris de código abierto y es el proyecto de la empresa de Red Hat Network Satellite.

  Esta aplicación funciona con RHEL, Fedora y otras distribuciones derivadas como RHEL CentOS, Scientific Linux, etc Hay esfuerzos en curso para conseguir que se distribuya para su inclusión en Fedora.

  Proporciona un inventario de sistemas (información de hardware y software, la instalación y las actualizaciones de software, recogida y distribución de paquetes de software personalizados en grupos manejables, los sistemas de abastecimiento, la gestión y la implementación de archivos de configuración, monitorización del sistema, creación huésped virtual, iniciar, detener y configurar estos elementos virtuales invitados y delegar todas estas acciones a los usuarios locales o usuarios de LDAP.

– STAF: El Marco de pruebas de software de automatización (STAF) permite a los usuarios crear en entornos multiplataforma, entornos de prueba de software distribuidos.

STAF elimina el problema de la construcción de una infraestructura de automatización, lo que permite a los usuarios centrarse en la construcción de su solución de automatización.

El marco STAF proporciona la base sobre la que construir soluciones de más alto nivel, y proporciona un enfoque de acoplamiento activo con el apoyo a través de una gran variedad de plataformas y lenguajes.

– System Version Concurrent (CVS), también conocido como el Sistema de versiones concurrente, es un software libre cliente-servidor de control de versiones de sistemas que se usa sobre todo en el campo de desarrollo de software libre.



Un **sistema de control de versiones** mantiene un registro de todo el trabajo y todos los cambios en un conjunto de archivos, y permite que varios desarrolladores (potencialmente muy separadas en el espacio y tiempo) para colaborar .



Dick Grune desarrolló CVS como una serie de scripts de shell en julio de 1986.

Además de entre los desarrolladores de software comercial, CVS se hizo popular con los desarrollos el software de código abierto alrededor mundo y fue liberado bajo la GNU General Public License .

Si bien hasta hace bastante poco hubo un desarrollo regular para agregar características y corregir errores incluyendo compilaciones y los resultados de pruebas, no ha habido nuevos lanzamientos desde el año 2008 . Se considera que el producto está maduro y por tanto las nuevas versiones no se producen hasta que no haya solicitudes de nuevas características o informes de error.

CVS utiliza una arquitectura cliente-servidor, un servidor almacena la versión actual de un proyecto y su histórico de versiones, y los clientes se conectan al servidor con el fin de obtener («check out»), una copia completa del proyecto, el trabajo se realiza sobre esta copia en esta copia y luego después de suben al servidor («check in»)los cambios realizados.

Normalmente, el cliente y el servidor se conectan a través de una LAN oa través de Internet, pero el cliente y el servidor pueden ejecutarse tanto en la misma máquina si CVS tiene la tarea de hacer el seguimiento del historial de versiones de un proyecto en el que solamente trabajan desarrolladores en modo local.

El software de servidor se ejecuta con normalidad en Unix o Linux, (aunque al menos la versión CVS-NT servidor también soporta varias versiones de Microsoft Windows), mientras que los clientes CVS pueden funcionar en cualquier sistema operativo importante.



Varios desarrolladores pueden trabajar en el mismo proyecto simultáneamente, cada una edición de archivos dentro de su propia «copia de trabajo» del proyecto, y enviar sus modificaciones a el servidor. Para evitar la posibilidad de que las personas sobrescriban el trabajo de otros y se produzcan conflictos, el servidor sólo acepta cambios realizados en la versión más reciente de un archivo.

Por lo tanto, se espera que los desarrolladores traten mantener su copia de trabajo al día con la incorporación de cambios de otras personas de manera regular.

Esta tarea es manejada automáticamente por el cliente CVS, que requiere intervención manual sólo cuando un conflicto de edición surge entre una modificación check-in y la versión local aún-sin control de un archivo.

Si la comprobación de la operación tiene éxito, entonces los números de versión de todos los archivos implicados se incrementan automáticamente y el servidor CVS escribe una descripción suministrado por el usuario, la fecha y el nombre del autor a sus registros de archivos.

CVS también puede ejecutar de manera externa, los scripts de procesamiento de registros especificados por el usuario después de cada subida de archivos.

Estos scripts se instalan de una entrada en el archivo de configuración de CVS, lo que puede conllevar por ejemplo una notificación al responsable del proyecto por correo electrónico o convertir los datos de registro en un formato basado en la Web.

Los clientes pueden también comparar versiones, solicitar una historia completa de los cambios, o explorar una foto histórica del proyecto en una fecha determinada o en un número de revisión.

Muchos proyectos de código abierto permiten lo que se denomina el «acceso de lectura anónimo», dentro de su repositorio CVS, una característica por primera vez utilizada por OpenBSD.

Esto significa que los clientes pueden obtener y comparar versiones, ya sea con una contraseña en blanco o contraseñas simples publicadas en la web. EN este caso sólo el check-in y actualización de cambios requiere una cuenta personal y clave en estos entornos.

Los clientes también pueden utilizar el comando «update» para obtener sus copias locales al día con la versión más reciente en el servidor. Esto elimina la necesidad de descargas repetidas de todo el proyecto, optimizando el uso del ancho de banda de los clientes y los servidores.



Sabías que

CVS también puede mantener distintas «ramas» de un proyecto. Por ejemplo, una versión comercial del proyecto de software puede formar una rama, que se utiliza para la corrección de errores, mientras que la versión actualmente en desarrollo, con grandes cambios y nuevas características, puede formar una rama separada.

CVS utiliza compresión de alto rendimiento para el almacenamiento eficiente de diferentes versiones de un mismo archivo.

Esto funciona bien con grandes archivos de texto con pocos cambios de una versión a la siguiente, como suele ser el caso de los archivos de código fuente y otra documentación asociada a un proyecto informático.

Por otro lado, cuando el CVS se le dice que guarde un archivo como binario, esto indica que se conservará una versión individual en el servidor, sin comprimir. El almacenamiento de archivos como binario es importante a fin de evitar la corrupción de archivos binarios.

En el mundo del software de código abierto, el Concurrent Version System (CVS) ha sido durante mucho tiempo la herramienta más elegida para el control de versiones.

Ya que CVS es software libre, su forma de trabajar no restrictiva y el apoyo para el funcionamiento en red - que permiten a docenas de programadores dispersos geográficamente compartir su trabajo, todo esto se ajusta muy bien a la naturaleza colaborativa del mundo de código abierto.

1.6.5. Herramientas para métricas

CVS y su modelo de desarrollo semi caótico se han convertido en piedras angulares de la dinámica de trabajo open-source

La medición precisa es un requisito previo para toda las disciplinas de la ingeniería, y el software no es una excepción. Durante décadas buscan ingenieros e investigadores para expresar características de software con de forma numérica para facilitar la evaluación de la calidad del software.



Se han desarrollado un gran conjunto de métricas de calidad de software, y existen numerosas herramientas para recopilar métricas a partir de los modelos de análisis, diseño o el programa terminado.

Esta gran variedad de herramientas permite a un usuario seleccionar la herramienta más adecuada, por ejemplo, dependiendo su manejo, herramientas de soporte, o el precio.

Hay que tener en cuenta que las estimaciones y métricas son cruciales a la hora de realizar correctamente la planificación de un proyecto software. Las estimaciones de esfuerzo son utilizadas como dato de entrada a la planificación, presupuestos, análisis de inversión, los procesos de fijación de precios y las rondas de licitación del proyecto.

Sin embargo, esto supone que todas las herramientas calculan la misma métrica de la misma manera. Esto a veces no es así y por tanto conviene estudiar este aspecto a la hora de seleccionar un métrica adecuada. Además hay que tener en cuenta que distinta medidas hechas con distintas herramientas pueden dar valores distintos para la misma métrica.

Algunas encuestas publicadas sobre la práctica de estimación sugieren que la estimación de expertos es la estrategia dominante en la estimación de esfuerzo de desarrollo de software.

Por lo general, las estimaciones de esfuerzo son demasiado optimistas y hay un fuerte exceso de confianza en su exactitud. El error por exceso medio del esfuerzo parece ser alrededor del 30% y no es decreciente en el tiempo ni utilizando mejores medidas.

Sin embargo, la medición del error de estimación no es problemático, sino que el problema esta motivado por el exceso de confianza en la exactitud de las estimaciones de esfuerzo.



Sabías que

Esto se ilustra con la constatación de que, en promedio, si un profesional de software estima en el 90% el rango de confianza de un estimación del esfuerzo real,(lo que podríamos decir «casi seguro»)en un intervalo mínimo-máximo la frecuencia observada en la que el esfuerzo realmente está en este ámbito es sobre un 60-70%.

Actualmente el término «estimación de esfuerzo» se utiliza para denotar como diferentes conceptos como el uso más probable de esfuerzo (valor modal), el esfuerzo que corresponde a una probabilidad de 50% de no superior a el esfuerzo planeado, el esfuerzo o presupuestado el esfuerzo utilizado para proponer una oferta o precio para el cliente.

Es necesario utilizar unas herramientas adecuadas para medir el esfuerzo de un proyecto ya que en muchas ocasiones el llevar a buen puerto este depende de que se haga una correcta estimación. Para ello las herramientas de generación de estimaciones son muy importantes ya que partan numerosas ventajas a los gestores del proyecto:

– Las métricas del software mejoran la capacidad del gestor para controlar y coordinar el proceso de ingeniería del software y la capacidad del ingeniero para mejorar la calidad del software que se produce lo que es necesario para evitar que se produzcan desviaciones importantes en el proyecto. Para su cálculo se utilizan distintos tipos de herramientas:

– Las herramientas de estimación calculan el esfuerzo estimado, la duración del proyecto y el número de personas recomendado para el proyecto.

– Las herramientas de planificación de proyectos hacen posible que el jefe de proyecto defina todas las tareas del proyecto, que cree una red de tareas, que represente las interdependencias entre tareas y que modele el paralelismo que sea posible para ese proyecto.

– Las que son orientadas a la gestión se sirven de métricas específicas del proyecto que proporcionan una indicación global de productividad o de calidad.

– Las herramientas con orientación técnica determinan las métricas técnicas que proporcionan una mejor visión del diseño o del código.

– Las Herramientas de análisis de riesgos se encargan de la fase de identificación y seguimiento de los riesgos de un proyecto.

– Las herramientas de análisis de riesgos hacen posible que el gestor del proyecto construya una tabla de riesgos proporcionando una guía detallada en la identificación y análisis de riesgos.



La identificación de posibles riesgos y el desarrollo de un plan para mitigar, monitorizar y gestionar esos riesgos tiene una importancia fundamental en los proyectos grandes.

Además, el gestor deberá utilizar las herramientas que recojan métricas que proporcionen una indicación de la calidad del producto del software.

Las herramientas de esta categoría suelen ser extensiones de herramientas de planificación de proyectos.El objetivo de las herramientas de seguimiento de requisitos es proporcionar un enfoque sistemático para el aislamiento de los requisitos, comenzando por lo indicado por el cliente o por la especificación



La planificación del proyecto y el plan del proyecto deben ser analizados de forma continua para evaluar la correcta consecución de los objetivos planificados. Para ello se usan las herramientas de gestión de proyectos.

UD1
Lo más importante

- La ingeniería del software busca formalizar los procesos de elaboración de sistemas informáticos, aplicando las técnicas de la ingeniería.

- El desarrollo del software se divide de manera clásica en varias fases, análisis de requisitos, diseño, codificación y pruebas y mantenimiento.

- En la fase de análisis de requisitos se trata de averiguar cuales son las necesidades reales del cliente, y crear un modelo de estas necesidades.

- La fase de diseño lo que se trata es de buscar la solución más adecuada a las necesidades que se han obtenido en la fase de análisis.

- Durante la codificación estas soluciones se plasman utilizando un lenguaje de programación. En la fase de pruebas se trata de Verificar que el software funciona correctamente y Validar que se cumplan con los requisitos. Por último la fase de mantenimiento busca eliminar errores no encontrados o ampliar el sistema a nuevas necesidades.

- Las fases genéricas se aplican durante lo que se llama el ciclo de vida. Hay varios modelos, de ciclo de vida, el modelo en cascada, el modelo evolutivo en espiral modelo Sahimi o el modelo orientado a objetos.

- Una metodología de desarrollo indica las tareas que se deben de realizar en cada una de las fases del ciclo de vida. Existen múltiples metodologías, como Métrica, Merisse o el Modelo Unificado del Rational.

– En una metodología Orientada a Objetos, se modelan los requisitos mediante casos de uso. De estos se obtiene un primer diagrama de clases, que se refina durante el diseño, mediante los diagramas de secuencias. EL resultado final es un modelo detallado de clases en el que se especifican los atributos y métodos que tendrá cada una de nuestras clases.

– Las herramientas CASE buscan aumentar la productividad durante el desarrollo del software, permitiendo automatizar las tareas que se producen a diferentes niveles y en varias fases del desarrollo .

– Las herramientas de ingeniería del software trabajan a alto nivel, formalizando la fase de análisis de requisitos, asignación de recursos y gestión del proyecto software. Destacamos por su importancia las herramientas de gestión y planificación de recursos (ERP).

– Los entornos de desarrollo proporcionan un marco de trabajo completo al programador en el que desarrollar su labor, incluyendo la codificación, depuración, pruebas y documentación en una misma herramienta.

– Las herramientas de prueba busca automatizar el proceso de probar un sistema software. Hay numeroso tipos entre las que destacamos las desarrolladas mediante script y las dirigidas por API.

– Las herramientas de gestión de la configuración automatizan las tareas de mantenimiento de los elementos de configuración del software y su trazabilidad. Destacan las herramientas de versionado que permiten a un equipo extenso trabajar sobre el mismo proyecto compartiendo los elementos sin conflictos.

– Las métricas son un factor muy importante en el desarrollo del software. Las herramientas para realizarlas permiten automatizar el proceso y sirven para calcular costes y esfuerzos.

UD1
Lo más importante

1. Las fases genéricas del desarrollo del software son:

    a. Análisis, modelado, codificación y testeo.
    b. Análisis, diseño, codificación, pruebas y mantenimiento.
    c. Diseño, escritura y verificación.
    d. Verificación y validación.

2. ¿Cuál de estas es una ventaja del ciclo de vida clásico?

    a. Es fácil arreglar los errores.
    b. Se obtienen el producto al final.
    c. La planificación es sencilla.
    d. Hay muchos indicadores del ritmo de trabajo.

3. ¿Cual no es una ventaja del ciclo de vida en espiral?

    a. Es difícil evaluar los riesgos.
    b. No necesita una definición completa de los requisitos al comienzo.
    c. El riesgo en general es menor.
    d. Es más fácil validar los requisitos.

4. ¿Cuál de las siguientes NO es una técnica de extracción de requisitos?

   a. El modelado de requisitos durante el análisis.
   b. Las entrevistas y Brainstorming.
   c. El Desarrollo Conjunto de Apliaciones (JAD).
   d. Los prototipos.

5. La matriz de trazabilidad:

   a. Se usa para comprobar que los requisitos del proyecto se están cumpliendo.
   b. Nos indica los errores de diseño.
   c. Sirve para que el usuario tenga una buena experiencia de uso de la aplicación.
   d. Indica el numero de clases de un proyecto.

6. La metodología métrica:

   a. Es la metodología oficial de las administraciones públicas en España
   b. Es obligatorio utilizarla en toda España, salvo en el País Vasco y Navarra.
   c. No tiene fase de análisis de requisitos.
   d. No es orientada a objetos.

7. ¿Cuáles no son objetivos de una herramienta CASE?

   a. Facilitar la realización de prototipos.
   b. Eliminar la necesidad de esfuerzo de programación.
   c. Mejorar y estandarizar la documentación.
   d. Facilitar la reutilización de componentes.

8. Las herramientas de ingeniería del software:

   a. Trabajan solamente a bajo nivel.
   b. Trabajan a nivel medio de la organización.
   c. Trabajan fundamentalmente a alto nivel
   d. Modelan la organización a nivel táctico.

9. Sobre un IDE:

    a. Está perfectamente claro qué es un IDE y qué no lo es .
    b. No suelen incorporar un navegador de clases.
    c. No están diseñados para aumentar la productividad de los programadores.
    d. Todas las anteriores son falsas.

10. Sobre los prototipos:

    a. No deben ser mostrados al usuario final.
    b. Deben centrarse en la arquitectura de sistema en vez de en su aspecto.
    c. No requieren de ninguna herramienta en especial.
    d. Su propósito es que los usuarios finales puedan evaluar las propuestas de los desarrolladores.

Área: informática y comunicaciones

UD2

Planificación y seguimiento

2.1. Realización de estimaciones

2.2. Planificaciones. Modelos de diagramado. Diagramas de Gantt

2.3. Análisis del proceso de seguimiento. Reuniones e informes

2.1. Realización de estimaciones

La estimación es un proceso de gestión de una importancia estratégica para todo proyecto software y comprende la evaluación del esfuerzo necesario, de la duración del proyecto y de los recursos necesarios para llevarlo a cabo.

Se realiza en las primera fase del ciclo de vida y generalmente es el primer paso para proceder a la planificación del proyecto.

Esto implica una cierta incertidumbre y por este motivo es conveniente realizar también un análisis de los riesgos asociados a un posible error en dicha estimación.

Los resultados del mismo se suelen dar en función de una probabilidad de cumplimiento del esfuerzo y del tiempo estimados inicialmente.

Existen multitud de métodos de estimación cada uno desarrollado con la intención de estimar diversos tipos de proyectos.

Una posible clasificación de dichos métodos permite distinguir entre cuatro grupos: Modelos matemáticos paramétricos,Estimación basada en la experiencia de experto, técnicas orientadas al aprendizaje y modelos dinámicos



Definición

Los modelos paramétricos relacionan un conjunto de variables dependientes (por ejemplo, esfuerzo ycalendario del proyecto) con una o más variables independientes denominadas factores de costo (por ejemplo el uso de herramientas CASE o la capacidad de los analistas).

De esta forma se realizan las estimaciones a través de unas expresiones matemáticas que han sido ajustadas mediante un conjunto de parámetros.

Las diferencias fundamentales entre los distintos modelos paramétricos están en el tipo de modelo matemático o formulación que se utiliza para establecer la relación entre las variables dependientes e independientes que se consideran.

De la definición de variables adecuadas y de su correcta ponderación o parametrización depende en gran medida la exactitud de las predicciones realizadas por el modelo.

La estimación basada en modelos paramétricos es una de las más extendidas y utilizadas. Más adelante se verá con detalle uno de los principales modelos paramétricos de estimación, el modelo COCOMO



Los modelos denominados como de **estimación basada en expertos** son aquellos que proporcionan una manera de extraer, de la forma más objetiva posible, los conocimientos de expertos para realizar las estimaciones.

Normalmente, lo hacen a través de un conjunto de pasos o mediante la utilización de cuestionarios.



Técnicas orientadas al aprendizaje: este conjunto de métodos se basa en el empleo de datos recogidos en proyectos anteriores para realizar las estimaciones y en este aspecto es similar a los modelos matemáticos paramétricos, los cuales utilizan también los datos de proyectos anteriores para determinar sus parámetros.

La diferencia que presentan los modelos clasificados en este grupo, con los paramétricos, aparte de no estar fundamentados en ecuaciones matemáticas, es que no pretenden ser de aplicación universal sino que su objetivo es realizar estimaciones precisas para aquellos proyectos del mismo tipo de los que se han empleado para aprender.



Definición

Modelos dinámicos: estos modelos consideran que los factores que afectan al coste de un proyecto software cambian a lo largo del tiempo y para su cálculo realizan estimaciones y simulaciones de dichas variaciones. Por lo tanto, el fundamento de este conjunto es muy diferente al de los otros tres, los cuales tienen una visión estática del sistema.



Sabías que

Investigadores y profesionales de software se han ocupado de los problemas de estimación de esfuerzo para los proyectos de desarrollo de software desde la década de 1960.

La mayor parte de la investigación se ha centrado en el desarrollo de modelos de estimación de esfuerzo del software formales. Los primeros modelos se basaban normalmente en el análisis de regresión oderivanmatemáticamente de las teorías de otros dominios.

Desde entonces, un gran número de métodos de construcción de modelos han sido evaluados, tales como los enfoques fundados en el razonamiento basado en casos, de clasificación y árboles de regresión, la simulación, las redes neuronales, la estadística bayesiana, análisis léxico de especificaciones de requisitos, la programación genética, la programación lineal, la producción económica modelos, soft computing, lógica difusa modelado, estadística bootstrapping, y combinaciones de dos o más de estos modelos.

Los métodos de estimación tal vez más comunes hoy en día son los modelos de estimación paramétrica COCOMO y SLIM. Ambos tienen su base en la investigación de la estimación realizada en la década de 1970 y 1980 y desde entonces se van actualizando con nuevos datos de calibración, y la última actualización importante ha sido COCOMO II en el año 2000.

La estimación basado enfoques que usan en medidas de tamaño desde el punto de vista de la funcionalidad, como por ejemplo, los puntos de función, también se basa en investigaciones realizadas en los años 1970 y 1980, pero se han re-calibrados con medidas de tamaño modificados y además se han propuesto diferentes enfoques de medida, como los «puntos de caso de uso» propuesto en la década de 1990 y COSMIC en la década de 2000.

Hay muchas maneras de clasificar los enfoques de estimación, dependiendo de la técnica que se use para ello y del tipo de datos de entrada que necesite la técnica. Generalmente la clasificación no es estricta habiendo métodos que se pueden incluir en una u otra categoría simultáneamente, y también existen métodos híbridos que combinan lo mejor de dos o mas categorías. Las categorías básicas son las que se nombran a continuación.

– Evaluación de expertos o peritos: El denominado paso de cuantificación, es decir, el paso donde se produce la estimación está basada en los procesos de razonamiento de un humano experto.

– Modelo de estimación formal: La etapa de cuantificación se basa en procesos mecánicos, por ejemplo, el uso de una fórmula derivada de los datos históricos.

– La estimación combinada: La etapa de cuantificación se basa en una combinación de juicio y mecánica de las estimaciones de diferentes fuentes.

La selección del método de estimación es de bastante importancia en cualquier organización.La evidencia sobre las diferencias en la precisión de la estimación de los diferentes enfoques y modelos de estimación sugiere que no hay una «mejor manera», y que la precisión relativa de un enfoque o modelo en comparación con otro depende en gran medida del contexto.

Esto implica que las diferentes organizaciones beneficiarse del uso de diferentes enfoques de estimación.Los resultados, que se resumen que pueden apoyar la selección de método de cálculo basado en la precisión esperada de un enfoque incluyen:

– Evaluación de peritos es, en promedio, tan precisa como la la estimación basada en modelos, aunque si hablamos de expertos en el modelo de negocio de la empresa la exactitud que se consigue es mayor.

- En particular, las situaciones con relaciones entre datos inestables e información de gran importancia, no incluida en el modelo pueden sugerir el uso de la estimación de expertos. Esto supone, por supuesto, que los expertos con experiencia relevante están disponibles.

- Los modelos formales de estimación que no están adaptados a un contexto de una organización en particular pueden ser muy imprecisos. Por ello es necesario que cada organización adapte el uso de sus técnicas a sus datos históricos.

- El uso de los datos históricos propios por tanto es crucial si uno no puede estar seguro de que las relaciones básicas del modelo de estimación como por ejemplo, los parámetros de la fórmula, se basan en contextos de proyectos similares.

- Los modelos de estimación formal puede ser particularmente útil en situaciones en las que el modelo se adapta al contexto de la organización, ya sea a través del uso de datos históricos propios o que el modelo se deriva de los proyectos y contextos similares, ya que es probable que las estimaciones de los expertos estén sujetas a un alto grado de optimismo.

- El hallazgo más contundente, en muchos dominios de predicción, es que la combinación de estimaciones de fuentes independientes, preferiblemente aplicando diferentes enfoques, en promedio, mejora sensiblementela precisión de la estimación.

- Además, otros factores como la facilidad de comprensión y comunicación de los resultados de este enfoque, la facilidad de uso de un enfoque, el costo de la introducción de un enfoque siempre deben ser consideradas en un proceso de selección.

La medida más común de la exactitud promedio de estimación es la MRE o magnitud del error relativo medio, donde el MRE de cada estimación se define como se indica en la fórmula

MRE = (esfuerzo real- esfuerzo estimado)/ esfuerzo real

Un error de estimación alto no puede automáticamente ser interpretada como un indicador de la capacidad baja de estimación.

Hay muchas razones que pueden hacer variar el costo de un proyecto, como la alta complejidad de los trabajos de desarrollo y que por ejemplo la funcionalidad que no se detalle con la suficiente precisión. Todo esto puede hacer que el esfuerzo necesario sea mucho mayor que el estimado al principio.

Hay muchos factores psicológicos potencialmente explican la fuerte tendencia a estimaciones de esfuerzo excesivamente optimistas que tienen que ser tratados para aumentar la precisión de las estimaciones de esfuerzo.

Estos factores son esenciales, incluso cuando se utilizan modelos formales de estimación, porque gran parte de la entrada a estos modelos están basado en el juicio de los involucrados.

Los factores que han demostrado ser importantes son: El optimismo a ultranza, la falacia de planificación y la disonancia cognitiva. Básicamente hay que tener en cuanta tres factores:

– Es fácil calcular lo que sabes.

– Es difícil estimar lo que sabes que no sabes.

– Es muy difícil estimar las cosas que no sabes que no sabes.



El Modelo Constructivo de Costes o COCOMO (Constructive Cost Model) fue desarrollado por B. W. Boehm a finales de los 70 y comienzos de los 80 y fue desarrollado como un conjunto de modelos para la estimación que toma como base análisis estadísticos de una serie de proyectos con características similares.



COCOMO está formado por una colección jerárquica de modelos de estimación de costes software que incluye tres submodelos **básico, intermedio y detallado.**

Las ecuaciones de estimación del esfuerzo de desarrollo tienen la forma genérica siguiente:

Td = 2.5 E*0.38

siendo:

– E: esfuerzo en personas/mes.

– td: Tiempo de desarrollo en meses

– S el número de miles de líneas de código fuente

– m(X) es un multiplicador que depende de 15 atributos (en el modelo básico toma el valor 1)

– ai, bi, ci y di son coeficientes que dependen del submodelo de estimación y del "modo" o tipo de proyecto que se está planificando. En la siguiente tabla se muestran los coeficientes para los diferentes modos y modelos:

Modelo	Básico				Intermedio			
Modo	ai	bi	ci	di	ai	bi	ci	di
Orgánico	2.4	1.05	2.5	0.38	3.2	1.05	2.5	0.38
Semiempotrado	3.0	1.12	2.5	0.32	3.0	1.12	2.5	0.32
Empotrado	3.6	1.2	2.5	0.35	2.8	1.2	2.5	0.35

Modelo Básico: Este modelo trata de estimar, de forma rápida, la mayoría de proyectos pequeños y de tamaño medianos dentro del COCOMO. Es una manera de realizar estimaciones rápidas sin necesidad de muchos factores, aunque en ocasiones las estimaciones no sean muy precisas.

Se consideran tres modos de desarrollo, o lo que es lo mismo, tres tipos de proyectos dentro de este modelo: los denominados modos orgánico, semiacoplado y empotrado.



Modo orgánico: en este modo, un grupo pequeño de programadores experimentados desarrollan software en un entorno familiar.

El tamaño del software varía desde unos pocos miles de líneas (tamaño pequeño) a unas decenas de miles de líneas de código (medio), mientras que en los otros dos modos el tamaño varía de pequeño a muy grandes (varios cientos de miles de líneas).



Modo Empotrado: en este modo, el proyecto tiene fuertes restricciones, que pueden estar relacionadas con el interface hardware o con otros sistemas con los que interaciona.

El problema a resolver es un problema **único** y por tanto es difícil basarse en la experiencia, puesto que puede no haber expertos en el tema. Las estimaciones de tiempo y coste se basan en las mismas ecuaciones que en el modo orgánico, pero con diferentes constantes.



Modo semiacoplado: es un modo intermedio entre los dos anteriores. Dependiendo del problema, el grupo puede incluir una mezcla de personas experimentadas y no experimentadas.

Ejemplo

Una gran empresa de productos químicos, está planeando desarrollar un nuevo programa informático para realizar un seguimiento de las materias primas.

El sistema será desarrollado por un equipo interno de programadores y analistas que se han venido desarrollando programas similares durante varios años. Un estudio inicial ha determinado que el tamaño del programa será de aproximadamente 32.000 líneas de código.

Este proyecto es un buen ejemplo de un proyecto de software de modo orgánico. Usando las ecuaciones básicas de COCOMO para este modo de desarrollo que tenemos:

Esfuerzo: E = 2.4 * (32) 1,05 = 91 personas-mes

Productividad: 32000lineas/ 91 personas-mes = 352 líneas por persona-mes /

Duración y dotación de personal: una vez que se obtiene una estimación de esfuerzo (en meses-personal), un gerente debe determinar el número de personas para poner en el trabajo.

De esta manera podrán determinar la duración del calendario del proyecto.

Es muy importante tener en cuenta que más personal no significa proporcionalmente menos tiempo del calendario. Más personal complican las comunicaciones y esta complejidad se traduce en una desaceleración del proyecto.

La segunda ecuación del modelo COCOMO utilizar el esfuerzo estimado del proyecto (SM) para sugerir la duración óptima del calendario del proyecto. Para el ejemplo anterior con un esfuerzo estimado de 91 SM tenemos:

Calendario: TDEM = 2.5 * (91) 0,38 = 14 meses

Después de estimar la duración del proyecto, el gerente puede determinar fácilmente el número de personas en el medio se deben poner en el proyecto:

Promedio de Personal: 91 personas-mes / 14 meses = 6.5 personas

Modelo Intermedio: En este modelo se introducen varios de los denominados atributos de coste que sirven para tener en cuenta el entorno de trabajo.

Estos atributos se utilizan para ajustar el coste del proyecto al entorno real, incrementando la precisión de la estimación.

Estos atributos lo que tratan es de capturar el impacto del entorno del proyecto en el coste total de desarrollo.

Los atributos se agrupan en cuatro categorías: atributos del producto, atributos del ordenador, atributos del personal y atributos del proyecto.

Cada atributo se cuantifica para un entorno de proyecto siguiendo la siguiente tabla:

Atributos	Valor				
	Muy bajo	Bajo	Nominal	Alto	Muy alto
Atributos de software					
Fiabilidad	0,75	0,88	1	1,15	1,4
Tamaño de base de datos		0,94	1	1,08	1,16
Complejidad	0,7	0,85	1	1,15	1,3
Atributos de hardware					
Restricciones de tiempo de ejecución			1	1,11	1,3
Restricciones de memoria virtual			1	1,06	1,21
Volatilidad de la máquina virtual		0,87	1	1,15	1,3
Tiempo de respuesta		0,87	1	1,07	1,15
Atributos de personal					
Capacidad de análisis	1,46	1,19	1	0,86	0,71
Experiencia en la aplicación	1,29	1,13	1	0,91	0,82
Calidad de los programadores	1,42	1,17	1	0,86	0,7
Experiencia en la máquina virtual	1,21	1,1	1	0,9	
Experiencia en el lenguaje	1,14	1,07	1	0,95	
Atributos del proyecto					
Técnicas actualizadas de programación	1,24	1,1	1	0,91	0,82
Utilización de herramientas de software	1,24	1,1	1	0,91	0,83
Restricciones de tiempo de desarrollo	1,22	1,08	1	1,04	1,1

Los atributos del producto que se tienen en cuenta en este modelo son los siguientes:

– RELY: garantía de funcionamiento requerida al software.

– DATA: tamaño de la base de datos.

– CPLX: complejidad del producto.

– RUSE: Índice de reutilización del producto.

– DOCU: Cantidad y complejidad de la documentación a desarrollar.

Los atributos del hardware que se han de considerar cuando se usa el modelo COCOMO intermedio son los que se enumeran a continuación:

– TIME: restricción de tiempo de ejecución.

– STOR: restricción del almacenamiento principal.

– PVOL: Cambios en la plataforma hardware.

Por otro lado también se han de tener en consideración los atributos del personal que se encuentra en el proyecto. Los que hay que prestar especial consideración son los siguientes:

– ACAP: capacidad del analista.

– AEXP: experiencia del analista.

– PCAP: capacidad del programador.

– PEXP: experiencia del programador.

– LEXP: experiencia en lenguaje de programación.

– PCONT: Continuidad del personal.

El último factor que tienen cuenta el modelo intermedio de COCOMO son lso atributos del mismo proyecto. Los que hay que medir son estos:

– SITE: Desarrollo en varias localizaciones.

– TOOL: utilización de herramientas CASE.

– SCED: restricciones del plan de desarrollo.

Los valores obtenidos se multiplican en la ecuación del esfuerzo y nos proporciona el esfuerzo ajustado al entorno del proyecto.

Modelo Detallado: Este modelo introduce como característica principal el hecho de utilizar multiplicadores de esfuerzo específicos para cada fase del proyecto que se está llevando a cabo.

El modelo COCOMO detallado incorpora todas las características de la versión intermedia con una evaluación de impacto del conductor de costos en cada paso (análisis, diseño, etc.) del proceso de ingeniería de software.



El modelo detallado utiliza diferentes multiplicadores de esfuerzo para cada atributo factor de costo. Estos multiplicadores de esfuerzo sensible a la fase son cada uno para determinar la cantidad de esfuerzo que se requiere para completar cada fase.

En COCOMO detallado, todo el software se divide en diferentes módulos y luego aplicamos COCOMO en diferentes módulos para estimar el esfuerzo y luego sumar el esfuerzo.

Es decir, el esfuerzo se calcula en función del tamaño del programa y un conjunto de factores de coste determinado de acuerdo a cada fase del ciclo de vida del software.

Algunas fases se ven más afectadas que otras por los atributos. El modelo detallado proporciona un conjunto de multiplicadores de esfuerzo para cada atributo. Esto ayuda a determinar la asignación del personal para cada fase del proyecto.

El desarrollo del software se lleva a cabo a través de cuatro fases consecutivas: Requisitos/planes, diseño del producto, programación y prueba/integración.

La primera fase es la fase de requisito y planificación del producto. Esta es la primera fase del ciclo de desarrollo, y es con la que debe comenzar cualquier proyecto.

En ella se analizan los requisitos, se muestra un Plan de proyecto y se general una especificación completa del producto. Esta fase consume del 6% al 8% del esfuerzo nominal E, y dura del 10% al 40% del tiempo nominal de desarrollo td. Estos porcentajes dependen del modo y del tamaño.

Diseño del producto: La segunda fase del ciclo de desarrollo COCOMO se preocupa de la determinación de la arquitectura del producto y de las especificaciones de los subsistemas. Esta fase requiere del 16% al 18% del esfuerzo nominal E, y puede durar del 19% al 38% del tiempo nominal de desarrollo td.

Programación: La tercera fase del ciclo de desarrollo COCOMO se subdivide en dos subfases: diseño detallado y prueba del código. Esta fase requiere del 48% al 68% del esfuerzo nominal E, y dura del 24% Al 64% del tiempo nominal de desarrollo.

Prueba e Integración: Esta última fase consiste principalmente en unir las diferentes unidades ya probadas.Se utiliza del 16% al 34% del coste nominal Kn y dura del 18% al 34% del td.



Sabías que

Los puntos de función se definieron en 1979 en el desarrollo de aplicaciones de medición de la productividad por Allan Albrecht en IBM.

Cuando se utilizan los puntos de función las necesidades funcionales de los usuarios del software se identifican y cada una se clasifican en uno de los siguientes cinco tipos: salidas, consultas, aportaciones, archivos internos, y interfaces externas.

Una vez que una función se identifica y se clasifican dentro de un tipo, se evalúa a continuación su complejidad y le asigna un número de puntos de función en función de lo complejo que resulte su desarrollo.

Cada uno de los requisitos funcionales del usuario se ha de asigna a una función de negocio tales como la entrada de datos para una entrada o una consulta de usuario para una consulta.



Esta distinción es importante, ya que tiende a hacer que las funciones medidas en puntos de función mapa fácilmente en los requisitos orientados a los usuarios, sino que también tiende a ocultar las funciones internas (por ejemplo los algoritmos que se están utilizando), que también requieren de recursos para poner en funcionamiento.

Actualmente no existe una norma ISO reconocida como **método de medición que incluye la complejidad algorítmica en el resultado d**e la identificación. Recientemente se han propuesto diferentes enfoques para hacer frente a esta debilidad percibida, implementado en varios productos de software comercial.

Las variaciones del método, denominado, IFPUG diseñado para compensar esta y otras debilidades de la metodología de planificación de puntos de función, e incluyen dos técnicas :

– "Easy Early Fucntion points" o puntos de función **"Pronto y fácil". En** este caso lo que se hace es que se realiza el ajuste para un problema, se mide la complejidad de datos con dos preguntas simples que dan una medida complejidad algo subjetiva, pero que simplifica la medición mediante la eliminación de la necesidad de contar elementos de datos.

– Puntos de la función de Ingeniería: Elementos como los nombres de variables y operadores (por ejemplo, la aritmética, la igualdad / desigualdad, la lógica booleana) se cuentan en el cálculo de los puntos de función.

De esta manera se consigue considerar la complejidad interna de el programa a resolver y no solamente desde el punto de vista del usuario final de la aplicación, sino que también e tenga en cuenta la complejidad de desarrollo de una determinada función.

La técnica de medidas de dispersión es otra variación de la técnica de los puntos de función, en la que se define una métrica función basada en doce entradasprimitivas y sencillas que afectan o que muestran la dispersión de los datos, que se define como «la medida de la verdadera función que debe de realizar el programa tal y como esta es percibida por lo usuarios finales de la aplicación».

Esta técnica es en realidad un método de medidas de medidas de dispersión y puede ser útil para evaluar el valor de una elemento de software en términos de la cantidad de funciones de utilidad que proporciona, aunque hay discrepancias entre distintos investigadores sobre si esto es realmente **útil** o no.

El uso de medidas de dispersión podría puede cuando se está considerando utilizar las técnicas de reingeniería ya sea un proyecto completo, o en varias partes de este.



Puntos de características: la técnica de los puntos de características **añade cambios para mejorar la aplicabilidad a los sistemas con proce**samiento interno significativo como por ejemplo, sistemas empotrados, sistemas operativos, si sistemas de comunicaciones.

Esto permite que la cuenta que se haga con las funciones no sean **fácilmente perceptibles por el usuario, pero esenciales para la operación** adecuada del sistema que se está desarrollando. Hay que tener en cuenta que muchas veces los sistemas con mas carga de proceso no interactúan mucho con el usuario.



Puntos tabulados de Micro Función: es uno de los modelos más recientes, desarrollado en 2009, en el cual ajusta los puntos de función utilizando ponderaciones derivadas de la complejidad del flujo del programa, del lenguaje de programación y sus operando y operadores, tiene en cuenta la utilización de objetos, y la complejidad algorítmica.

El uso de los puntos de función en lugar de las de líneas de código como media de complejidad pretende solucionar varias cuestiones adicionales como las siguientes:

El riesgo de la «inflación» de las líneas creadas de código, y por lo tanto reduciendo el valor del proceso de medición, así los desarrolladores están incentivados a ser más productivos.

Los Defensores de PF se refieren a las líneas como una medida del tamaño de la solución en lugar de una medida de la magnitud del problema

Las Líneas de código favorecen a lenguajes de bajo nivel porque se necesitan más líneas de código para realizar una cantidad similar de funcionalidad a un lenguaje de alto nivel.

Las medidas basadas en líneas de código no son útiles en las fases tempranas del proyecto, donde la estimación del número de líneas de código que se entregarán es un problema que puede ser extremadamente complicado.

Sin embargo, los puntos de función se pueden derivar de los requisitos y por lo tanto son útiles en métodos tales como la estimación por poder.

Sin embargo Albrecht observó en su investigación que los puntos de función están altamente correlacionados con líneas de código, lo que ha dado lugar a cuestionar el valor de una medida de este tipo, si una medida más objetiva, como por ejemplo, contar líneas de código, se puede hacer de forma factible y esta disponible.

Además, ha habido varios intentos de abordar las deficiencias detectadas con la medida al aumentar el régimen de cuenta.

El Método MK II es uno de los métodos de medición del tamaño del software que se engloba el grupo funcional de mediciones mediante PF.Este es un método para el análisis y la medición de aplicaciones de procesamiento de información basado en la vista funcional usuario final del sistema.



El Método MK II se denomina oficialmente Prácticas de Análisis de cuenta de Punto Función según la norma ISO / IEC 20968 utilizando el manual de software de ingeniería-Mk II.

Este método es, en la actualidad,una de los cinco estándares ISO reconocidos actualmente para dimensionamiento del software

El Método MK II fue definido por Charles Symons, el cual esahora responsable del método y su continuo desarrollo. Las necesidades de los usuarios funcionales del software se identifican y cada uno se clasifican en uno de estos tipos: entradas, las salidas y objetos.

Con el fin de determinar el tamaño funcional de sistema se cuentan estos requisitos funcionales.

MkII FPA cuenta siguiendo una serie de normas que a continuación de detallan. El procedimiento de conteo MkII contiene varios pasos descritos a continuación.

En primer lugar hay que determinar el punto de vista, el objetivo y tipo del sistema a dimensionar.

Posteriormente se pasa a definir el límite del sistema consiste en separar lo que será el sistema de los usuarios.

La línea de cobertura del sistema representa la línea lógica que separa a los usuarios de un sistema.

Se utiliza para determinar las transacciones lógicas tales como entradas y salidas que cruza límite durante la interacción entre el usuario y el sistema.El segundo paso es identificar las transacciones lógicas.



Se definen las **transacciones lógicas** como procesos elementales desencadenadas por eventos únicos. Las transacciones pueden ser de tres tipos, entrada,operaciones y salida.

Las transacciones de entrada que introduce información a través del límite del sistema, las operaciones son las que mantienen internamente el sistema para el procesamiento de los datos y las transacciones de salida las que devuelve información de vuelta al usuario.

Cada transacción debe dejar el sistema en un estado coherente. Se definen además dos tipos de transacción lógica que cruzan los límites del sistema, entradas y salidas.

Definición 

Entradas: procesos que introducen información en el sistema. Ejemplos de estas operaciones son la creación de nuevos registros, la actualización de los registros o la eliminación de registros.

Salidas: procesos que obtienen información del sistema y la muestran al usuario. Ejemplos de estas operaciones son los informes, notificaciones o registros.

El tamaño total de aplicación se define como la suma de las transacciones lógicas, teniendo en cuenta que cada transacción se cuenta una vez a pesar de que puede ser ejecutada desde más de un lugar de la aplicación.

Una vez determinado el tamaño de la aplicación se pasa a determinar la complejidad e esta, para ello se realizan las siguientes acciones: Identificar y categorizar los tipos entidades de datos.

Los tipos de entidad de datos además de las entradas y las salidas son estructuras de datos lógicas que contiene información significativa para el usuario.

En el método MarkII **sólo hay un tipo** de tipos de entidad de datos, los objetos. Los objetos deben ser identificados correctamente y de manera unívoca de tal manera que puedan contar.

Una vez identificadas las entradas hay que realizar la cuenta de los tipos de datos de cada entrada, de cada salida, y los tipos de datos referenciados en cada operación.

Posteriormente, se pasa a una fase en la que se identifican las transacciones y los objetos del sistema, y una vez estos se han contabilizado, se puede usar esta cuenta para obtener de el tamaño funcional del sistema.



Tamaño funcional del sistema se representa como recuentos ponderados de las transacciones y los objetos dentro de los límites del sistema en la entrada/salida. El tamaño del sistema se puede expresar de la siguiente manera:

Tamaño = Wi **ΣNi** * + * Nos **ΣNe** + Wo * **ΣNo**

En la ecuación **ΣNi**, **ΣNe** y **ΣNo** son el número total de entradas únicas, salidas y los objetos dentro del sistema.

Actualmente los valores recomendados para los coeficientes de ponderación son Wi = 0.58, We = 1.66, y Wo = 0.26.

Estos coeficientes se han ajustado para adaptarse a la mayoría de los proyectos se software de manera empírica. Se ha utilizado para ello una gran base de datos de valores de esfuerzo de proyectos y se han ajustado los mejores coeficientes.

El modelo de Putnam es un modelo empírico de estimación del esfuerzo necesario para el desarrollo de un sistema software. De esta manera se recoge lo que es la experiencia de la organización en proyectos similares a la hora de planificar y programar un proyecto nuevo.



El artículo original de Lawrence H. Putnam publicado en 1978 se considera como un trabajo pionero en el campo de la modelización de procesos de software.

Como grupo, los modelos empíricos trabajan mediante la recopilación de datos de muchos proyectos de software, por ejemplo, datos de esfuerzo y ta-

maño y tratan de ajustar una curva que nos indique cual es la estimación del esfuerzo una curva a estos datos.

La estimaciones de esfuerzo futuros se hacen tomando como base un tamaño estimado y se calcula el esfuerzo asociado con la ecuación a la que se ajustan a los datos obtenidos en otros proyectos similares, por lo general con algún margen de error.

El modelo Putnam describe el tiempo y el esfuerzo necesarios para terminar un proyecto de software de tamaño especificado. Es uno de los primeros de este tipo de modelos desarrollados, y está entre los más ampliamente utilizados.



Sabías que

SLIM (Software Lifecycle Management) es el nombre dado por Putnam a un conjunto de de herramientas de propiedad de su compañía QSM, Inc. ha desarrollado sobre la base de su modelo.

Mientras que desarrollaba proyectos de I + D para el ejército y más tarde en la compañía GE, Putnam notó que el perfil de esfuerzo del personal de un proyecto de software siguen la conocida distribución de Rayleigh. Esta distribución nos indica como se va a repartir probabilísticamente el esfuerzo en cada unoa de las fases del proyecto

Putnam utilizó sus observaciones acerca de los niveles de productividad para derivar la ecuación de software:

$$\frac{B^{1/3} - \text{Tamaño}}{\text{Productividad}} = \text{Esfuerzo}^{1/3} * \text{Tiempo}^{4/3}$$

donde:

– El tamaño es el tamaño del producto (sea cual sea el tamaño estimado es utilizado por su organización es apropiado).Putnam utiliza ESLOC (Líneas de código fuente efectivas) a lo largo de sus libros.

– B es un factor de escala y es una función del tamaño del proyecto.

– La productividad es la productividad de los procesos, la capacidad de una organización de software en particular para producir software de un tamaño dado a una velocidad defecto particular.

– El esfuerzo es el esfuerzo total aplicado al proyecto en años-persona.

– El tiempo es la duración total del proyecto en años.

Utilizando la ecuación del software, en el modelo de Putnam, a partir de del tamaño se puede determinar el esfuerzo necesario y el número total de personas.

También se puede utilizar a la inversa y partir de los datos de una organización, determinar su capacidad para poder tener un proyecto en un tiempo determinado. De esta forma también puede ser utilizado por los contratistas para determinar si una subcontrata realmente tendrá la capacidad necesaria como para acabar un proyecto a tiempo.

En la práctica, cuando se hace una estimación para una tarea de software de la ecuación se resuelve para el esfuerzo.

El tamaño del software se estima tras la finalización del proyecto y se utiliza la productividad para refinar los procesos de medida de la organización. De esta manera la organización va refinando sus mediciones para futuros proyectos.

Trazando de esfuerzo como una función del tiempo se obtiene la curva tiempo-esfuerzo. Los puntos de la curva representan el esfuerzo total estimado para completar el proyecto en algún momento. De esta manera el método también nos devuelve los valores estimados de esfuerzo para cada uno de los instantes del proyecto, lo que permite a los gestores determinar que recursos son necesarios destina a este durante cada momento.

Uno de los rasgos distintivos del modelo de Putnam es que el esfuerzo total disminuye a medida que el tiempo para completar el proyecto se extiende. Es decir, que cuanto más dure un proyecto las personas que se ven involucradas en el disminuyen su rendimiento.

Esto es normalmente representado en otros modelos paramétricos con un parámetro llamado de relajación que nos indica que cuanto más se extienda el proyecto menor será la productividad de este. Esto es debido a factores psicológicos como organizaciones.

Este método de estimación es bastante sensible a la incertidumbre, tanto en tamaño y productividad del proceso. Putnam defiende la obtención de productividad de los procesos de calibración.

Putnam hace una clara distinción entre «productividad convencional», definida de la manera clásica como capacidad dividida entre esfuerzo y productividad del proceso. La diferencia está en que una es la productividad teórica y la segunda es la productividad en una de las fases del proyecto.

Una de las principales ventajas de este modelo es la simplicidad con la que está calibrado. La mayoría de las organizaciones de software, independientemente del nivel de madurez pueden recoger fácilmente el tamaño, esfuerzo y duración (tiempo) de proyectos anteriores.

El índice de productividad de los procesos, siendo exponencial en la ecuación, en la realidad se suele convertir en un índice de productividad lineal una organización puede utilizar para realizar un seguimiento de sus propios cambios en la productividad y aplicar en futuras estimaciones de esfuerzo.

De esta forma las organizaciones van obtenido una realimentación de sus propios proceso internos, pudiéndose utilizar en la mejora continua.

El método COSMIC es un método estandarizado internacionalmente dimensionamiento software ampliamente utilizado. Más específicamente, es un método de medición del «tamaño funcional» de software. ‹



COSMIC significa Measurement Software Common International Consortium, una agrupación de expertos en medición de software de todo el mundo que, en 1998, vio la necesidad de mejorar los métodos tradicionales basados en el punto de función del software.

Los desarrolladores de métodos describen como un método de segunda generación, ya que, en contraste con los métodos tradicionales de punto de función, el método COSMIC tiene unas característica **más avanzadas** entre las que destacan las siguientes:

Esel primer método de medición de tamaño funcional desarrollado para ajustarse a la norma ISO / IEC 14143/1: 2003 sobre este tema.

El método ha sido diseñado en base a los principios fundamentales de la ingeniería de software y la teoría de la medición, utiliza conceptos de esta ramas de

tal manera que es fácilmente aplicable a aplicaciones de negocios, software en tiempo real y infraestructuras de software.

Es un método completamente abierto y yoda la documentación sobre el **método** (manuales me medida, directrices, casos de estudios, etc) puede ser descargadode forma gratuita, desde la web.



Sabías que

El manual de medición está disponible en árabe, chino, holandés, Inglés, francés, japonés y español. Alemán, italiano, polaco y traducciones turcas está siendo preparadas lo que lo convierte en método que puede ser extensamente utilizado alrededor del mundo independientemente el origen de la persona y del proyecto.

El método COSMIC se puede considerar como el más objetivo de todos los métodos de dimensionamiento de software:

Para proporcionar una medida del tamaño de un desarrollo de software o de trabajo de salida del proyecto de mejora que se puede utilizar para derivar medidas de rendimiento tales como «productividad» (= tamaño / esfuerzo).

Como la medida de tamaño depende únicamente de la necesaria funcionalidad y es independiente de cualquier tecnología utilizada, las medidas se pueden utilizar para comparar el desempeño entre proyectos utilizando diferentes tecnologías.

Puede ser utilizada para su uso como la entrada primaria a métodos para la estimación de esfuerzo del proyecto. La capacidad de medir el tamaño del software parte de sus requisitos y es especialmente valiosa y oportuna en la vida de un proyecto de software.

Teniendo en cuenta estos usos de gran importancia económica, es vital contar con un método de medición de tamaño del software que está bien fundado, sea fiable y fácil de usar. Por ello el método COSMIC es ahora de uso generalizado en todo el mundo.



El método COSMIC se basa en el principio de que los requisitos de los usuarios finales de cualquier desarrollo de software consisten en procesos funcionales.

Cada proceso funcional se activa cuando un usuario del software, sea este una persona, un dispositivo de hardware o de otra pieza de software reconoce un evento y envía un mensaje para iniciar el proceso.

El proceso se completa cuando el software ha hecho todo lo que se requiere para responder al evento, es decir, cuando se ha procesado correctamente los datos y se ha generado al salida adecuada.

Ejemplo

Ejemplos de eventos de activación y los procesos funcionales correspondientes de software podrían ser los siguientes

En una aplicación de negocio ejemplos de procesos funcionales serian los siguientes:

– Si se recibe un pedido, el proceso Funcionar sería introducir una orden de pedido para que sea resuelta por el departamento correspondiente.

– Por ejemplo un cambio de codicilo en un empleado: el proceso funcional sería Actualizar un registro de personal.

– Llegar a fin de mes el proceso sería producir la lista con las nómicas.

En una aplicación en tiempo real, por ejemplo el piloto automático de un avión, también existen proceso funcionales como los que se describen a continuación:

– Evento tic de reloj : Iniciar el ciclo para leer los instrumentos y ajustar el control de un proceso industrial.

– Comando del piloto : Levante las ruedas en el despegue.

– Recepción de un mensaje de marcación: establecer llamada telefónica.

Los procesos funcionales de software se analizan en **más** detalle teniendo en cuenta en los movimientos de datos que estos procesos producen. Para ello hay que hacer un recuento de las entradas y salidas de datos desde y hacia

el sistema. De esta amera la entrada de datos de método será la información que recibe y devuelve el sistema



El recuento de los movimientos de datos entradas y salidas del software hacia el exterior, y hacia y desde dispositivos de almacenamiento persistente se toma como medida del tamaño funcional del software, en unidades denominadas de Puntos función Cosmic (PPC).

Desde un punto de medición del tamaño puro a "ojo" o utilizando estimaciones más o menos satisfactorias, las mejoras más importantes del método COSMIC en comparación con el uso de Puntos de Función tradicionales.

El método COSMIC fue diseñado para medir los requerimientos funcionales del software en los ámbitos de aplicación de negocio, en tiempo real y software de infraestructura (por ejemplo, sistemas operativos, componentes Web, etc), en cualquiera de las capas de una arquitectura multicapa y en cualquier nivel de descomposición.

Los puntos de Función tradicionales fueron diseñadas para medir sólo la funcionalidad "vista" por los usuarios humanos de software empresarial en la capa de aplicación. Además los puntos de Función tradicionales utilizan una escala de tamaño con una gama limitada de tamaños posibles para cada componente.

Sin embargo, los procesos funcionales COSMIC se miden en una escala de tamaño continuo con un mínimo de 2 PPC y sin límite de tamaño superior.



El software moderno puede tener procesos extremadamente grandes., por ejemplo procesos funcionales individuales de aproximadamente 100 PPC se han medido en software de aviónica y sistemas en los sistemas nacionales de seguros públicos.

Por lo tanto, los puntos de función tradicionales pueden dar tamaños equivocados para ciertos tipos de software que significa que al usar estos tamaños

para la medición del rendimiento o estimación,. Los resultados han de ser tomados con bastante cuidado.

El método COSMIC da una medida mucho más fina del tamaño de los cambios que deben introducirse en el software en comparación con los puntos de función tradicionales. El cambio más pequeño que se puede medir con el método COSMIC es 1 PPC.

Hay múltiples ventajas en el hecho de utilizar el método COSMIC, a continuación se enumeran algunas de las principales de estas:

– Los usuarios del método COSMIC han comunicado los siguientes beneficios, en comparación con el uso de métodos de 'primera generación'.

– El método es fácil de aprender y estable debido al enfoque basado en principios, por lo tanto, " a prueba del futuro" y rentable de implementar.

– Suele ser bien aceptado por el personal del proyecto debido a la facilidad de la cartografía de los conceptos del método a los métodos de documentación de requisitos de software modernas, y su compatibilidad con modernas arquitecturas de software.

– Mejora la estimación de la precisión, especialmente para los proyectos de software más grandes.

– Tiene posibles aplicaciones para medir el tamaño de los requisitos que son manejados de forma automática que se realizan utilizando herramientas CASE.

– Su uso muestra una mejora del rendimiento real, donde el uso de puntos de función tradicionales no ha indicado ninguna mejora debido a su incapacidad para reconocer cómo los procesos de software han aumentado de tamaño con el tiempo.

– El dimensionamiento con COSMIC es una excelente manera de controlar la calidad de los requisitos en todas las etapas del desarrollo, a medida que estos requisitos evolucionan.

2.2. Planificaciones. Modelos de diagramado. Diagramas de Gantt

Cuando se comienza un proyecto, la primeras fases tratan de delimitar el trabajo a realizar, para ello el alcance del proyecto se define y los métodos apropiados para la realización del proyecto se determinan.

Después de esta etapa, las duraciones de las distintas tareas necesarias para completar el trabajo se enumeran y se agrupan en una estructura de desglose del trabajo.

La planificación de proyectos a menudo se utiliza para organizar las diferentes áreas de un proyecto, incluyendo los planes de proyecto, cargas de trabajo y la gestión de los equipos y las personas.

Las dependencias lógicas entre tareas se definen mediante un diagrama de red de actividad que permite la identificación de la ruta crítica.



La planificación del proyecto es una técnica inherentemente incierta, ya que debe hacerse antes de que realmente se ponga en marcha el proyecto.

Por lo tanto la duración de las tareas se estima a menudo mediante una media ponderada de los casos optimistas, normales, y pesimistas. El método de camino crítico agrega lo que se denominan "buffers" o tiempos de holgura en la planificación para anticipar y gestionar los posibles retrasos en la ejecución del proyecto.

El tiempo de holgura en el calendario de ejecución se pueden calcular utilizando el software de gestión de proyectos. Entonces los recursos necesarios pueden ser estimados y los costos para cada actividad se puede asignar a cada recurso, dando el costo total del proyecto.

En esta etapa, el cronograma del proyecto puede ser optimizado para lograr el equilibrio adecuado entre el uso de recursos y la duración del proyecto para cumplir con los objetivos del proyecto.

Una vez establecido y acordado, el cronograma del proyecto se convierte en lo que se conoce como la línea base del cronograma. El progreso se medirá contra el programa de línea de base en toda la vida del proyecto. Analizar el progreso en comparación con la línea base del cronograma se conoce como gestión del valor ganado.

Las entradas de la fase de planificación del proyecto incluyen la carta de aceptación del proyecto y la propuesta de concepto.

Las salidas de la fase de planificación del proyecto incluyen los requisitos del proyecto, el cronograma del proyecto y el plan de gestión del proyecto. La

planificación del proyecto se puede hacer manualmente. Sin embargo, en la gestión de varios proyectos, por lo general es más fácil y rápido de usar software de gestión de proyectos.



Una vez determinado la duración y los recursos necesarios para realizar un proyecto, es necesario planificar este, es decir, determinar que tareas se irán realizando cada vez y con qué recursos se completarán estas.

La planificación ha de contemplar dos aspectos, planificación temporal (que se hace y cuando se hace) y organizativa (quien lo hace).

A continuación vamos a centrarnos en cada uno de estos aspectos, la planificación temporal: la primera acción que hay que llevar a cabo a la hora de afrontar una planificación temporal, es organizar el proyecto y dividirlo en subtareas. Para ello se puede utilizar como herramienta la WBS.



Un WBS o "Work Breakdown Structure", es una representación gráfica de las diferentes actividades que se han de llevar a cabo para la realización de un proyecto.

Dichas actividades estarán agrupadas por paquetes de trabajo. El propósito de un WBS es dividir el proyecto en porciones que posteriormente sean fáciles de planificar en tiempo, coste, recursos, etc.



En el nivel más alto de la estructura WBS o primer nivel, se tendrá el principal objetivo del proyecto o fase que estemos gestionando.

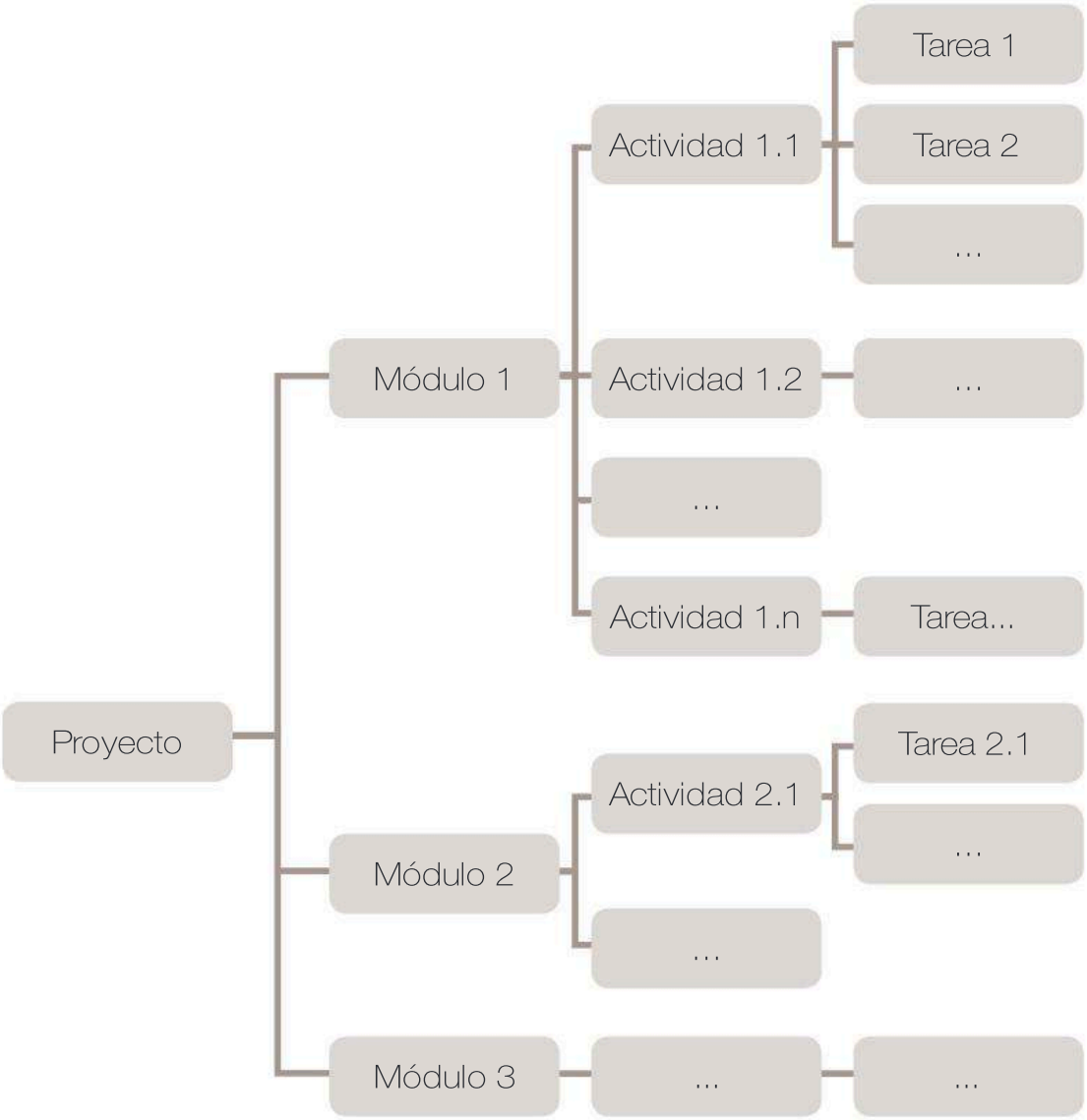
En el segundo nivel se tienen las metas más significativas a alcanzar y en el tercer nivel una división de estas metas en hitos más pequeños.

Así la estructura se divide en niveles y sub-niveles hasta alcanzar las llamadas unidades de trabajo.

Todas las tareas tienen asignado un código numérico gracias al cual es posible identificar a que paquete de trabajo pertenece cada tarea y en qué nivel se encuentra situada dentro de la estructura.

Las unidades de trabajo son actividades pequeñas, de una duración en torno a una semana y que son asignables a una persona.

Una WBS genérica puede ser la siguiente:





La **actividad de planificación** o también llamada de programación de un proyecto, consiste en establecer la estructura temporal de las fases, actividades y tareas del proyecto, en función de los recursos de que se disponga.

Las planificaciones son llevadas a cabo en diferentes momentos del ciclo de vida del proyecto, apoyándose éstas a su vez en diversas técnicas de estimación de la duración de las actividades y tareas, así como de la carga de recursos en las diferentes fases.

La planificación del proyecto se apoya en la fase previa de organización en la que se habrán construido los diagramas WBS.

De forma genérica, la programación del proyecto implica las siguientes actividades:

– Establecer la red del proyecto: consiste en modelar el proyecto a partir de la WBS construida en la fase de organización del proyecto, utilizando una red PERT e Identificando para cada actividad las fechas más tempranas y más tardías de inicio y fin, así como las holguras asociadas.

– Definir la ubicación en el tiempo de las actividades y recursos asociados: a partir del PERT, se realizará el diagrama GANTT del proyecto, así como el histograma de recursos.

Una vez llegados a este punto se verán las dos técnicas de planificación, las técnicas PERT y GANTT, que sirven de ayuda a la hora de planificar.



La **planificación** trata básicamente de programar en el tiempo, todos los elementos identificados como tareas que se deben llevar a cabo durante el proyecto.

El motivo de programar las actividades se debe al hecho de que entre las actividades identificadas en la organización del proyecto, habrá una serie de dependencias que es preciso identificar, ya que si todas las actividades discurren libremente sin fechas ni costes ni recursos asignados, es seguro que el proyecto no tendrá éxito. Estas dependencias se pueden representar gracias a los diagramas que presentaremos a continuación.

El establecer el calendario de trabajo no es una actividad en absoluto trivial y muchas veces la tarea de generar el calendario del proyecto, tarea propia del jefe del proyecto, es crucial para el propio proyecto.



Definición

En la gestión de proyectos, una **estructura de desglose del producto** (PBS) es una herramienta para analizar, documentar y comunicar los resultados de un proyecto, y forma parte de la planificación de productos a base técnica.

El PBS ofrece una, jerárquica exhaustiva estructura de árbol de los entregables (físicos, funcionales o conceptuales) que componen el proyecto, dispuesto en forma de relación en todo-parte.

Esta representación esquemática de los resultados del proyecto proporciona una declaración clara e inequívoca de lo que el proyecto debe entregar al final.

El PBS es idéntica en formato a la estructura de desglose del trabajo (WBS), pero es una entidad independiente y se utiliza en cada etapa del proceso de planificación.

El PBS precede a la WBS y se centra en la catalogación de todos los productos deseados o productos, necesarios para lograr el objetivo del proyecto. Esto conlleva la creación de la WBS, que identifica las tareas y actividades necesarias para lograr esos resultados.

Los partidarios de la planificación con base de productos sugieren que esto supera las dificultades que surgen de supuestos acerca de qué hacer y cómo hacerlo, centrándose en cambio en las metas y objetivos del proyecto.

Una analogía muy citado es que PBS define dónde quiere ir, la WBS se indica cómo llegar hasta ese objetivo.

Ejemplo

Ejemplo de PBS de un ordenador podría ser la que se detalla a continuación:

- Unidad principal:
  - Chasis.
  - Motherboard.
  - CPU.
  - Chips de RAM.
  - ...
  - FDD.
  - HDD.
  - Tarjeta de vídeo.
  - Tarjeta de sonido.
  - Tarjeta de red.
  - Tarjeta de puerto LPT.
  - Monitor:
    - CRT.
    - Chasis.
    - Componentes electrónicos.
  - Ratón.
- Cable.
- Teclado.
- ...



Estas técnicas tienen una serie de ventajas como son el mostrar de una forma explícita y clara la dependencia entre actividades.

Posteriormente esta dependencia de modela utilizando métodos de análisis como el PERT (Program Evaluation and Review Technique), Técnica de Revisión y Evaluación de la Programación, que se verán a continuación.



Los diagramas PERT forman parte de una familia de diagramas de tipo red, que fueron desarrollados en los Estados Unidos durante los años 50, para su aplicación en proyectos de defensa a gran escala.

Hay varias posibilidades para representar estos gráficos, aunque básicamente constan de unas líneas unidas por nodos, donde los nodos representan, en general, acontecimientos de la programación del proyecto.

La t**écnica de evaluación y revisión de programa** (o proyecto), abreviada normalmente como PERT, es una herramienta estadística, que se utiliza en la gestión del proyecto, el cual fue diseñado para analizar y representar las tareas involucradas en la realización de un determinado proyecto.

Desarrollado por primera vez por la Marina de los Estados Unidos en la década de 1950, que se utiliza comúnmente en conjunción con el método del camino crítico (CPM).

Se aplica a muchos proyectos de investigación y desarrollo a gran escala, proyectos de un solo desarrollo, que requieren compleja infraestructura.

Este modelo de proyecto fue el primero de su tipo, un tras el nacimiento de la denominada gestión científica, fundada por Frederick Taylor (taylorismo) y más tarde perfeccionado por Henry Ford (fordismo). **El método del camino crítico se inventó más o menos al mismo tiempo que PERT.**

Un ejemplo de esto fue su aplicación para los Juegos Olímpicos de Invierno de 1968 en Grenoble en el que se aplicó PERT desde 1965 hasta la inauguración de los Juegos 1968.



Para poder llevar a cabo los diagramas PERT es necesario conocer las dependencias entre las distintas actividades.

Esto de obtiene de la WBS y de la PBS.

Como desventaja del método PERT cabría destacar lo difícil y poco flexible que resulta el hacer ajustes y modificaciones, y además no es una buena herramienta para hacer seguimiento del proyecto.

A continuación de describen algunos términos usados en la construcción de diagramas PERT y su forma de representación:



Actividad: también denominada elemento de trabajo. Una actividad tiene una duración relacionada con el alcance de ésta. Se representan por cajas que contienen el nombre de la actividad y alguna información adicional.

Subactividad PERT: una actividad PERT se puede descomponer aún más en un conjunto de sub-actividades.

Ejemplo

Por ejemplo, la actividad A1 se puede descomponer en A1.1, A1.2 y A1.3 por ejemplo. Sub-actividades tienen todas las propiedades de las actividades, en particular una subactividad tiene eventos predecesoras o sucesoras como una actividad. Un sub-actividad se puede descomponer de nuevo en sub-actividades de grano más fino.

Ejemplos de actividades:

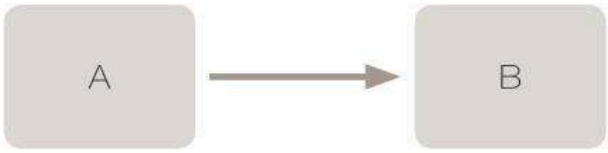


Definición

Enlaces entre actividades: los enlaces representan la dependencia entre actividades, se representan por flechas que conectan las actividades.

Definición

Actividad paralela: se trata de una actividad que debe ejecutarse en paralelo con otra u otras actividades de la red. La imagen muestra que la actividad "A"y la actividad "B" se pueden llevar a cabo en paralelo.



Definición

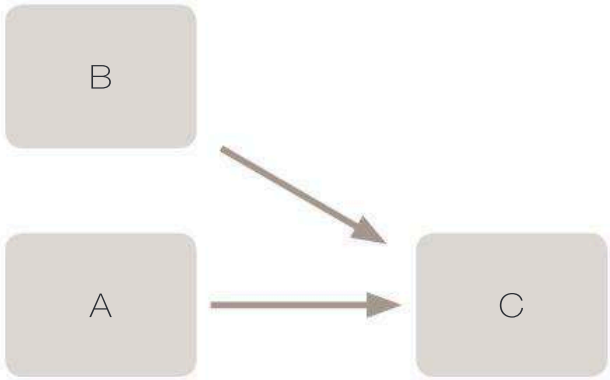
Evento o hito: un evento o hito es un punto en el proyecto que tiene especial relevancia.

Todos los proyectos tienen al menos dos hitos: el de comienzo y el de fin. Otros hitos típicos son la aprobación de presupuestos, la producción de un resultado importante, etc. Los hitos se representan mediante triángulos equiláteros o rombos.

Definición

Suceso externo: la ocurrencia de un suceso externo tiene el mismo sentido que la finalización de una actividad de la red, y señala la disponibilidad de un recurso que viene de fuera del proyecto que se está modelizando.

Los sucesos se representan asimismo mediante "cajas" conteniendo su nombre.Los enlaces entre sucesos y actividades se representan mediante una flecha discontinua oblicua como se puede ver en la imagen, entre el suceso A y la actividad C.



El modelado de una red PERT consta básicamente de los siguientes pasos:

– Se identifican los enlaces de las actividades representadas en el WBS.

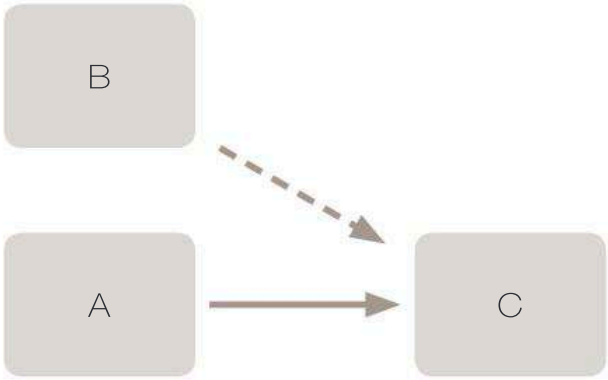
– Se asignan duraciones estimadas a las actividades.

– Se determinan las fechas de comienzo y finalización, así como las holguras o márgenes para cada actividad.

En el caso de los proyectos software es aconsejable realizar estos tres pasos de forma iterativa, realizando redes PERT por niveles de la jerarquía representada en el WBS. Es decir, establecer un diagrama PERT general al comienzo del proyecto, a nivel de fases de desarrollo.

Posteriormente ir generando submodelos detallados a medida que se van realizando las fases del proyecto e incluirlos en el modelo general para poder recalcular el conjunto de datos del proyecto. Así se puede ir generando el proceso global de modelado del proyecto.

Un ejemplo de diagrama PERT es el siguiente:



Definición

El **método del camino crítico** (CPM) es un algoritmo para programar un conjunto de actividades del proyecto.

El método del camino crítico (CPM) es una técnica de modelado proyecto desarrollado a finales de 1950 por Morgan R. Walker de DuPont y James E. Kelley, Jr. de Remington Rand.

Kelley atribuyó el término "ruta crítica" para los desarrolladores de la técnica de evaluación y revisión de programa que se desarrolló casi al mismo tiempo por Booz Allen Hamilton y la Marina de Estados Unidos.

CPM es de uso general con todas las formas de proyectos, incluyendo la construcción, la industria aeroespacial y de defensa, desarrollo de software,

proyectos de investigación, desarrollo de productos, ingeniería y mantenimiento de la planta, entre otros.

Cualquier proyecto con actividades interdependientes puede aplicar este método de análisis matemático. Aunque ya no se utiliza el programa enfoque CPM original l término se aplica generalmente a cualquier método utilizado para analizar un diagrama lógico de la red del proyecto.

La técnica fundamental para el uso de la CPM es construir un modelo de proyecto que incluya lo siguiente:

Una lista de todas las actividades necesarias para completar el proyecto (típicamente categorizados dentro de una estructura de desglose de trabajo.

El tiempo (duración) que cada actividad se llevará para ser completada

Las dependencias entre las actividades, es decir, las actividades que ha de hacerse antes que se inicie una actividad determinada.,

Puntos finales lógicos como hitos o elementos entregables.

Con estos valores, CPM calcula la ruta más larga de las actividades previstas para los puntos finales lógicos o al final del proyecto, y la primera y la última que cada actividad puede empezar y terminar sin hacer el proyecto más largo.



Este proceso determina qué actividades son “críticas” (es decir, pasan por el camino más largo) y que “holgura total” tienen (es decir, hasta que punto se pueden retrasar sin hacer el proyecto más largo).



En la gestión de proyectos, una **ruta crítica** es la secuencia de actividades de la red de proyectos que se suman a la duración global más larga.

Esto determina el menor tiempo posible para completar el proyecto. Cualquier retraso de una actividad en la ruta crítica afecta directamente la fecha de finalización del proyecto prevista.



Un proyecto puede tener varias rutas críticas, que sean paralelas.

Un camino paralelo adicional a través de la red con las duraciones más cortas que el total de la ruta crítica se llama un camino sub-crítica o no crítica.

Las herramientas de análisis de CPM permiten al usuario seleccionar un punto final lógico en un proyecto y rápidamente identificar su serie más larga de actividades dependientes (su trayectoria más larga).

Estas herramientas pueden mostrar la ruta crítica (y cerca de las actividades de la ruta crítica, si se desea) como una cascada de agua que fluye desde el inicio del proyecto (o fecha de estado actual) hasta el punto final lógico seleccionado.

Aunque el diagrama de actividad-en-flecha (“diagrama PERT”) todavía se usa en muchas organizaciones, por lo general, ha sido reemplazado por el diagrama de actividad-en-nodo, donde se muestra cada actividad, como una caja o un nodo y las flechas representan las relaciones lógicas que van de predecesor al sucesor, como se muestra aquí en el “diagrama de actividad-en-nodo”.

El diagrama de actividades en nodo es un diagrama que muestra programación de ruta crítica, junto con holgura total y cálculos críticos camino de arrastre. De esta manera se serpentea con mayor efectividad la información que en lo diagramas de actividades en arco.

En el diagrama de la figura 5 las, Actividades A, B, C, D, y E comprenden la ruta crítica o más largo, mientras que las actividades F, G y H están fuera de la ruta crítica con Margen Libre de 15 días, 5 días, y 20 días respectivamente.

Considerando que las actividades que están fuera de la ruta crítica tienen flotador y por lo tanto no están retrasando la finalización del proyecto, los de la ruta crítica por lo general tienen arrastre ruta crítica, es decir, retrasar la finalización del proyecto.



Si una actividad de la ruta crítica no tiene nada en paralelo, su arrastre es igual a su duración. Así, A y E tienen lastres de 10 días y 20 días, respectivamente.

Si una actividad de la ruta crítica tiene otra actividad en paralelo, su arrastre es igual a lo que sea menor: su duración o la holgura total de la actividad paralela a la flotación menos total.

Así, ya que B y C son ambas paralelo a F (Margen Libre de 15) y H (Margen Libre de 20), B tiene una duración de 20 y arrastre de 15 (igual a flotador de F), mientras que C tiene una duración de sólo 5 días y por lo tanto arrastre de sólo 5. Actividad D, con una duración de 10 días, es paralela a G (Margen Libre de 5) y H (Margen Libre de 20) y por lo tanto su arrastre es igual a 5, el flotador de G.

Los resultados del cálculo CPM sirven para la gestión del proyecto y de los posibles retrasos. Los **cálculos de arrastre** permiten a los administradores priorizar las actividades para la gestión eficaz de la finalización del proyecto, y para acortar la ruta crítica prevista de un proyecto mediante la poda de actividades de la ruta crítica, por «vía rápida» (es decir, la realización de más actividades en paralelo), y o por lo que se denomina «crash por el camino crítico» (es decir, acortar la duración de las actividades de la ruta crítica mediante la adición de recursos).



«Duración del Crash» es un término que se refiere a la mayor anticipación posible la para que se puede programar una actividad.

Esto se logra desplazando más recursos hacia la realización de esa actividad, lo que resulta en menor tiempo gastado y con frecuencia una reducción de la calidad del trabajo, se usa cuando se prima sobre todo la velocidad.

La duración del crash es típicamente modelada como una relación lineal entre el coste y la duración de la actividad, sin embargo en muchos casos una función convexa o una función de umbral suele ser **más aplicable.** Originalmente, el método del camino crítico considera dependencias **lógicas** entre los elementos terminales.

Desde entonces, se ha ampliado para permitir la inclusión de los recursos relacionados con cada actividad, a través de procesos llamados asignaciones de recursos basados en la actividad y la redistribución de recursos.



Un planificación nivelada de recursos puede evitar retrasos debidos a los cuellos de botella de recursos (es decir, falta de disponibilidad de un recurso en el momento necesario), y puede causar un camino más corto previamente se convierta en el camino más largo debido a la ausencia de un «recurso crítico».

Un concepto relacionado es la llamada cadena crítica, que intenta proteger la actividad y proyectos duraciones de retrasos imprevistos debido a las limitaciones de recursos.

Desde programas de proyectos cambian de forma regular, CPM permite la monitorización continua de la programación, lo que permite que el director del proyecto para realizar un seguimiento de las actividades críticas, y alerta al administrador de proyectos a la posibilidad de que las actividades no críticas pueden retrasarse más allá de su holgura total, por tanto, la creación de una nueva ruta crítica y retrasar la finalización del proyecto.

Además, el método puede incorporar fácilmente los conceptos de predicciones estocásticas, utilizando la técnica de la evaluación del programa y revisión (PERT) y la metodología de la cadena de eventos.

Actualmente, hay varias soluciones de software disponibles en la industria que utilizan el método CPM de programación. Casi todas las herramientas CASE de gestión de proyectos y de programación y estimación permite que se utilice sobre ellas las técnicas del método CMP sobre el proyecto que se esté planificando.

El diagrama Gantt o de Barras es una de las técnicas de planificación más utilizadas. Se suele usar en conjunción con la técnica PERT o CPM con la intención de poder ir visualizando gráficamente el estado y el avance de un proyecto.

En este diagrama cada una de las tareas se representa como una barra y el progreso teóricos de esta se indica por la longitud de la barra. Además e añaden en el eje un calendario que indique cuando debe realizarse.

Al contrario que las anteriores también es una técnica que nos permite ir haciendo el seguimiento d e un proyecto a la misma vez que se visualiza su cronograma de actividades. Este tipo de diagramas es más fácil de entender que las redes de precedencia como por ejemplo la red PERT.

El principal inconveniente de este tipo de representaciones estriba en la incapacidad para representar interrelaciones entre tareas, es decir, no queda reflejado el hecho de que un producto que fue originado por una tarea sea utilizado como entrada en otra u otras.

Por otro lado sí se puede reflejar muy bien el paralelismo y las precedencias en el tiempo entre tareas. De esta forma se puede usar como una herramienta muy potente a la hora de plantear como se han de repartir los recursos disponibles entre als diversas tares que constituyen un proyecto.

A continuación de describen algunos términos usados en la construcción de los diagramas Gantt.



Definición

Fecha de comienzo más temprana: es la fecha en la que todas las actividades que preceden a una actividad estarán finalizadas teniendo en cuenta los desfases.

Es igual a la mayor de las fechas de finalización más temprana de todas las actividades que preceden a una dada.



Definición

Fecha de finalización más temprana: es la fecha de comienzo más temprana de la actividad aumentada en su duración.



Fecha de finalización más tardía: es la fecha a la que como mucho se puede retrasar la actividad sin afectar a la fecha más tardía de finalización del proyecto.

Es igual a la menor de las fechas de comienzo más tardías de las actividades siguientes.



Fecha de comienzo más tardía: es la fecha de finalización más tardía disminuida en la duración de la tarea.



Holgura (Margen Total): representa lo más que se puede retrasar una actividad sin afectar a la fecha de fin de proyecto.

La holgura es la diferencia entre la fecha de finalización más tardía y la fecha de finalización más temprana de la actividad.



Margen Libre: representa el tiempo máximo que se puede retrasar una actividad sin afectar a la fecha de comienzo de las actividades siguientes.

Es la diferencia entre la fecha de finalización más temprana de la actividad y la menor de las fechas de comienzo más temprana de las actividades que le siguen.

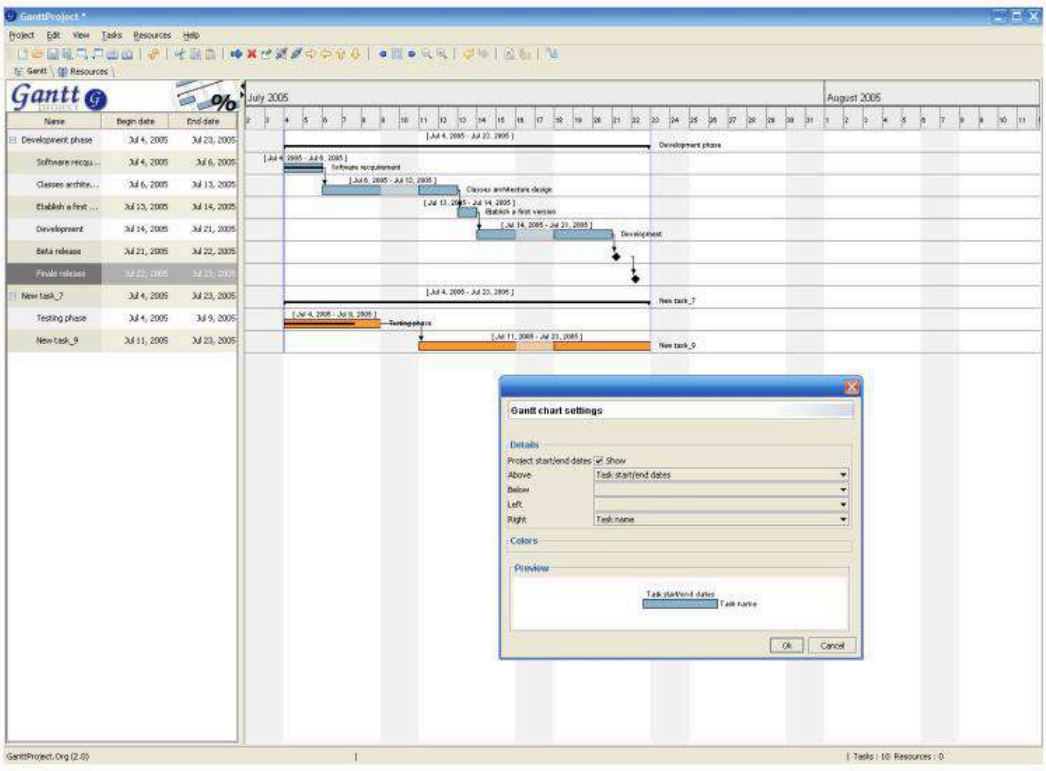
Definición

Actividades críticas: son las actividades cuya holgura vale 0.

Hay que tener especial cuidado en la planificación y seguimiento de estas actividades pues cualquier retraso en su consecución determinarán un retraso de todo el proyecto. El formato básico de un diagrama de Gantt consta de una lista de actividades a las que se asigna fecha de comienzo y finalización.

El nombre de la actividad suele aparecer a la izquierda del diagrama y la escala temporal en la parte superior del diagrama. Cada tarea tiene una línea horizontal que cubre el período de tiempo que dura esa actividad. La escala temporal suele venir dada en días, semanas o meses.

Un ejemplo de diagrama de Gantt, correspondiente al diagrama PERT anterior, es el siguiente:



Ejemplo de diagrama de GANTT



Definición

El **plan de proyecto** de software es un documento que se produce al comienzo del proyecto, a la vez que otros planes.

Recoge el presupuesto inicial contenido en una posible oferta, además de la programación de las actividades de desarrollo. El plan de proyecto de software se considera un documento dinámico.

A lo largo del proyecto se irán contrastando los resultados reales del proyecto (avance del proyecto, esfuerzo dedicado, presupuesto gastado, etc.) con los datos estimados en el Plan del Proyecto.

Si como resultado de esta comparación se considera por los responsables del proyecto actualizar el plan del proyecto en algún aspecto (actividades, recursos, productos, etc.,), éste se modifica para registrar el nuevo compromiso.

El propósito del plan de proyecto es el establecimiento preciso, desde el comienzo de las actividades del proyecto, de las tareas a realizar y sus características (duración, elementos a entregar, restricciones, etc.), así como la organización para asegurar el desarrollo del software.

Un posible formato del plan de proyecto puede ser en el que se muestra a continuación.

– Introducción: nos servirá par contextualizar el documento en la situación en la que se va desarrollar y debería incluir al menos la siguiente información:

  · Objetivo: este apartado contendrá una breve presentación del proyecto, describiendo los aspectos generales que limitan el contexto y el ámbito del mismo. Asimismo, este párrafo describe genéricamente la funcionalidad del software, en sus diferentes entregas, los subsistemas de que consta, el entorno de operación, así como su forma de realización

  · Alcance: este apartado describirá a quién está dirigido el documento, así como los aspectos cubiertos y no cubiertos por el mismo. Introducirá brevemente la estructura del plan de proyecto del software, resumiendo el contenido de cada párrafo. También se deberán indicar las condiciones de actualización del documento. Se indicará también el

procedimiento a seguir en caso de que ocurran cambios, indicando en quién recaerán las responsabilidades concretas de modificación y validación del documento.

- Estimación de esfuerzo y duración: en este párrafo del plan de desarrollo del software se incluirá el presupuesto del desarrollo que tendrá en cuenta todas las hipótesis y restricciones que tienen influencia sobre el proceso de estimación.

- Hipótesis y restricciones: hipótesis técnicas y organizativas que tienen influencia en las estimaciones. Es necesario que se ofrezca una lista sintetizada de las consideraciones realizadas. También se deben indicar las hipótesis de cálculo, dependiendo de los métodos de estimación utilizados.

  Las restricciones técnicas pueden afectar a diversos factores del proyecto. Entre estos factores se pueden citar los siguientes:

- Periodo de vida del sistema a desarrollar:

  › Condiciones de uso e impacto en la fiabilidad.

  › Tecnología impuesta por el cliente.

  › Fiabilidad.

  › Lenguaje de programación.

- Restricciones organizativas, que pueden afectar también a varios aspectos del proyecto. Esas restricciones vienen provocadas por la propia organización en las que se incluye en proyecto y las principales que hay que contemplar son las siguientes:

  › Medios y recursos humanos.

  › Coste.

  › Duración.

  › Entorno.

  › Calidad.

  › Cualificación del personal.

- Estimaciones: este párrafo incluirá los resultados de las estimaciones del presupuesto de desarrollo, es decir: esfuerzo de desarrollo total y duración total, esfuerzos y duraciones de cada fase, teniendo en cuenta todas las restricciones del proyecto.

- Las Hipótesis del riesgo asociado al proceso de estimación. También han de incluirse en este apartado para identificar posibles orígenes del riesgo de una estimación hecha.

- Organización del proyecto: en primer lugar, se debe especificar la aproximación de desarrollo que se va a emplear para lograr los objetivos del proyecto, especificando qué tipo de ciclo de vida se va utilizar.

- Descomposición detallada de actividades: en cuanto a las actividades, debe especificar las actividades que componen cada paquete de trabajo. Ha de usarse un diagrama que muestre la división de actividades en subactividades.



En cuanto al presupuesto, debe especificar la estructura de costes, es decir, la dedicación del presupuesto a las actividades Estructura y organización de productos: Debe recoger todos los ítems a entregar al cliente. Los productos se dividen de tal manera que se pueda seguir su avance técnico y contable, y se descomponen de acuerdo a la granularidad necesaria para abarcar los objetivos.

- Recursos necesarios: en este párrafo se detallan todos los recursos que estarán destinados a la realización del proyecto. Recursos Humanos: Se deben identificar, definir y establecer los roles y responsabilidades.

  También se deben estimar los recursos humanos necesarios para la realización del proyecto, especificando el número y tipo de personal. Por tanto habría que incluir una definición de los niveles de conocimiento, fechas de inicio, duración, métodos de contratación, entrenamiento, retención y despido del personal.

- Otros recursos: estimación de los recursos necesarios para realizar el proyecto, tales como software de soporte, hardware, facilidades de oficina,

comunicaciones, lenguajes de programación, notaciones, herramientas, técnicas y métodos empleados para la producción de entregas.

– Organigrama: se deben describir la estructura interna y externa del proyecto en términos de mecanismos gráficos, tales como diagramas de organización.

– Relación entre productos y actividades del proyecto: en este apartado se debe especificar la relación de los elementos pertenecientes a la estructura y organización del producto con las actividades de alto nivel consideradas para el proyecto.

– Planificación: en este apartado se incluirán los resultados de las actividades de planificación:

– Red PERT: se muestran las relaciones de dependencia entre las actividades descritas en la Estructura de Descomposición de Actividades

– Diagrama GANTT: planificación de las distintas actividades, teniendo en cuenta las relaciones de precedencia y las fechas de los puntos de decisión requeridos.

– Coste y plazos del proyecto. Se documentan las fuentes, razonamientos y suposiciones realizadas en las estimaciones. Se preparan las distribuciones de esfuerzo, coste y equipo en las tareas.

– Plan de Entregas: un diagrama de GANTT de entregas al cliente.

– Estrategia de seguimiento y control de proyectos: en este apartado se definen los mecanismos para informar y los formatos de los informes, flujo de información, mecanismos de revisión y auditoría y otras técnicas a emplear para el control de adherencia al plan.

  Se precisará particularmente: Métodos de seguimiento, informes de avance, tipo de reuniones planificadas…

– Identificación y análisis de los riesgos: se deben identificar los riesgos del proyecto, incluyendo riesgos técnicos, económicos, de soporte a la operación y de planificación. Estos riesgos deben analizarse para determinar los factores que pueden prevenir o paliar los mismos.

2.3. Análisis del proceso de seguimiento. Reuniones e informes

La planificación de la organización de proyectos software tiene por objeto representar la organización de recursos humanos del proyecto, su estructura, responsabilidades, etc.; así como la estructura de recursos tecnológicos y materiales.

Una vez realizada las estimaciones adecuadas se deben de identificar los recursos necesarios para llevar a cabo el proceso.

Además de una lista de recursos, una técnica para poder organizar estos es el diagrama RBS que nos sirve para representar un proyecto desde el punto de vista organizativo.

La RBS (Resource Breakdown Structure) es un árbol jerárquico que representa los recursos humanos y materiales del proyecto.

Los principales objetivos de la RBS son los siguientes:

– Mostrar gráficamente la organización (humana) del proyecto (organigrama).

– Maximizar el uso de los conocimientos y experiencia del personal disponible.

– Reflejar la estructura de recursos materiales necesarios para la realización del proyecto (herramientas, plataformas, etc.), así como sus costes asociados.

En general, la estructura jerárquica que se refleja en la parte correspondiente a los recursos humanos de la RBS viene marcada por las políticas propias de la organización en la que se vaya a desarrollar el proyecto software.

Aun así, existen algunas reglas generales aplicables a todos los desarrollos software y que son independientes de la organización en la que se esté llevando a cabo el desarrollo software.

A continuación se pasa a identificar dichas reglas:

– El responsable de calidad y el jefe de proyecto no deben depender uno del otro, ya que sus tareas deben ser absolutamente independientes y no deben estar supeditadas las unas a las del otro.

– Un jefe de proyecto no debería gestionar a un equipo formado por más de siete personas.

– Ello implica que debería existir un jefe de proyecto por cada siete personas que se precisen para llevar a cabo un proyecto software.

En la figura aparece un ejemplo de RBS en el que se pueden apreciar el resultado de aplicar las dos reglas anteriores para un proyecto software.



El seguimiento y control es una parte fundamental de la Gestión de Proyectos; se ha observado que el éxito de los proyectos radica en un adecuado seguimiento y control del mismo.Una vez que se ha planificado el proyecto, hay que controlar su desarrollo para comprobar que se está siguiendo el plan previsto.

Definición

El seguimiento se puede definir como asegurar que el proyecto satisface sus objetivos en calidad, coste y tiempo.

Normalmente el seguimiento de proyectos, a nivel de gestión, incluye una serie de tareas, reuniones e informes sobre el estado, revisiones de hitos y demás gestiones.

A nivel técnico, el control y seguimiento debe incluir inspecciones y revisiones técnicas que controlan si las entregas se consideran correctas y terminadas.

La práctica diaria de seguimiento de proyectos básicamente consiste en recopilar datos reales consiste en recoger los datos del avance del proyecto en cuanto a tiempo, esfuerzo y coste.

Posteriormente estos datos han de ser contrastados con los datos estimados, que se encuentran documentados en el plan de proyecto, y de esta forma detectar la existencia de desviaciones significativas.

Para aquellas desviaciones que sean consideradas como significativas se han de tomar acciones correctivas, las cuales deberán ser gestionadas hasta su conclusión. **No sólo se tienen que tomar acciones correctivas sino que además se tiene que ver que estas acciones funcionan.**

La gestión de los cambios a los compromisos es la práctica de asegurar que, antes de cambiar cualquier acuerdo con el cliente, el progreso del proyecto y las acciones correctivas que se tomen en su caso son revisados y ratificados por los responsables del proyecto.

De esta forma se asegura que los responsables del proyecto tomen decisiones que puedan ser asumidas por todos los actores participantes.

Así se garantiza que cualquier cambio podrá ser implementado y todos los miembros de la organización asumen como suyos los acuerdos y compromisos de esta.



La combinación de estas tres prácticas: recopilar, acciones correctivas y la gestión de los cambios, y no sólo el cumplimiento de unos formularios de seguimiento, es lo que realmente forma el núcleo del proceso de seguimiento y control de proyectos software.

Una reunión de stand-up (o simplemente «stand-up») es una reunión de equipo realizada para proporcionar una revisión de cómo avanza el proyecto de esta a los miembros del equipo.

El hecho de que sea de «en tiempo semi-real» permite a los participantes conozcan los problemas desafíos potenciales, así como para coordinar los esfuerzos para resolver cuestiones difíciles y / o que consumen mucho tiempo.

Este tipo de reuniones tiene especial valor en el proceso desarrollo de software ágil, como Scrum, pero realmente puede ser utilizado en cualquier metodología de desarrollo.



Sabías que

El término «stand-up» se deriva de la práctica de que los asistentes no se sientan durante la reunión, ya que la incomodidad de estar de pie durante largos períodos ayuda a mantener las reuniones cortas.

Las reuniones suelen estar limitadas a a 5-15 minutos y se mantienen de pie para recordarle a la gente que debe mantener la reunión corta y e ir al grano. La reunión de pie a veces también se conoce como el 'stand-up', 'nominal por la mañana' o 'diario scrum'.

Hay tres preguntas que hacer y contestar en la reunión stand-up diaria. A pesar de que puede no ser práctico limitar toda discusión a estas tres preguntas, el objetivo es seguir lo más fielmente posible a las siguientes preguntas:

– ¿Logré los objetivos de ayer?

– ¿Qué voy a hacer hoy?

– ¿Qué obstáculos están impidiendo mi progreso?

La reunión suele tener lugar al a la misma hora y lugar cada día de trabajo. Además se anima a todos los miembros del equipo para que asista, pero las reuniones no se posponen si algunos de los miembros del equipo no están presentes.

Una de las características importantes es que el encuentro está pensado como un vehículo de comunicación entre los miembros del equipo y no una actualización de estado a los gestores u otros grupos de interés.

Aunque se denomina a veces como un tipo de reunión de estado, la estructura de la reunión tiene la intención de promover el seguimiento de conversa-

ción, así como para identificar los problemas antes de que sean demasiado problemático.

La práctica diaria también promueve las relaciones de trabajo más estrechas por su frecuencia, la necesidad de conversaciones de seguimiento y la estructura de la reunión, que a su vez dar lugar a una mayor tasa de transferencia de conocimientos entre los miembros del equipo.

En la reunión de stand-up, todos los miembros del equipo se turnan para hablar, a veces pasando a de uno a otro un token u objeto para indicar la persona que actualmente se le tiene concedida la palabra..

Cada miembro expone sobre el progreso desde la última stand-up, el trabajo previsto hasta el próximo stand-up y los obstáculos, teniendo además la oportunidad de pedir ayuda.

Los miembros del equipo pueden también pedir aclaraciones cortas y hacer declaraciones breves, del tipo «Vamos a hablar de esto más después de la reunión», pero el stand-up no suele consistir en debates largos entre ellos sino más bien un estilo brainstorming.



Aunque se habla de que el sistema de notificación y seguimiento pertenece a la Fase de Implementación, en realidad la gestión comienza desde el momento en que usted se involucre con el proyecto.

Por lo tanto, un sistema de notificación deberá ser negociado por adelantado con el patrocinador del proyecto y debe estar en su lugar desde el principio del proyecto.

El contenido, el formato y la frecuencia pueden variar a medida que el proyecto avanza a través de su ciclo de vida, pero los informes regulares deben de ir desarrollándose durante todas las fases del proyecto.

La primera pregunta que se debe tratar de responder, incluso durante la fase de planificación, es qué información necesitan los interesados en el proyecto o stakeholders.

Ha de ser recabada información de los grupos de interés del proyecto lo que tienen que informar a sus propios grupos de interés. Esta pregunta guiará al responsable del proyecto en relación con el nivel de detalle necesario en los informes y la frecuencia con la que estos deben ser hechos.

Hay que tener en cuenta que es una pérdida de tiempo para informar de los detalles innecesarios, pero también es una pérdida de tiempo para informar detalles insuficientes.

La propuesta de proyecto se desarrolla en la conclusión de la fase de iniciación del proyecto y requiere autorización de todos los interesados para proceder a la siguiente fase.

Contiene los resultados resumidos del proceso de planificación de alto nivel inicial. En los grandes proyectos de la propuesta de proyecto puede ser sustituido por un caso de negocio.

Los informes de gestión de proyectos es un importante vehículo de comunicación entre los miembros del equipo del proyecto, los departamentos involucrados, la alta dirección y los clientes.



Aprender a escribir un informe eficaz es una habilidad importante y requiere atención a los aspectos más sobresalientes de un proceso de gestión.

A continuación se exponen las estrategias más útiles para la redacción de un informe de gestión de proyectos.

– El primer paso es Identificar la audiencia. Hay que tener en cuenta si el informe sólo va a ser leído por el equipo del proyecto o distribuido ampliamente a múltiples departamentos, propietarios de la empresa y los clientes.

  La identificación de los lectores desde el principio, proporcionará indicaciones útiles para el contenido de la memoria del proyecto así como para los informes de seguimiento.

– Hay que decidir sobre la extensión del informe entes de escribir. El autor no puede que no necesariamente puede adherirse a la longitud exacta,

pero el haber tomado esta decisión le impedirá escribir un informe de 50 páginas, si la gestión requiere un informe mucho más corto.

Cuando se escribe un informe se ha de ser lo más conciso posible sin sacrificar la calidad o detalles necesarios. En general, los lectores tienen lapsos de atención y así como limitaciones de tiempo.

Si se puede transmitir un punto en dos párrafos, en vez de en cinco, utilice la opción más corta. Para ello se debe dividir el contenido del informe en categorías claramente etiquetados. De esta forma cada persona podrá ir a la sección de su interés y obviar el resto.

Realizar un resumen al comienzo del informe es de mucha importancia. El resumen ejecutivo se presenta al comienzo de un informe de gestión del proyecto y le da al lector una visión general concisa del propósito del proyecto, resultados, avances, problemas y recomendaciones.

Los lectores pueden utilizar el resumen ejecutivo para decidir si deben leer todo el informe o no, o bien solamente centrarse en una parte de él. Siempre conviene escribir una introducción. Esta sirve para explicar la naturaleza del proyecto, su objetivo y cualquier información de antecedentes elaborar sobre la necesidad del proyecto

- Hay que incluir una sección de metodología y describir esta, en ella se debe describir por ejemplo la mano de obra, las herramientas y recursos que se utilizan para lograr los objetivos del proyecto.

- Hay que detenerse y elaborar con detalle los resultados del proyecto. Si el proyecto es un estudio, incluir estadísticas relevantes y observaciones cualitativas. Si el proyecto incluye el desarrollo de productos, explicar los hallazgos relacionados con las elaboraciones de productos.

- Hay que explicar los éxitos preliminares del proyecto. El proyecto puede estar progresando antes de lo previsto o satisfacer una necesidad inesperada. Incluya toda noticia positiva que se considere relevante o no.

- Describir los desafíos y los obstáculos del proyecto. Desafíos podrían incluir la insuficiencia de recursos, obstáculos inesperados, fallos mecánicos u otros obstáculos.

- Proponer recomendaciones y soluciones. Si su proyecto está obteniendo resultados importantes, discutir las implicaciones de los hallazgos para la organización o el cliente. Conviene incluir también una sección de soluciones como un remedio potencial para los desafíos del proyecto.

- Por último finalizar el informe con un capítulo de conclusiones. Se debe de utilizar este espacio de la conclusión para llamar la atención del lector sobre los temas más importantes, las soluciones y los pasos siguientes.

- Convienen técnicas de formato para guiar la atención de los lectores. Proporcionar categorías claras con subtítulos y viñetas en todo el informe para ahorrar tiempo para los lectores que sólo están interesados en ciertos aspectos del documento de proyecto.

- Hay que hacer circular el informe preliminar a los miembros del equipo del proyecto. Este es un paso muy importante antes de distribuir el informe del proyecto a otros departamentos, altos mandos o clientes. Los miembros del equipo pueden llamar su atención a detalles importantes, inexactitudes o soluciones.

  Revisar el informe para incluir las contribuciones de los miembros del equipo. El criterio del responsable deben ser el que decida con respecto a qué detalles desea incluir.

- Detalles tediosos. Pueden ser importantes para un técnico pero puede no ser interesante para los lectores interesados en luna información con un zoom más amplio como los gerentes y gestores de la organización.

- Conviene revisar el informe para evitar errores tipográficos y erratas. Conviene tener en cuenta que el leer el mismo documento varias veces puede conducir a pasar por alto los errores evidentes.

  Para evitar esto es necesario advertir a los lectores que ojean el documento por primera vez que se encarguen de señalar los errores o imprecisiones. Estos además podrán ayuda a cerrar la redacción final con sus sugerencias.

  Es importante corregir todos los errores tipográficos y problemas de formato antes de distribuir el informe. Un informe bien organizado y sin errores reducirá las distracciones y permitir que sus lectores se concentren en los puntos principales de su informe.



Definición

El **método Delphi** es una técnica de comunicación estructurada, originalmente desarrollado como un proceso sistemático, interactivo previsión método que se basa en un panel de expertos.

Los expertos responden cuestionarios en dos o más rondas. Después de cada ronda, un facilitador proporciona un resumen anónimo de las previsiones de los expertos de la ronda anterior, así como las razones que proporcionan para sus juicios.

Por lo tanto, se alienta a los expertos a revisar sus respuestas anteriores a la luz de las respuestas de los otros miembros de su panel.

Se considera que durante este proceso el rango de las respuestas disminuirá y el grupo convergerá hacia la respuesta «correcta».Por último, el proceso se detiene después de un criterio predefinido de parada

Criterios de parada más o menos estandarizados son por ejemplo, número de rondas, el logro de un consenso, la estabilidad de los resultado y los resultados de las medias o medianas de las rondas finales son los que sirven en la determinación los resultados.

Delphi se basa en el principio de que las previsiones (o decisiones) a partir de un grupo estructurado de individuos son más precisos que los de los grupos no estructurados.



Sabías que

La técnica también puede ser adaptado para su uso en las reuniones cara a cara, y se llama entonces mini-Delphi o Estimation-Talk-Estimation (ETE).

Delphi ha sido ampliamente utilizado para la previsión de negocio y desarrollos del software, y tiene ciertas ventajas sobre otros enfoques estructurados de pronóstico en los mercados de predicción



Sabías que

El nombre de «Delphi» deriva del Oráculo de Delfos. Los autores del método no estaban contentos con este nombre, ya que implica algo oracular, algo obtenido un poco de lo oculto».

El método Delphi se basa en la suposición de que los juicios de grupo son más válidos que los juicios individuales. El **método Delphi** fue desarrollado a principios de la Guerra Fría para pronosticar el impacto de la tecnología en la guerra.



En 1944, el general Henry H. Arnold ordenó la creación del informe para el Cuerpo Aéreo del Ejército de Estados Unidos sobre las futuras capacidades tecnológicas que podría ser utilizado por los militares.

Se probaron diferentes enfoques, pero las deficiencias de tradicionales de predicción métodos, como enfoque teórico, los modelos cuantitativos o la extrapolación de tendencias, en las zonas donde las leyes científicas precisas no se han establecido aún, rápidamente se hizo evidente.

Se ha utilizado desde entonces, junto con varias modificaciones y reformulaciones, tales como el procedimiento Imen-Delphi.

Se pidió los expertos que dieran su opinión sobre la probabilidad, la frecuencia y la intensidad de posibles ataques enemigos. Otros expertos podrían dar información de manera anónima. Este proceso se repitió varias veces hasta que un consenso.

Las siguientes características clave del método Delphi ayudan a los participantes a concentrarse en los temas en cuestión y por separado Delphi de otras metodologías:

El anonimato de los participantes: Por lo general, todos los participantes permanecen en el anonimato. Su identidad no se revela, incluso después de la terminación del informe final. Esto evita que la autoridad, la personalidad, o la reputación de algunos participantes de dominar a los demás en el proceso.

Podría decirse que también libera los participantes (hasta cierto punto) de sus prejuicios personales, minimiza el « efecto de contagio «o» efecto de halo «, permite la libre expresión de opiniones, alienta la crítica abierta, y facilita la admisión de errores al revisar sentencias anteriores.



Estructuración del flujo de información: las contribuciones iniciales de los expertos se recogen en forma de respuestas a los cuestionarios y sus comentarios a estas respuestas.

El director de panel controla las interacciones entre los participantes mediante el procesamiento de la información y la filtración de contenido irrelevante. Esto evita los efectos negativos de los paneles de discusión cara a cara y resuelve los problemas habituales de la dinámica de grupos.

Retroalimentación regular: Los participantes comentan sus propias previsiones, las respuestas de los demás y sobre los progresos del panel en su conjunto. En cualquier momento un experto puede revisar sus declaraciones anteriores.

Mientras que en las reuniones regulares de los grupos participantes tienden a adherirse a las opiniones anteriormente expuestas ya menudo conformar demasiado a la líder del grupo; el método Delphi se lo impide.

Papel del facilitador: la persona encargada de coordinar el método Delphi se conoce generalmente como un facilitador o líder, y facilita las respuestas de su panel de expertos.

Los expertos son seleccionados por una razón, por lo general que posean conocimientos sobre una opinión o punto de vista.El facilitador envía cuestionarios, encuestas, etc. y si el panel de expertos aceptan, que siga las instrucciones y presentar sus puntos de vista.

Las respuestas se recopilaron y analizaron, a continuación, se identifican puntos de vista comunes y en conflicto. Si no se alcanza el consenso, el proceso continúa a través de la tesis y la antítesis, trabajar gradualmente hacia la síntesis, y la creación de consenso.

De esta forma el método sigue trabajando de forma iterativa hasta que se logar llegar a un acuerdo.

UD2
Lo más importante

- Las estimaciones tienen una importancia estratégica crucial en cualquier proceso de desarrollo del software

- Hay varios modelos de estimación, los paramétricos, los basados en la opinión de expertos y los dinámicos.

- El modelo COCOMO es uno de los más utilizados. Hay varios submodelos dependiendo de la complejidad del proyecto y la precisión que se desee en la estimación

- Los puntos de función son otra menara de medir la complejidad de un proyecto, basado en las estimaciones de un experto de las entradas, salidas y procesos que ha de realzar el sistema.

- El método COSMIC es un método estandarizado internacionalmente dimensionamiento software ampliamente utilizado

- Un WBS o "Work Breakdown Structure", es una representación gráfica de las diferentes actividades que se han de llevar a cabo para la realización de un proyecto.

- Los diagramas PERT son una familia de diagramas que se usan para modelar las actividades que se han de llevar a cabo en un proyecto así como sus dependencias.

– Los diagramas de GANTT representan la misma información que un diagrama PERT., pero en forma de barras, siendo de esta manera más sencillo hacer el seguimiento de un proyecto.

– El método del camino crítico indica cuales son las actividades más importantes de un proyecto en cuenta su impacto en el tiempo, y de esta forma se puede mejorar el seguimiento con a asignación correcta de recursos para evitar su retraso.

– El plan de proyecto es el documento en el que se indican todos y cada uno de los aspecto más importantes de las tareas a realizar y su planificación, se ha de realizar al comienzo del proyecto

– Los entornos de desarrollo proporcionan un marco de trabajo completo al programador en el que desarrollar su labor, incluyendo la codificación, depuración, pruebas y documentación en una misma herramienta.

– La planificación de la organización de proyectos software tiene por objeto representar la organización de recursos humanos del proyecto, su estructura, responsabilidades, etc.; así como la estructura de recursos tecnológicos y materiales.

– Las reuniones de seguimiento diarias o semanales son una herramienta crucial para que se puedan detectar a tiempo problemas en el proyecto. Se recomienda seguir al metodología de reuniones de stand-up

– Los informes de seguimiento son otra herramienta fundamental de del seguimiento de un proyecto. Por ello es necesario cuidar su redacción, estructurarlos adecuadamente y cuidar la longitud así como determinar la audiencia a la que serán enviados.

UD2
Autoevaluación

1. Los tipos de modelos de estimación que hay son:

    a. Modelos de determinación de análisis, modelado, codificación y testeo.
    b. Modelos matemáticos paramétricos, Estimación basada en la experiencia de experto, técnicas orientadas al aprendizaje y modelos dinámicos.
    c. Modelos matemáticos paramétricos, Estimación basada en la experiencia de experto, técnicas orientadas al aprendizaje y modelos de verificación.
    d. Modelos de Verificación y validación.

2. Cual de estos no es un método de estimación:

    a. COCOMO.
    b. Método de PUTNAM.
    c. PERT.
    d. MK II.

3. Sobre el método COSMIC, indica la respuesta falsa:

    a. Es difícil evaluar los riesgos en él.
    b. Es un método de evaluación de segunda generación.
    c. Es un método abierto.
    d. Se ajusta a la normativa ISO.

4. La actividad de planificación o también llamada de programación de un proyecto, consiste en establecer la estructura temporal de…

   a. Las fases, actividades y tareas del proyecto, en función de los recursos de que se deseen.
   b. Las fases del proyecto, en función de los recursos de que se disponga.
   c. Las fases, actividades y tareas del proyecto, en función de los recursos de que se disponga.
   d. Las fases de programación así como sus, actividades y tareas del proyecto, en función de los recursos de que se disponga.

5. Las actividades del método PERT:

   a. Se representa mediante cajas.
   b. Se representa mediante flechas.
   c. Se representan mediante rombos.
   d. Nos e indican explícitamente en el diagrama.

6. El seguimiento y control:

   a. Es fundamental en la gestión de un proyecto.
   b. Se hace mediante el método COCOMO.
   c. No es necesario en proyectos pequeños.
   d. Se realiza mediante herramientas de dimensionamiento.

7. El resumen ejecutivo:

   a. Se presenta al comienzo del proyecto.
   b. Se comienza con la planificación.
   c. Se presenta al final de proyecto.
   d. Se presenta al finalizar la fase de diseño.

8. En la presentación de un informe:

   a. No hay que añadir la metodología.
   b. Conviene estructurarlo adecuadamente.
   c. Se puede obviar el formato para mayor velocidad.
   d. No es necesario difundirlo entre el equipo.

9. Las técnicas de formato a la hora de elaborar un informe:

    a. No son necesarias para hacer un informe.
    b. Dependen de la organización y no del autor.
    c. Son necesarias para poder presentarlo adecuadamente.
    d. Modelan la organización a nivel táctico.

10. La retroalimentación regular se refiere a:

    a. El director del proyecto indica quien lo hace bien y quien lo hace mal.
    b. Deben centrarse en los fallos cometidos.
    c. No requieren de reuniones específicas.
    d. Los participantes comentan sus propias previsiones y las de los demás.

Área: informática y comunicaciones

UD3

Diagramado

3.1. Identificación de los principios básicos de UML

3.2. Empleo de diagramas de uso

3.1. Identificación de los principios básicos de UML



Definición

El Unified Modeling Language™ o l Lenguaje de Modelado Unificado (UML®) es un lenguaje de modelado visual estándar destinado a ser utilizado para el modelado de procesos de negocio y similares,análisis, diseño e implementación de sistemas basados en software.

UML es un lenguaje común para los analistas de negocios, arquitectos y desarrolladores de software que se utilizan para describir, especificar, diseñar, y documento existente o nuevos procesos de negocio, la estructura y el comportamiento de los elementos de sistemas de software.

UML se puede aplicar a diversos dominios de aplicación (por ejemplo, banca, finanzas, internet, aeroespacial, salud, etc.) Puede ser utilizado con todos los objetos y el componente de los métodos de desarrollo de software y por varias plataformas de aplicación (por ejemplo, J2EE, NET).

UML realmente es un estándar de modelado lenguaje, no un proceso de desarrollo de software. La especificación UML 1.4.2 detalló como debía de ser un proceso basado en UML, el cual debe cumplir con las siguientes características:

- Debe proporcionar orientación en cuanto a la orden de las actividades de un equipo.
- Ha de especificar qué objetos deben ser desarrollados.
- Debe dirigir las tareas de los desarrolladores individuales y el equipo en su conjunto.

– Tiene que ofrecer criterios para el seguimiento y la medición de los productos y las actividades de un proyecto.

UML se la creado intencionalmente de manera independiente del proceso y puede aplicarse en el contexto de diferentes procesos. De esta forma puede ser utilizado con múltiples metodologías tanto Orientadas a Objetos como a procesos.

Sin embargo, es más adecuado para modelos basado en casos de uso, l así como os procesos de desarrollo iterativos e incrementales. Un ejemplo de tal proceso es Proceso Unificado del Rational.



Un diagrama UML no es un documento completo y no es completamente visual, es decir, dado un diagrama UML, no podemos estar seguros de entender parte representada o comportamiento del sistema a partir del diagrama solo.

Parte de la información podría ser omitido intencionadamente del diagrama, alguna información representada en el diagrama podría tener diferentes interpretaciones, y algunos conceptos de UML no tienen notación gráfica en absoluto, así que no hay manera de representar los de diagramas.

Por ejemplo, la semántica de la multiplicidad de actores y multiplicidad de casos de uso en los diagramas de casos de uso no se define con precisión en la especificación UML y podría significar el uso simultaneo o sucesiva de casos de uso.

O por ejemplo el nombre de un clasificador abstracto se muestra en cursiva, mientras clasificadores finales no tiene notación gráfica específica, lo que no hay manera de determinar si clasificador definitiva o no, a partir del diagrama.



Las primeras versiones de UML fue creado porl"os tres amigos", esto es, Grady Booch (creador del método de Booch), Ivar Jacobson (Object-Oriented Software Engineering, OOSE), y JimRumbaugh. (ObjectModelingTechnique-, OMT).

La versión actual de UML es UML 2.4.1, y fue lanzada en agosto de 2010.

La especificación UML estándar se actualiza y gestionado por el Object Management Group (OMG).En el siguiente esquema se muestra como ha sido la evolución de las diferentes versiones UML:

Versión	Fecha	Descripción
1.1	11-1997	UML 1.1 como propuesta es adoptada por el OMG.
1.3	03-2000	Una actualización menor a la propuesta original.
1.4	09-2001	Definiciones refinadas de los componentes y conceptos relacionados
1.5	03-2003	Añadidos a la versión
1.4.2	01-2005	Esta versión fue aceptada como la especificación ISO de estándar ISO / IEC 19501. UML 1.5 fue lanzado 2 años antes.
2.0	08-2005	Nuevos diagramas: diagramas de objetos, de paquete, diagramas de estructura, diagramas de descripción de la interacción, diagramas de tiempo.
2.1	04-2006	Revisión menor a UML 2.0 - correcciones y mejoras de consistencia.
2.1.1	02-2007	Revisión menor a la UML 2.1
2.1.2	11-2007	Revisión menor a la UML 2.1.1
2.2	02-2009	Numerosos problemas y consistencia de menor importancia añaden aclaraciones a UML 2.1.2
2.3	05-2010	Revisión menor a la UML 2.2, clarificados asociaciones y clases de asociación, añadió clasificador,actualizar el diagrama de componentes, estructuras de materiales compuestos, acciones, etc.
2.4.1	08-2011	Versión actual de UML, una revisión menor de la UML 2.3, con algunas correcciones.
2.5 Beta	9/2013	Trabajo en curso en el que se van a añadir algunas aclaraciones y modificaciones a la versión actual. Aún no está vigente.



Un **diagrama UML** es una representación gráfica parcial (vista) de un modelo de un sistema que se encuentra diseño, ejecución, o ya existentes.

Un diagrama UML contiene elementos gráficos o símbolos, también llamados nodos UML conectados con líneas (también conocidos como caminos o flujos, que representan elementos en el modelo UML del sistema diseñado.

El modelo UML del sistema también podría contener otros documentos, tales como casos de uso escritos como textousando una plantilla.

El tipo del diagrama se define por los símbolos gráficos primarios mostrados en el diagrama.

Por ejemplo, un diagrama donde los símbolos primarios en el área de contenido son clases es diagrama de clases Un diagrama que muestra los casos de uso y actores es un diagrama de casos de uso, o un secuencia muestra la secuencia de intercambio de mensajes entre líneas de vida de objetos.

La Especificación UML no excluye la mezcla de diferentes tipos de diagramas, por ejemplo, para combinar elementos estructurales y de comportamiento para mostrar una máquina de estado anidado dentro de un caso de uso. En consecuencia, los límites entre los diferentes tipos de diagramas no se aplican estrictamente.

Al mismo tiempo, algunas herramientas UML restringen conjunto de elementos gráficos disponibles que podrían utilizarse cuando se trabaja en determinado tipo de diagrama.

La Especificación UML define dos grandes tipos de diagrama UML: diagramas de estructura y diagramas de comportamiento.

Los diagramas de estructura muestran la estructura estática del sistema y de sus componentes en diferentes abstracción e implementación niveles y cómo se relacionan entre sí.



Los elementos en un diagrama de estructura representan conceptos significativos de un sistema, y pueden incluir elementos abstracto, del mundo real y conceptos de aplicación.

Los diagramas de comportamiento muestran el comportamiento dinámico de los objetos en un sistema, que puede ser descrito como una serie de cambios en el sistema a lo largo del tiempo.

Diagramas de estructura muestran la estructura estática del sistema y de sus componentes en diferentes niveles de abstracción y de ejecución y cómo las partes se relacionan entre sí.

Los elementos en un diagrama de estructura representan los conceptos significativos de un sistema, y pueden incluirelementos abstractos, elementos del mundo real y conceptos del sistema o de la aplicación.

Estructura de los diagramas no indican como se usan durante el tiempo los conceptos relacionados, y por tanto no muestran los detalles del comportamiento dinámico. Sin embargo, pueden mostrar las relaciones a los comportamientos de los clasificadores que se exhiben en los diagramas de estructura.

En las siguientes tablas se muestran los diversos tipos de diagramas que integran el estándar de UML, especificándose para cada una de ellos el propósito o la finalidad del diagrama correspondiente, y los diversos elementos que componen este diagrama:

Diagrama	Propósito	Elementos
Diagrama de clases	Muestra la estructura del sistema diseñado, subsistema o componente como clases e interfaces relacionadas, con sus características, limitaciones y relaciones-Asociaciones, generalizaciones, dependencias, etc	Clase, interfaz, característica, restricción, generalización, dependencia.

Diagrama de objetos	Diagrama de nivel de instancia de clase que muestra las especificaciones de instancias de clases e interfaces (objetos), las ranuras con las especificaciones de valores, y los enlaces (instancias de la asociación). Diagrama de objetos se define en la ahora obsoleta especificación UML1.4.1 como "un gráfico de instancias, incluyendo los objetos y los valores de los datos Un diagrama de objeto estático es un ejemplo de un diagrama de clases;. muestra una instantánea del estado detallado de un sistema en un punto en el tiempo. "También se dice que diagrama de objeto es "un diagrama de clases de objetos y no hay clases". UML 2.5 especificación simplemente ofrece ninguna definición de diagrama de objeto.	Especificación de instancia, objeto, ranura, enlace
Diagrama de paquetes	Muestra los paquetes y las relaciones entre los paquetes.	Paquete elementos que se puedan incluir en paquetes, dependencia, elementos importables
Diagrama de modelo	Diagrama UML de la estructura auxiliar que muestra una abstracción o consulta específica de un sistema, para describir los aspectos arquitectónicos, lógicos o de comportamiento del sistema. Se podría mostrar, por ejemplo, la arquitectura de una aplicación de varias capas (también conocido como de varios niveles)	Modelo, paquete, dependencias
Diagrama de estructura compuesta	Diagrama podría utilizarse para mostrar la estructura interna en un clasificador o un comportamiento en una colaboración	
Diagrama de estructura interna	Muestra la estructura interna de un clasificador-una descomposición del clasificador en sus propiedades, partes y relaciones.	Clase estructurada, propiedad, parte, puerto, conector, uso

Diagrama de uso y colaboración	Muestra los objetos en un sistema que cooperan entre sí para producir un cierto comportamiento del sistema.	La colaboración, la propiedad, conector, parte, la dependencia.
Diagrama de componentes	Muestra los componentes y las dependencias entre ellos. Este tipo de diagramas se utiliza para el desarrollo basado en componentes (CDB), para describir los sistemas con arquitectura orientada a servicios (SOA).	Componente, interfaz, proporcionado interfaz, interfaz necesaria, clase, puerto, conector, artefacto, realización de componentes, el uso.
Diagrama de Manifestación	Mientras los diagramas de componentes muestran los componentes y las relaciones entre los componentes y los clasificadores y diagramas de despliegue que despliegues de artefactos a los destinos de despliegue algunas diagrama intermedio que falta es diagrama de manifestación que se utilizará para mostrar la aplicación de los componentes de los artefactos en la estructura interna de los artefactos. Debido a que los diagramas de manifestación no se definen por UML 2.5 especificación, manifestación de componentes por los artefactos se puede mostrar ya sea utilizando diagramas de componentes o diagramas de despliegue.	Manifestación, componente, artefacto.

Diagrama de despliegue	Muestra la arquitectura del sistema en el despliegue (distribución) de software artefactos a los destinos de despliegue. Tenga en cuenta, que los componentes fueron desplegados directamente a los nodos de diagramas de despliegue 1.x UML. En UML 2.x artefactos se implementan en los nodos, y los artefactos podrían manifestarse (aplicar) componentes. Los componentes se implementan en los nodos indirectamente a través de los artefactos. Diagrama de despliegue nivel de especificación (también llamado nivel de tipo) muestra cierta visión general de despliegue de artefactos a los destinos de despliegue, sin hacer referencia a casos concretos de artefactos o nodos. Diagrama de despliegue a nivel de instancia muestra la implementación de instancias de artefactos a los casos específicos de despliegues Podría ser utilizado por ejemplo para mostrar las diferencias en los despliegues a entornos de desarrollo, puesta en escena o de producción con los nombres / identificadores de acumulación específico o servidores o dispositivos de despliegue.	Despliegue, artefacto, destino de despliegue, el nodo, dispositivo, el entorno de ejecución, ruta de comunicación, especificación de implementación.
Diagrama de la arquitectura de red	Los diagramas de despliegue podrían utilizarse para mostrar lógico o físico arquitectura de red del sistema. Este tipo de diagramas de despliegue-no formalmente definidos en UML 2.5- podría ser llamado diagramas de arquitectura de red.	Nodo, conmutadores de red, enrutadores, firewalls, elementos de comunicación, segmentos de red, etc.
Diagrama de perfil	Diagrama UML auxiliar que permite definir estereotipos personalizados, valores y limitaciones etiquetada como un mecanismo de extensión ligera con el estándar UML. Los perfiles permiten adaptar el metamodelo UML para diferentes. Plataformas (tales como J2EE o NET). Dominios (por ejemplo, en tiempo real o el modelado de procesos de negocio). Perfil diagramas se introdujeron por primera vez en UML 2.0.	Perfil, estereotipo, de extensión, referencias, perfiles de la aplicación.

Los diagramas de comportamiento muestran el comportamiento dinámico de los objetos en un sistema, que puede ser descrito como una serie de cambios en el sistema durante un período de tiempo.

Diagrama	Propósito	Elementos
Diagrama de casos de uso	Describe un conjunto de acciones denominada que algún sistema o sistemas (sujeto) deben o pueden llevar a cabo con la colaboración de uno o más usuarios externos del sistema (actores) para proporcionar unos resultados observables y valiosas a los actores u otras partes interesadas de la sistema (s). Tenga en cuenta, que UML 2.4.1 especificación indicó que los diagramas de casos son una especialización de diagramas de clases de tal manera que los clasificadores que se muestran están restringidos a ser tanto actores o casos de uso. Los diagramas de clases son diagramas de estructura.	Caso de uso, actores, sujetos, extensiones, inclusiones y asociaciones
Diagrama de flujo de información	Muestra el intercambio de información entre las entidades del sistema en algunos altos niveles de abstracción. Los flujos de información pueden ser útiles para describir la circulación de información a través de un sistema mediante la representación de aspectos de los modelos aún no plenamente expresados o con menos detalles.	Flujos de información, elementos de información, actores y clases.
Diagrama de actividad	Muestra la secuencia y las condiciones para la coordinación de las conductas de menor nivel, en lugar de que los clasificadores son dueños de esos comportamientos. Estos son comúnmente llamados flujo de control y flujo de objeto modelos.	Actividad, partición, acción, objeto, el control, borde actividad.

Diagrama	Propósito	Elementos
Diagramas de máquinas de estados	Se utiliza para modelar la conducta discreta a través de transiciones de estados finitos. Además de expresar el comportamiento de una parte del sistema, máquinas de estado también se pueden usar para expresar la protocolo de uso de una parte de un sistema. Estos dos tipos de máquinas de estados se conocen como máquinas de estado de comportamiento y las máquinas de estado	
Diagrama de interacción	Los diagramas de interacción incluyen varios tipos diferentes de diagramas: Diagramas de secuencia. Diagramas de comunicación (conocidos como diagramas de colaboración en UML 1.x). Diagramas de temporización y un descripción de la interacción.	
Diagrama de secuencia	La mayor parte de los diagramas de interacción que se centra en el intercambio de mensajes entre líneas de vida (objetos).	Líneas de vida, especificación ejecución, mensaje, fragmento combinado, el uso de interacción, estado invariante, destrucción ocurrencia.
Diagrama de Comunicación (también conocido como Diagrama de colaboración en UML 1.x)	Se centra en la interacción entre líneas de vida donde la arquitectura de la estructura interna y cómo esto se corresponde con el mensaje que pasa es central. La secuenciación de los mensajes se da a través de una secuencia de numeración de esquema.	Líneas de vida, mensajes
Diagrama de tiempos	Muestra interacciones cuando un propósito principal del diagrama es de razonar acerca de tiempo. Los diagramas de temporización se centran en las condiciones cambiantes dentro y entre las líneas de vida a lo largo de un eje de tiempo lineal.	Línea de vida, estado o condición de línea de tiempo,evento de destrucción, duración restricción, limitación de tiempo.

Diagrama general de Interacción	Define las interacciones a través de una variante de los diagrama de actividad de una manera que promueve la visión general del flujo de control. Diagramas de descripción de la interacción se centran en la visión general del flujo de control, donde los nodos son interacciones o utiliza la interacción. Las líneas de vida y los mensajes no aparecen en este nivel de resumen.	Nodo inicial, flujo, último nodo, nodo final de la actividad, nodo de decisión, fusionar nodo, nodo tenedor, unirse a nodo, la interacción, el uso de interacción, duración restricción, limitación de tiempo.

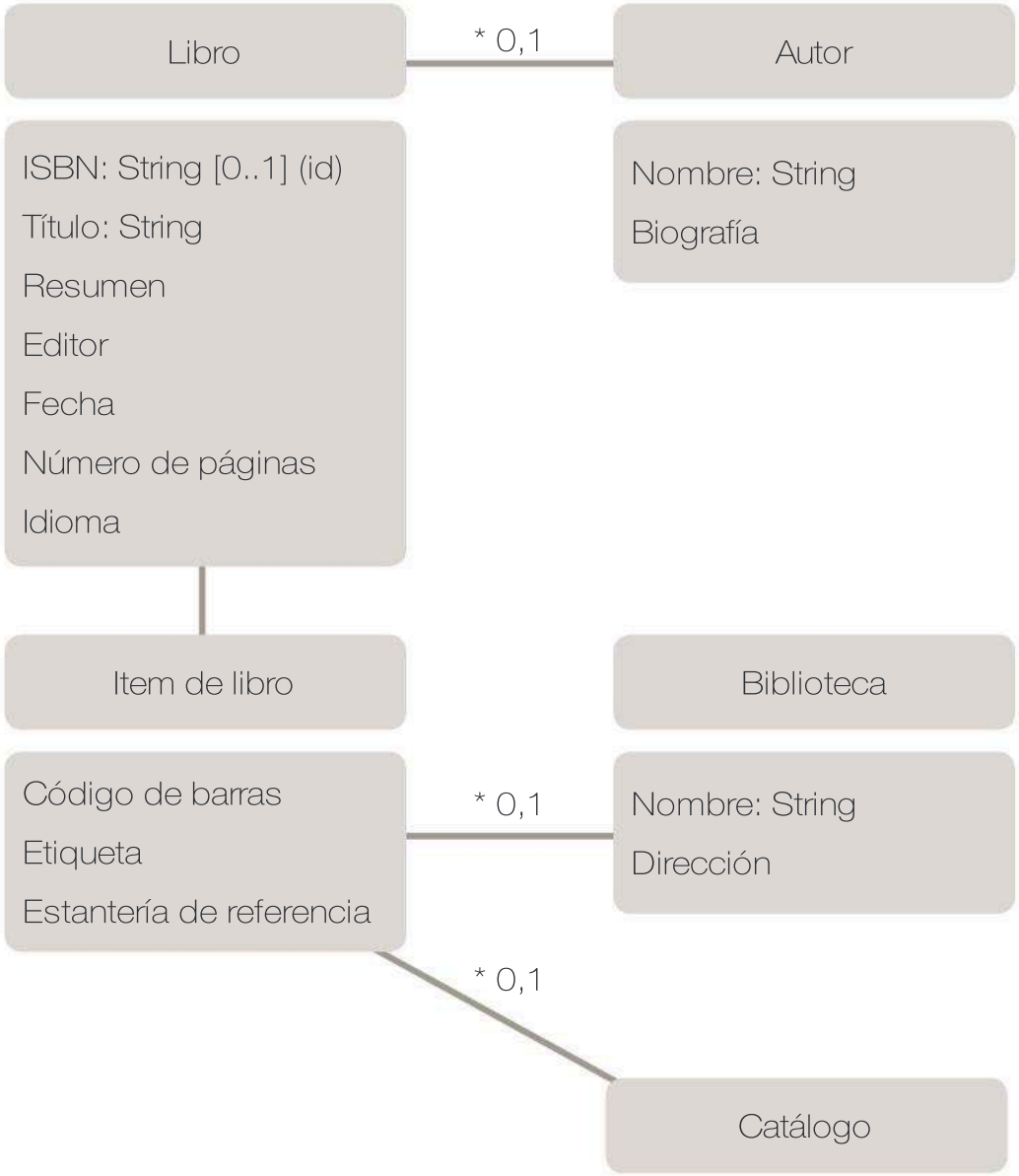
3.2. Empleo de diagramas de uso

Los **diagrama de clases** son un tipo de diagrama UML de estructura que muestra la estructura del sistema diseñado en el nivel de las clases y las interfaces, muestra sus características, estructura y relaciones asociaciones, generalizaciones, dependencias, etc.

Algunos tipos comunes de diagramas de clases son, por ejemplo, el diagrama de modelo de dominio, o el diagrama de clases de implementación.

El **Diagrama de objeto** podría ser considerado como diagrama de clases de nivel de instancia, que muestra las especificaciones de instancias de clases e interfaces (objetos), las ranuras con las especificaciones de valores y Enlaces (instancias de la asociación).

(Ver diagrama en la página siguiente)



El diagrama de modelo de dominio muestra un plano general de dominio, es decir, las clases, las interfaces, las asociaciones, el uso, la realización y la multiplicidad.

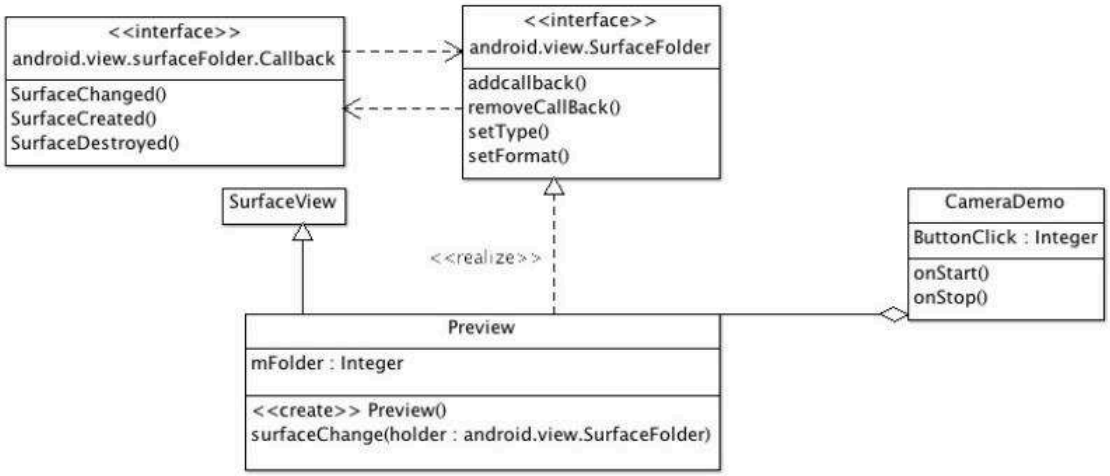


Diagrama de clases de implementación

Los Elementos del diagrama de clase de implementación son las clases, interfaces, asociaciones, de uso, y asociaciones de realización. Muestra como será la implementación real del sistema que se esta desarrollando.

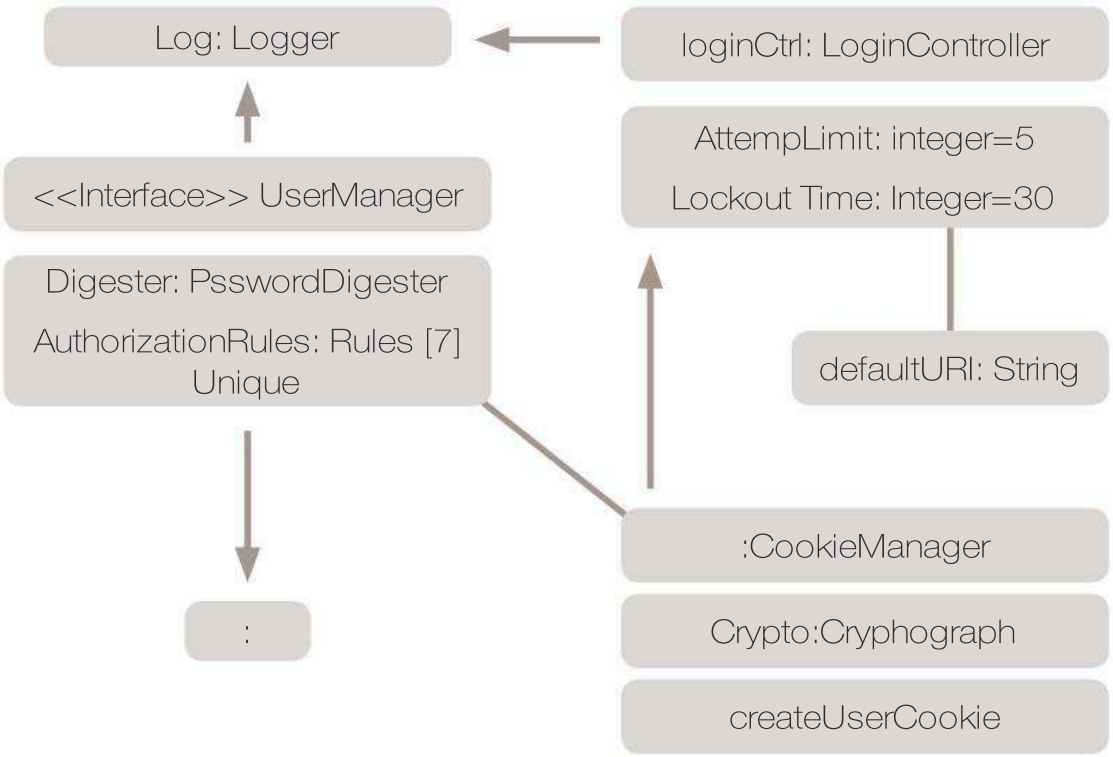
Definición

Los **diagramas de objetos** se definen en la ahora obsoleta Especificación UML 1.4.2 como "un gráfico de instancias, incluyendo los objetos y los valores de los datos.

Un diagrama de objeto estático es un ejemplo de un diagrama de clases, muestra una instantánea del estado detallado de un sistema en un punto en el tiempo. También se puede decir que un diagrama de objeto es "un diagrama de clases donde en lugar de clases lo que aparecen son instancias de clases.

Importante

La especificación UML 2.4 simplemente no ofrece ninguna definición de diagrama de objeto, excepto una mención a que "los siguientes nodos y los bordes se extrae típicamente en un diagrama de objeto: Instancia, Especificación y Enlace (es decir, de la Asociación).



Hay que tener en cuenta que jerarquía estándar de diagramas de UML 2.5 muestra diagramas de clases y diagramas de objetos como elementos totalmente ajenos, sin relación.

Algunas otros autores señalan que los diagramas de componentes y diagramas de despliegue que sólo contienen las especificaciones de instancia son realmente también un tipo especial de diagramas de objetos.

El esquema anterior algunos de los elementos principales de diagrama de objetos, tanto con nombre y anónimos indicando especificaciones de instancia para los objetos, las ranuras para recibir mensajes con las especificaciones de valores, y los enlaces (instancias de una asociación).

Definición

Diagramas de Clases: diagrama de clases es un diagrama que muestra la estructura del sistema diseñado en el nivel de las clases y las interfaces, muestra sus características, limitaciones y relaciones, es decir, asociaciones, generalizaciones, dependencias, etc.

Clase: una clase es un clasificador que describe un conjunto de objetos que comparten las mismas características, restricciones y semántica o significado.

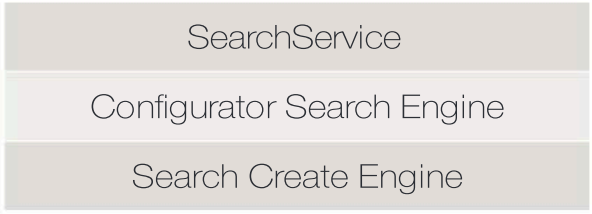
Una clase se muestra como un esquema con un rectángulo sólido que contiene el nombre de la clase, y opcionalmente con compartimentos separados por líneas horizontales que contienen características o otros miembros de la clasificadorComo la clase es el clasificador más utilizado, no hay necesidad de añadir la palabra clave "class" encima del nombre de la clase.

El nombre de la clase debe aparecer centrado y en negrita, con la primera letra del nombre de la clase en mayúscula (si el conjunto de caracteres compatible con mayúsculas).



Las Características de una clase son los atributos y operaciones. Cuando la clase se muestra con tres apartados, el hueco central contiene una lista de atributos y el compartimento inferior contiene una lista de operaciones.

Los Atributos y operaciones deben ser justificados a la izquierda en el espacio correspondiente, con la primera letra de los nombres en minúsculas. El siguiente esquema representa una clase llamada SearchService a nivel de detalle de análisis:



Las características de la clase representadas por valores pueden ser o bien de instancias del clasificador considerados individualmente (no estático) o del propio clasificador (estática).

La misma característica no puede ser estático en un contexto y no estática en otro. Con respecto a las características estáticas, se reconocen dos semántica alternativas.

Las Características estáticas puede tener, o bien diferentes valores para diferentes clasificadores o el mismo valor para todos los clasificadores al que pertenecen. De acuerdo con esta semántica, la herencia es de valores de características estáticas está permitida, aunque no es obligatorio en UML2.

Las Características estáticas se representan mediante un subrayado , pero sólo se subraya el valor del nombre. Los puntos suspensivos (...) como el elemento final de una lista de características indica que existen características adicionales, pero se no se muestran en dicha lista.

Los atributos de una clase están representadas por las instancias de la propiedad que se pertenecen a la clase. Algunos de estos atributos se pueden representar en os extremos navegables de asociaciones binarias.



Los objetos de una clase debe contener valores para cada atributo que es un miembro de esa clase, de acuerdo con las características del atributo, por ejemplo, su tipo y multiplicidad.

El siguiente diagrama incluye la clase anterior pero esta vez a nivel de detalle de aplicación, teniendo en cuenta que el método createEngine va a ser una operación estática.

SearchService
-config: Configurator -search: SearchEngine
+search (query: SearchRequest: SearchResult -CreateEngine(): SearchEngine

Los atributos o las operaciones pueden ser agrupados por la visibilidad. Una palabra clave de visibilidad o símbolo en este caso se puede dar una vez para múltiples funciones con la misma visibilidad.

Compartimientos adicionales se pueden añadir para mostrar otros detalles, tales como restricciones, o simplemente para dividir características.

Una clase en el UML puede ser utilizado como un espacio de nombres para otros clasificadores incluyendo clases, interfaces, casos de uso, etc. Los clasificadores anidados son visibles sólo dentro del espacio de nombres de la clase que contiene.

En el siguiente diagrama se muestra la clase anterior con los atributos y métodos agrupados por visibilidad:

SearchService
Private: -config: Configurator -search: SearchEngine
Private: CreateEngine(): SearchEngine Public: search(query: SearchRequest: SearchResult



Una **clase abstracta** se define en UML 1.4.2 como clase que no se puede instanciar directamente.

La clase abstracta existe sólo para otras clases hereden desde y para apoyar la reutilización de las características declaradas por el mismo. Ningún objeto puede ser una instancia directa de una clase abstracta, aunque un objeto puede ser una instancia de una indirecta a través de una subclase que es no abstracta.

UML 2.4 menciona el concepto de clase abstracta en su especificación, pero no proporciona ninguna definición. Probablemente lo que se pretende es relacionar la definición de clasificador abstracto a clase abstracta de tal manera que lo que se aplica a un clasificador abstracto también se puede aplicar a una clase abstracta.

Podemos suponer que en UML 2.x clase abstracta no tiene declaración completa y «normalmente» no se pueden crear instancias.

Una clase abstracta está destinado a ser utilizado por otros clasificadores (por ejemplo, como el objetivo de las relaciones de generalización)

UML 2.4 no proporciona ninguna explicación de la «declaración incompleta de clase» y si está relacionado con el concepto de la operación abstracta - que también estaba presente en UML 1.4.2 y está ausente en UML 2.x.

El intento de crear una instancia de una clase abstracta es indefinido ya que algunos lenguajes pueden realizar esta acción aunque se considere ilegal, otros pueden crear una instancia parcial para fines de prueba.El nombre de una clase abstracta ha de ponerse en cursiva.

Un clasificador abstracto también puede indicarse utilizando la palabra clave “abstract”después o debajo del nombre.Estereotipos de clase UML estándar puede ser las palabras clave «Focus»«Auxiliar»«Tipo» o «Utilidad»

Hay que tener en cuenta, que la convención de nominación de los estereotipos fue cambiado en UML 2.4.1 y ha de tenertener la primera letra en mayúscula. Posiblemente se seguirán viendo estereotipos en minúsculas, por ejemplo, «foco», «tipo», en todas partes, que tomará algún tiempo para cambiar a la nueva notación.



Definición

Focus: focus o foco es una clase que define la lógica de la base o el flujo de control de una o más clases de apoyo.

Las clases de apoyo se pueden definir de forma explícita el uso de clases auxiliares o implícitamente por dependencia relaciones. Estas clases se centran normalmente se utilizan para especificar la lógica de negocio de núcleo o de control de flujo de los componentes durante la fase de diseño.



Auxiliar: auxiliar es una clase que soporta otra clase más central o fundamental, por lo general mediante la implementación de la lógica secundaria o flujo de control.

La clase que los soportes auxiliares se pueden definir de forma explícita el uso de un enfoque de clase o implícitamente por una dependencia de relación. Las clases auxiliares se utilizan normalmente para especificar la lógica de negocio o de control de flujo secundario de los componentes durante la fase de diseño.



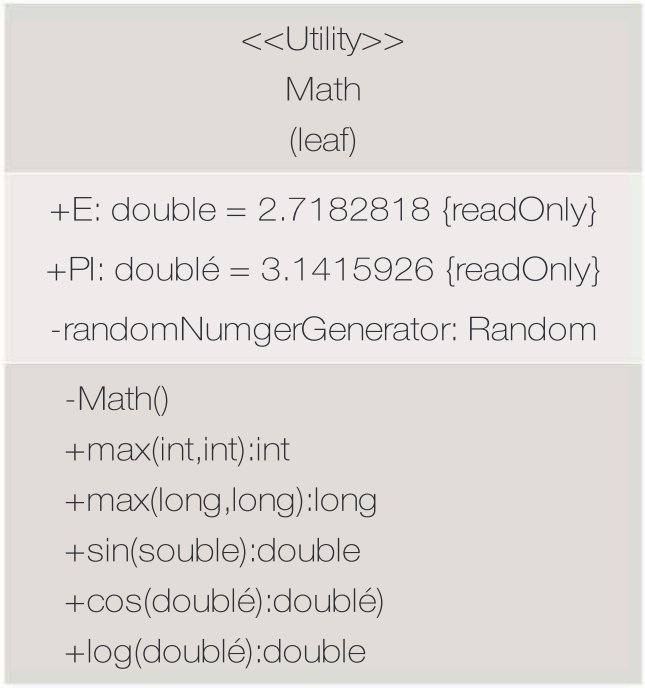
Tipo: tipo es de clase que especifica un dominio de objetos, junto con las operaciones aplicables a los objetos, sin definir la implementación física de esos objetos.

Una clase tipo puede tener atributos y asociaciones. Especificaciones de comportamiento para las operaciones de tipo pueden ser expresadas utilizando, por ejemplo, los diagramas de actividad. Un objeto puede tener como máximo una clase de aplicación, sin embargo, puede adaptarse a varios tipos diferentes.



Utilidad: utilidad es la clase que tiene única clase con ámbito atributos y operaciones estáticas. Como tal, clase de utilidad por lo general no tiene instancias.

Hay varios estereotipos no estándar pero utilizados de forma rutinaria de clase disponibles en varias herramientas UML como «Límite», «Control» «Entidad» Estos estereotipos de clase son parte del perfil de análisis en el modelo Unificado del Rational que se aplica de forma predeterminada durante el análisis y por eso son de uso muy común.



Estos estereotipos corresponden a las partes del patrón de diseño Modelo-Vista-Controlador (MVC), donde "Boundary" representa a la vista, Control es controlador, y la Entidad corresponde al modelo.

Definición

«Límite» o "boundary" es una clase estereotipada u objeto que representa algún límite del sistema, por ejemplo, un objeto de la pantalla de interfaz de usuario, la interfaz del sistema o interfaz de dispositivo.

Podría ser utilizado en el análisis o la fase conceptual del desarrollo para capturar usuarios o sistemas externos que interactúan con el sistema en desarrollo. A menudo se utiliza en los diagramas de secuencia que muestran interacciones del usuario con el sistema.

Boundary se dibuja como un círculo conectado con una línea corta a una línea vertical a la izquierda Podría también mostrarse como una clase con el estereotipo **«Límite»**.



«Control»: el control es una clase estereotipada o un objeto que se utiliza para modelar el flujo de control o algún tipo de coordinación en el comportamiento.

Una o varias clases de control podrían describir realización de casos de uso. Los controles del sistema representan la dinámica del sistema diseñado y por lo general describen algunos «lógica de negocio».

El control se dibuja como un círculo con la flecha incrustada en la parte superior. Se podría también muestra como una clase con el «control» estereotipo.

Conviene subrayar que el estereotipo estándar «Focus **»** es aplicable a estas las clases y podría ser utilizado para especificar la lógica de negocio de núcleo o de control de flujo de los componentes durante el diseño.



«Entidad»: entidad es una clase estereotipada u objeto que representa alguna información o datos, por lo general, pero no necesariamente persistente.

Las clases de tipo entidad se dibujan como un círculo con línea conectada a la parte inferior del círculo. Se podría también muestra como una clase con el estereotipo «Entidad».

Las entidades empresariales representan algunas «cosas», objetos, documentos o información que se maneja, utilizados o transformados por los trabajadores de negocios, mientras que están trabajando en alguna parte de la lógica de negocio de la entidad.

Ejemplos de entidades de negocios son por ejemplo la receta en la consulta del doctor, un menú en el restaurante, o billetes en un aeropuerto.

Las entidades del sistema representan alguna información o datos, por lo general, pero no necesariamente persistente, que es procesada por el sistema. Hay que tener en cuenta que UML tiene el estereotipo «Entidad» aplicable a los componentes y que representa un concepto de negocio de una información persistente.



Plantilla de Clase: Clases UML podrían utilizar plantillas que se substituyen por los correspondientes objetos cuando se realiza la instanciación.

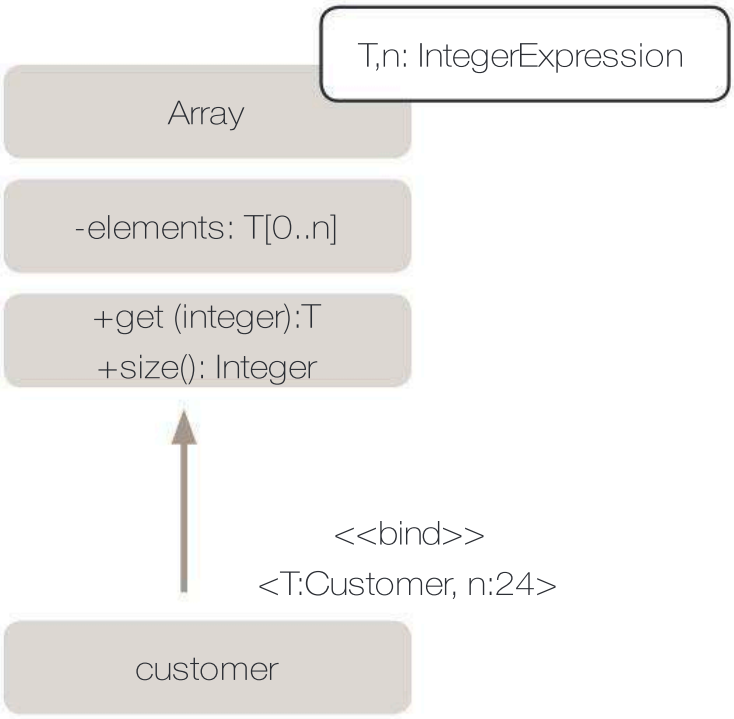
Ejemplo

El siguiente ejemplo muestra la plantilla de la clase Array con dos parámetros de plantilla formales.

El primer parámetro de plantilla T es una clase sin restricciones de parámetros. El segundo parámetro de plantilla, n, es un parámetro de plantilla que debe ser una expresión entera.

La clase Clientes sustituye la clase T no está restringida a la clase Cliente y n límite con valor entero 24 Por lo tanto, Se almacena en el array una lista de clientes limitada a 24 elementos.

En el siguiente diagrama se representa a la clase Array que utiliza como plantilla una clase Customer.



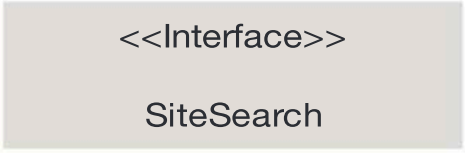
Definición

Interfaz: una interfaz es un clasificador que declara de un conjunto de funciones públicas coherentes y obligaciones para la clase que los implemente.

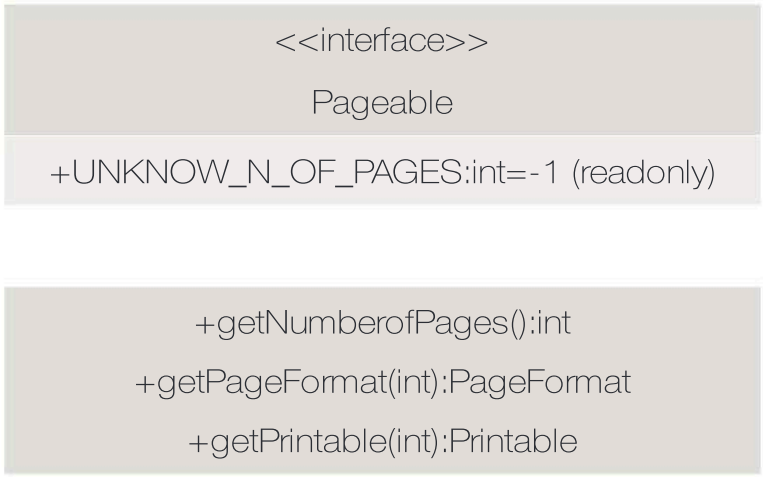
Una interfaz especifica un contrato, es decir, cualquier instancia de un clasificador que utiliza (o implementa) la interfaz debe cumplir ese contrato y por lo tanto proporciona los servicios descritos en el contrato.

En UML 1.4 interfaz era formalmente equivalente a una clase abstracta sin atributos ni métodos y sólo operaciones abstractas. Una interfaz se puede demostrar usando un símbolo rectangular con la palabra clave «interfaz» que precede al nombre

UN ejemplo de interfaz aparece en la siguiente figura:



Las obligaciones que pueden estar asociados con una interfaz están en la forma de diversos tipos de restricciones o especificaciones de protocolo, que puede imponer restricciones sobre las interacciones a través de la interfaz.





Hay que tener muy presente que las interfaces son las declaraciones, de las que no pueden crearse instancias.

En lugar de ello, una especificación de interfaz se implementa mediante una instancia de un clasificador instanciable, lo que significa que el clasificador instanciable presenta una fachada pública que se ajusta a la especificación de dicha interfaz.

Cualquier clasificador dado puede implementar más de una interfaz. Y una interfaz puede ser implementado por un número cualquiera de diferentes clasificadores.

Una asociación entre una interfaz y cualquier otro clasificador implica que una asociación conforme debe existir entre cualquier aplicación de esa interfaz y aquel otro clasificador que la implemente. En particular, una asociación entre

las interfaces implica que una asociación similar debe existir entre las implementaciones de las interfaces.

Las interfaces realizadas o implementadas por un clasificador son sus interfaces proporcionadas, y representan las obligaciones que las instancias de ese clasificador tienen con sus clientes. Por tanto esas describen los servicios que las instancias de esa oferta clasificador a sus clientes.

La interfaz que participa en la realización de una relación de dependencia se muestra como un círculo o bola, etiquetado con el nombre de la interfaz y unido por una línea sólida para el clasificador que se relaciona utilizando esta interfaz.

Una Interfaz necesaria especifica los servicios que un clasificador necesita para realizar su función y cumplir con sus propias obligaciones con sus clientes. Se especifica por una dependencia de uso entre el clasificador y la correspondiente interfaz.

La dependencia de uso de un clasificador por una interfaz se muestra mediante la representación de la interfaz por un medio-círculo o zócalo, etiquetado con el nombre de la interfaz, unida por una línea sólida para el clasificador que requiere esta interfaz.

A menudo es el caso práctico en el que dos o más interfaces están acoplados mutuamente a través de las dependencias específicas de la aplicación. En tales situaciones, cada interfaz representa un papel específico en un multipartidismo «protocolo.» Este tipo de protocolo acoplamientos papel que pueden ser capturadas por las asociaciones entre interfaces.

Una interfaz en UML podría ser utilizado como un espacio de nombres para otros clasificadores incluyendo clases, interfaces, casos de uso, etc. los clasificadores anidados son visibles sólo dentro del espacio de nombres de la interfaz que contiene.



Objeto: objeto es una instancia de una clase o una interfaz. El objeto no es un elemento UML por sí mismo. Los objetos se representan como especificaciones de instancia, por lo general en diagramas de objetos.



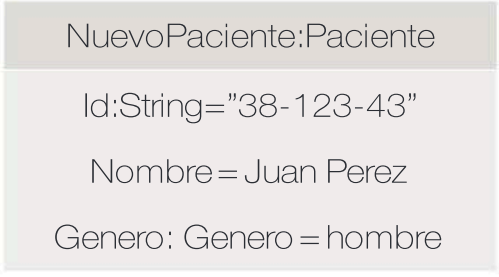
En algunos casos, la clase de la instancia es desconocido o no especificado. Cuando también no se proporciona el nombre de instancia, la notación para tal caso de un clasificador sin nombre es simplemente dos puntos subrayados.



El nombre del objeto puede incluir el nombre de instancia, la clase y espacio de nombres (el paquete) en el que se encuentre. Si una instancia tiene algún valor, se muestra la especificación de valor, ya sea después de un signo de igual (‹=›) tras el nombre de la instancia, o sin el signo igual debajo del nombre.



Los slots o ranuras de las propiedades se muestran como características estructurales con el nombre de la función seguido de un signo de igual (‹=›) y una especificación de valor. I tipo del clasificador de la propiedad puede ser también mostrado en el cuadro.



Definición

Un **tipo de datos** es un clasificador, similar a una clase , cuyas instancias se identifican sólo por su valor.

Ejemplo

Un uso típico de los tipos de datos sería la de representar los tipos de valor de dominio de negocio, tipos primitivos o tipos estructurados de un lenguaje de programación.

Por ejemplo, la fecha / hora, el género, la moneda, la dirección se podrían definir como tipos de datos. Todas las copias de una instancia de un tipo de datos y cualquier instancia de ese tipo de datos con el mismo valor se consideran iguales casos.

<<dataType>>

DateTyme

Un tipo de datos puede contener atributos y operaciones para apoyar el modelado de tipos de datos estructurados. Un tipo de datos se muestra el uso de símbolo rectángulo con la palabra clave «dataType».

Importante

Las instancias de un tipo de datos estructurado se consideran iguales si la estructura es la misma y los valores de los atributos correspondientes son iguales.



Cuando el tipo de datos hace referencia a, por ejemplo, como el tipo de un atributo de clase, se muestra simplemente como el nombre del tipo de datos.

Definición

Tipo Primitivo: un tipo primitivo es un tipo de datos que representa los valores de datos atómicos, es decir, los valores que no tienen partes o estructura interna.

Un tipo de datos primitivo puede tener la semántica y operaciones precisas definidas fuera de UML, por ejemplo, definidos matemáticamente. Los Tipos primitivos UML estándar incluyen:Booleano, Entero, Natural,Cadenas o Strings. Un tipo primitivo se indica por la palabra «primitivo» encima o delante del nombre del tipo.

Las instancias de tipos primitivos no tienen identidad. Si dos instancias tienen la misma representación, entonces son indistinguibles.

Definición

Enumeración: una enumeración es un tipo de datos cuyos valores se enumeran en el modelo como literales de enumeración definidos por el usuario.

Una enumeración se muestra usando la notación clasificador (un rectángulo) con la palabra clave «enumeración **».** El nombre de la enumeración se coloca en el compartimiento superior.

Un compartimento de la lista de los atributos para la enumeración se coloca debajo del compartimiento de nombre. Un compartimento lista las operaciones para la enumeración se coloca debajo del compartimiento de atributo.

Una lista de literales de enumeración se puede colocar, uno a una línea, en el compartimiento inferior. Los atributos y los compartimentos de las operaciones pueden ser suprimidos, y por lo general se suprimen si estarían vacíos.



Definición

Operación: una operación es una característica del comportamiento de un clasificador que especifica el nombre, tipo, parámetros y limitaciones para invocar un comportamiento asociado.

Una Operación se invoca en una instancia del clasificador para el cual la operación es una característica. Si la operación se indica como isQuery **,** la ejecución de la operación sale del estado del sistema sin cambios, aunque se pueden producir efectos secundarios.

Por defecto, El valor predeterminado es falso, por lo que se asume el funcionamiento de una operación tienen efectos secundarios y puede cambiar el estado del sistema.

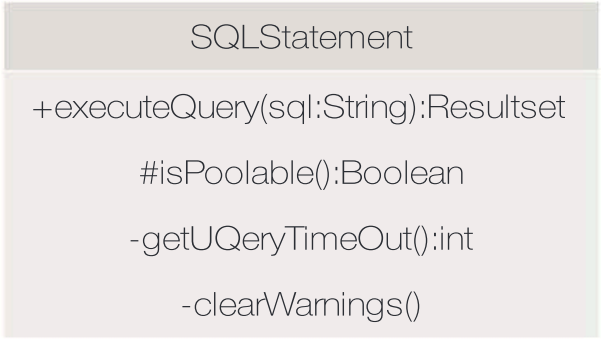
Una operación en UML puede tener como máximo un parámetro de retorno. La operación puede provocar también una excepción durante su invocación. Una operación puede ser redefinida en una especialización del clasificador mostrado.

En esta redefinición puede cambiarse los tipos de los parámetros de propiedad, añadir nuevas condiciones previas o post-condiciones, añadir nuevas excepciones, o de otra manera refinar la especificación de la operación.

Cuando se muestra el funcionamiento de un diagrama, el texto debe ajustarse a la sintaxis definida en la especificación de UML.

La sintaxis sigue la siguiente especificación: operación :: = [visibilidad] rúbrica [Propiedades]. La Visibilidad de la operación es opcional, y si está presente, debe ser uno de estos valores: ‹+› | ‹-› | ‹#› |‹~›

En la imagen, la operaciónexecuteQuery es pública, isPoolable, protegida, getQueryTimeout con visibilidad de paquete y clearWarnings es privada.



La rúbrica de la operación contiene la lista de parámetros opcionales y especificación de retorno, según la siguiente definición:

rúbrica:: = nombre ‹(‹ [parámetro-lista] ‹)› [‹:› retorno]

Nombre es el nombre de la operación. La lista de parámetros es una lista de los parámetros de la operación en el siguiente formato:

Lista-parametros:: = parámetro [‹,› parámetro] *

parámetro :: = [dirección] Parm-nombre ‹:› tipo-expresión [‹[‹ multiplicidad ‹]›] [‹=› default] [Propuedades_parametro.

Parm-nombre es el nombre del parámetro y tipo-expresión es una expresión que especifica el tipo de parámetro.

El parámetro de multiplicidad nos sirve para indicar la multiplicidad del parámetro. El valor predeterminado es una expresión que define la especificación de valor para el valor predeterminado del parámetro. A continuación se muestran imágenes de ejemplo:

File
+getName():String
+créate(parent:String, child:String):File
+listFiles:File[0..*]
-slahify(path:String, isDir:Boolean):String

File tiene dos operaciones estáticas, create y slashify. Create cuenta con dos parámetros y devuelve un File.

Slashify es una operación privada. La Operación listFiles devuelve un array de archivos. Las operaciones getName y listFiles pueden o bien no tener parámetros o bien sus parámetros ha sido suprimidos. El tipo de parámetro se indica como uno de los siguientes tipo:: = ‹in› | ‹out› | ‹inout› | ‹return› y por defecto es in en el caso de que se omita.

Las propiedades de parámetros describen los valores de propiedad adicionales que se aplican al parámetro. parm-propiedades :: = ‹{‹ Parm-propiedad [‹,› Parm-propiedad] * ‹}›

La Especificación de retorno si presente se define como:

Especificación de retorno:: = [tipo de retorno] [‹[‹ multiplicidad ‹]›]

Tipo de valor devuelto es el tipo del resultado, si se ha definido para la operación. Especificación de retorno también tiene multiplicidad opcional dentro del tipo de retorno.

Thread
+setDaemon(in isDaemon: Boolean)
+changeName(inout name: char[0..*]
+enumerate(out treads:Thread[0..*]):int
+isDaemon(return: Boolean)

En la imagen, la Operación setDaemon tiene un parámetro de entrada, mientras que el parámetro de changename es a la vez parámetro de entrada y de salida.

Definición

Propiedad UML: una propiedad es una característica estructural que podría representar un atributo de un clasificador, o un extremo miembro de la asociación, o una parte de un clasificador estructurado.

Como una característica estructural, la propiedad representa una parte llamada de la estructura de un clasificador. Por ejemplo, la clase paciente podría tener id, nombre, sexo, fecha de nacimiento, etc como sus propiedades.

Paciente
Id:String
Nombre:String
Sexo:Genero



Cuando se crea una instancia de un clasificador, cada propiedad no estática se convierte en una parte del estado de esa instancia, y se implementa por algún método de mapeo del nombre de la propiedad a un valor específico o valores que el estado de los que la instancia se compone.

Los valores de cada propiedad tienen tipo específico y se asignan en los slots o ranuras de la instancia o asociados a esa instancia. Por ejemplo:

nuevoPaciente:Paciente
Id:String =”345-54-567”
Nombre:String = “Juan Perez”
Sexo:Genero = masculino

La sintaxis general de propiedades se muestra a continuación:

– propiedad :: = [visibilidad] [‘/’] nombre-propiedad [‘:’ -tipo de propiedad] [‘[‘ multiplicidad ‘]’] [‘=’ default-valor] [propiedad modificadores]

– Visibilidad :: = ‘ + ‘|’ ~ ‘|’ # ‘|’ - ‘

– propiedad modificadores :: = ‘{‘ propiedad modificador [‘,’ propiedad modificador] * ‘}’

– propiedad modificador :: = ‘id’ | ‘soloLectura ‘|’ ordenado ‘| (‘ ss ‘|’ secuencia ‘) |’ única ‘|’ no único ‘|’ unión ‘

La visibilidad de la propiedad es opcional. Hay que tener en cuenta en cuenta que no existe un visibilidad predeterminada.

Además, la visibilidad puede ser suprimida cuando se muestre en un diagrama, incluso si tiene algún valor en el modelo (por ejemplo, almacenado por la herramienta de UML). Así que, si la visibilidad no se muestra en un diagrama, puede ser que o bien no está especificada o esta ha sido suprimida.

La barra inclinada ‹/› significa que la propiedad se deriva. UML ofrece una definición vaga de propiedades derivadas, indica que son así cuando **su valor o valores se pueden calcular a partir de otra información.**



El tipo de propiedad está representada por el nombre del clasificador. Una Propiedad podría tener multiplicidad. Los límites multiplicidad limitan el tamaño de la colección de valores de propiedad. Por defecto, el máximo consolidado es de 1.

El valor por defecto- opción es una expresión para el valor por defecto o valores de la propiedad.

La propiedad puede tener modificadores opcionales:

Modificador	Descripción
Identificación del	La propiedad es parte del identificador para la clase propietaria de la propiedad.
readOnly	Propiedad de sólo lectura (isReadOnly = true).
ordenado	Propiedad es ordenada (isOrdered = true).
única	Propiedad con varios valores no tiene valores duplicados (isUnique = true).
No unica	Multi-valorada propiedad puede tener valores duplicados (isUnique = false).
secuencia (o siguientes)	La propiedad es una bolsa ordenado (isUnique = false y isOrdered = true).
unión	La propiedad es una unión derivada de sus subconjuntos.

redefine nombre-propiedad	Propiedad redefine una propiedad con nombre heredado nombre-propiedad.
subconjuntos nombre-propiedad	La propiedad es un subconjunto de la propiedad denominada nombre-propiedad.
propiedad en restricciones	Una restricción que se aplica a la propiedad



Atributo: la especificación UML tiene definido atributo como una ranura llamada dentro de un clasificador que describe un rango de valores que las instancias del clasificador contendrán en el.

En UML 2.xa una propiedad poseída por un clasificador representa un atributo del clasificador. El este Contexto el atributo es propiedad del clasificador.

Por ejemplo, la clase paciente puede tener id, nombre, sexo, fecha de nacimiento, etc, como sus atributos.

Paciente
Id : String
Nombre : String
Sexo : Genero
Fecha_nacimiento : DateTime
Direción : Address
Visitas: Visists[1..*]

La Especificación UML describe muy vagamente la propiedad atributo (no estático) como algún tipo de relación cuando el valor o valores relacionados con la instancia del clasificador se encuentran incluidos en las ranuras de la instancia.



Definición

Slot o ranura es elemento UML que indica que un caso determinado tiene un valor o valores de una característica estructural específica.

La especificación UML también dice que una ranura da el valor o valores de una característica estructural de la instancia. Una instancia va a tener una ranura por característica estructural de sus clasificadores, incluyendo las características heredadas.

En programación llamaríamos a este tipo de mapeo como ranuras de memoria o almacenamiento. Por tanto, atributos del clasificador se almacenan en las ranuras de dicho clasificador.

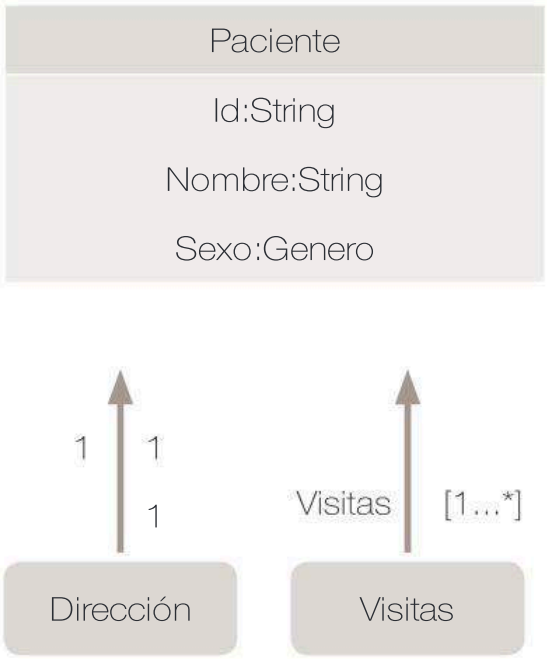
Cuando se crea una instancia de un clasificador, cada propiedad no estática se convierte en una parte del estado de esa instancia, y se implementa por algunos mapeo del nombre de la propiedad a un valor específico o valores de los que el estado de la instancia se compone.

Los valores de cada propiedad tienen tipo específico y se asignan en las ranuras de la instancia o asociados a esa instancia.

nuevoPaciente:Paciente
Id : String =”345-54-567”
Nombre : String = “Juan Perez”
Sexo : Genero = masculino

Las especificaciones y las guías de usuario de las versiones 1.x consideradas atributo como un acceso directo o equivalente semántico de la composición.

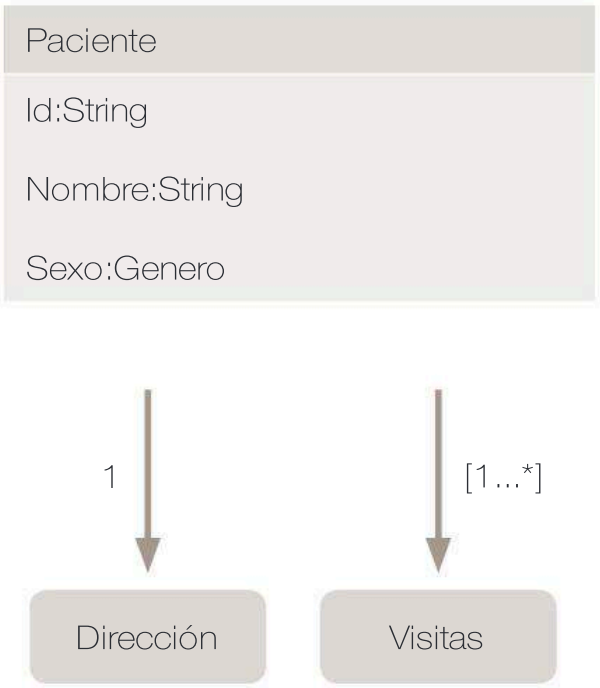
En UML 1.x también se denomina a esto “maneras diferentes de mostrar la composición”, por lo que el ejemplo de la clase de Pacientes con atributos que se muestran arriba podría ser también presentado utilizando la composición en lugar de algunos atributos.



Por ejemplo, UML 1.4.2 indica que “hay que tener en cuenta que un atributo es semánticamente equivalente a una asociación de composición; sin embargo, la intención y uso es normalmente diferente”.El símbolo de la clase puede ser visto como un ejemplo de la notación de composición por anidación (con algunas propiedades especiales de composición).

Sin embargo, la notación atributo subordina los atributos fuertemente dentro de la clase; Por lo tanto, se debe utilizar cuando la estructura y la identidad del atributo propios objetos no es importante fuera de la clase

Las especificaciones 2.x UML permiten mostrar un atributo usando la notación de asociación, por lo que ese ejemplo anterior de la clase paciente podría ser traducida en UML 2.5, como se muestra a continuación.



Hay que tener en cuenta, que la propiedad de la asociación termina por la clase Paciente asociada se indica gráficamente por unos pequeños círculos rellenos.

Definición

Asociación final o extremo: cuando una propiedad es un extremo de la asociación, el valor o valores de la propiedad están relacionados con la instancia o las instancias en el otro extremo (s) de la asociación.

Una propiedad de una asociación representa un atributo de la asociación y se califica como miembro final. En el caso de un extremo de la asociación el contexto es el conjunto de tipos en el otro extremo o extremos de la asociación.

El final de una asociación puede ser propiedad ya sea por pertenecer al clasificador, o a la asociación en sí. El estándar UML no impone el uso de la notación de la propiedad final explícito, pero define un valor predeterminado.

Cuando la notación de puntos no se utiliza en asociación binaria, se supone extremos de asociación que se poseído por asociación.

Atributo clasificador puede representar un extremo de la asociación cuando este extremo de la asociación es de propiedad del clasificador. Dependiendo del contexto en el que se encuentre la asociación esto se interpretará de una manera u otra.

La propiedad de la asociación termina en un clasificador asociado puede estar indicada gráficamente por un pequeño círculo relleno o "punto".

El punto se dibuja en el punto donde la línea se encuentra con el clasificador. Podría interpretarse como muestra de que el modelo incluye una propiedad del tipo representado por el clasificador indicado por el punto. Esta propiedad pertenece al clasificador del otro extremo.



qbuilderquery

Definición

Calificador: una propiedad puede tener otras propiedades (atributos) que sirven como clasificatorios.

Un calificador es una propiedad que define una partición del conjunto de instancias asociadas con respecto a un ejemplo en el extremo cualificado.

Los calificadores se utilizan para modelar mapas de hash en Java, los diccionarios en C #, tablas de **índice,** etc., donde el acceso rápido a objeto (s) relacionado se proporciona mediante calificador como clave hash, argumento de búsqueda o índice.

Un calificador se muestra como un pequeño rectángulo unido al clasificador cualificado también llamada la fuente. Este rectángulo calificador es parte de la asociación y no es parte del clasificador. UML no permite calificador a ser suprimida.

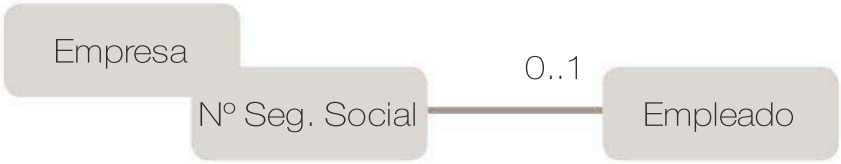
Uno o varios atributos del calificador se dibujan dentro del cuadro, uno en cada línea. Los atributos de calificador tienen la misma notación que atribuye clasificador, excepto que los valores iniciales no se utilizan en este caso.

UML permite tener un partido de clasificación en cada extremo de una sola asociación, es lo que Multiplicidad de objetivo o de Target

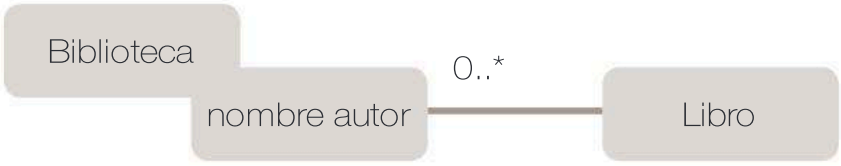
Dado un objeto cualificado y una instancia de clasificación, el número de objetos en el extremo destino de la asociación se ve limitada por la declarada multiplicidad del objetivo.

La multiplicidad del objetivo debe permitir 0 el valor "no hay resultados" si dominio de la búsqueda o clasificación es demasiado grande o infinito, por ejemplo, número, de cadena larga, la fecha. También puede ser exactamente 1 para los calificadores con dominio enumerado o restringido.

En el caso en el que la multiplicidad de destino sea 0..1, el valor calificador es único con respecto al objeto cualificado, y los enlaces a lo sumo contendrán un objeto asociado.



Por ejemplo, Dada una empresa y un número de seguridad social, como máximo un empleado se podrá encontrar. En el caso de multiplicidad 0.. *, el conjunto de instancias asociadas se divide en subconjuntos, cada uno seleccionado por una instancia de clasificación dado



Por ejemplo imaginemos que dada una biblioteca y el nombre del autor de varios libros no se pudo encontrar ninguno de esos libros al hacer un a búsqueda sobre el catalogo de la biblioteca, como se representa en la imagen.



Por ejemplo, Dado un tablero de ajedrez y rango específico y el archivo que vamos a localizar exactamente 1 cuadrado.

La multiplicidad en UML permite especificar cardinalidad - es decir, el número de elementos - de alguna colección de elementos. Los elementos de la multiplicidad define alguna colección de elementos, e incluye tanto la multiplicidad, así como la especificación del orden y la singularidad de los elementos de la colección.

Definición

Multiplicidad: la multiplicidad es una definición de cardinalidad, es decir, el número de elementos, de alguna colección de elementos, proporcionando un intervalo inclusivo de enteros no negativos para especificar el número permitido de instancias de elemento descrito.

El intervalo de multiplicidad ha de tener alguna cota inferior y límite superior, aunque este puede ser infinito y se denota por un asterisco (*)

Los límites inferior y superior podrían ser constantes naturales o expresiones constantes evaluados a un número natural (no negativo). El Límite superior puede ser también especificado como un asterisco ‹*› que denota número ilimitado de elementos. El límite superior debe ser mayor que o igual que el límite inferior.

Hay que tener en cuenta, que en la especificación la multiplicidad que describe no es muy consistente. Por ejemplo, se dice que el límite inferior debe ser un número entero no negativo literal, y más tarde - que puede ser especificado por la especificación de valor, tales como una constante libre de efectos secundarios o una expresión.

Otra cuestión es que asterisco como límite superior en un lugar significa ilimitada (y no infinito) número de elementos, mientras que en la otra se puede leer el (posiblemente infinita) límite superior.

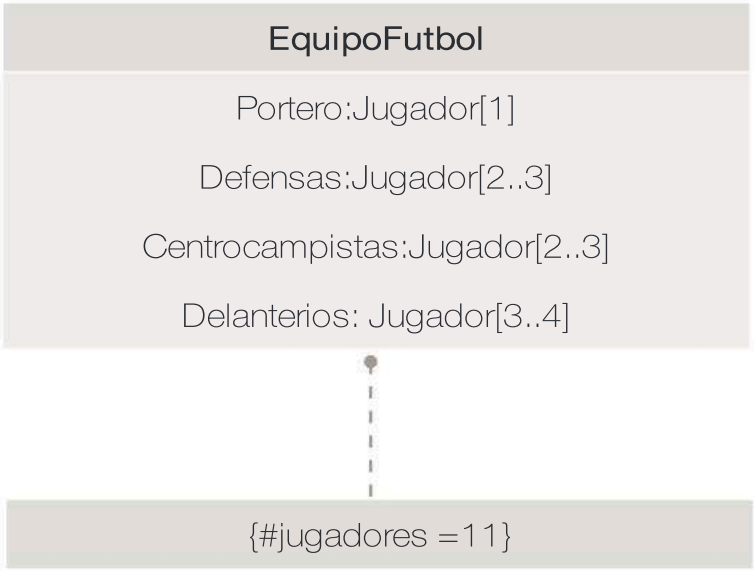
Por ejemplo en UML se expresarían igual las dos frases: «Un médico puede tener un número ilimitado de los pacientes.» y «Un médico puede tener infinitos pacientes.»

Algunos ejemplos típicos de multiplicidad se muestran en el siguiente esquema:

Multiplicidad	Opción	Cardinalidad
0..0	0	Colección debe estar vacío
0..1		No hay instancias o una instancia
1..1	1	Exactamente un ejemplo
0.. *	*	Cero o más instancias
1.. *		Al menos en un caso
5..5	5	Exactamente 5 casos
m..n		Al menos m pero no más de N instancias



Si la multiplicidad se asocia con un elemento cuya notación es una cadena de texto, tal como un atributo de una clase, el rango de multiplicidad se coloca dentro de corchetes como parte de esa cadena de texto.



Definición

La visibilidad en UML: visibilidad permite restringir el uso de un elemento llamado, ya sea en los espacios de nombres o en el acceso al elemento.

Se utiliza con las clases, paquetes, generalizaciones, elemento de importación y paquetes de importación.UML cuenta con los siguientes tipos de visibilidad :pública,paquete,protegida y privado

Hay que tener en cuenta que si un elemento con nombre no es propiedad de ningún espacio de nombres, entonces no tiene visibilidad

Un elemento público es visible a todos los elementos que pueden acceder a los contenidos del espacio de nombres que lo posee. Visibilidad Pública es representado por literal ‹+›.

Un elemento con visibilidad paquete es propiedad de un espacio de nombres que no puede ser un paquete, y es visible a los elementos que se encuentran en el mismo paquete al que pertenece su espacio de nombres.

Sólo elementos con nombre que no son propiedad de los paquetes se pueden marcar para tener visibilidad de paquete.

Cualquier elemento marcado como que tiene visibilidad de paquete es visible para todos los elementos dentro del paquete que encierra más cercano (ya que otros elementos que poseen tienen una visibilidad adecuada).

Fuera del paquete envolvente más cercano, un elemento marcado como tener visibilidad de paquete no es visible. La visibilidad de paquete es representado por ‹~›. Un elemento protegido elemento es visible a los elementos que tienen una relación de generalización al espacio de nombres que lo posee. Esta visibilidad está representado por el símbolo›#›.

Un elemento privado elemento sólo está visible en el interior del espacio de nombres que lo posee. Esta visibilidad está representado por ‹-›.

SQLStatement
+executeQuery(sql:String):Resultset
#isPoolable():Boolean
-getUQeryTimeOut():int
-clearWarnings()

Si algún elemento con nombre parece tener múltiples visibilidades, por ejemplo, al ser importados en múltiples ocasiones, la visibilidad pública anula visibilidad privada.

Si un elemento se importó dos veces en el mismo espacio de nombres, una vez mediante una importación pública y otra vez utilizando una importación privada, el resultado de la visibilidad es pública.



Una restricción es un elemento que representa alguna condición, restricción o afirmación relacionada con algún otro elemento (que se denomina propietario de la restricción) o varios elementos.

Una restricción por lo general se especifica mediante una expresión booleana que debe evaluar a un verdadero o falso.

La restricción, cuando es definida, debe ser satisfecha (o sea evaluada a verdadero) para un correcto diseño del sistema. Las limitaciones y restricciones son de uso general para los diversos elementos en los diagramas de clases.

En general hay muchos tipos posibles de los propietarios de una restricción. Ser propietario de elemento implica tener acceso a los elementos necesarios para verificar la restricción. El propietario de la restricción determinará cuando se evalúa la restricción.

Por ejemplo, una operación puede tener una pre-condición y / o una post-condición. Las restricciones podría tener un nombre opcional, aunque normalmente es anónimo.

Una restricción se muestra como una cadena de texto entre llaves de acuerdo con la siguiente sintaxis:

restricción:: = ‹{‹ [nombre›:›]{Expresion booleana›}›

La Especificación UML no restringe el lenguaje que podrían ser usados para expresar restricción. Algunos ejemplos de lenguajes de restricción son:OCL, Java, lenguaje natural,etc.

Definición

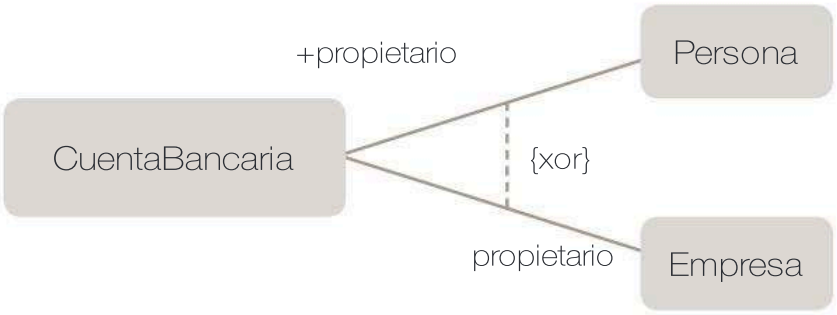
OCL es un lenguaje de restricción predefinidos en UML pero si alguna herramienta UML se utiliza para dibujar diagramas, cualquier lenguaje de restricción con el apoyo de esa herramienta se podría utilizar.

Para un elemento cuya notación es una cadena de texto (por ejemplo, una clase de atributos), la cadena de restricción puede seguir la cadena de texto elemento entre llaves.

CuentaBancaria
+propietario:String {propietario ->notEmpty()}
+balance:Number {Balance >=0}

Para una restricción que se aplica a un solo elemento (tal como una clase o una ruta de asociación), la cadena de restricción puede ser colocado cerca del símbolo para el elemento, preferiblemente cerca del nombre, si hubiere.

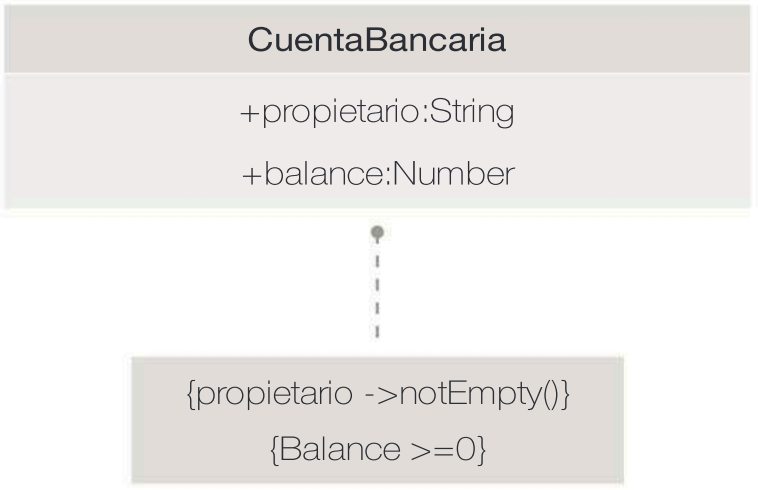
Para una restricción que se aplica a los dos elementos (por ejemplo, dos clases o dos asociaciones), la restricción se puede mostrar como una línea de trazos entre los elementos marcados por la cadena de restricción entre llaves.



Si la restricción se muestra como una línea discontinua entre dos elementos, a continuación, una punta de flecha puede ser colocado en un extremo.

La dirección de la flecha es la información relevante dentro de la restricción. El elemento a la cola de la flecha se asigna a la primera posición y el elemento a la cabeza de la flecha se asigna a la segunda posición en la colección.

Si hay tres o más trayectorias en la misma clase (tales como caminos o caminos de generalización de asociación), la restricción puede estar unido a una línea de trazos que cruza todos los caminos. La cadena de restricción puede ser colocado en una nota (el mismo que el utilizado para los comentarios) y unido a cada uno de los símbolos de los elementos limitados por una línea discontinua.





Asociación UML: la asociación es una relación entre clasificadores que se utilizan para mostrar que los casos de los clasificadores pueden ser ya sea vinculados entre sí o combinada lógica o físicamente en alguna agregación.

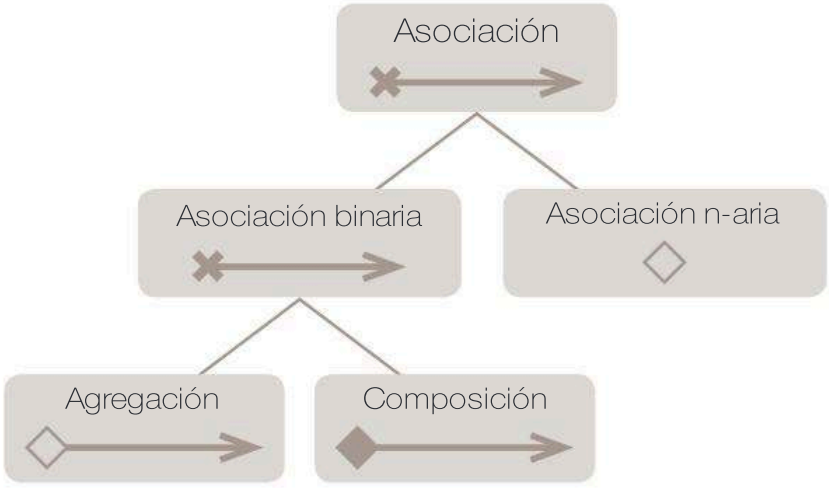
La Especificación UML categoriza a la asociación como una relación semántica. Algunas otras fuentes UML también categorizan asociación como una relación estructural. La asociación es una relación **a nivel de instancia y las asoci**aciones sólo pueden ser mostradas en los diagramas de clases.

Sin embargo siguiendo explícitamente la especificación UML la asociación podría ser utilizado en diferentes tipos de diagramas de estructura UML, no solamente en diagramas de clases. De hecho se puede utilizar en casi todos los diagramas de UML ya que la especificación no prohíbe en absoluto.

La especificación 2.4 de UML establece que en el caso de la definición de una asociación hay que tener en cuenta que el tipo de agregación, la navegabilidad y la propiedad final de la asociación deben ser considerados como conceptos ortogonales, lo que es en muchísimos casos una estricción demasiado severa sobre las propiedades de la asociación..

Ortogonal realmente significa totalmente independiente. Es decir, que un término debe ser considerado de manera completamente independiente a los otros dos. Mientras la notación para el tipo de agregación , la navegabilidad así como la propiedad final de la asociaciónpodría aplicarse de forma independiente, los propios conceptos en sí mismos no son ortogonales, sino que están relacionados, ya que en muchas ocasiones una navegabilidad condiciona la propiedad final que debe tener la asociación.

Por ejemplo, en UML 2.4 propiedad final de asociación de una clase final es navegable, lo que hace que la asociación claramente dependiente de la propiedad de navegabilidad.



Una asociación se dibuja generalmente como una línea continua que conecta dos clasificadores o un solo clasificador a sí mismo. El nombre de la asociación puede ser mostrado en algún lugar cerca de la mitad de la línea de asociación, pero no demasiado cerca de cualquiera de los extremos de la línea.

Cada extremo de la línea podría ser dibujado con el nombre del extremo de la asociación.



El extremo de Asociación es una conexión entre la línea que representa una asociación y el icono que representa el conectado al clasificador.

El nombre del extremo de asociación se coloca cerca del final de la línea. El nombre de extremo de asociación se refiere comúnmente como rol o papel (pero no se define como tal en el estándar UML 2.4). El nombre es opcional y suprimible.

La idea de la función es que el mismo clasificador puede reproducir los mismos o diferentes papeles en otras asociaciones. Por ejemplo, el profesor puede ser un autor de algunos libros o un editor.

Una asociación final podría ser propiedad ya sea por el clasificador final, o la asociación en sí. La asociación acaba de asociaciones con más de dos extremos deben ser propiedad de la asociación.

La propiedad de la asociación termina por un asociado clasificador puede estar indicada gráficamente por un pequeño círculo lleno o punto.

El punto se dibuja en el punto donde la línea se encuentra con el clasificador. Esto debe de interpretarse como muestra de que el modelo incluye una propiedad del tipo representado por el clasificador tocado por el punto. Esta propiedad pertenecerá al clasificador en el otro extremo.



qbuilderquery

El punto "propiedad" se puede utilizar en combinación con las otras anotaciones en rutas de acceso de línea gráfica de las propiedades de las asociaciones y de terminal de asociación. Estos incluyen el tipo de agregación y navegabilidad.

El Estándar UML no impone el uso de la notación de la propiedad explícito, sino que define una notación que se aplicarán en modelos en los que se elige dicho uso.

Definición

La **agregación** es una asociación que representa una relación todo / parte. Hay diferentes tipos de agregación: La agregación propiamente dicha, o agregación compartida y la composición.

La agregación o también agregación compartida es una forma «débil» de la agregación cuando copia de una parte es independiente del compuesto: la misma parte (compartida) podría incluirse en varios materiales compuestos, y si se elimina el elemento compuesto, todavía pueden seguir existiendo partes compartidas.

Agregación compartida se muestra como asociación binaria dibujada con un hueco de diamante como un terminal en el extremo agregado de la línea de asociación. El diamante debe ser notablemente menor que la notación de diamantes para las asociaciones n-arias.



Una composición o agregación compuesta es una forma "fuerte" de agregación.

Los Requisitos de composición son las características enumeradas en la especificación UML son los siguentes:

– Se de detratar de una relación de todo / parte.

– Debe una asociación binaria, es decir, solamente pueden intervenir dos clases en ella.

– Las partes podría incluirse en como máximo un elemento compuesto o conjunto a la vez.

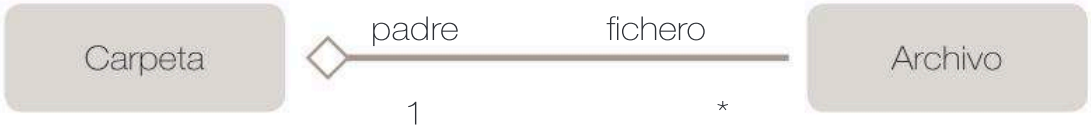
– En el caso de que se elimine un elemento compuesto (todo), todas sus componentes son «generalmente» eliminadas con él.

Importante

Hay que tener en cuenta que UML no define cómo, cuándo y el orden específico en el que se crean partes del compuesto.

También, en algunos casos, una parte puede retirarse de un material compuesto antes de eliminar el material compuesto, y así no es necesariamente elimina como parte del compuesto.

Una composición es representado como una asociación binaria decorado con un diamante negro lleno al final del conjunto agregado.



En el ejemplo, una carpeta puede contener muchos archivos, mientras que cada archivo tiene exactamente una carpeta. Si se elimina la carpeta, todos contenían archivos se eliminan también. Este es un ejemplo clásico de una composición UML, ya que los ficheros no pueden estar almacenados en varias carpetas simultáneamente.

Definición

Clase Asociación: una asociación puede ser refinada para tener su propio conjunto de características ; es decir, características que no pertenecen a ninguno de los clasificadores conectados sino más bien a la propia asociación. Esta asociación se denomina una clase de asociación.

Es a la vez una asociación ya que es la conexión de un conjunto de clasificadores y una clase, y como tal podría tener características y podría ser incluido en otras asociaciones. Una clase de asociación puede ser visto como una asociación que también tiene propiedades de la clase, o como una clase que también tiene propiedades de asociación.

Una clase de asociación se muestra como un símbolo de la clase unida a la vía de la asociación mediante una línea discontinua. La ruta de asociación y el símbolo de la clase asociación representan el mismo elemento del modelo subyacente, sobre el cual tiene un nombre.

El nombre de la asociación puede ser colocado en la línea que une ambos elementos, en el símbolo de la clase, o en ambos, pero debe ser el mismo nombre.

Definición

Enlace: un enlace o link es una instancia de una asociación. El enlace se considera una tupla con un valor para cada extremo de la asociación, donde cada valor es una instancia del tipo del final de la asociación.

Una asociación debe de tener al menos dos extremos, representados por propiedades (propiedades finales).Los enlaces se representa utilizando la misma notación que para una asociación general.

La línea continua conecta a estos en lugar de a clasificadores. Nombre de la relación podría mostrarse subrayado aunque no es necesario. Nombres finales (funciones) y las flechas de navegación se pueden indicar también.

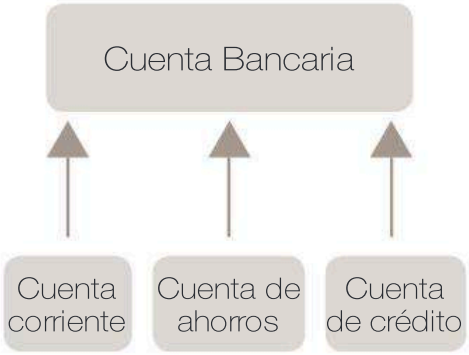


Definición

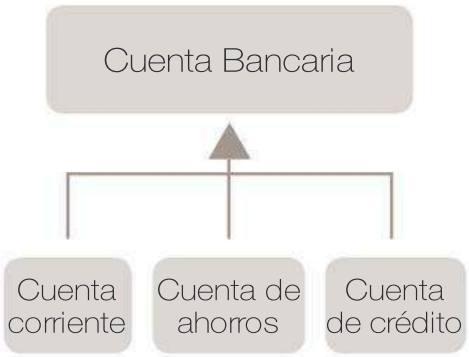
Una **generalización o herencia**es una relación binaria es decir, relacionada con la clasificación entre un tipo más general de un clasificador o superclase y un clasificador más específico o subclase.

Cada instancia del clasificador específico es también una instancia indirecta del clasificador general, por lo que podemos decir “Paciente es una persona”, “cuenta de ahorros es una cuenta”,

Una generalización se muestra como una línea con un triángulo hueco como una punta de flecha entre los símbolos que representan los clasificadores involucrados. La punta de flecha señala al símbolo que representa el clasificador general.



Las relaciones de generalización que hacen referencia a la misma clasificador general, también pueden conectarse entre sí con lo que se denomina “estilo delinea destino compartido.»



Definición

Herencia: en Análisis y diseño Orientado a Objetos, la herencia se define generalmente como un mecanismo por el cual las clases más específicas (llamadas subclases o clases derivadas) incorporan la estructura y comportamiento de las clases más generales (llamados superclases o clases base).

La herencia se explica en UML 1.4.2 usando los conceptos de un descriptor completo y un descriptor de segmento.



Un **descriptor** completo contiene una descripción de todos los atributos, asociaciones, operaciones, y las limitaciones que el objeto contiene, y es normalmente implícito porque está construido de segmentos incrementales combinados entre sí mediante herencia.

En un lenguaje orientado a objetos, la descripción de un objeto se construye fuera de los segmentos adicionales que se combinan utilizando la herencia para producir un descriptor completo de un objeto.



Los **segmentos** son los elementos de modelado que en realidad se declaran en un modelo. Incluyen elementos tales como la clase y otros elementos generalizables.

Cada elemento generalizable contiene una lista de características y otras relaciones que se suma a lo que hereda de sus antecesores. Cada tipo de elemento generalizable tiene un conjunto de características heredables. Para cualquier elemento del modelo, estos incluyen restricciones

Para clasificadores, estos incluyen características (atributos, operaciones, recepciones de señales y métodos) y la participación en asociaciones.

Si un elemento generalizable tiene más de uno de los padres (herencia múltiple), a continuación, su descriptor completo contiene la unión de las características de su propio segmento descriptor y los descriptores de segmentos de todos sus predecesores.

Los Atributos en UML 1.4 no podían ser redefinidos, sin embargo un método pueden ser declarados en más de una subclase. Un método declarado en un segmento reemplaza y sustituye un método con la misma firma declarada en cualquier antepasado.

Las especificaciones UML 2.x indican que con la generalización y especialización, un clasificador hereda todas las características del clasificador más general. Además, cualquier restricción que haya que aplicar a los objetos de el clasificador general, también se aplica a las instancias de la clasificador específico.

UML 2.5 ofrece una explicación un tanto vaga e incompleta de cómo trabaja la herencia en UML, la especificación indica que Cuando un clasificador es generalizado, algunos miembros de sus generalizaciones son heredados, es decir que se comportan como si estuvieran definidos en el propio clasificador del que heredera.

Por ejemplo, un miembro heredado que sea un atributo tiene un valor o conjunto de valores en cualquier instancia del clasificador heredar, y un miembro heredado que sea una operación puede ser invocado en una instancia del clasificador heredera.

Definición

Herencia Múltiple: la herencia múltiple se permite implícitamente estándar UML, mientras que la norma no proporciona ninguna definición de lo que es.

Un clasificador en UML puede tener cero, una o muchas relaciones de generalización a los clasificadores más generales. En Análisis y diseño Orientado a Objetos, la herencia múltiple se refiere a la capacidad de una clase a heredar comportamientos y características de más de una superclase.



Ejemplo de herencia múltiple

Definición

Dependencia en UML: dependencia es una relación dirigida que se utiliza para mostrar que algún elemento UML o un conjunto de elementos requiere, necesita o depende de otros elementos de modelo para la especificación o aplicación.

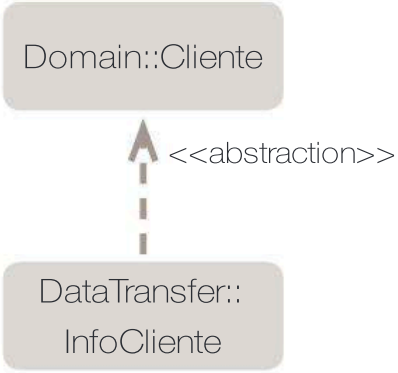
Debido a esto, la dependencia se denomina relación cliente-proveedor, donde el proveedor ofrece algo para el cliente, y por lo tanto el cliente es, en cierto sentido dependiente estructuralmente en el elemento o elementos del proveedor.

Hay que tener en cuenta que una modificación del proveedor puede afectar los elementos de cliente.



Definición

Abstracción en UML: abstracción es una relación de dependencia que relaciona dos elementos o conjuntos de elementos(llamados cliente y proveedor) que representa el mismo concepto pero en diferentes niveles de abstracción o desde diferentes puntos de vista.



Por ejemplo, una clase de nivel de análisis podría servir como una abstracción de una o varias clases de nivel de diseño. Caso de uso podría ser la abstracción para la colaboración.

Una relación de abstracción se muestra como una flecha de dependencia desde el cliente en la cola para el proveedor en la punta de flecha, con la «abstracción»palabra clave o algún otro nombre estereotipo predefinido unido a. Podría ser pegadizo para decidir qué elemento es el cliente y cuál es el proveedor.

En general,el cliente depende de proveedor, y es una convención común en el Análisis y diseño orientado a objetos que para la relación abstracción se ha de tener el elemento más abstracto como proveedor.

Sin embargo, el modelador puede decidir que para algún dominio o tarea que es más apropiado tener un dependiente en el elemento más específico elemento más abstracto.

Por ejemplo, un cliente de clase de nivel de análisis (proveedor) puede ser implementado como clase de nivel de diseño como InfoCliente (cliente).

Definición 

La interfaz de realización es una relación de realización especializada entre un clasificador y una interfaz. Esta relación significa que el clasificador se ajusta a la realización de contrato especificado por la interfaz.

Un clasificador que implementa una interfaz especifica los casos que se ajusten a la interfaz y para cualquiera de sus ascendientes.

Importante 

Un clasificador puede implementar una o varias interfaces.

El conjunto de interfaces implementadas por el clasificador son sus interfaces proporcionadas. Las Interfaces proporcionadas describen el conjunto de servicios que el clasificador ofrece a sus clientes.

Un clasificador implementar una interfaz compatible con el conjunto de características que posee la interfaz. Además de apoyar las funciones, un clasificador debe cumplir con las limitaciones de propiedad de la interfaz.

Una relación de interfaz de realización entre un clasificador y una interfaz implica que el clasificador admite el conjunto de características de propiedad de la interfaz, y cualquiera de sus interfaces padres.

Para las características de comportamiento, el clasificador tendrá la aplicación de una operación o de recepción para cada operación o recepción, respectivamente, definida por la interfaz.

Para las propiedades, el clasificador realización proporcionará funcionalidad que mantiene el estado representado por la propiedad.

Aunque tal puede hacerse mediante mapeo directo a una propiedad del clasificador, también puede ser apoyado por la máquina de estado del clasificador o por un par de operaciones que apoyan la recuperación de la información de estado y una operación que cambia la información de estado.

La interfaz de realización dependencia de un clasificador a una interfaz se muestra mediante la representación de la interfaz por un círculo o bola, etiquetado con el nombre de la interfaz y unido por una línea sólida para el clasificador que se da cuenta de esta interfaz.



En el diagrama, Interfaz SiteSearch se dio por utilizada o ejecutada por la clase SearchService.

Si la interfaz se representa usando la notación rectángulo, interfaz realización dependencia se denota con interfaz realización flecha. El clasificador en la cola de la flecha implementa la interfaz a la cabeza de la flecha.

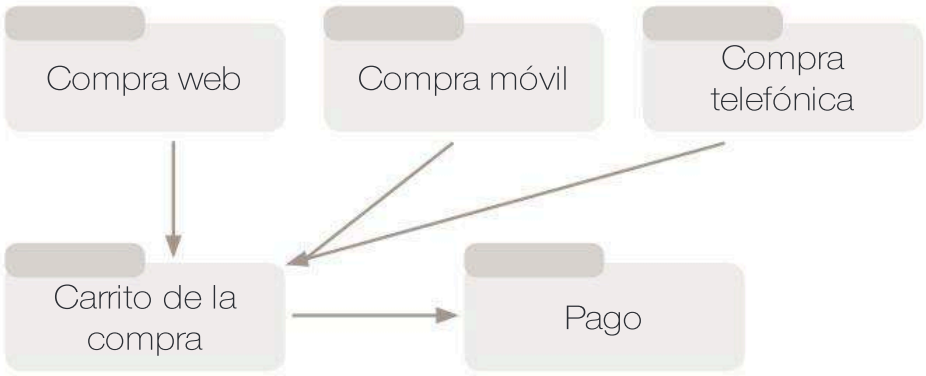


Definición

Diagrama de paquetes es un diagrama de estructura UML que muestra la estructura del sistema diseñado en el nivel de paquetes.

Los siguientes elementos suelen aparecer típicamente en un diagrama de paquetes son: paquete, elemento empaquetable, dependencia, elemento de importación, importación de paquetes,paquete de combinación.

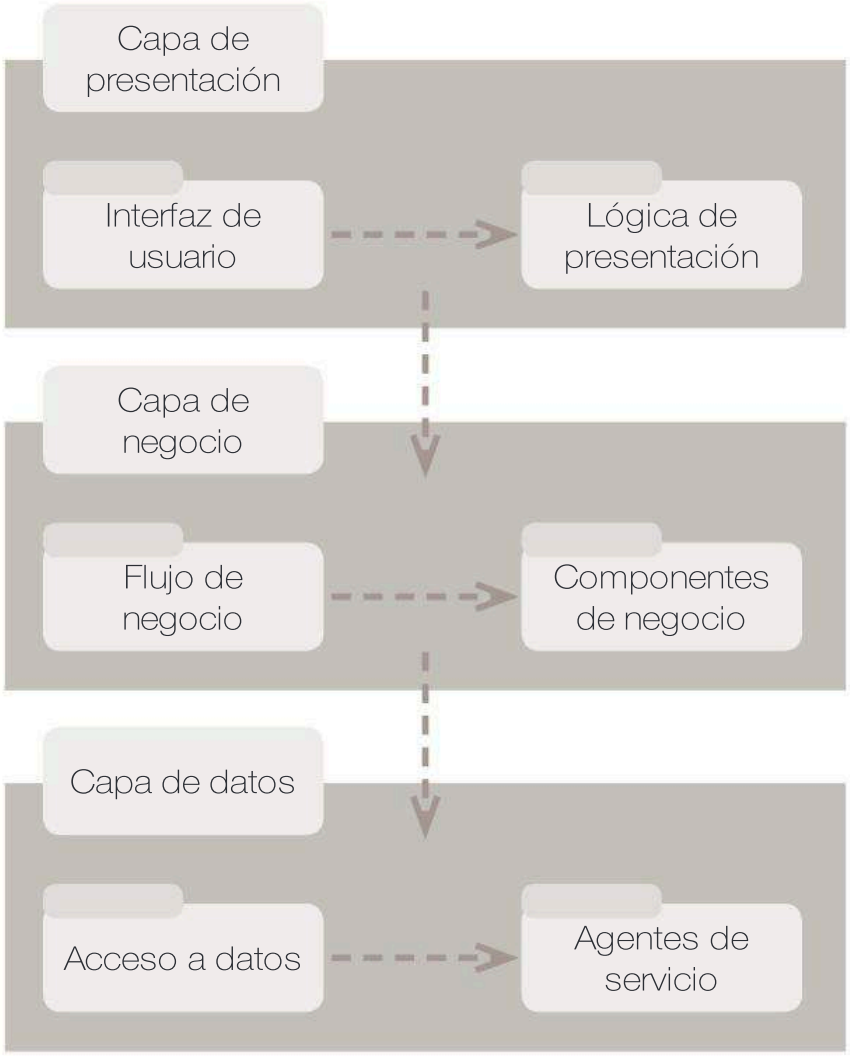
El diagrama de modelo UML es un diagrama auxiliar de estructura que muestra una abstracción o vista específica de un sistema, para describir los aspectos arquitectónicos, lógicos o de comportamiento del sistema. Se podría mostrar, por ejemplo, la arquitectura de una aplicación de varias capas (también conocido como de varios niveles).



Definición

Diagrama de modelo: Diagrama de modelo UML es un diagrama de estructura auxiliar que muestra una abstracción o vista específica de un sistema, para describir algunos aspectos arquitectónicos, lógicos o de comportamiento del sistema.

El esquema siguiente mostrado abajo muestra algunos de los elementos principales del diagrama del modelo.



El paquete es un espacio de nombres utilizado para agrupar los elementos que están semánticamente relacionados y podrían cambiar juntos Es un mecanismo de propósito general para organizar elementos en grupos para proporcionar una mejor estructura para el modelo de sistema.

Los Miembros de un paquete deben estar todos lo elementos encapsulados. Si un paquete se elimina de un modelo, por lo que son todos los elementos de propiedad por el paquete.

El paquete por sí mismo es elemento empaquetable, por lo que cualquier paquete podría ser también un miembro de los otros paquetes

Debido a que el paquete es un espacio de nombres, elementos de tipo similar y la misma deben tener nombres únicos dentro del paquete envolvente. Sin embargo, e permiten a elementos diferentes tipos tener el mismo nombre.

Como espacio de nombres, un paquete puede importar cualquiera de los miembros individuales de otros paquetes o todos los miembros de otros paquetes. El paquete también puede ser fusionado con otros paquetes. Un paquete se representa como una carpeta con pestañas - un rectángulo con una pequeña pestaña unido al lado izquierdo de la parte superior del rectángulo.

Si los miembros del paquete no se muestran en el interior del paquete rectángulo, entonces el nombre del paquete que se debe colocar en el interior. Un paquete además se puede utilizar como una plantilla para otros paquetes.



Un elemento empaquetable puede ser utilizado como un parámetro de plantilla. Un parámetro de plantilla paquete puede referirse a cualquier elemento de propiedad o utilizados por la plantilla de paquete, o plantillas anidadas dentro de ella.



Diagramas de componentes UML: un Diagrama de componentes muestra los componentes, interfaces provistas y requeridas, puertos, y las relaciones entre ellos.

Este tipo de diagramas se utiliza en el Desarrollo Basado en Componentes (CDB) para describir los sistemas con arquitectura orientada a servicios (SOA).

Desarrollo basado en componentes se basa en suposiciones de que los componentes previamente construidos podrían reutilizarse y que los componentes podrían ser sustituidos por otros componentes «equivalentes» o «conformes», si es necesario.

Los artefactos que implementan componente están destinados a ser capaz de ser desplegado y re-desplegado de forma independiente, por ejemplo, para actualizar un sistema existente. Los componentes en UML podrían representar tanto componentes lógicos como físicos.

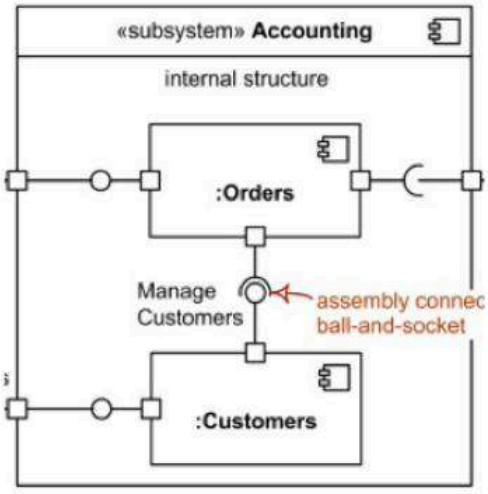
Los componentes lógicos son por ejemplo, componentes de negocio, o componentes de proceso.

Componentes físicos serían por ejemplo, componentes CORBA, componentes EJB, COM + y NET componentes, componentes de WSDL, etc.

Además en el diagrama se representan junto con los artefactos que los desarrollen y los nodos en los que se implementan y ejecutan. Se prevé que se desarrollarán perfiles basados alrededor de los componentes de tecnologías de componentes específicos y entornos de hardware y de software asociados.

Definición

Componente: un componente es una clase estructurada que representa una parte de un sistema modular con el contenido encapsulado y cuya manifestación es reemplazable dentro de su medio ambiente.



Ejemplo de diagrama de componentes

Un componente tiene su comportamiento definido en términos de las interfaces proporcionadas y las interfaces que necesita, las cuales quedan expuesta a través de los puertos.



Un Componente sirve como un tipo cuya conformidad se define por estas interfaces provistas y requeridas (que abarcan tanto la estática, así como semántica dinámica).Por lo tanto, un componente puede ser sustituido por otro sólo si los dos son de tipo conforme.

Partes más grandes de la funcionalidad de un sistema se pueden ensamblar mediante la reutilización de componentes como piezas de un componente o conjunto de componentes que abarca, y de cableado juntos sus interfaces necesarias y proporcionadas.

Un componente es un elemento que se usa como modelo a lo largo del ciclo de vida de desarrollo y refinado sucesivamente en el despliegue y tiempo de ejecución.



Un componente puede estar formado por uno o más artefactos.

Indirectamente, un componente instanciado se define en tiempo de diseño, pero no existe como objeto direccionable en tiempo de ejecución.

El comportamiento de tiempo de ejecución del componente y sus puertos se define por el comportamiento de ejecución de los clasificadores o partes se den cuenta. La partes interna de un componente suelen están ocultos y de difícil acceso que no sea conforme a lo dispuesto por sus interfaces.

Aunque puede ser dependiente de otros elementos en términos de interfaces que se requieren, un componente se encapsula y sus dependencias están diseñados de tal manera que se puede tratar lo más independientemente posible.Un componente se muestra como un rectángulo clasificador con la palabra clave «componente».



Definición

Diagramas general de Despliegue: Diagrama de despliegue es un diagrama de estructura que muestra la arquitectura del sistema en el despliegue (distribución) de artefactos de software en los destinos de despliegue.

Los artefactos representan elementos concretos en el mundo físico que son el resultado de un proceso de desarrollo. Algunos ejemplos de artefactos son archivos ejecutables, bibliotecas, archivos, esquemas de bases de datos, archivos de configuración, etc.

Los destinos de despliegue está generalmente representado por un nodo que es uno de los dispositivos de hardware o algún entorno de ejecución de software.Los nodos pueden estar conectados a través de elementos de comunicaciónpara crear sistemas en red de complejidad arbitraria.

Los diagramas de despliegue podrían describir la arquitectura a nivel de especificación (también llamado nivel de tipo) o al nivel de instancia (similar a los diagramas de clases y diagramas de objetos).



Definición

El **diagrama de despliegue** a nivel de especificación muestra cierta visión genera de despliegue de elementos a los destinos de despliegue, sin hacer referencia a casos concretos de artefactos o nodos.

El diagrama de despliegue a nivel de instancia muestra el despliegue de instancias de artefactos a los casos específicos de destinos de despliegue.

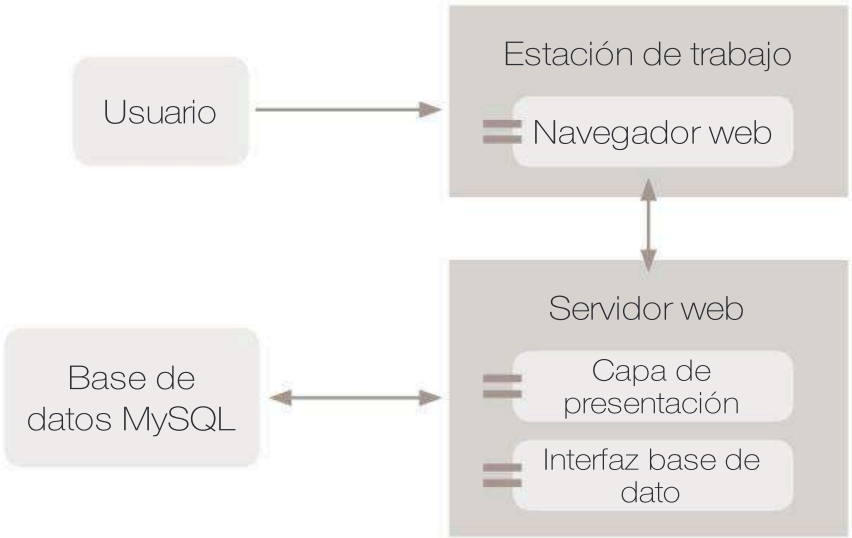
Importante

Un diagrama de despliegue podría ser utilizado por ejemplo para mostrar las diferencias en los despliegues a entornos de desarrollo, puesta en escena o de producción con los nombres e identificadores específicos o servidores o dispositivos de despliegue.

Mientras diagramas de componentes muestran los componentes y las relaciones entre los componentes y los clasificadores. Los diagramas de despliegue muestran como se realiza la instalación y despliegue de artefactos en los destinos de despliegue.

Algún diagrama intermedio que falta es diagrama el que a veces se denomina diagrama de manifestación que se utiliza para mostrar la manifestación o aplicación de los componentes de artefactos y la estructura interna de los artefactos.

En la figura se muestra un diagrama de despliegue de una aplicación web:





Nivel de Especificación del Diagrama de Despliegue, también llamado nivel de tipo), este muestra algunas visión general de despliegue de artefactos a destinos del despliegue, sin hacer referencia a algunos caso concretos de artefactos o nodos.

Diagrama de implementación de Nivel de Instancia: los Diagrama de despliegue a nivel de instancia muestra la implementación de instancias de artefactos a los casos específicos de los destinos de despliegue.

Esto Podría ser utilizado por ejemplo para mostrar las diferencias en los despliegues para el desarrollo, puesta en escena o entornos de producción con los nombres e identificadores de servidores o dispositivos de despliegue específicas.

Los diagramas de despliegue podrían utilizarse para mostrar el aspecto lógico o físico de la arquitectura de red del sistema. El denominado Diagrama de la arquitectura de red podría no mostrar artefactos o despliegues en absoluto o sólo los más importantes.



Diagramas de casos de uso: diagramas de casos de uso UML se refieren generalmente como un diagrama de comportamiento utilizado para describir un conjunto de acciones, llamadas caso de uso que algún sistema o sistemas deben o pueden llevar a cabo, con la colaboración de uno o más usuarios externos del sistema, llamados actores.

Cada caso de uso debe proporcionar algún resultado observable y valioso para los actores que intervienen u otros actores del sistema.

Hay que tener en cuneta que las especificaciones también describen el diagrama de casos de uso como una especialización de un diagrama de clases, y el diagrama de clases es un diagrama de estructura.

Los diagramas de casos de uso tienen de hecho de hecho una doble clasificación: son diagramas de comportamiento, ya que describen el comportamiento del sistema, y también son diagramas de estructura, como un caso especial de los diagramas de clase, donde los clasificadores están restringidos a ser tanto actores o casos de usos relacionados con cada uno mediante asociaciones.

Los diagramas de casos de negocio: si bien el apoyo para el modelado de negocios fue declarada como una de las metas de la UML, especificación UML ofrece ninguna anotación específica a las necesidades empresariales.

Los casos de uso de negocio se introdujeron en el Proceso Unificado del Rational para representar la función empresarial, proceso o actividad que se realiza en el modelo de negocio

Un actor de negocio representa un papel desempeñado por una persona o un sistema externo al negocio modelado, y la interacción con el negocio. El uso comercial caso debe producir un resultado de valor observable para un actor de negocio.

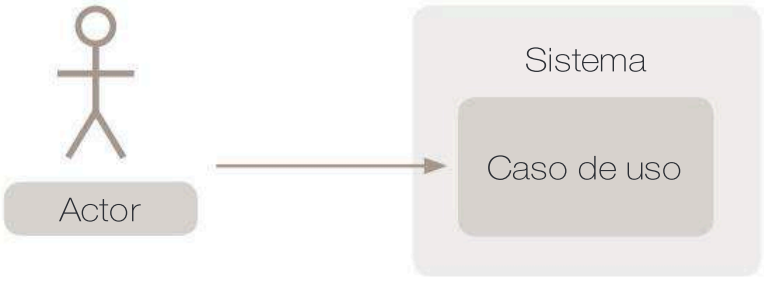
Tanto los casos de uso del negocio, así como el actor de negocio no están definidos en el estándar UML, por lo que sea que tenga que utilizar alguna herramienta UML apoyando a aquellos o crear sus propios estereotipos de modelado de negocios.

Los diagramas de casos de uso de un sistema se utilizan para especificar el sistema a nivel externo e interno.

(A nivel externo, la utilización de los casos de uso sirve para determinar un sistema en fase de diseño o análisis y para capturar lo que se supone que el sistema debe hacer;

la funcionalidad ofrecida sobre un uso, lo que el sistema puede hacer indica los requisitos específico que se plantea sobre su entorno por la definición de cómo el entorno debe interactuar con el objeto de modo que será capaz de realizar sus servicios.

Los principales elementos del diagrama de casos de uso de UML se muestran en la imagen siguiente:



Definición

Escenario de un caso de uso: un sujeto de un caso de uso define y representa los límites de una empresa, el sistema de software, sistema físico o dispositivo, subsistema, componente o incluso una sola clase en relación con la recopilación y el análisis de requisitos.

En términos de UML, un escenario es un clasificador de jugar el papel de «sujeto».UML 2.5 establece que un objeto de un caso de uso podría ser un sistema o cualquier otro elemento que puede tener un comportamiento, como un componente o Clase.

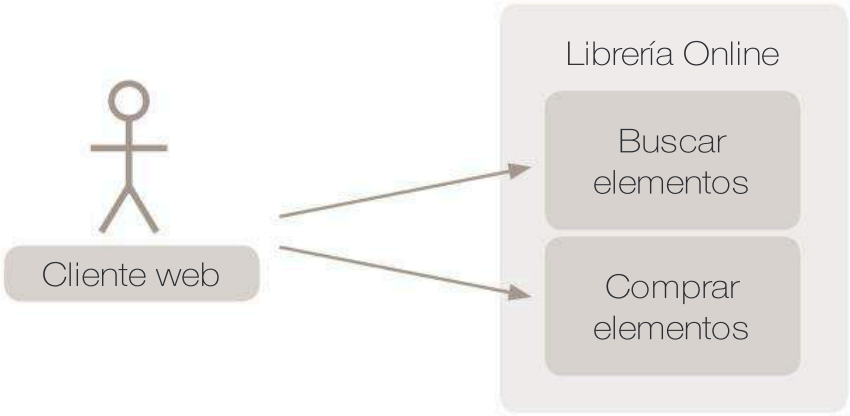
Un escenario es un clasificador (incluyendo subsistemas, componentes, o incluso clases) en representación de una organización, el sistema de software, sistema físico o dispositivo bajo análisis, diseño, o consideración, tener algún comportamiento, y para el que un conjunto de casos de uso se aplica.

El motivo está relacionado con casos de uso aplicables y es diferente de los casos de uso de clasificadores poseer. El escenario (a veces llamada frontera del sistema) es presentado por un rectángulo con el nombre del sujeto, palabras clave asociadas y los estereotipos en la esquina superior izquierda.

Los casos de uso aplicables al escenario se encuentran en el interior del rectángulo y los actores fuera de los límites del sistema. Por tanto los primero que se encarga de definir un caso de uso es qué es el sistema y qué son elementos externos al sistema.

Hay que considerar que el requisito de que nombre del escenario debe estar en la esquina superior izquierda se añadió en UML 2.5.Las versiones ante-

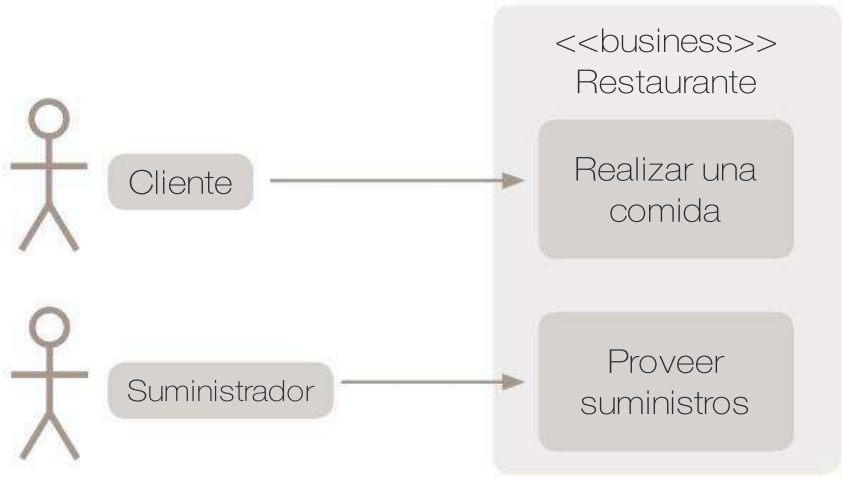
riores de UML permite usar y tenía ejemplos con el nombre de una de las esquinas superiores.



Escenario de Modelo de Negocio : Los casos de uso se podrían utilizar para modelar negocios, para analizar los procesos de negocio, reconocen los problemas que atraviesa, determinar las oportunidades para servir mejor a los clientes.

Los escenarios de casos de uso en este caso con negocio, empresas, sociedades o parte de estas, como departamento, equipos.UML no ofrece ninguna estereotipo estándar para modelar los procesos de negocio, pero podemos utilizar estereotipos personalizados como «negocio» o«departamento**»,** si es necesario, para mayor claridad.

El Ejemplo muestra un ejemplo de un «negocio» Restaurante con los actores empresariales de los clientes, el Anunciante y el Proveedor y relacionados con casos de uso de negocio.





Un agente o actor es un clasificador con comportamiento que especifica un papel desempeñado por una entidad externa que interactúa con el escenario.

Un actor interactúa con el escenario por ejemplo, mediante el intercambio de señales y datos y puede ser un usuario humano del sistema diseñado, algún otro sistema o hardware utilizando los servicios de la asignatura.

El término « papel « o rol se utiliza de manera informal como algún tipo, grupo o particular, las facetas de los usuarios que requieren servicios específicos del sujeto modelado con casos de uso asociados. Cuando una entidad externa interactúa con el escenario, desempeña el papel de un actor específico.

Esta entidad física puede desempeñar varias funciones diferentes, y un rol específico puede ser desempeñado por una o múltiples instancias diferentes. Un actor es externo al escenario se define típicamente en el mismo clasificador o el paquete que incorpora el escenario.



Todos los actores deben tener nombres de acuerdo con el rol asumido.

Ejemplo

Ejemplos de nombres de actores (roles de usuario):

- Cliente.
- Cliente Web.
- Estudiante.
- Pasajeros.
- Sistema de Pagos.

La notación UML estándar para el actor es mediante un " hombre del palillo ", esto es, un icono con el nombre del actor por encima o debajo del icono. Los nombres de Actor deben seguir las directrices de mayúsculas y de puntuación para las clases. Los nombres de los actores abstractos deben consignarse en letra cursiva.

Los iconos personalizados que transmiten el tipo de actor también se pueden utilizar para denotar un actor, como el uso de un icono independiente (s) para los agentes no humanos.

Un actor también se puede mostrar como una clase **rectángulo con la nor**ma palabra clave «agente**»,** siguiendo la notación habitual para compartimientos de clase, si es necesario.



Un actor sólo puede tener asociaciones binarias con los casos de uso, los componentes y las clases.

Un actor de negocio, definidos en el Proceso Unificado del Rational sirve para apoyar el modelado de negocios y representa un papel desempeñado por una persona o un sistema externo al modelo de negocios que interactúa con el negocio.

Hay que tener en cuenta que el concepto actor negocio está no definido en estándar UML.

Ejemplo

Algunos ejemplos típicos de agentes de negocios son:

- Cliente.
- Proveedor.
- Pasajeros.
- Autoridad.
- Banco.

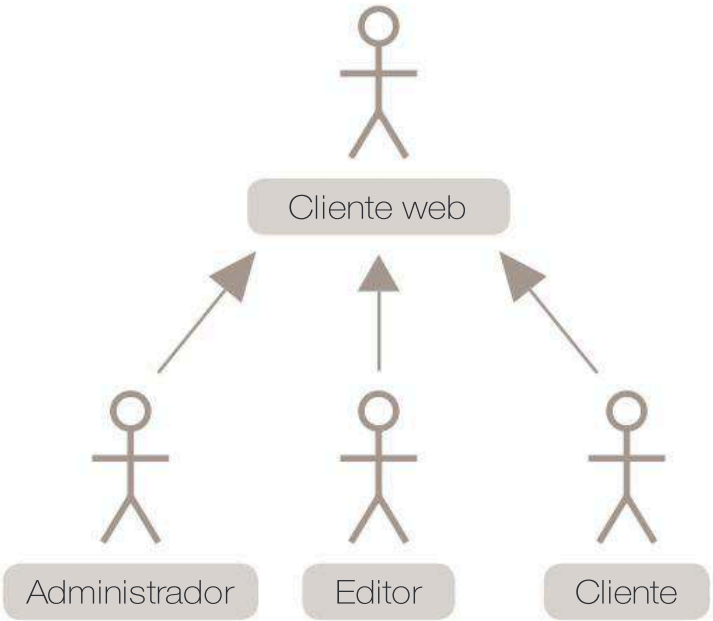
Cada actor de negocio representa algo que está fuera del negocio modelado y debe participar con al menos un caso de uso del negocio . Los agentes de negocios está representado en la metodología del rational por un «hombre de palo «con una línea que cruza su cabeza.



Definición

Relaciones entre los actores: se pueden definir actores abstractos y concretos y especializar un actor utilizando relaciones de generalización.

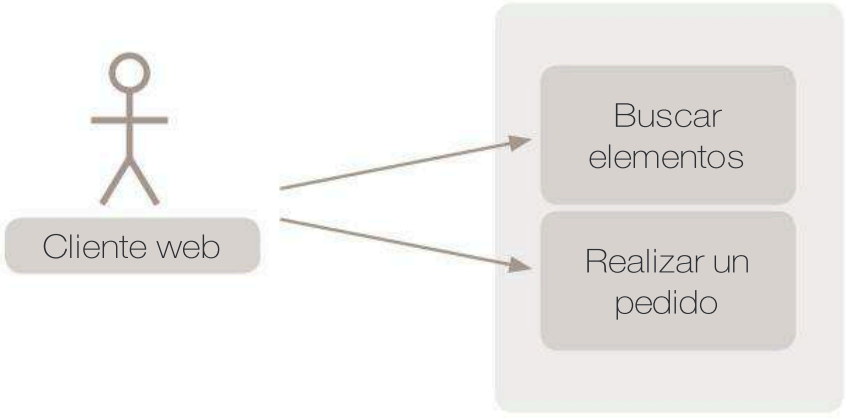
La generalización entre actores se representa como una línea dirigida sólido con una punta de flecha grande (lo mismo que para la generalización entre clases).



Asociación UML entre actor y caso de uso: Cada caso de uso representa una unidad de funcionalidad útil que los escenarios proporcionan a los actores.

Una asociación entre un actor y un caso de uso indica que el actor y el caso de uso de alguna manera interactúan o se comunican entre sí.

Sólo se permiten asociaciones entre actores y casos de uso. Un actor puede estar asociada a uno o varios casos de uso. Un caso de uso puede tener uno o varios actores asociados.



Si hay varios actores asociados al mismo caso de uso, puede que no sea evidente a partir del diagrama de casos de uso que el actor inicia el caso de uso, es decir, es un " agente iniciador".

Definición

Multiplicidad de un caso de uso: cuando un actor tiene una asociación con un caso de uso con una multiplicidad que es mayor que uno en la línea final de los de casos de uso, esto significa que un determinado actor puede estar involucrado en múltiples casos de uso de ese tipo.

Multiplicidad de un Actor: el actor se requiere puede ser denotado explícitamente usando multiplicidad 1 o mayor. UML 2.5 también permite que el actor sea opcional. Multiplicidad 0..1 del actor significa que el actor puede o no puede participar en cualquiera de sus casos de uso asociados.

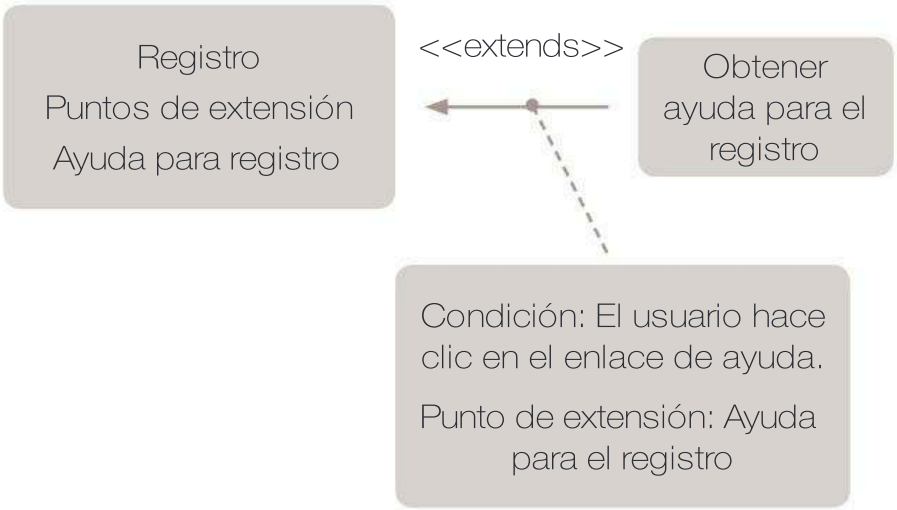
Definición

Caso de Uso UML Entendido: extender es una relación dirigida que especifica cómo y cuándo el comportamiento definido en lo general es complementado de manera opcional por otro caso que se extiende y puede ser insertado en el comportamiento definido en el caso de uso extendido.

Un caso de uso extendido debe ser significativo por sí solo o sea que es independiente del caso de uso del que extiende. Un caso de uso extendido define normalmente comportamiento opcional que no es necesariamente significativo por sí mismo.

La relación de extensión es propiedad del caso de uso extendido. El mismo caso de uso extendido se puede extender de más de un caso de uso. La extensión se lleva a cabo en uno o más puntos de extensión definidas en el caso de uso extendido.

Extender relación se muestra como una línea discontinua con una punta de flecha abierta dirigida desde el caso de uso se extiende al caso base que extiende caso de uso. La flecha se etiqueta con la palabra clave «extend**».**





Punto de Extensión: un punto de extensión es una característica de un caso de uso que identifica (referencias) un punto en el comportamiento del caso de uso donde ese comportamiento puede ser prorrogado por otro caso (se extiende) el uso, definidas por extender relación.

Caso de Uso UML Incluir: el caso de uso de inclusión es una relación dirigida entre dos casos de uso que se utiliza para mostrar que el comportamiento de un caso de uso se inserta en el comportamiento de otro caso de uso en el que esta el comportamiento que se ha incluido.

Al caso de uso en el que se incluye otro caso de uso se le llama caso de uso base.

La relación de inclusión podría ser utilizado fundamentalmente para dos propósitos:

– Para simplificar un caso de uso grande dividiéndolo en varios casos de uso.

– Para extraer las partes comunes de los comportamientos de dos o más casos de uso.

Un caso de uso general podría tener algún comportamiento que puede ser separadas en los casos de uso distintos de menor tamaño que se incluirán de nuevo en el caso de uso base usando la relación de inclusión.

El propósito de esta acción es la modularización de los comportamientos, haciéndolos más manejables.

Cuando dos o más casos de uso tienen algún comportamiento común, esta parte común podría ser extraído en un caso de uso separado que se incluirán vuelta por los casos de uso con el uso de una relación UML de inclusión. Por ejemplo un caso de uso C se puede extraer de los casos de uso A y B para ser reutilizado por ambos casos de uso utilizando UML incluye relación. La ejecución del caso de uso incluido es análogo a una llamada de subrutina o macro de comandos de programación.

Todo el comportamiento del caso de uso incluido se ejecuta en un solo lugar en el caso de uso que lo incluye, para posteriormente pasar a reanudar la ejecución del caso de uso que ha hecho uso del proceso de inclusión

Importante

Mientras UML 2.x define los puntos de extensión para la extender la relación, no hay «puntos de inclusión» para especificar la ubicación o condición de inclusión donde se incluirá.



El caso de uso que incluye depende de la adición del caso de uso incluido, que siempre se requiere y no opcional. Esto significa que la inclusión de casos de uso es una operación no completa por sí misma, y por lo que tendría sentido para referirse a los casos de uso incluidos como casos de uso abstractos.

Importante

La relación incluir entre dos casos de uso se muestra mediante una flecha discontinua con una punta de flecha abierta desde el caso que incluye (base) hacia el caso de uso incluido.

La flecha se etiqueta con la palabra clave «incluir».

En el siguiente esquema se especifica cuando utilizar una relación u otra:

Generalización	Extensión	Inclusión
Caso de uso base podría ser abstracto (incompleto) o concreto (completa).	Caso de uso base es completo por sí mismo, de tal forma que se define de forma independiente.	Caso de uso base es incompleta, se trata de un caso de uso abstracto
Se requiere caso de uso especializado, no es opcional, si el caso base es abstracto.	La extensión de casos de uso es opcional, complementaria.	Caso de uso incluido siempre se requiere, no opcional.
Sin lugar explícito para usar la especialización.	Tiene al menos una ubicación de extensión explícita.	Sin ubicación explícita para inclusión, pero se incluye en algún lugar.
Ninguna condición explícita para usar la especialización.	Podría tener la condición de extensión opcional.	Ninguna condición inclusión explícita.



Diagramas de actividad: Diagrama de actividad es un diagrama de comportamiento UML que muestra flujo de control o flujo de objetos con énfasis en la secuencia y las condiciones del flujo.

Las acciones coordinadas por los modelos de actividad puede iniciarse porque otras acciones terminan la ejecución, ya que los objetos y los datos estén disponibles, o porque algunos eventos externos al flujo ocurren.

Una Actividad es un comportamiento parametrizado es representado como flujo coordinado de acciones. El flujo de ejecución es modelado como nodos de actividad conectados por líneas de actividad.

Un nodo puede ser la ejecución de un comportamiento subordinado, tal como un cálculo aritmético, una llamada a una operación o manipulación del contenido del objeto.

Los Nodos de actividad también incluyen el flujo de las construcciones de control, como la sincronización, decisión y control de concurrencia.

Las actividades pueden formar jerarquías de invocación invocando otras actividades, en última instancia, la resolución de las acciones individuales.



En un modelo orientado a objetos, actividades suelen invocarse indirectamente como métodos vinculados a las operaciones que se invocan directamente.

Las actividades pueden incluir acciones de diversa índole como por ejemplo:

– Ocurrencias de las funciones primitivas, como las funciones aritméticas.

– Las invocaciones de comportamiento, tales como actividades.

– Acciones de comunicación, como el envío de señales.

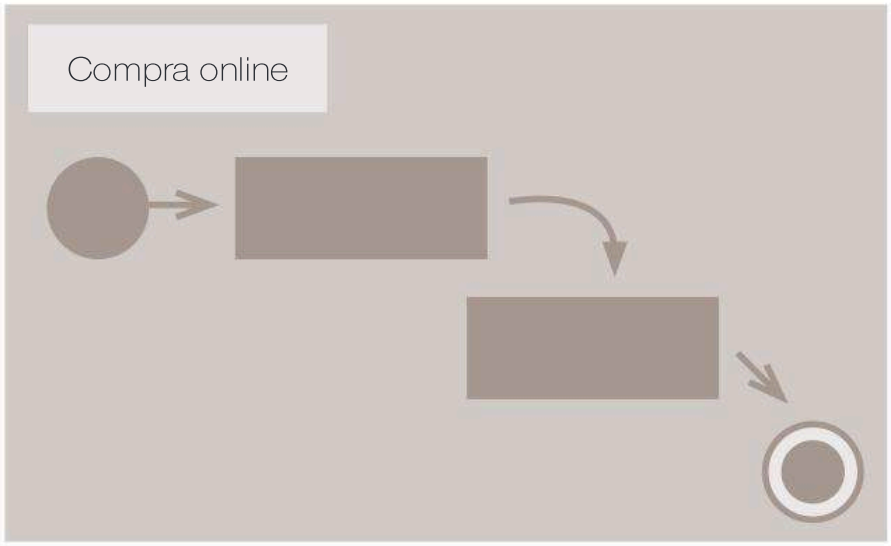
– Manipulaciones de objetos, tales como atributos o asociaciones de lectura o escritura.

Hay acciones que invocan actividades,y a sea directamente a través de la acción comportamiento de llamadas o indirectamente con la acción de operación llamada.

Una actividad podría ser representada como un rectángulo con esquinas redondeadas nombre de la actividad en la esquina superior izquierda y los nodos y los bordes de la actividad dentro del limite.

Particiones se utilizan en los diagramas de actividad y a menudo corresponden a unidades organizativas o los actores empresariales en un modelo de negocio.

Las particiones proporcionan una visión limitada sobre los comportamientos invocados en las actividades.



Por ejemplo, las particiones pueden representar clasificadores específicos. En este caso las acciones en cada partición deben ser operaciones o señales de orientación de objetos que son instancias del clasificador correspondiente

Por ejemplo, una partición puede representar el lugar en el que una actividad se lleva a cabo. Las particiones se representan mediante la notación de “calle” o“swimlane” con dos líneas normalmente paralelas, ya sea horizontal o vertical, y un nombre de etiquetado de la partición en una caja en un extremo.

Cualquier nodos de actividad, por ejemplo, las acciones y los bordes colocados entre estas líneas se consideran para ser contenida dentro de la partición.





Diagramas de Secuencia UML: diagrama de secuencia es el tipo más común de diagramas de interacción, que se centra en el mensaje de intercambio entre un número de líneas de vida.

Un Diagrama de secuencia describe una interacción centrándose en la secuencia de mensajes que se intercambian. Junto con los mensajes se detallan las especificaciones correspondientes de estos en las líneas de vida del objeto que tiene asociado.



La **línea de vida** es un elemento con nombre que representa un participante individual en la interacción. Sirve para representar el comportamiento de un elemento a lo largo de su existencia.

Mientras que partes y características estructurales pueden tener multiplicidad mayor que 1, líneas de vida representar sólo una entidad con la que interactuar. En general una línea de vida representa una sola ocurrencia de un objeto particular en el sistema.

Si hay **múltiples valores** de referencia del elemento conectable (es decir, tiene una multiplicidad> 1), a continuación, la línea de vida debetener una expresión que especifica qué parte particular está representado por esta línea de vida.

Una línea de vida se muestra el uso de un símbolo que consiste en un rectángulo que forman su «cabeza», seguido de una línea vertical (que podrá ser discontinua).

Esta línea de vida lo que representa es el tiempo de vida del participante en la interacción correspondiente que se especifica en el diagrama. Por lo general,

la cabeza se representa por un símbolo constituido por un rectángulo blanco que contiene el nombre de la clase o del objeto.



Un **mensaje** es un elemento con un nombre que define un tipo específico de la comunicación entre líneas de vida de una interacción.

El mensaje especifica no sólo el tipo de comunicación, sino también el emisor y el receptor del mensaje representado por la línea.

El Emisor y el receptor son normalmente dos especificaciones de ocurrencia, mostradas por puntos en los extremos de los mensajes. Un mensaje se muestra como una línea del mensaje de fin emisor al extremo receptor mensaje. Los argumentos de un mensaje pueden ser los siguientes:

– Atributos de la línea de vida que lo envía.

– Constantes.

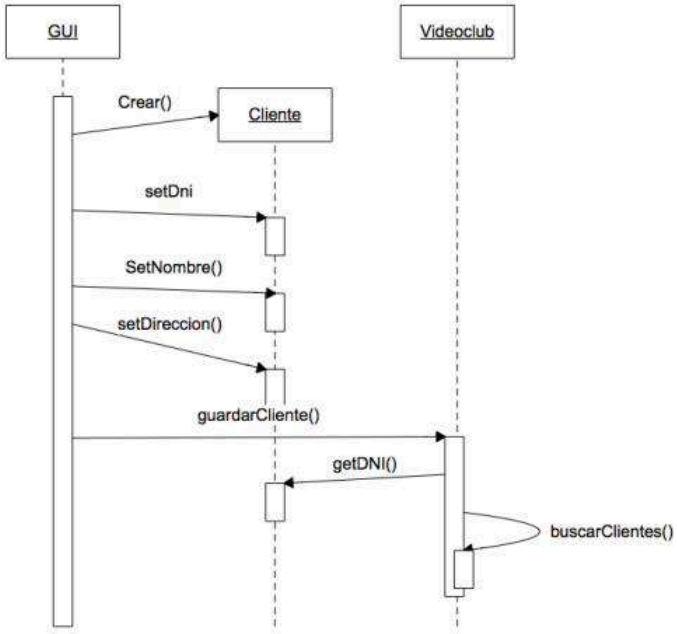
– Valores simbólicos.

– Parámetros explícitos de la interacción que encierra.

– Los atributos de la clase propietaria de la interacción.



La línea debe ser tal que cada fragmento de la línea es horizontal o hacia abajo cuando atravesado desde el evento de envío para recibir eventos.



Paso de mensajes

El envío y recepción de eventos pueden ser ambos en la misma línea de vida. La forma de la línea o de punta de flecha refleja las propiedades del mensaje.

Definición

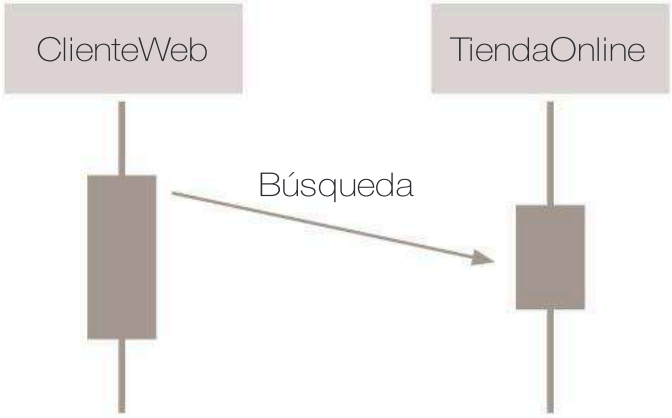
Un mensaje refleja o bien una operación de llamada y el inicio de la ejecución o de un envío y la recepción de una señal.

Cuando un mensaje representa una llamada de la operación, los argumentos del mensaje son los argumentos de la operación.Cuando un mensaje representa una señal, los argumentos del mensaje son los atributos de la señal.

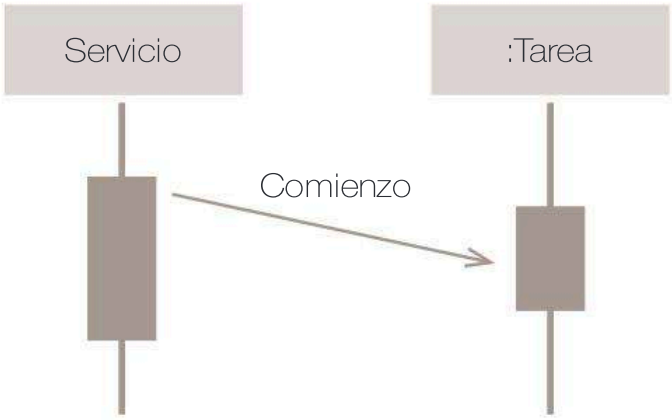
Importante

Dependiendo del tipo de acción que se utilizó para generar el mensaje, el mensaje podría ser uno de llamada síncrona, asíncrona, crear, eliminar o responder.

Una llamada sincrónica representa típicamente llamada operación, se envía el mensaje y se suspende la ejecución mientras se espera respuesta. Los mensajes de llamada síncronos se muestran con punta de flecha rellena.



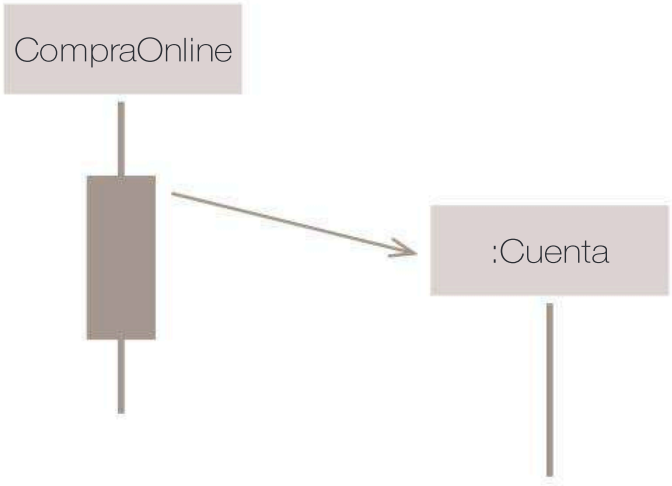
La llama asincrónica significa enviar mensaje y la tarea inmediatamente,sin esperar a que el valor de retorno. Posteriormente el valor de retorno será o no enviado por la tarea asíncrona cuando acabe. Generalmente las tareasasíncronas se ejecutarán mensajes asíncronos se representan mediante una punta de flecha abierta.



Definición

Un **mensaje de creación** se envía a un objeto para generar una instancia de el mismo.

Se muestra como una línea discontinua con punta de flecha abierta y apuntando a la cabeza de la línea de vida creada.



Importante

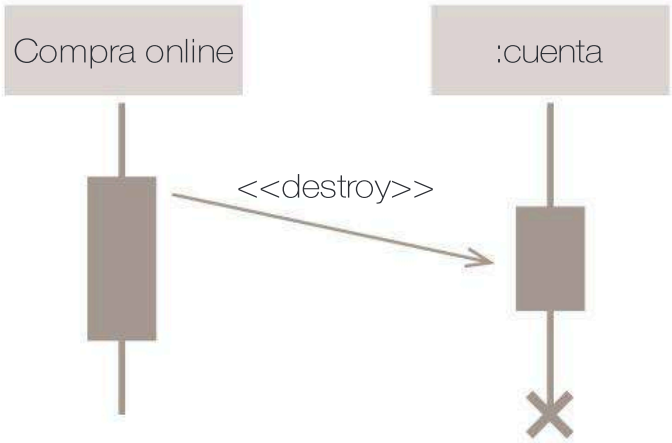
Hay que tener en cuenta, que es raro que en un lenguaje haya que enviar un mensaje a un objeto no existente para crearlo, aunque es la convención que realmente se utiliza tanto en UML 1.x como 2.x.

En la práctica, en programación Orientada a Objetos, el mensaje, probablemente, se debe enviar a la clase correspondiente. Así que una manera de interpretar en la notación UML el mensaje de crear es probablemente como un acceso directo para estas acciones.

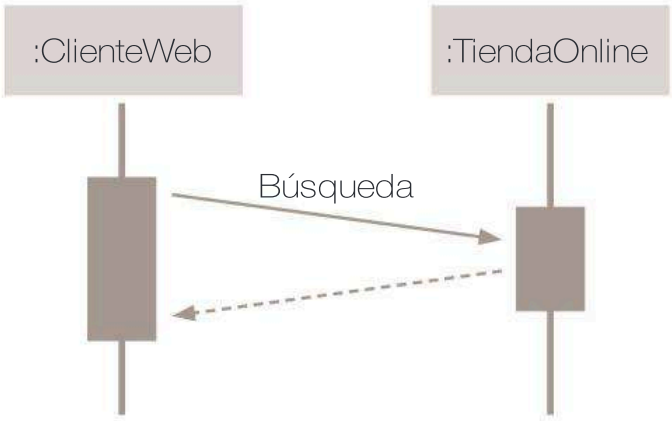
Importante

El mensaje eliminar (denominado también parada en las versiones anteriores de UML) es enviado a terminar otra línea de vida.

La línea de vida por lo general termina con una cruz en forma de X en la parte inferior que indica la destrucción de la ocurrencia correspondiente.



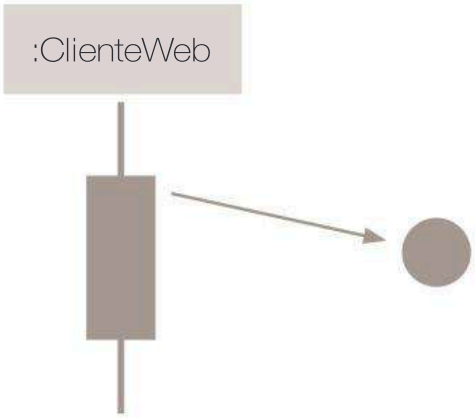
Un mensaje de respuesta a una operación llamada se muestra como una línea discontinua con punta de flecha abierta, de forma similar a la del mensaje de creación.



Definición

Un **mensaje perdido** es un mensaje donde se sabe que este se ha enviado, pero no hay ningún evento de recepción. Se interpreta como que el mensaje nunca ha llegado a su destino.

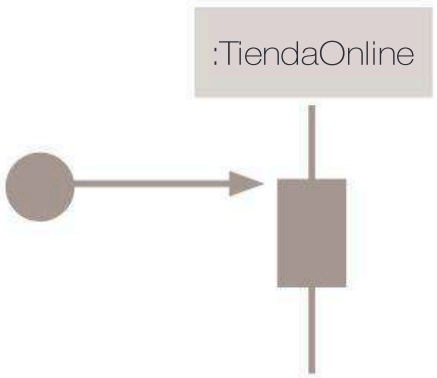
Los Mensajes perdidos se denotan con un pequeño círculo negro en el extremo flecha del mensaje, como se indica en el diagrama, en el que la clase ClienteWeb manda un mensaje de búsqueda que se pierde.



Definición

Un **mensaje encontrado** es un mensaje donde se sabe que el evento de recepción, pero no hay un emisor conocido. Se interpreta como si el origen del mensaje está fuera del alcance del entorno de descripción del diagrama. Esto puede ser por ejemplo el ruido u otra actividad que no se quiera para describir en detalle.

Los mensajes encontrados se indican con un pequeño círculo negro en el extremo inicial del mensaje, como se indica en el diagrama, en el que la clase tienda recibe un mensaje de búsqueda “encontrado”.



UD3
Lo más importante

- UML es un lenguaje común para los analistas de negocios, arquitectos y desarrolladores de software que se utilizan para describir, especificar, diseñar, y documento existente o nuevos procesos de negocio, la estructura y el comportamiento de los elementos de sistemas de software.

- Un diagrama UML contiene elementos gráficos o símbolos, también llamados nodos UML conectados con líneas llamadas caminos o flujos, que representan elementos en el modelo UML del sistema diseñado.

- Los diagrama de clases son un diagrama UML de estructura que muestra la estructura del sistema diseñado en el nivel de las clases y las interfaces, muestra sus características, estructura y relaciones asociaciones, generalizaciones, dependencias, etc.

- Un diagrama de objetos es un ejemplo de un diagrama de clases, muestra una instantánea del estado detallado de un sistema en un punto en el tiempo.

- Las características estáticas puede tener, o bien diferentes valores para diferentes clasificadores, o el mismo valor para todos los clasificadores al que pertenecen.

- La clase abstracta existe sólo para otras clases hereden desde y para apoyar la reutilización de las características declaradas por el mismo.

- Una interfaz es un clasificador que declara de un conjunto de funciones públicas coherentes y obligaciones para la clase que los implemente.

- Un objeto es una instancia de una clase o una interfaz. El objeto no es un elemento UML por sí mismo. Los objetos se representan como especificaciones de instancia, por lo general en diagramas de objetos.

- Cuando se crea una instancia de un clasificador, cada propiedad no estática se convierte en una parte del estado de esa instancia, y se implementa por algún método de mapeo del nombre de la propiedad a un valor específico o valores que el estado de los que la instancia se compone.

- La visibilidad permite restringir el uso de un elemento llamado, ya sea en el espacio de nombres o en el acceso al elemento

- Una asociación es una relación entre clasificadores que se utilizan para mostrar que los casos de los clasificadores pueden ser ya sea vinculados entre sí o combinada lógica o físicamente en alguna agregación.

- La herencia se define como un mecanismo por el cual las clases más específicas, llamadas subclases o clases derivadas, incorporan la estructura y comportamiento de las clases más generales, llamados superclases o clases base

- El diagrama de modelo es un diagrama de estructura auxiliar que muestra una abstracción o vista específica de un sistema, para describir algunos aspectos arquitectónicos, lógicos o de comportamiento del sistema.

- Un Diagrama de componentes muestra los componentes, interfaces provistas y requeridas, puertos, y las relaciones entre ellos.

- Diagramas de casos de uso UML se definen como un diagrama de comportamiento utilizado para describir un conjunto de acciones, llamadas caso de uso que algún sistema o sistemas deben o pueden llevar a cabo, con la colaboración de uno o más usuarios externos del sistema, llamados actores.

- Un Diagrama de actividad es un diagrama de comportamiento UML que muestra flujo de control o flujo de objetos con énfasis en la secuencia y las condiciones del flujo.

- El diagrama de secuencia es el tipo más común de diagramas de interacción, y se centra en el intercambio de mensajes entre líneas de vida.

- Un Diagrama de secuencia describe una interacción centrándose en la secuencia de mensajes que se intercambian.

UD3
Autoevaluación

1. UML es:

   a. Un lenguaje de programación estructurado.
   b. Un conjunto de especificaciones que indican como construir unos diagramas que sirve para intercambiar información.
   c. Una metodología de desarrollo orientada a objetos.
   d. Un ciclo de vida de desarrollo del software.

2. Los diagramas de comportamiento:

   a. Muestran el comportamiento dinámico de las clases del sistema.
   b. Muestran como se desarrolla un determinado caso de uso.
   c. Muestran el comportamiento dinámico de los objetos del sistema.
   d. Ninguna de las anteriores.

3. El diagrama de clases:

   a. es un diagrama que muestra la estructura del sistema diseñado en el nivel de las clases y las interfaces.
   b. Es un diagrama que muestra los objetos del sistema.
   c. Muestra el comportamiento de un sistema.
   d. Se ajusta a la normativa ISO.

4. Una clase abstracta:

    a. Es una clase de la que no pueden heredar otras.
    b. Es una clase con solo un objeto asociado.
    c. Es una clase de la que no se pueden crear objetos.
    d. Ninguna es correcta.

5. Una operación:

    a. Especifica nombre, tipo, parámetro y restricciones.
    b. Especifica nombre, tipo y parámetros.
    c. Especifica un nombre.
    d. Especifica parámetros in y parámetros out.

6. La visibilidad en UML:

    a. Puede ser publica, de paquete, protegida y privada.
    b. Puede ser publica, protegida y privada.
    c. Se usa solamente en clases.
    d. Se indica en atributos y métodos.

7. Una asociación:

    a. Es una relación entre clasificadores.
    b. Es una relación entre clases.
    c. Es una relación entre objetos.
    d. Es una relación entre casos de uso.

8. Un actor en un caso de uso:

    a. Es parte del sistema software modelado.
    b. Representa un elemento externo al sistema.
    c. Solamente se indica mediante un hombre de palo.
    d. Ninguna de las respuestas es correcta.

9. Un diagrama de actividad:

    a. Representa la actividad de un actor en el sistema.
    b. Es un diagrama estático.
    c. Es un diagrama de comportamiento.
    d. Modelan la organización a nivel medio.

10. Un mensaje:

    a. Ha de tener una visibilidad pública o privada dependiendo del tipo de clasificador al que apunte.
    b. Sirve para indicar la creación o finalización de un objeto.
    c. Indica la terminación de dos o más líneas de vida.
    d. Es un elemento que define un tipo específico de comunicación entre dos líneas de vida.

Área: informática y comunicaciones

UD4

Desarrollo de la GUI

4.1. Análisis del modelo de componentes y objetos

4.2. Identificación de los elementos de la GUI

4.3. Presentación del diseño orientado al usuario. Nociones de usabilidad

4.4. Empleo de herramientas de Interfaz Gráfica

4.1. Análisis del modelo de componentes y objetos

Hoy en día las interfaces de los programas son cada vez más sofisticadas y atractivas para el usuario. Son intuitivas y cada vez más fáciles de usar: pantallas táctiles, etc.



Sin embargo, no siempre ha ocurrido así. Hace ya algunos años, antes de que surgieran y se popularizaran las interfaces gráficas de usuario, en las que el usuario trabaja interaccionando con ventanas y botones, etc. se trabajaba en modo consola, o modo carácter, es decir, se le daban las ordenes al ordenador con comandos por teclado, de hecho, por entonces no existía el ratón.

Así que, con el tiempo, con la idea de simplificar el uso de los ordenadores para extender el uso a un cada vez mayor espectro de personas,no sólo para los expertos, se ha convertido en una técnica habitual el utilizar interfaces gráficas de usuario (IGU ó GUI en inglés), para que el usuario trabaje, interacciones y establezca un contacto más fácil e intuitivo con el ordenador.



En ocasiones, interfaz se define de otra manera, como la que define una interfaz como un dispositivo que permite comunicar dos sistemas que no hablan el mismo lenguaje.

Por esta razón se suele añadir la coletilla “de usuario” cuando nos referimos a la interfaz entre una persona y un ordenador.

También se emplea el término interfaz para definir el conjunto de conexiones y mecanismos que hacen posible la comunicación entre dos sistemas.

Cuando hablamos de interfaz nos referimos a la parte “visible” de los programas tal y como se muestran a los usuarios para que interactúen con ellos.

La interfaz gráfica implica la presencia de un monitor de ordenador, en el que veremos la interfaz, que está constituida por una serie de menú, iconos y elementos que representan las acciones que el usuario puede realizar dentro del sistema.

Las interfaces gráficas de usuario proporcionan al usuario ventanas, cuadros de diálogo, barras de herramientas, botones, listas desplegables y muchos otros más elementos. Las aplicaciones son programadas mediante por eventos y se desarrollan haciendo uso de clases que para ello ofrece el API de Java.



Sabías que

En 1981 aparecieron los primeros ordenadores personales, los llamados Personal Computer o PC, pero hasta el año 1993 no se generalizaron las interfaces gráficas de usuario.

El escritorio del sistema operativo MS Windows y su sistema de ventanas sobre la pantalla se ha estandarizado y universalizado, pero fueron los Macintosh de la compañía Apple los pioneros en introducir las interfaces gráficas de usuario.

Cuando apareció Java, los componentes gráficos disponibles para el desarrollo de GUI se encontraban en la librería denominada Abstract Window Toolkit (AWT).

Las clases AWT se desarrollaron usando código nativo (o sea, código asociado a una plataforma concreta). Eso dificultaba la portabilidad de las aplicaciones.

Al usar código nativo, para poder conservar la portabilidad era necesario restringir la funcionalidad a los mínimos comunes a todas las plataformas donde se pretendía usar AWT. Como consecuencia, AWT es una librería con una funcionalidad muy pobre.

La estructura básica de la librería gira en torno a componentes y contenedores. Un componente es un elemento gráfico y un contenedor un componente que los alberga.



Los contenedores albergan componentes y son componentes a su vez, de forma que los eventos pueden tratarse tanto en contenedores como en componentes.

AWT es una librería adecuada para desarrollar interfaces gráficas sencillas, pero no para proyectos más complejos

Más tarde, cuando apareció Java 2 surgió una librería mucho mejor elaboradora, versátil y flexible: Swing. AWT aún sigue estando disponible, de hecho se usa por los componentes de Swing.

Nombre de la clase AWT	Utilidad del componente
Applet	Ventana para una applet que se incluye en una página web.
Button	Crea un botón de acción.
Canvas	Crea un área de trabajo en la que se puede dibujar. Es el único componente AWT que no tiene un equivalente Swing.
Checkbox	Crea una casilla de verificación.
Label	Crea una etiqueta.
Menu	Crea un menú.
ComboBox	Crea una lista desplegable.
List	Crea un cuadro de lista.
Frame	Crea un marco para las ventanas de aplicación.
Dialog	Crea un cuadro de diálogo.
Panel	Crea un área de trabajo que puede contener otros controles o componentes.

PopupMenu	Crea un menú emergente.
RadioButton	Crea un botón de radio.
ScrollBar	Crea una barra de desplazamiento.
ScrollPane	Crea un cuadro de desplazamiento.
TextArea	Crea un área de texto de dos dimensiones.
TextField	Crea un cuadro de texto de una dimensión.
Window	Crea una ventana.

Cuando se determinó que era importante mejorar las características que ofrecía AWT, muchas empresas empezaron a sacar sus controles desarrolladas por ellas para mejorar algunas de las características de AWT. Casi todos los desarrolladores hicieron sus propias librerías particulares, pero una especialmente comenzó a cobrar más relevancia.

Así, la compañía Netscape sacó una librería de clases llamada Internet Foundation Classes para usar con Java, y eso obligó a Sun (todavía no adquirida por Oracle) a adaptar el lenguaje a las nuevas necesidades

AL nueva librería se desarrolló en colaboración con Netscape todo el conjunto de componentes Swing y estos se añadieron a la JFC, por esta razón a veces los copnnetes swing tabíne se les llama componentes de Foundatiosn

Swing es una librería de Java para la generación del interfaz gráfico en aplicaciones de escritorio. Esta se apoya sobre AWT y añade los denominados JComponents.



La arquitectura de los componentes de Swing facilita la adaptación de apariencia y comportamiento, si es comparado con los componentes AWT, muchos menos felxibles.

Por cada componente AWT (excepto Canvas) existe un componente Swing equivalente, cuyo nombre empieza por J, que permite más funcionalidad siendo menos pesado.

Ejemplo

Así, por ejemplo, para el componente AWT Button existe el equivalente Swing JButton, que permite crear botones con distintas formas (rectangulares, circulares, etc), cosa que no se podría hacer en Swing, e incluir imágenes en el botón, además de tener distintas representaciones para un mismo botón según esté seleccionado, o bajo el cursor, etc.

El motivo por el que que no existe JCanvas es que los paneles de la clase JPanel ya soportan todo lo que el componente Canvas de AWT soportaba. Esta es la razón por la que no se consideró necesario añadir un componente Swing JCanvas por separado.

Algunas características más de Swing parate de las que ya se han detallado, podemos mencionar las siguientes:

- Es completamente independiente de la arquitectura (Usa una metodología no nativa propia de Java).
- Proporciona todo lo necesario para la creación de entornos gráficos completos, tales como elaboración de menús, botones, cuadros de texto, manipulación de eventos, y todo lo necesario.
- Los componentes Swing no necesitan una ventana propia del sistema operativo cada uno, sino que son visualizados dentro de la ventana que los contiene directamente mediante métodos gráficos, por lo que utilizan bastantes menos recursos.
- Las clases Swing están escritas en Java en su totalidad, con lo que la portabilidad es total, a la vez que no hay obligación de restringir la funcionalidad a los mínimos comunes de todas las plataformas



Por ello las clase Swing aportan una muy alta gama de funciones que haciendo uso de la funcionalidad básica propia de AWT aumentan las posibilidades de diseño de interfaces gráficas.

Debido a sus características, los componentes AWT son denominados componentes “de peso pesado” por la cantidad de recursos que usan, y los componentes Swing se nombran como componentes “de peso ligero” por hacer un uso más efectivo de los recursos ya que por ejemplo no necesitar que el sistema operativo les proporcione una ventana, y por este motivo consumen muchos menos recursos del sistema.

Aunque todos los componentes Swing derivan de componentes AWT y de hecho se pueden mezclar en una misma aplicación componentes de ambos tipos, no se aconseja hacerlo.

Sin embargo es preferible desarrollar aplicaciones enteramente utilizando Swing, que requieren menos recursos y son más portables. De hecho en la actualidad casi no se desarrollan interfaces de usuario utilizando AWT.



Definición de evento: es todo hecho que ocurre mientras se ejecuta una aplicación. Normalmente, llamamos evento a cualquier manipulación que realiza el usuario con la aplicación.

Ejemplo

Ejemplos de eventos pueden ser:

- Pulsar un botón con el ratón.
- Hacer doble clic.
- Pulsar y arrastrar.
- Pulsar una combinación de teclas en el teclado.
- Pasar el ratón por encima de un componente.
- Salir el puntero de ratón de un componente.
- Abrir una ventana, etc.



La **programación guiada** por eventos es un modo de programar en el que el programador determina que acción se va a realizar al responder a un evento que se ha generado, por ejemplo que tiene que hacer el programa cuando el usuario presione, o arrastre un botón o elemento visual por la pantalla.

Ejemplo

Por ejemplo imagina la ventana de cualquier programa, por ejemplo la de un procesador de textos.

En esa ventana aparecen una gran cantidad de elementos gráficos interactivos, de forma que no es posible que el programador haya previsto todas las posibles entradas que se pueden generar por parte de un usuario en cada momento.

Con el control de flujo de ejecución de la programación imperativa, el programador tendría que estar continuamente leyendo todas las entradas (de teclado, o ratón, etc.).

Tras esto tendría que verificar para cada entrada o interacción producida por el usuario, de que se trata de entre todas las posibles, usando estructuras de flujo condicional (if then-else, switch) para ejecutar el código conveniente en cada caso.

Si piensas que para cada opción del menú, para cada botón o etiqueta, para cada lista desplegable, y por tanto para cada componente de la ventana, incluyendo la propia ventana, habría que comprobar todos y cada uno de los eventos posibles, nos damos cuenta de que las posibilidades son casi infinitas, y desde luego impredecibles.

Por tanto, de ese modo es prácticamente imposible solucionar el problema.

Para abordar el problema de tratar correctamente las interacciones del usuario con la interfaz gráfica de la aplicación hay que cambiar de estrategia, y la programación guiada por eventos es una buena solución, veremos cómo funciona el modelo de gestión de eventos.

Hoy en día, la mayoría de sistemas operativos utilizan interfaces gráficas de usuario. Este tipo de sistemas operativos están continuamente monitorizando su entorno para capturar y tratar los eventos que se producen.

El sistema operativo informa de estos eventos a los programas que se están ejecutando y entonces cada programa decide, según lo que se haya programado, qué hace para dar respuesta a esos eventos.

Cada vez que el usuario realiza una determinada acción sobre una aplicación que estamos programando en Java, un clic sobre el ratón, presionar una tecla, etc, se produce un evento que el sistema operativo transmite a la máquina virtual de Java.



Java crea un objeto de una determinada clase de evento, y este evento se transmite a un determinado método para que lo gestione.

El modelo de eventos de Java está basado en el concepto de delegación, es decir, la responsabilidad de gestionar un evento que ocurre en un objeto fuente la tiene otro objeto oyente.

Las fuentes de eventos (event sources) son objetos que detectan eventos y notifican a los receptores que se han producido dichos eventos. Ejemplos de fuentes:

– Botón sobre el que se pulsa o pincha con el ratón.

– Campo de texto que pierde el foco.

– Campo de texto sobre el que se presiona una tecla.

– Ventana que se cierra.

– Etc.

Cuando se crea una interfaz de usuario con ayuda de los asistentes del IDE, qie se sencillo que es realizar este tipo de programación, ya que el IDE hace muchas tareas, genera código automáticamente para el programador.

Pero también podrí hacer todo esto el programador, si no dispusiera un IDE como NetBeans, o porque simplemente deseare tener el control completo y hacerlo todo desde código, sin usar asistentes ni diseñadores gráficos.

En este caso, los pasos a seguir se pueden resumir en:

– Crear la clase oyente que implemente la interfaz.

  · Ej. ActionListener: pulsar un botón.

– Implementar en la clase oyente los métodos de la interfaz.

  · Ej. void actionPerformed(ActionEvent).

Tras esto hay que crear un objeto de la clase oyente y registrarlo como oyente en uno o más componentes gráficos que proporcionen interacción con el usuario.

Cada vez que se presiona un botón, se genera un ActionEvent y se pasa al método actionPerformed del listener, el cual, ejecuta el código que nosotros dispusimos en el manejador de evento para este evento.

Para ser capaz de responder, cada componente GUI interactivo necesita registrar a un oyente de eventos y necesita implementar un manejador de eventos. El manejador de eventos será el que responda al evento que se produce.

El IDE que se utilice, normalmente hace esta tareas. Se recomienda usar NetBeans el cual se encarga de enlazar el oyente de evento, de manera que el programador puede concentrarse en implementar la lógica de negocio que se tenga que ejecutar por el evento .

A continuación se va a desarrollador un ejemplo muy sencillo para ver estos tres pasos:

– En primer lugar se crea el programa con su método main.

– Este programa contiene en un panel dos etiquetas, la etiqueta uno y la etiqueta dos. Posteriormente tiene dos botones que se pueden presionar por el usuario.

– Se crea un objeto de la clase oyente y ese será el que se encargue de realizar las acciones cuando se produzca el evento correspondiente.

Ejemplo

```
public class Ventanilla extends JFrame {
JLabel etiqueta; // Declarar variables
JButton botonUno, botonDos;
JPanel panel;
public Ventanilla(){
// Creamos los componentes: una etiqueta,
// dos botones y un JPanel.
etiqueta = new JLabel(“La etiqueta: ”);
botonUno = new JButton(“Uno”);
botonDos = new JButton(“Dos”);
panel = new JPanel();
// Añadimos los componentes al panel
panel.add(etiqueta);
panel.add(botonUno);
panel.add(botonDos);
// Añadir el panel al Frame
getContentPane().add(panel);
// Crear objeto de la clase oyente para cuando se pulse el
botón
OyenteAccion oyenteBoton = new OyenteAccion();
// Registrar el objeto como oyente en los dos botones
botonUno.addActionListener(oyenteBoton);
botonDos.addActionListener(oyenteBoton);
}
}
```

Se implementa la clase oyente. Esta implementa el interfaz actionListener Además realiza las acciones que se van a llevar a cabo cuando se lance el evento correspondiente.

En este caso lo que hace es obtener el objeto botón que se ha pulsado. A partir de este objeto, se obtiene el valor del texto que contiene.

Este texto posteriormente se escribe en el cuadro de texto que había creado la aplicación, de esta manera el funcionamiento de programa consiste en que cuando el usuario pulsa un botón, se muestra en el cuadro de texto el botón pulsado.

Ejemplo

```
// Implementación en la clase oyente
class OyenteAccion implements ActionListener{
// Cuando se pinche en el botón
public void actionPerformed(ActionEvent evento){
// Obtener el botón que disparó el evento
JButton botón = (JButton) evento.getSource();
// Escribir en la etiqueta, el botón que se pulsó
etiqueta.setText("Botón pulsado: " + botón.getText());
}
```

4.2. Identificación de los elementos de la GUI

En el diseño de su interfaz, conviene ser consistente y predecible en la elección de elementos de la interfaz. Esto es que la interfaz presente un aspecto unificado en cada una de sus partes.

Ya sea que estén conscientes de ello o no, los usuarios se han familiarizado con los elementos que actúan de una manera determinada, por lo que decidir adoptar esos elementos en su caso ayudará con la realización de tareas, eficiencia y satisfacción.

Los elementos de la interfaz incluyen, pero no se limitan a:

– Controles de entrada: casillas de verificación, botones de radio, listas desplegables, cuadros de lista, botones, alternativos, campos de texto, campo de fecha.

– Componentes de navegación: barras, slider, campos de búsqueda, paginación, etiquetas o iconos.

– Componentes informativos: información sobre herramientas, iconos, barra de progreso, notificaciones, cuadros de mensaje, ventanas modales.

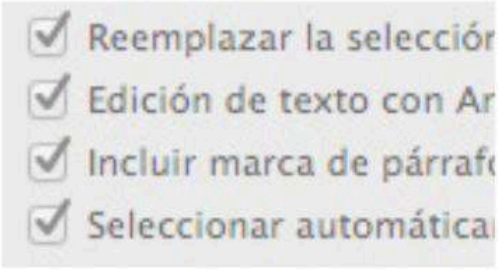
– Contenedores: elementos tipo acordeón, etc.

A continuación vamos a ir detallando cada uno de ellos por separado, especificando sus principales características y detallando una imagen de ellos.

Definición

Las **casillas de verificación:** las casillas de verificación permiten al usuario seleccionar una o más opciones de un conjunto.

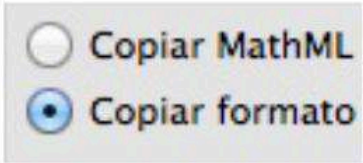
Por lo general, es mejor presentar casillas de verificación en una lista vertical. Más de una columna es aceptable también si la lista es lo suficientemente larga que podría requerir el desplazamiento o bien si podría ser necesario hacer comparaciones de términos.



Casilla de verificación

Definición

Los botones de radio: los botones de radio se utilizan para permitir a los usuarios seleccionar un elemento cada vez.

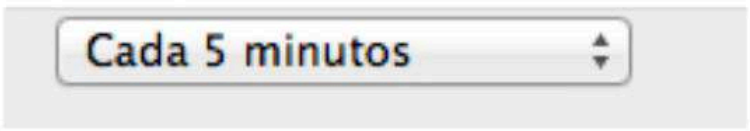


Botón de radio

Definición

Listas desplegables: listas desplegables permiten a los usuarios seleccionar un elemento cada vez, de manera similar a los botones de radio, pero son más compactos lo que permite ahorrar espacio.

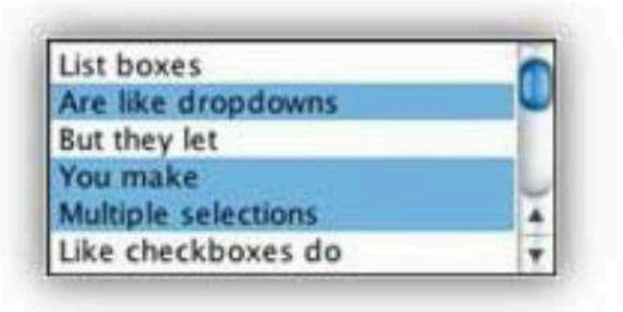
Conviene agregar texto al campo, como por ejemplo la frase “Seleccione uno” para ayudar al usuario a reconocer las acciones necesarias



Lista desplegable

Definición

Las listas de cuadro: estas listas, como las casillas de verificación, permiten a los usuarios seleccionar múltiples elementos a la vez, pero son más compactos y pueden soportar una lista de opciones más tiempo si es necesario.



Lista de cuadro

Definición

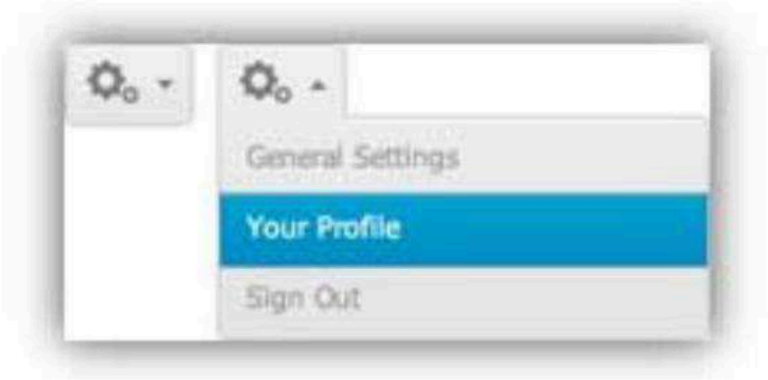
Botones: un botón indica una acción que se realiza al presionar y lleva una etiqueta que es generalmente de texto, un icono o ambos.



Ejemplos de botones

Definición

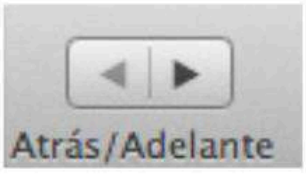
Botón desplegable: el botón desplegable consiste en un botón que al hacer clic muestra una lista desplegable de elementos mutuamente excluyentes.



Ejemplos de botones desplegables

Definición

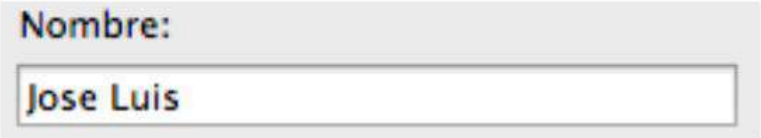
Alternativos: un botón de activación o alternativos permite al usuario cambiar la configuración entre dos estados. Estos son más eficaces cuando los estados de encendido / apagado son visualmente distintos.



Botones alternativos

Definición

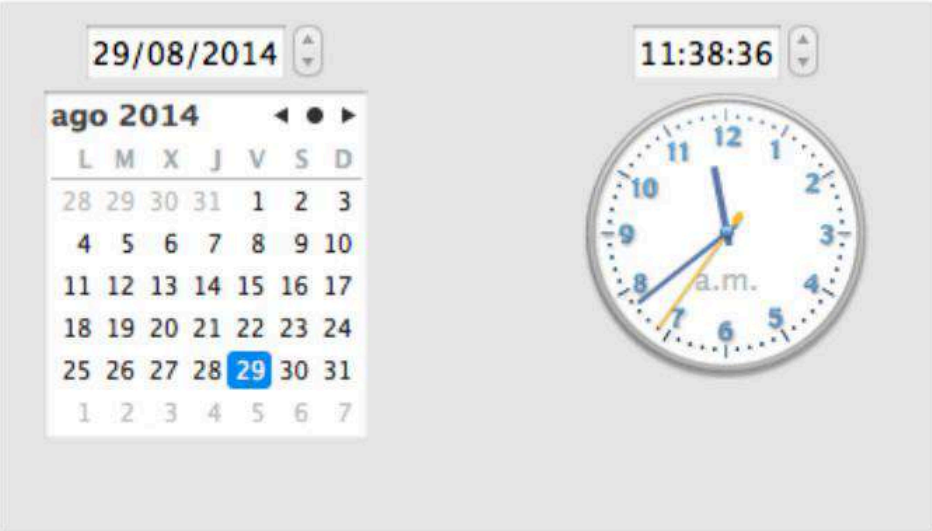
Los campos de texto: los campos de texto permiten a los usuarios introducir texto. Se puede definir, ya sea teniendo una sola línea o bien con varias líneas de texto.



Campos de texto

Definición

Selector de Fecha y hora: un selector de fechas permite a los usuarios seleccionar una fecha y / u hora. Al utilizar el selector, el programador se asegura que la información obtenidos tiene el formato adecuado y la entrada es consistente con el sistema.



Selector de fecha y hora

Los componentes de navegación indican al usuario donde se encuentra dentro de la aplicación, y a obtener información, correspondiente.

Definición

Campo de búsqueda: un cuadro de búsqueda permite a los usuarios introducir una palabra clave o frase (query) y lo utilizará para buscar en el índice o en la base de datos, con la intención de devolver los resultados más relevantes.

Normalmente los campos de búsqueda son de una sola línea cuadros de texto y, a menudo se acompañan de un botón de búsqueda.



Campo de búsqueda

Barra de situación: La barra de situación o "Breadcrum" ("miguitas de pan") permite a los usuarios identificar su ubicación actual dentro del sistema, proporcionando un rastro por el que volver si es necesario



Barra de situación

Paginación: paginación divide el contenido en las páginas, y permite a los usuarios saltar entre las páginas o ir en orden a través del contenido.

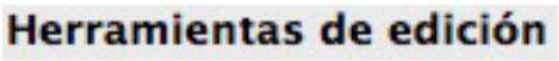


Paginación

Definición

Etiquetas: las etiquetas permiten a los usuarios encontrar contenido en la misma categoría.

Algunos sistemas de etiquetado también permiten a los usuarios aplicar sus propias etiquetas a los contenidos mediante la introducción en el sistema.



Etiquetas

Definición

Sliders: un control deslizante, también conocida como barra de seguimiento o Slider, permite a los usuarios configurar o ajustar un valor.



Control deslizante

Cuando el usuario cambia el valor, esto generalmente no cambia el formato de la interfaz u otra información en la pantalla.

Definición

Iconos: un icono es una imagen simplificada para servir como un símbolo intuitiva que se utiliza para ayudar a los usuarios a navegar por el sistema.



Icono del sistema

Por lo general, los iconos contienen hipervínculos o están asociados a un botón.

Definición

Carrusel de imágenes: los carruseles de imágenes permiten a los usuarios navegar a través de un conjunto de artículos o imágenes y hacer una selección de uno si así lo desean.



Carrusel de imágenes

Definición

Notificaciones: una notificación es un mensaje de actualización que anuncia algo nuevo para que el usuario lo vea.

Las notificaciones se suelen utilizar para indicar elementos tales como la finalización con éxito de una tarea o un mensaje de error o advertencia.

Definición

Barras de progreso: una barra de progreso indica a un usuario cómo es el avance a medida que avanza a través de una serie de pasos en un proceso.

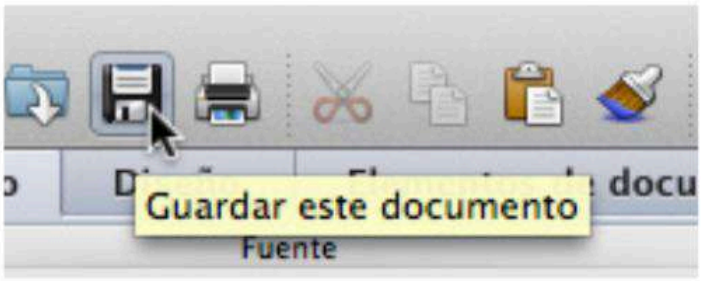
Por lo general, en las barras de progreso no se puede hacer clic.



Barra de progreso

Definición

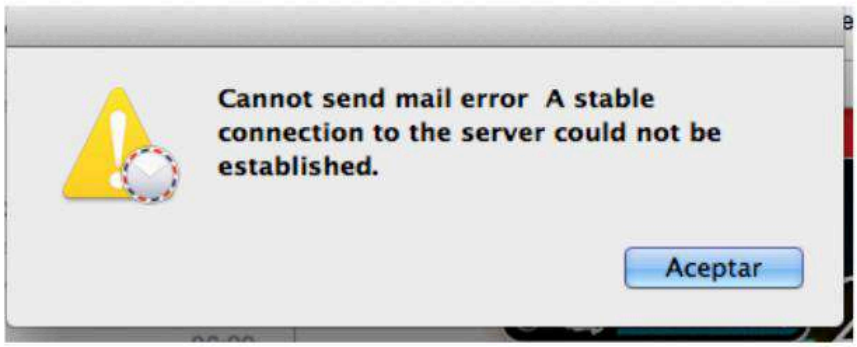
Sugerencias de herramientas: una información sobre herramientas le permite al usuario ver consejos cuando pone el cursor sobre un elemento, este generalmente indica el nombre o el propósito del elemento



Sugerencia

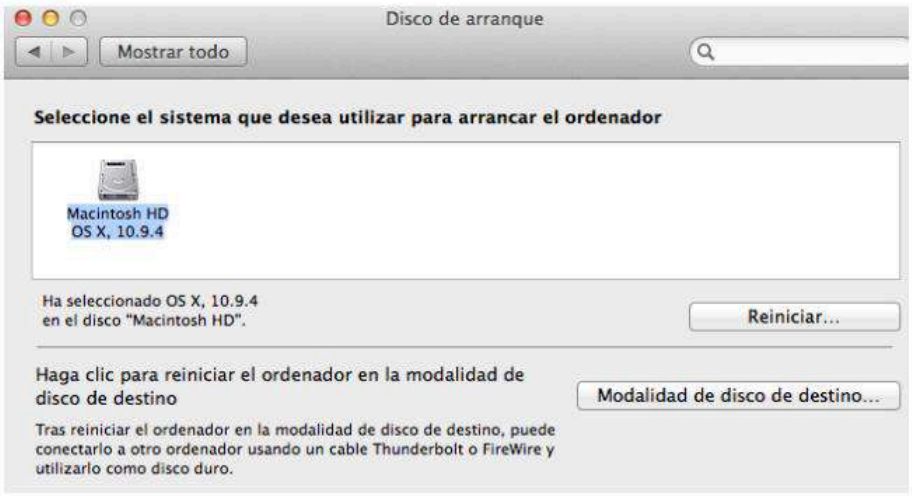
Definición

Cuadros de mensaje: un cuadro de mensaje es una pequeña ventana que ofrece información a los usuarios y les obliga a realizar una acción antes de que puedan avanzar.



Cuadro de mensaje

Ventana modal o pop-up: una ventana modal requiere que los usuarios interactuar con ella de alguna manera antes de que puedan volver al sistema. No permite que los usuarios cambien de ventana hasta que no realicen determinada acción.



Ventana modal

Definición

Un acordeón es una lista de apilamiento vertical de los objetos que utiliza un modo mostrar / ocultar para visualizar su funcionalidad.

Cuando se hace clic en una etiqueta, se expande la sección que muestra el contenido almacenado en su interior.

No puede tener uno o más elementos que muestran a la vez y pueden tener estados predeterminados que revelan una o más secciones sin el usuario haga clic.



Acordeón

Importante

Java incluye las bibliotecas para proporcionar soporte multiplataforma para los objetos de interfaz gráfica de usuario.

El aspecto "multi-plataforma" de esto es que usted puede escribir un programa en un Macintosh y tienen los mismos objetos gráficos que aparecen cuando el programa se ejecuta bajo UNIX o Windows (salvo algunos problemas menores con el ajuste de objetos en la pantalla).

Componentes GUI de Java incluyen etiquetas, campos de texto, áreas de texto, botones, etc

El Abstract Windowing Toolkit (AWT) también incluye recipientes que pueden incluir estos componentes. Contenedores incluyen marcos (ventanas), lienzos (que se utilizan para dibujar en ellos), y paneles (que se utilizan para los componentes de grupo).

Paneles y lienzos están contenidas en los marcos (ventanas), mientras que los botones y otros componentes se pueden colocar directamente en cuadros o en paneles dentro de los marcos.

Estos componentes GUI se dibujan automáticamente cada vez que la ventana en la que se encuentran se dibuja.

Por lo tanto no es necesario indicar de forma explícita en ellos los comandos para pintarlos. De esta manera el propio lenguaje de programación se encarga de refrescar el componente cada vez que se modifica la ventana.

Estos componentes GUI son manejados usando modelo de eventos de Java.

Cuando un usuario interactúa con un componente (hace clic en un botón, cambia los tipos en un campo, elige de un menú pop-up, etc), se genera un evento por el componente con el que ha interactuado.



Para cada componente de su programa, se requiere que el programador designe uno o más objetos que deben "escuchar" los acontecimientos de ese componente.

Por lo tanto, si su programa tiene un botón con la etiqueta "inicio" debe asignar uno o más objetos que serán notificados cuando un usuario hace clic en el botón.

Comenzamos nuestro análisis de la programación en Java GUI con una discusión de los botones con más detalle.



Botones: button es una clase en el paquete java.awt que representa botones en la pantalla.

El constructor es:

```
Button public (String etiqueta);
```

que, cuando se ejecuta, crea un botón con “etiqueta” impreso en ella.

Generalmente, el botón es lo suficientemente grande para mostrar la etiqueta. También hay un constructor sin parámetros que crea un botón sin marcar.

Los botones responden a una variedad de mensajes. Por ejemplo, puede agregar un “ActionListener” a un botón con el método:

```
public void addActionListener (ActionListener listener);
```



Agregar botones a un marco o panel: podemos añadir un botón a un marco o panel enviando el mensaje:

```
myFrame.add (startButton);
```

Normalmente incluimos dicho código en el constructor que crea el marco o panel para que estos sean agregados desde el principio.

De ahí que por lo general sólo escribimos this.add (startButton) o simplemente añadimos add(startButton).

A continuación se muestra un código de ejemplo para una clase, ButtonDemo, lo que representa una especialización de Marco con dos botones.

La clase se extiende Frame, que es parte de java.awt.

El constructor de Frame toma un parámetro de cadena y crea una ventana con la cadena en la barra de título.

El constructor de ButtonDemo llama al constructor de la superclase, y luego establece el tamaño del nuevo marco a ser de 400 x 200

El comando setLayout define el nuevo marco en el que se deben agregar nuevos componentes de izquierda a derecha en el panel.

Si se queda sin espacio se añadirán nuevos componentes en las nuevas filas, de nuevo de izquierda a derecha.

El código para la de botones creación es autoexplicativo. Los comandos añaden a los botones para el marco. Explicamos la clase buttonListener continuación.



Esta clase crea los objetos que realizan acciones en respuesta a las pulsaciones de botón.

Por lo tanto les decimos a los dos botones (con el método addActionListener) para notificar myButtonListener cuando el usuario hace clic en ellos.

Por último, tenga en cuenta que teníamos que importar java.awt. * Porque Frame, Button y FlowLayout son clases del paquete java.awt.

Ejemplo

```
import java.awt.*;

public class ButtonDemo extends Frame

{

 protected Button startButton, stopButton;

 //EL constructor añade las características del botón y lo
coloca

 public ButtonDemo()

 {

 super("Button demo"); // Llama al constructor del marco

 setSize(400,200); // Coloca el tamaño de la ventana

 // crea dos botones con las etiquetas start y stop

 startButton = new Button("Start");
```

```
 stopButton = new Button(“Stop”);
 add(startButton); // se los añade al frame
 add(stopButton);
 // crea un objeto que escucha ambos botones
 ButtonListener myButtonListener = new ButtonListener();
  // tle indica a los botones a quien deben notificar los
eventos
 startButton.addActionListener(myButtonListener);
 stopButton.addActionListener(myButtonListener);
      setVisible(true); // muestra la ventana por pantalla
 }
 // Programa main
 // crea el objeto y lo muestra
 public static void main(String args[]){
 // Create an instance of Buttons
 ButtonDemo app = new ButtonDemo();
 }
}
```

El programa principal asociado con la clase simplemente crea un objeto de la clase, que se mostrará en la pantalla en sí.



Los programas principales asociados con marcos casi siempre hacen esta cantidad mínima de trabajo.

ActionListeners para Botones: Los objetos que implementan la interfaz Action-Listener están obligados a implementar un método actionPerformed que toma un parámetro de tipo ActionEvent.

Cuando se produce una acción a un botón, todos los objetos que se han añadido como ActionListener de ese botón se notifica a llamando a su método actionPerformed con un parámetro con información sobre el evento exacto que ocurrió.

Así, el sistema crea automáticamente el objeto ActionEvent y lo envía junto con el oyente.

Los métodos más útiles de ActionEvent son:

- `public String getActionCommand ();`
- `getSource public Object ();`



Si envía getActionCommand a un ActionEvent que representa un clic de un botón, se devolverá una cadena correspondiente a la etiqueta del botón.

getSource devuelve una referencia al objeto que se produjo el evento. El buttonListener clase a continuación implementa ActionListener, y por lo tanto un objeto de esta clase se puede añadir a un botón como un oyente.

El método actionPerformed es muy simple. Utiliza getActionCommand para obtener la etiqueta del botón se hace clic y luego dependiendo de lo que era en el botón, se imprime un mensaje diferente.

Recordemos que el mismo ActionListener esta en cada uno de los dos botones en ButtonDemo, por lo getActionCommand es el que tiene que averiguar cuál se hizo clic.

Hay que tener en cuenta que debido a que esta clase es tan simple que ni siquiera tiene variables de instancia y por tanto no necesita un constructor explícito.

En su lugar, tiene un constructor predeterminado, sin parámetros, que sólo asigna almacenamiento para nuevos objetos construidos.

Hay que considerar que se debe importar el paquete java.awt.event porque ActionEvent y la interfaz ActionListener son a la vez en ese paquete.

Ejemplo

```
import java.awt.event.*;

public class ButtonListener implements ActionListener{

	public void actionPerformed(ActionEvent evt)

	{

		// Obtiene la etiqueta del botón que se le ha pasado como parámetro

	String arg = evt.getActionCommand();

	if (arg.equals(“Start”))

		System.out.println(“Start button”);

	else if (arg.equals(“Stop”))

		System.out.println(“Stop button”);

	}

}
```



Clases Internas: Puede parecer un poco pesado tener que crear una clase completamente separada (que va en un archivo separado) con el fin de crear un oyente para los dos botones en nuestra clase ButtonDemo.

Se puede optar por dos alternativas: Una es dejar que el propio marco sea el ActionListener para el botón. Podemos hacer esto cambiando el encabezado de clase original para que diga:

```
public class ButtonDemo extends Frame implements ActionLis-
tener
```

También debemos copiar el método actionPerformed de la clase buttonListener, y por último, cambiar las dos declaraciones añadiendo el ActionListener que debe tener el objeto en sí lo hacen que participe en la escucha:

```
// Indicar a los botones que el objeto this debe ser noti-
ficado

startButton.addActionListener (this);

stopButton.addActionListener (this);
```

Este es el estilo sugerido en Java para el manejo de eventos de acción en casos sencillos. La ventaja es que ahora sólo tenemos una clase en lugar de dos.



Sin embargo, en general es un buen diseño para separar el código para la construcción y el manejo de componentes GUI desde el código que en realidad hace el trabajo generado por los eventos de acción.

Además, hay otro estilo que es casi tan simple, pero más general. Esto implica el uso de las llamadas “clases internas”.

Java permite que las clases pueden anidar. A menudo, cuando se trata de eventos tendremos una clase especial que es tan única que realmente sólo va a ser usada en conjunto con otra clase.

En lugar de hacer pública esta clase auxiliar, puede dejarse en el interior de la otra clase.

Así podríamos incluir la clase buttonListener interior de ButtonDemo.

El código ahora tendría la siguiente estructura:

Ejemplo

```
 import java.awt.*;
import java.awt.event.*;
public class ButtonDemo extends Frame
{
 protected Button startButton, stopButton;
 public ButtonDemo()
 {
 ...
 // crea el objeto que escucha ambos botons
 ButtonListener myButtonListener = new ButtonListener();
 // tell buttons that myButtonListener should be notified
 startButton.addActionListener(myButtonListener);
 stopButton.addActionListener(myButtonListener);
 }
 public static void main(String args[]){
 ... // igual que antes
 }
 protected class ButtonListener implements ActionListener{
 public void actionPerformed(ActionEvent evt)
 {
 // obtiene la referencia al objeto que ha recibido
 Object source = evt.getSource();
 if (source == startButton)
 System.out.println("Start button");
 else if (source == stopButton)
```

```
 System.out.println("Stop button");
 }
 }
}
```

Observe que buttonListener ahora declaró protegida, lo que significa que sólo se puede utilizar dentro de la clase que contiene, ButtonDemo.

El método actionPerformed es todavía público, sin embargo.

Si fuera declarada protegido, entonces sólo estaría disponible desde dentro buttonListener y no dentro ButtonDemo (y tenemos que usarlo en el constructor de ButtonDemo).

Estas clases anidadas ahora se encuentran dentro de un solo archivo llamado ButtonDemo.java.



Otra ventaja de utilizar clases anidadas es que todas las variables de instancia (y métodos) de la clase externa son visibles dentro de la clase interna.

Esto puede ser útil para recoger información de otros componentes de la clase externa.

Tenga en cuenta especialmente que ahora podemos comprobar el origen real del evento en lugar de para la etiqueta del botón.

Otros componentes de la GUI se detallan a continuación



Las etiquetas: una etiqueta es un componente muy simple que contiene una cadena.

Los constructores son:

```
Label public () // crea etiqueta sin texto

Label public (String text) // crear etiquetas con texto
```

Los métodos disponibles son

```
public String getText () // devolver texto de la etiqueta

public void setText (String s) // establece el texto de la etiqueta
```

Sin embargo, rara vez cambiamos el texto de las etiquetas.



Los campos de texto: un TextField es una sola área con una línea en la que el usuario puede escribir.

Es una buena manera de conseguir la entrada de texto por parte del usuario se utiliza cuendo un usuario debe de introducir un texto que no sea demasiado grande. Los constructores de la clase son:

```
TextField public ()

// crea campo de texto sin texto

TextField public (int columnas)

// Crear campo de texto conun numero de columnas

TextField public (String s)

// Crear campo de texto con la cadena que se pasa

TextField public (String s, int columnas)

// Crear un campo de texto con cadena y columnas
```

Y los métodos más comunes que se suelen usar a la hora de interactuar con un cuadro de texto son los que se indican a continuación:

```
 public setEditable( boolean b)

// Si b es falso el TextField no es editable por el usuario

 public String getText ()

// devolver texto de la etiqueta

 public void setText (String s)

// establece el texto de la etiqueta
```

Muchos otros métodos también están disponibles para este componente (consulte también la documentación de su superclase, TextComponent).

Cuando el usuario escribe en un campo de texto y luego presiona la tecla Retorno o Intro, se genera un evento que puede ser manejado por el mismo tipo de ActionListener utilizado con botones.

Sin embargo, en este caso no hay ninguna cadena útil para ser de regresar de getActionCommand. Aquí se utiliza el método getSource.

Así, con el fin de responder a las entradas del usuario de la tecla de retorno mientras se escribe en un TextField myField, podemos escribir:

Ejemplo

```
 public void actionPerformed (ActionEvent evt)

 {

 if (evt.getSource () == myField) {

 contenido = myField.getText ();

 System.out.println (“El campo contenía:” + contenido);

 } Else ...

 }
```



Conviene observar que este uso de myField en actionPerformed sólo tiene sentido si bien tienen una clase interna y por lo tanto tener acceso a todas las variables de instancia de la clase externa, o si el marco o panel donde se define myField es el ActionListener asociado.



Áreas de Texto: TextArea es una clase que proporciona un área para mantener varias líneas de texto.

Es bastante similar al TextField salvo que ningún evento especial se genera pulsando la tecla de retorno.

Los constructores son.

```
TextArea public (int filas, columnas int)

// Crear área de texto con filas y columnas apropiadas

TextArea public (String s, int filas, en las columnas)

// Crear área de texto con filas, columnas, y s en su interior
```

Métodos mas utilizados de TextArea son los siguientes:

```
public setEditable (boolean s)

// Si es falso el TextArea no es editable por el usuario

public String getText () // devuelve texto en TextArea

public void setText (String s) // establece el texto
```

4.3. Presentación del diseño orientado al usuario. Nociones de usabilidad

El diseño del software parte de la especificación de requisitos del software obtenidas durante el análisis y a partir de ahí en esta fase he de construirse una guía efectiva, un "plano" que permita llevar a cabo la implementación del sistema.

Hay que tener en cuenta que es la etapa más crítica para la construcción de un sistema de calidad pues es en el diseño donde se convierten los requisitos del cliente en un sistema software.

Un buen diseño es imprescindible para implementar un software de calidad. Las características que indican que un diseño es de calidad son las siguientes:

– El diseño debe implementar todos los requisitos explícitos del software así como ajustarse también a los requisitos implícitos.

– El diseño deberá servir de guía legible y comprensible para los programadores lleven a cabo la fase de codificación.

– El diseño deberá proporcionar una imagen completa del software que se está desarrollando, es decir, igual que durante el análisis se modelaba el problema a solucionar, durante el diseño debe de crearse un modelo de la solución adoptada para este problema (el sistema).

– El proceso de diseño es una secuencia de pasos que hacen posible que el diseñador describa todos los aspectos del software que va a construir, representando en un principio la totalidad y posteriormente refinando en visiones parciales y más específicas el sistema.

A la hora de diseñar, conviene seguir una serie de "principios de diseño" enunciados por Davis:

– Durante el diseño siempre hay que tener en cuenta enfoques alternativos.

– El diseño y cada uno de sus elementos deberá poder rastrearse hasta el modelo de análisis.

– El diseño no debería inventar nada que no esté inventado, conviene recurrir a patrones de diseño (situaciones de diseño tipificadas para las que se da una solución estándar) siempre que sea posible.

- El diseño deberá minimizar la distancia intelectual entre el el software y el problema, es decir, la estructura del diseño del software deberá imitar todo lo que sea posible la estructura del dominio del problema.

- El diseño deberá presentar uniformidad e integración entre sus partes, como si lo hubiera realizado completo una sola persona.

- El sistema diseñado debe de estructurarse para que se degrade poco a poco ante datos, recursos o condiciones aberrantes.

- Diseñar NO es codificar, no codificar es diseñar.

Debe de revisarse el diseño para tratar de minimizar errores conceptuales.

A la hora de realizar un diseño debemos de trabajar con dos conceptos básicos, abstracción y modularidad:



La **abstracción** permite concentrar el esfuerzo de diseño en un problema a un nivel de generalización sin necesidad de considerar los detalles de más bajo nivel.

El **refinamiento** es el concepto opuesto a la abstracción; consiste en descomponer una abstracción de un nivel determinado (del tipo que sea) en varias abstracciones simples de un nivel inferior.

Modularidad: un sistema software debe de dividirse en componentes nombrados, que permitan ser diseñados (e implementados) por separado; a estos componentes se les denomina módulos, que posteriormente se integrarán para satisfacer los requisitos del problema.

La tarea de diseño debe de producir varios resultados:

- El diseño de datos debe de transformar el modelo del dominio de la información que se crea durante el análisis de requisitos en estructuras de datos implementables en el sistema.

- El diseño arquitectónico se encarga de definir la relación entre los principales elementos estructurarles del software (los módulos) y que estas relaciones cumplan los requisitos definidos en el análisis.

– El diseño de la interfaz debe describir como se comunica el software internamente, con otros sistemas y con los usuarios que lo van a manipular.

– El diseño a nivel de componentes se encarga de transformar los elementos estructurales de la arquitectura del software (los módulos)en una descripción procedimental detallada de estos elementos que sirva de base a la tarea de codificación.

El diseño de la interfaz de usuario es la fase del diseño que trata de crear un medio de comunicación entre el hombre y la máquina.

Su correcto desarrollo es de vital importancia para la aceptación del sistema final por las personas que lo van a utilizar, ya que cualquier sistema informático, independientemente de su potencia o funcionalidad, si no está acompañado por una buena interfaz no será del agrado de los usuarios.



Nunca hay que olvidar cuando se diseña que la interfaz de usuario es la percepción que tiene estos del sistema software.

A la hora de diseñar una buena interfaz conviene seguir una serie de criterios (las tres “reglas de oro” de Mantel):

– Dar el control al usuario: en muchas ocasiones el diseñador introduce restricciones un la interfaz para simplificar su construcción que pueden dar lugar a sistemas frustrantes de utilizar. Dar el control al usuario puede detallarse en:

  · Definir los modos de interacción de manera que no obligue al usuario a realizar acciones innecesarias y no deseadas. Por ejemplo forzar a que el usuario siempre corrija la ortografía de un mensaje de correo que se va a enviar.

  · Tener en consideración una interacción flexible, por ejemplo una aplicación debe poder manejarse solamente con el teclado, solamente con el ratón o bien con una combinación de ambos.

  · Permitir que la interacción que está realizando el usuario en un momento determinado pueda interrumpirse y deshacerse.

- Aligerar la interacción a medida que avanza el nivel de conocimientos del usuario, utilizando por ejemplo atajos de teclado.

- Ocultar al usuario los entresijos técnicos de la aplicación.

- Permitir interaccionar directamente con todos los objetos que aparezcan en la pantalla (si hay un botón, que se pueda pulsar).

– Reducir la carga de memoria del usuario: cuantas más cosas tenga que recordar el usuario, más propenso a errores será su interacción con el sistema. Puede desarrollarse en:

  - Reducir la carga de memoria a corto plazo, evitando que el usuario tenga que recordar datos entre una interacción y otra.

  - Establecer valores por defecto útiles.

  - Definir las deficiencias para que sean intuitivas: Cuando sea necesario implementar una acción basada en un mnemónico, conviene hacerlo de la manera más intuitiva posible.

    Aunque parezca una contradicción, con un ejemplo se visualiza mejor esta regla: Utilizar CTRL-C para copiar es una acción nada intuitiva que se basa en un mnemónico, sin embargo es mucho mas intuitiva (Control-Copiar) que por ejemplo CTRL-V para pegar.

  - El formato visual de la interfaz debe basarse en una metáfora del mundo real.

    Un ejemplo de esto es la “metáfora de escritorio” usado por muchos sistemas operativos, en la que el ordenador es una “mesa” o “escritorio” sobre el que el usuario despliega “carpetas” y “documentos”

– La información debe de desglosarse de manera progresiva. La construcción de una interfaz consistente: la información deberá ser presentada y adquirida por la interfaz de forma consistente, esto es:

  - La información estará organizada en base a un criterio estándar en toda la aplicación.

  - Los mecanismos de entrada se limitarán a un conjunto definido y se utilizarán coherentemente a lo largo de toda la aplicación.

  - Los mecanismos para ir pasando de tarea en tarea estará definidos e implementados correctamente.

A la hora de construir la interfaz de usuario entran en juego cuatro modelos diferentes, el modelo de diseño, el modelo de usuario, la percepción de usuario y el modelo del sistema.

– Modelo de diseño: modelo del sistema completo que crea el ingeniero del software, que incluye diseño arquitectónico, de dotas, de procedimientos y de interfaces.

  En este modelo se determinan las entradas y salidas que debe recibir o enviar el sistema a sus usuarios

– Modelo de usuario: representa el perfil de los usuarios del sistema que tienen los analistas e ingenieros del software.

  Para construir una interfaz eficaz, es necesario comenzar por conocer la mayor cantidad posible de características de los usuarios finales.

  Los usuarios pueden además catalogarse en varias categorías, y es conveniente que la interfaz de usuario se adapte a todas las categorías aunque dependiendo del estudio del perfil se podrá incidir más en una u otra:

  · Principiantes: No tienen conocimientos ni a nivel sintáctico ni a nivel semántico de la utilización de la aplicación, es decir, no saben ni como se hacen las acciones ni porqué se hacen las acciones.

  · Usuarios esporádicos y con conocimientos: Poseen un razonable conocimiento semánticos (es decir, saben en principio que acciones debe permitir el sistema) pero pocos conocimientos sintácticos (no saben como se hacen las cosas), además de una baja retención de la información necesaria para manejar la interfaz.

  · Usuarios frecuentes y con conocimientos: Poseen conocimientos elevados tanto a nivel semántico como sintáctico y es posible que busquen como sacar el máximo partido a la interfaz, utilizando por ejemplo modos abreviados de interacción.

– Percepción del usuario (en alguna bibliografía también se denomina “modelo de usuario”, pero el concepto es completamente diferente al anterior): Consiste en la imagen mental que se crea el usuario de cómo es y como funciona el sistema a través de lo que le transmite la interfaz.

  Esta percepción también está muy influida por el tipo de usuario que se trate. Dependiendo de su el usuario tiene o no conocimientos sintácticos y semánticos de la aplicación, esta se verá influida en un sentido u otro.

– Imagen del sistema: es el modelo del sistema que el diseñador de la interfaz quiere transmitir al usuario a través de ésta y de la información suministrada junto con el sistema (ayuda, manuales de usuario,…).



Un buen diseño de la interfaz es aquel en el que le Imagen del sistema y la percepción del usuario coinciden adecuadamente, lo que hace que los usuarios se sientan a gusto con el software y con su funcionamiento.

Para conseguir esto, el diseñador de la interfaz debe de unir la información procedente del modelo de diseño con el modelo de los usuarios para crear una imagen del sistema que refleje de manera veraz y entendible para el perfil del usuario definido la información tanto a nivel sintáctico como semántico.

Los modelos de diseño de la interfaz anteriormente descritos son "abstracciones de lo que el usuario está haciendo o de lo que cualquier otra persona (ingenieros, analistas, diseñadores,…) piensa que debería estar haciendo cuando usa un sistema interactivo"

Plantearse dichos modelos básicamente ayuda a la tarea básica y esencial del diseño de la interfaz, conocer al usuario.

El objetivo del proceso de diseño de la interfaz es definir un conjunto de objetos y acciones sobre estos objetos que posibiliten al usuario llevar a cabo todas las tareas que se consideren necesarias para que el sistema cumpla con los objetivos de usabilidad obtenidos en el análisis de requisitos.

El proceso a seguir, brevemente enunciado, comienza por la identificación de los requisitos desde el punto de vista del usuario.

Estos s sirven para crear e identificar los escenarios de usuario que posteriormente se utilizan como base para definir el conjunto de objetos y de acciones de la interfaz.

A partir de este conjunto de objetos se creará el formato de la pantalla y se especificarán los elementos principales de éste.

Por último se generará un prototipo que será validado por los usuarios finales.

El proceso anteriormente descrito puede iterarse tantas veces como sea necesario hasta diseñar una interfaz que cumpla con las características que se han comentado anteriormente.

El proceso de diseño, desglosado tiene cuatro partes:

Análisis y modelado de las tareas: Un sistema interactivo basado en computadora se utiliza para reemplazar una actividad manual o semi-manual.



Es necesario entender las tareas que se realizan de manera manual como primer paso, para posteriormente hacerlas corresponder con un conjunto de tareas similares que se implementan en la interfaz de usuario.

Este primer paso de diseño busca estas tareas, o escenarios de usuario, que se utilizaran como punto de partida para el siguiente paso de diseño.

Identificación de los objetos y acciones a realizar: En esta fase se analizan los escenarios de usuario anteriormente identificados buscando objetos que el usuario va a utilizar.

Además e identifican también las acciones que se puedan realizar sobre estos objetos para llevar a cabo la tarea.

Formato de pantalla: A cada objeto identificado se le dota de un diseño gráfico (un icono, un botón, un menú…) asociando las acciones a eventos que el usuario produzca sobre este objeto.

Los objetos se distribuyen por la pantalla siguiendo una lógica (como un estándar o bien de manera coherente con el resto del sistema).

En esta fase además de define el texto explicativo que aparecerá en la interfaz, el tamaño, ubicación y título de las ventanas, texto descriptivo de los menús y en general todos los elementos visuales y sus características que aparezcan en la interfaz.

Implementación de un prototipo: Una vez realizado un modelo del diseño de la interfaz este debe ser implementado como prototipo y evaluado por los usuarios que validarán el diseño o sugerirán mejoras y críticas que sirvan para refinar este.



A la hora de construir una interfaz de usuario, o un prototipo de ésta suelen utilizarse lenguajes de muy alto nivel (4ª Generación) que permiten definir fácilmente la interfaz de usuario.

Estos lenguajes y las herramientas que los soportan permiten crear la interfaz sin necesidad de escribir código, sino simplemente arrastrando, soltando y distribuyendo por la pantalla los elementos del modo que se desee. Un ejemplo de este tipo de lenguajes es Visual Basic.

Para la construcción de prototipos también pueden utilizarse componentes reusables de software, como por ejemplo los javaBeans, que hacen que la interfaz pueda construirse a base de encapsular componentes ya desarrollados uniéndolos o bien de manera gráfica con una herramienta adecuada (en caso de javaBeans se pueden citar VisualAge o JBuilder) o bien escribiendo código pero reutilizando la mayoría de estructuras lo que permite una construcción muy rápida.

A medida que la interfaz de usuario se va desarrollando suelen surgir casi siempre cuatro tipos de problemas comunes a casi todos los diseños de interfaces, el tiempo de respuesta, los servicios de ayuda, la información de error y el etiquetado de órdenes.



Tiempo de respuesta: está caracterizado por dos valores, su duración y su variabilidad.

La primera asociada con la media de los tiempos de respuesta y la segunda con su varianza.

Si la duración es muy larga, el usuario puede frustrarse y si es demasiado corta puede ocasionar que el usuario se precipite y cometa errores.

La variabilidad se refiere a la desviación del tiempo de espera promedio, una variabilidad baja permite al usuario mantener un ritmo de trabajo constante.

Es conveniente incluso aumentar la duración del tiempo de respuesta si se consigue con eso disminuir su variabilidad.



Servicios de ayuda al usuario: Los tipos de ayuda más frecuentes son ayudas integradas y ayudas complementarios.

Las ayudas integradas se añaden al software desde el principio de su construcción y son sensibles al contexto en el que se encuentra el usuario. Es por esto son mucho más útiles que el resto de ayudas

De hecho su principal característica es que le permite es buscar rápidamente ayuda sobre las acciones que está llevando a cabo en este momento. De tal forma que la ayuda se va a ir integrando en el trabajo cotidiano de la aplicación si estorbar al usuario ni interrumpir su flujo de trabajo.

Una función de ayuda complementaria se añade a la interfaz tras su construcción y se parece más a un manual de usuario en línea y es posible que el usuario tenga que buscar entre largos índices antes de encontrar lo que busca.

Siempre es preferible el primer tipo de ayuda al segundo ya que es sensible al contexto y permite optimizar la productividad del usuario. De hecho el segundo tipo de ayuda conviene que sea solamente un apoyo al primero, ya que si no seguramente no va a cumplir con su labor.



Información de errores: Los mensajes de error deben cumplir tres características: describir el error, ayudar a resolverlo y describir las consecuencias:

– Describir el error ocurrido de manera que el usuario lo entienda.

– Proporcionar conejos útiles para recuperarse del error.

– Indicar las consecuencias o las posibles consecuencias del error.



Etiquetado de órdenes: en caso de que se proporcione la posibilidad de interaccionar con teclado y ratón, hay que plantearse la forma de las órdenes de teclado(teclas de función, combinación de teclas, comandos,...), que haya órdenes para todos los elementos de los menús y la posibilidad de que el sistema recuerde las últimas órdenes enviadas.



El documento que se produce durante la fase de diseño y que recopila todas las actividades llevadas a cabo en el se denomina “especificación de diseño” y debe de irse completando a medida que los diseñadores van realizando este.

Los aspectos que deben de tratarse en este documento son los siguientes:

– Describir el ámbito global del diseño, básicamente un resumen de la información recogida en la fase de análisis que sirve de punto de partida.

– Diseño de datos: Diseño de las bases de datos que se van a utilizar así como de las estructuras de los archivos y estructuras de datos que use el sistema.

– Diseño arquitectónico, en el que se describirá como se ha derivado desde el modelo de análisis la arquitectura del programa

– Diseño de las interfaces tanto externas como internas del sistema.

  Además aquí se describirá como se ha diseñado la interfaz de usuario.

– Diseño de componentes: cada uno de los componentes del sistema se describirá haciendo uso de una narrativa de proceso y utilizando también alguna de las técnicas descriptivas que se conozcan

– Referencias al análisis de requisitos, normalmente en forma de matriz de manera que se establezca claramente que se han implementado todas los requisitos.

  Además es conveniente que se indique que componentes son críticos a la hora de implementar un requisito específico.

– Restricciones de diseño, como limitaciones físicas, de memoria o necesidades de una interfaz especializada.

Como último apartado pueden añadirse una serie de anexos con datos complementarios, descripciones de algoritmos, alternativas de diseño, resúmenes de otros documentos relevantes.

Como apéndice sería interesante desarrollar también un manual de usuario preliminar que además dará una idea bastante clara del sistema que se va a construir en la siguiente fase.

4.4. Empleo de herramientas de Interfaz Gráfica

Crear aplicaciones que incluyan interfaces gráficos de usuario, con por ejemplo el IDE NetBeans, es muy sencillo debido a las facilidades que nos proporcionan sus asistentes y herramientas.



El IDE de programación nos proporciona un diseñador para crear aplicaciones de una manera fácil, sin tener que preocuparse el programador en exceso del código. Simplemente nos debemos centrar en el funcionamiento de las mismas.

Un programador experimentado, probablemente no utilizará los asistentes, sino que escribirá, casi siempre, todo el código de la aplicación.

De este modo, podrá indicar por código la ubicación de los diferentes controles, de su interfaz gráfica, en el contenedor (panel, marco, etc) en el que se ubiquen.

Pero en el caso de programadores novatos, o simplemente si se quiere ahorrar tiempo y esfuerzo, aparece la la opción de aprovecharde las capacidades que nos brinda un entorno de desarrollo visual para diseñar e implementar formularios gráficos.

Algunas de las características de NetBeans, que ayudan a la generación rápida de aplicaciones, y en particular de interfaces son:

– Modo de diseño libre (Free Design): permite mover libremente los componentes de interfaz de usuario sobre el panel o marco, sin atenerse a uno de los layouts por defecto.

– Independencia de plataforma: diseñando de la manera más fácil, es decir, arrastrando y soltando componentes desde la paleta al área de diseño visual.

– NetBeans sugiere alineación, posición, y dimensión de los componentes, de manera que se ajusten para cualquier plataforma, y en tiempo de ejecución el resultado sea el óptimo, sin importar el sistema operativo donde se ejecute la aplicación.

– Soporte de internacionalización.

– Se pueden internacionalizar las aplicaciones Swing, aportando las traducciones de cadenas de caracteres, imágenes, etc, sin necesidad de tener que reconstruir el proyecto, sin tener que compilarlo según el país al que vaya dirigido.

– Se puede utilizar esta características empleando ficheros de recursos (ResourceBundle files). En ellos, deberíamos aportar todos los textos visibles, como el texto de etiquetas, campos de texto, botones, etc.

– NetBeans proporciona una asistente de internacionalización desde el menú de herramientas (Tools).

– Una enorme facilidad de uso que permite que usuarios que no estén familiarizados con el entorno puedan utilizarlo sin problemas.

UD4
Lo más importante

- Cuando hablamos de interfaz nos referimos a la parte “visible” de los programas tal y como se muestran a los usuarios para que interactúen con ellos. Las interfaces gráficas de usuario proporcionan al usuario ventanas, cuadros de diálogo, barras de herramientas, botones, listas desplegables y muchos otros más elementos.

- La programación guiada por eventos es un modo de programar en el que el programador determina que acción se va a realizar al responder a un evento que se ha generado, por ejemplo que tiene que hacer el programa cuando el usuario presione, o arrastre un botón o elemento visual por la pantalla.

- En el diseño de su interfaz, conviene de ser consistente y predecible en su elección de elementos de la interfaz, ya a sea que estén conscientes de ello o no, los usuarios se han familiarizado con los elementos que actúan de una manera determinada, por lo que decidir adoptar esos elementos ayudará con la realización de tareas, eficiencia y satisfacción.

- : Button es una clase en el paquete java.awt que representa botones en la pantalla. El constructor es: **Button public (String etiqueta);** ue, cuando se ejecuta, crea un botón con “etiqueta” impreso en ella.

- Cuando se produce una acción a un botón, todos los objetos que se han añadido como ActionListener de ese botón se notifica a llamando a su método actionPerformed con un parámetro con información sobre el evento exacto que ocurrió.

– En general en un buen diseño se suele separar el código para la construcción y el manejo de componentes GUI del código que en realidad hace el trabajo generado por los eventos de acción.

– El diseño del software parte de la especificación de requisitos del software obtenidas durante el análisis y a partir de ahí en esta fase he de construirse una guía efectiva o un "plano" que permita llevar a cabo la implementación del sistema.

– El diseño de la interfaz de usuario es la fase del diseño que trata de crear un medio de comunicación entre el hombre y la máquina.

– Su correcto desarrollo es de vital importancia para la aceptación del sistema final por las personas que lo van a utilizar, ya que cualquier sistema informático, independientemente de su potencia o funcionalidad, si no está acompañado por una buena interfaz no será del agrado de los usuarios.

– En muchas ocasiones el diseñador introduce restricciones un la interfaz para simplificar su construcción que pueden dar lugar a sistemas frustrantes de utilizar.

– Un atributo es definido como una ranura llamada dentro de un clasificador que describe un rango de valores que las instancias del clasificador contendrán en él.

– Percepción del usuario (en alguna bibliografía también se denomina "modelo de usuario", pero el concepto es completamente diferente al anterior): Consiste en la imagen mental que se crea el usuario de cómo es y como funciona el sistema a través de lo que le transmite la interfaz. Imagen del sistema: Es el modelo del sistema que el diseñador de la interfaz quiere transmitir al usuario a través de ésta y de la información suministrada junto con el sistema (ayuda, manuales de usuario…).

– Un buen diseño de la interfaz es aquel en el que le Imagen del sistema y la percepción del usuario.

– El objetivo del proceso de diseño de la interfaz es definir un conjunto de objetos y acciones sobre estos objetos que posibiliten al usuario llevar a cabo todas las tareas que se consideren necesarias para que el sistema cumpla con los objetivos de usabilidad obtenidos en el análisis de requisitos.

UD4
Autoevaluación

1. Un contenedor de AWT:

    a. Solo puede albergar componentes y no contenedores.
    b. Es a su vez un componente.
    c. Sirve de almacén de botones
    d. Se almacena en una clase JComponente.

2. Un evento:

    a. Se produce al pulsar un botón.
    b. Ocurre cuando el programador llama al método ActionPerformed.
    c. Es todo hecho que ocurre mientras se ejecuta una aplicación.
    d. Ninguna de las anteriores.

3. Las casillas de verificación:

    a. Junto con las listas desplegables permiten al usuario escoger múltiples elementos a la vez.
    b. Permiten a usuario escoger un solo elemento a la vez, junto con los radioButton.
    c. Muestran una lista de tres columnas.
    d. Han de mostrar listas de texto con fechas y horas.

4. Los componentes de la GUI de Java:

    a. Se almacenan en clases privadas.
    b. Un ejemplo de ellos es JCanvas.
    c. Son manejados por el modelo de eventos de Java.
    d. Ninguna es correcta.

5. Cuando se produce un evento:

    a. Se llama al método “ActionPerformed” de los oyentes registrados.
    b. El programador debe detectarlos con un esquema if.
    c. Se generan tres objetos asociados, el oyente, el evento y la interrupción.
    d. Se recibe un parámetro en el programa.

6. Son métodos de la clase textArea:

    a. getText y setText.
    b. getManyValues y getProportions.
    c. deleteWindows y setWindowsDefault.
    d. Ninguno de los anteriores.

7. Cual es una de tres “reglas de oro” de Mantel:

    a. Dar el control al usuario.
    b. Utilizar ventanas deslizantes siempre que se pueda.
    c. Todos los botones deben llevar Icono.
    d. Siempre hay que incluir una ayuda on-line.

8. El tiempo de respuesta:

    a. Lo importante es que su duración sea mínima.
    b. Se caracteriza por dos valores, su duración y su variabilidad.
    c. Ha de tratarse de minimizar su media ponderada.
    d. Ninguna de las respuestas es correcta.

9. El etiquetado de órdenes:

    a. No hay que plantearse la forma de estas.
    b. Debe haber órdenes para al menos la mitad de los elementos del menú.
    c. Debe darse la posibilidad que el sistema recuerde las últimas órdenes.
    d. Ninguna es correcta.

10. El IDE de programación:

    a. Los programadores expertos son los que más lo usan.
    b. Siempre se trabajará mediante asistentes en él.
    c. Fuerza al programador a preocuparse por su código.
    d. Proporciona al diseñador una manera fácil de crear aplicaciones.

Área: informática y comunicaciones

UD5

Calidad en el desarrollo del software

5.1. Enumeración de los criterios de calidad

5.2. Análisis de métricas y estándares de calidad

5.1. Enumeración de los criterios de calidad

Si hablamos de calidad lo primero que debemos es definir el concepto. Pressman define software de calidad de la siguiente manera:



Software de calidad es aquel que está en consonancia con los requisitos sean estos tanto funcionales como de rendimiento que hayan sido explícitamente establecidos, y además con los estándares de desarrollo explícitamente documentados y con las características implícitas que se desean de todo software desarrollado profesionalmente.

Para conseguir desarrollar software de calidad es necesario llevar a cabo durante todo el desarrollo del proyecto una serie de actividades que permita asegurar que están consiguiendo los objetivos de calidad marcados para dicho proyecto.

Estas actividades, denominadas de Aseguramiento de la Calidad (SQA) consisten en una serie de inspecciones, revisiones y pruebas realizadas a lo largo de todo el ciclo de vida del software para cerciorarse de que el producto cumple con los requisitos expuestos anteriormente.

El hecho de llevar a cabo una serie de actividades para poner en marcha un sistema de aseguramiento de la calidad conlleva un coste.

El coste de calidad puede dividirse en tres aspectos:

– Coste de prevención: es el coste que conlleva implantar un sistema de calidad que trate de evitar que se produzcan defectos en el desarrollo que conlleven una menor calidad del producto.

– Coste de evaluación: es el coste que se produce al determinar si en todo o en parte del desarrollo se han producido errores en la calidad del software.

– Coste de fallo: es el coste de corregir los fallos y defectos detectados en el software o en alguna de las fases de su desarrollo

Estos costes tal y como se han enunciado están ordenados de menor a mayor. Es decir el coste de prevención siempre será menor que el coste de un fallo, lo que pasa es que como el coste de fallo se ve implícitamente oculto por el coste de prevención, a veces las organizaciones se resisten a asumir, y cuestionan los costes de prevención



Además hay que tener en cuenta que el coste de fallo crece en un orden de magnitud cada vez que el desarrollo cambia de fase.

Por ejemplo si corregir un error de diseño cuando se está realizando este tiene un coste 1, corregir ese mismo fallo en la fase de pruebas tiene un coste 100.

Por este motivo es importante que el objetivo de las actividades de aseguramiento de la calidad sea en primer lugar no cometer fallos, y en segundo lugar tratar de detectar cuanto antes los fallos que se hayan cometido.

A largo plazo siempre es rentable para cualquier organización de desarrollo implantar actividades de aseguramiento de la calidad

A veces, a corto plazo, y si se imputa todo el coste de prevención a un solo proyecto, este puede no resultar ser rentable



Si se trabaja con esta idea, bastante limitada, es posible que se posponga el utilizar actividades de SQA debido a que muchas veces el coste de fallo, aunque sea mucho mayor, no es tan "evidente" como el coste de prevención.

Desarrollar software de calidad no depende de un solo factor, sino de muchos y muy diversos.

Básicamente puede hacerse una clasificación de los factores de calidad en dos grandes grupos:

– **Factores directos:** son aquellos factores que se pueden medir directamente sobre el producto desarrollado,

  Ejemplos de factores directos pueden ser el tiempo de ejecución, tiempo de respuesta, ocupación de la memoria o de espacio en disco,…

– **Factores indirectos:** son factores que no se pueden medir directamente sobre el producto, aunque generalmente afectan en mayor medida a la apreciación de la calidad por el usuario final.

  Pueden ser por ejemplo la facilidad de uso, la claridad de la documentación.

El estándar ISO 9126 ha sido desarrollado en un intento de identificar los atributos clave de calidad pira el software. Este estándar identifica seis atributos clave de calidad:

– Funcionalidad. El grado en que el software satisface las necesidades indicadas por los siguientes subatributos: idoneidad, corrección, interoperatividad, conformidad y seguridad.

– Confiabilidad. Cantidad de tiempo que el software está disponible para su uso. Está referido por los siguientes subatributos: madurez, tolerancia a fallos y facilidad de recuperación.

– Usabilidad. Grado en que el software es fácil de usar. Viene reflejado por los siguientes subatributos: facilidad de comprensión, facilidad de aprendizaje y operatividad.

– Eficiencia. Grado en que el software hace Óptimo el uso de los recursos del sistema. Está indicado por los siguientes subatributos: tiempo de uso y recursos utilizados.

– Facilidad de mantenimiento. La facilidad con que una modificación puede ser realizada. Está indicada por los siguientes subatributos: facilidad de análisis, facilidad de cambio, estabilidad y facilidad de prueba.

– Portabilidad. La facilidad con que el software puede ser llevado de un entorno a otro. Está referido por los siguientes subatributos: facilidad de instalación, facilidad de ajuste, facilidad de adaptación al cambio.

Mcall propone dividir en tres vertientes los factores que determinan la calidad de un producto software:

– Operación del producto: son factores que afectan al uso diario del producto por los usuarios finales. Dentro de este encontramos:

  · Corrección. Hasta dónde satisface un programa su especificación y logra los objetivos propuestos por el cliente.

  · Fiabilidad. Hasta dónde se puede esperar que un programa lleve a cabo su función con la exactitud requerida.

  · Eficiencia. La cantidad de recursos informáticos y de líneas de código necesarias para que un programa realice su función.

  · Integridad Es una medida de hasta dónde se controla el acceso al software o a los datos por personas no autorizadas.

  · Usabilidad (facilidad de manejo). El esfuerzo necesario para aprender a operar el sistema por lo usuarios, preparar los datos de entrada e interpretar las salidas de un software.

– Transición del producto: factores que influyen en la capacidad para cambiar el producto de entorno de operación, como pueden ser caminos en la infraestructura hardware o software. Puede dividirse en:

  · Portabilidad. El esfuerzo necesario para transferir el programa de un entorno hardware o Software a otro entorno diferente.

  · Reusabilidad o capacidad de reutilización: es una medida de se puede volver a utilizar un programa, o partes de un programa, en otras aplicaciones, esta relacionada con el empaquetamiento y alcance de las funciones que realiza el programa.

  · Interoperatividad. El esfuerzo necesario para acoplar un sistema con otro.

– Revisión del producto: son aquellos factores que afectan a la facilidad con la que se puede modificar el sistema software. Dentro de este grupo se encuentran los siguientes:

  · Facilidad de mantenimiento. El esfuerzo necesario para localizar y solucionar un error en un programa, aunque esta se considera una definición muy limitada.

  · Flexibilidad: El esfuerzo necesario para modificar un programa que ya está en producción y se está utilizando.

  · Facilidad de prueba: El esfuerzo necesario para probar un programa y garantizar de que este está realizando correctamente su función.

En el contexto de la ingeniería de software, la calidad del software se refiere a dos conceptos interrelacionados pero que son distintos que se aplican allí donde la calidad se define en un contexto de negocios: calidad funcional y la calidad estructural

– Calidad funcional de software refleja lo bien que cumple o está conforme a un modelo de requisitos determinado, tomando como base los requisitos funcionales o especificaciones.

  Ese atributo también se puede describir como la aptitud para el uso de una pieza de software o cómo se compara con sus competidores en el mercado como un producto valioso.

– Calidad estructural de software se refiere al modo en que este cumple con los requisitos no funcionales que sirven de apoyo a la prestación de los requisitos funcionales, como la robustez o de mantenimiento, el grado en que el software se ha producido correctamente.

Calidad estructural se evalúa mediante el análisis de la estructura del software interno, su código fuente, a nivel de unidad, el nivel de la tecnología y el nivel de sistema, que es, en efecto, cómo la arquitectura se adhiere al principio de buena arquitectura de software descritos en un documento estándar sobre este tema.

Por el contrario, la calidad funcional se suele cumplir y se mide a través de las pruebas de software, por esta razón conviene centrarse también en los aspectos de la calidad estructural además de en cumplir con los requisitos del sistema que se desarrolla.



Sabías que

Históricamente, la estructura, la clasificación y la terminología de los atributos e indicadores aplicables a la gestión de la calidad del software se han derivado o extraigan de la ISO 9126-3y el posterior ISO 25000:2005.

Tomando como base de estos modelos, el Consorcio para la Calidad de Software (CISQ) ha definido cinco grandes características estructurales deseables necesarias para una pieza de software para proporcionar valor de negocio: Fiabilidad, eficacia, seguridad, facilidad de mantenimiento y (adecuado) Tamaño.

La medición de la calidad del software cuantifica en qué medida un software o sistema cumple a lo largo de cada una de estas cinco dimensiones.

Una medida agregada de la calidad del software se puede calcular mediante una medición cualitativa o un sistema de puntuación cuantitativa o una mezcla de ambos y a continuación aplicar algún sistema de ponderación que refleja las prioridades de la organización.

Desde este punto de vista de la calidad del software está posicionado en un continuo línea que l se complementa con el análisis de “errores de programación críticos” que son aquellos que en determinadas circunstancias pueden llevar a cortes de catastróficas o degradaciones de rendimiento que hacen que un sistema dado inadecuado para su uso independientemente de clasificación basada en mediciones.

Tales errores de programación críticos los cuales se suelen encontrar a nivel del sistema representan genéricamente hasta el 90% de los problemas del software en fruncimiento, mientras que a nivel de unidad, incluso cuando son mucho más numerosos, los errores de programación representan menos del 10% de los problemas de producción.

Como consecuencia, la calidad del código sin el contexto de todo el sistema, como W. Edwards Deming describió, tiene un valor limitado.

Para ver, explorar, analizar y comunicar las medidas de calidad de software, conceptos y técnicas de la visualización de la información como facilitar los medios visuales e interactivas útiles, en particular, si hay varias medidas de calidad de software tienen que estar relacionados entre sí o con componentes de un software o sistema.

Por ejemplo, los mapas de software presentan un enfoque especializado que puede expresar y combinar la información sobre el desarrollo de software, la calidad del software, y la dinámica del sistema.



"Una ciencia se considera tan madura como sus herramientas de medición". (Louis Pasteur,).

La Medición de la calidad del software debe ser motivada por al menos dos razones:

– Gestión de Riesgos: fallos de software ha causado algo más de molestias. Algunos errores de software han causado en ocasiones muertes de seres humanos.

  Esto ha dado lugar a que los requisitos para el desarrollo de algunos tipos de software, en particular e históricamente de software embebido en dispositivos médicos y otros que regulan las infraestructuras críticas sea muchísimo más detallado.

  Se dice que los ingenieros que escriben software embebido, ven las dilaciones de tiempo de un tercio de segundo para realizar la recolección de basura y actualizar la interfaz de usuario que corren en programas Java, imaginan aviones que caen del cielo.

  En los Estados Unidos, dentro de la Administración Federal de Aviación (FAA), el Servicio de Certificación de Aeronaves ofrece programas de software, define la política, la orientación y la formación, para un enfoque en el desarrollo software y hardware que tiene un efecto sobre el producto "en el aire" (con "producto" se refieren a una aeronave, un motor, o una hélice) ".

– Gestión de costes: Al igual que en cualquier otro campo de la ingeniería, una aplicación con una buena calidad de software de manera estructural cuesta menos mantener y es más fácil de entender .

  Además esta aplicación soportará mejor el cambio en respuesta a apremiantes necesidades de negocio.

Estudios hechos en la industria demuestran que la mala aplicación de calidad estructural en entorno de grandes centros de aplicaciones empresariales como los sistemas de la planificación de recursos empresariales (ERP), o de gestión de relaciones con clientes (CRM) tienen un impacto global elevado en la organización.

Por ejemplo tiempos excesivos de procesamiento en transacciones en los sistemas de servicios financieros se traduce en sobrecostos y el hecho de rehacer los programas lleva a desperdiciar muchos recursos, hasta el 45% del tiempo de desarrollo en algunas organizaciones

Por otra parte, la mala calidad de la infraestructura está fuertemente correlacionada con alto impacto que tiene interrupciones en los negocios debido a los datos dañados, cortes de aplicación, las brechas de seguridad y problemas de rendimiento.

Sin embargo, la distinción entre la medición y mejora de la calidad del software en un sistema embebido (haciendo más énfasis en la gestión de riesgos) y la calidad del software en sistemas de negocios (con énfasis en la gestión de costes y facilidad de mantenimiento) se está convirtiendo en algo irrelevante.

Los sistemas embebidos hoy en día a menudo incluyen una interfaz de usuario y sus diseñadores están tan preocupados por los temas que afectan la usabilidad y la productividad del usuario como sus contrapartes que se centran en las aplicaciones de negocio.

Estos últimos están a su vez observando ERP o sistema de CRM como un sistema nervioso empresarial cuyo tiempo de actividad y rendimiento son vitales para el bienestar de la empresa.

Esta convergencia es más visible en la informática móvil: un usuario que desea acceder a una aplicación ERP en su teléfono depende de la calidad del software en todos los tipos de capas de software.



Ambos tipos de software ahora utilizan elementos de tecnología en varias capas y una arquitectura compleja por lo que el análisis de calidad del software y la medición tienen que ser manejados de una manera integral y coherente, desvinculado de último destino o utilización del software.

En ambos casos, los ingenieros y la gestión tienen que ser capaces de tomar decisiones racionales basadas en la medición y el análisis basado en los hechos.



Hay muchas **definiciones** diferentes de calidad. Para algunos es la "capacidad de un producto de software para cumplir con los requisitos." (ISO / IEC 9001) mientras que para otros puede ser sinónimo del "valor para el cliente" que tiene el sistema o incluso a nivel de defectos que se encuentran en el sistema.

La primera definición de calidad de la que se encuentra documentada fue hecha por Shewhart al comienzo del siglo 20: En ella se indica que hay dos aspectos comunes de calidad: uno de ellos tiene que ver con la consideración de la calidad de una cosa como una realidad objetiva independiente de la existencia del hombre .

El otro tiene que ver con lo que pensamos, sentimos o percibimos como resultado de la realidad objetiva. En otras palabras, hay un lado subjetivo de la calidad.

La calidad se puede ver desde diversas perspectivas:

- La perspectiva trascendental se ocupa del aspecto metafísico de la calidad. En este punto de vista de la calidad, se trata de "algo hacia el cual nos esforzamos como un ideal, pero nunca puede implementar por completo.

- La perspectiva del usuario se refiere a la conveniencia del producto para un determinado contexto de uso. Considerando que el punto de vista trascendental es etéreo, la vista del usuario es más concreta, basada en las características de los productos que satisfagan las necesidades de los usuarios.

- La perspectiva de fabricación representa la calidad como la conformidad con los requisitos. Este aspecto de la calidad se destacó por normas como la ISO 9001, que define la calidad como "el grado en que un conjunto de características inherentes cumple con los requisitos" (ISO / IEC 9001).

– La perspectiva del producto implica que la calidad puede ser apreciado por la medición de las características intrínsecas del producto.

– La perspectiva final de la calidad se basa en valores. Esta perspectiva reconoce que las diferentes perspectivas de la calidad pueden tener diferente importancia o valor, a las diversas partes interesadas.

La calidad del software de acuerdo a Deming : la dificultad en la definición de la calidad es traducir las necesidades futuras de los usuarios en las características mensurables, para que un producto pueda diseñar y el resultado sirva para dar satisfacción a un precio que el usuario va a pagar.

Esto no es fácil, y tan pronto como uno se siente bastante éxito en el esfuerzo, se encuentra con que las necesidades de los consumidores han cambiado, los competidores han mudado, etc .

La calidad del software de acuerdo con Feigenbaum: “La calidad es una determinación del cliente, no es la determinación de un ingeniero, no una determinación de marketing, ni una determinación general de gestión se basa en la experiencia real del cliente con el producto o servicio, medida contra sus requisitos.

Estos requisito puede ser establecidos o no, consciente o simplemente un sentimiento, técnicamente operacionales o enteramente subjetivo, y esto representa siempre un objetivo en movimiento en un mercado competitivo “

La calidad del software según Juran: la palabra calidad tiene múltiples significados Dos de estos significados dominan el uso de la palabra:

– Calidad consiste en aquellas características de los productos que respondan a las necesidades de los clientes y por lo tanto proporcionan satisfacción del producto.

– Calidad consiste en la ausencia de deficiencias Sin embargo, en un manual como este, es conveniente estandarizar una breve definición de la palabra calidad como “la aptitud para el uso” “.

A pesar de que “la calidad es un atributo de la percepción, condicionado y un tanto subjetivo y puede ser entendido de manera diferente por diferentes personas de características de software de calidad estructural han sido claramente definidos por el Consorcio para IT Calidad de Software (CISQ).



Bajo la dirección de Bill Curtis, co-autor del Modelo marco de Madurez de Capacidad y primer Director del CISQ, junto con su equipo ha definido cinco principales características deseables de una elemento de software que se necesita para proporcionar valor de negocio.

Estos cinco elementos de calidad son fiabilidad, eficiencia, seguridad, mantenibilidad y tamaño:

– Fiabilidad: un atributo de resistencia y solidez estructural. Confiabilidad mide el nivel de riesgo y la probabilidad de fallos potencial de las aplicaciones.

  También mide los defectos inyectados debido a las modificaciones introducidas en el software (la "estabilidad", como se llamó.

  La meta para el control y seguimiento de Confiabilidad es reducir y prevenir el tiempo de inactividad de aplicaciones, interrupciones de aplicación y los errores que afectan directamente a los usuarios y mejorar la imagen de las TI y su impacto en el rendimiento del negocio de una empresa.

– Eficiencia: el código fuente y los atributos de arquitectura software son los elementos que aseguran un alto rendimiento una vez que la aplicación está en modo de tiempo de ejecución.

  La eficiencia es especialmente importante para aplicaciones en entornos de alta velocidad de ejecución, tales como procesamiento algorítmico o transaccional donde el rendimiento y la escalabilidad son primordiales.

  Un análisis de la eficiencia del código fuente y la capacidad de ampliación proporciona una imagen clara de los riesgos de negocio latentes y el daño que pueden causar a la satisfacción del cliente debido a la degradación del tiempo de respuesta.

– Seguridad: es una medida de la probabilidad de posibles brechas de seguridad debido a las prácticas de codificación pobres y la arquitectura.

Esto cuantifica el riesgo de encontrar vulnerabilidades críticas que dañan el negocio.

– Mantenibilidad: mantenibilidad incluye la noción de adaptabilidad, portabilidad y capacidad de transferencia (de un equipo de desarrollo a otro).

  La medición y seguimiento de mantenimiento es una necesidad para las aplicaciones de misión crítica donde el cambio es impulsado por apretados límites de tiempo hasta el mercado y en los que es importante para que siga siendo sensible a los cambios del negocio que impulsa.

  También es esencial para mantener los costos de mantenimiento bajo control.

– Tamaño: Aunque no es un atributo de calidad de por sí, el tamaño del código fuente es una característica de software que, obviamente, los impactos de mantenimiento.

  En combinación con las características de calidad anteriores, tamaño del software se puede utilizar para evaluar la cantidad de trabajo producido y para ser hecho por los equipos, así como su productividad a través de la correlación con los datos de tiempo de hojas, y otras métricas relacionadas-.

Calidad funcional del software se define como la conformidad con los requisitos funcionales que se indique expresamente, identificados por ejemplo durante el análisis y documentada a través de casos de uso y el nivel de satisfacción experimentado por los usuarios finales.

Este último se conoce como la usabilidad y tiene que ver con la forma intuitiva y sensible la interfaz de usuario es, la facilidad con que se pueden realizar operaciones simples y complejas, y como de útiles los mensajes de error son.

Por lo general, las prácticas de pruebas de software y herramientas garantizan que una pieza de software se comporta de acuerdo con el diseño original, la experiencia del usuario planificada y deseada la capacidad de prueba, es decir, de la disposición del software para apoyar los criterios de aceptación.

La doble dimensión estructural / funcional de la calidad del software es consistente con el modelo propuesto que divide las características de software en dos partes: características internas y externas de calidad.



Las características de calidad externa son las partes de un producto que se presenta a sus usuarios, donde las características de calidad internos son aquellos que no lo hacen.

Uno de los retos en la definición de la calidad es que "todo el mundo siente que lo entiende" y otras definiciones de la calidad del software podría basarse en la ampliación de las diversas descripciones del concepto de calidad aplicado en los negocios.

Tom DeMarco ha propuesto que "la calidad de un producto es una función de cuánto cambia el mundo para mejor". Esta quizás sea una definición un poco ambigua e incluso filosófica, pero nos da una idea muy intuitiva de lo que debe significar la calidad, como algo que parte una mejora en el producto que se está desarrollando

Esto puede ser interpretado en el sentido de que la calidad funcional y la satisfacción del usuario es más importante que la calidad estructural en la determinación de software calidad. De hecho hoy en día se habla de la "experiencia de usuario" como un valor en alza dentro de lo que es la medida de la calidad del software.



Otra definición, acuñada por Gerald Weinberg en Gestión de la "Calidad del software: Pensamiento Sistémico", es. "La calidad es el valor que de el producto a alguna persona".

Esta definición hace hincapié en que la calidad es la gente inherentemente subjetivos-diferente experimentarán la calidad del mismo software de manera diferente.

Un punto importante de esta definición son las preguntas que invita a los equipos de software a considerar, tales como “¿Quienes son las personas que queremos valorar nuestro software?” Y “¿Qué será de gran valor para ellos?”.

Aunque los conceptos que se presentan en esta sección son aplicables a la calidad del software, tanto estructural como funcional, la medición de este último se lleva a cabo fundamentalmente a través de las pruebas .

La calidad se puede ver como una relación entre las características deseables de software y atributos medibles.



Definición

La medición de la calidad del software es cuantificar en qué medida un sistema o software posee características deseables.

Esto se puede realizar a través de medios cualitativos o cuantitativos o una mezcla de ambos.

En ambos casos, para cada característica deseable, debe haber un conjunto de atributos medibles la existencia de que en una pieza de software o sistema tienden a estar correlacionados y asociados con esta característica.

Por ejemplo, un atributo asociado con la portabilidad es el número de declaraciones son objeto dependiente en un programa.

De manera más precisa, utilizando el enfoque de calidad de funcionamiento del software, estos atributos mensurables son los “cómo” que se deben cumplir para que el “qué” en la definición de calidad de software anteriormente.

La estructura, la clasificación y la terminología de los atributos e indicadores aplicables a la gestión de la calidad del software se han derivado o extraído de la ISO 9126-3 y el modelo de calidad ISO / IEC 25000:2005. La atención se centra en la calidad estructural interna.

Varias subcategorías se han creado para manejar áreas específicas como de aplicaciones de negocios de arquitectura y características técnicas tales como el acceso y manipulación de datos o la noción de transacciones.

El árbol de la dependencia entre las características de calidad de software y sus atributos medibles se representa en el siguiente esquema donde cada una de las cinco características de calidad que son importantes para el usuario o el propietario del sistema depende de una serie de atributos mensurables

- Prácticas de Arquitectura de aplicaciones.
- Prácticas de codificación.
- Complejidad Aplicación.
- Documentación.
- Portabilidad.
- Volumen Técnico y Funcional.



Sabías que

Las relaciones entre los errores de programación y los defectos de producción revelan que los errores básicos de código representan el 92% del total de errores en el código fuente.

Estos problemas a nivel de código con el tiempo cuentan para sólo el 10% de los defectos en la producción. Es decir, que los errores de programación realmente tienen un impacto menos significativo en producción que otros errores producidos durante otras fases de ingeniería del software.

Las malas prácticas de ingeniería de software a nivel arquitectónico solamente representan el 8% del total de defectos, pero consumen más de la mitad el esfuerzo que hay que invertir en la solución de problemas, y dar lugar a un 90% de fallos de fiabilidad grave, problemas de seguridad y problemas de eficiencia en el sistema en producción.

La Medición de la calidad del software consiste en cuantificar en qué medida un sistema de software se encuentra a lo largo de estas dimensiones.

El análisis se puede realizar utilizando un enfoque cualitativo o cuantitativo o una mezcla de ambos para proporcionar una vista agregada, utilizando por

ejemplo medias ponderadsa que refleja nla importancia relativa entre los parámetros que se está midiendo.

Este punto de vista de la calidad del software en un continuo lineal tiene que complementarse con la identificación de los discretos errores de programación críticos .



Definición

Los errores de programación críticos: estas vulnerabilidades no pueden simplemente hacer fallar un caso de prueba, sino que son el resultado de malas prácticas que en determinadas circunstancias pueden conducir a cortes catastróficos, degradaciones de rendimiento, brechas de seguridad, datos dañados, y una miríada de otros problemas.

Estos errores hacen que un sistema dado no sea apto para su uso, independientemente de su clasificación basada en otras mediciones agregadas. Por tanto los errores críticos hacen el sistema completamente inservible, eliminando cualquier otra cuestión sobre si realmente tiene algún otro eleemeneto de calidad o no.

La medición de las características clave de las aplicaciones consiste en medir los atributos estructurales de la arquitectura de la aplicación, la codificación y la documentación en línea.

Por lo tanto, cada característica se ve afectada por los atributos en numerosos niveles de abstracción en la aplicación y todos los cuales se debe incluir el cálculo de la medida de la característica si ha de ser un predictor importante de los resultados de calidad que afectan el negocio.

Estos atributos pueden ser medidos a partir de los resultados analizados de un análisis estático del código fuente de la aplicación. Incluso las características dinámicas de aplicaciones tales como la fiabilidad y la eficiencia en el desempeño tienen sus raíces causales en la estructura estática de la aplicación.

El análisis estructural de calidad y la medición se realiza a través del análisis del código fuente, la arquitectura, plataforma de software, esquemas de bases de datos en relación a los principios y normas que en conjunto definen la arquitectura conceptual y lógico de un sistema.

Esto es distinto del análisis básico, búsqueda de errores locales a nivel de componente de código normalmente realizado por las herramientas de desarrollo, que están en su mayoría preocupados por consideraciones de implementación y son cruciales durante la depuración y pruebas de actividades.

Las causas fundamentales de la falta de fiabilidad se encuentran en una combinación de la falta de cumplimiento de las buenas prácticas arquitectónicas y de codificación.

Esta falta de cumplimiento puede ser detectada mediante la medición de los atributos estáticos de calidad de una aplicación.

La evaluación de los atributos estáticos subyacentes a la fiabilidad de una aplicación proporciona una estimación del nivel de riesgo del negocio y la probabilidad de fallos de las aplicaciones potenciales y defectos de la aplicación va a experimentar cuando esta se ponga pone en funcionamiento.

La evaluación de la confiabilidad requiere controles de, al menos, la siguiente la ingeniería de software las mejores prácticas y los atributos técnicos:

- Prácticas de Arquitectura de aplicaciones.
- Prácticas de codificación.
- La complejidad de los algoritmos.
- La complejidad de las prácticas de programación.
- El cumplimiento de las mejores prácticas de programación orientada a objetos y estructurada.
- De componentes o patrón de relación de re-utilización.
- Error y Control de excepciones.
- Multi-capa de diseño.
- Gestión de límites de recursos.
- Software que evita patrones que conduzcan a comportamientos inesperados.
- Software que gestiona integridad de los datos y la consistencia.
- Nivel de complejidad de transacciones.

En función de la arquitectura de la aplicación y los componentes de terceros utilizados (por ejemplo, las bibliotecas o los marcos externos), los controles personalizados deben ser definidos a lo largo de las líneas trazadas por la anterior lista de las mejores prácticas para asegurar una mejor evaluación de la fiabilidad del software entregado.



Al igual que con la fiabilidad, las causas de la ineficiencia de rendimiento se encuentran a menudo en violaciones de la buena práctica de la arquitectura y la codificación que puede ser detectado midiendo los atributos estáticos de calidad de una aplicación.

Estos atributos estáticos predicen posibles cuellos de botella de rendimiento operativo y los futuros problemas de escalabilidad, especialmente para aplicaciones que requieren alta velocidad de ejecución para el manejo de algoritmos complejos o grandes volúmenes de datos.

La evaluación de la eficiencia de rendimiento requiere la comprobación, al menos, los siguientes programas de ingeniería las mejores prácticas y los atributos técnicos:

– Prácticas de Arquitectura de aplicaciones.

– Interacciones adecuadas con los recursos costosos y / o remotas.

– Gestión de rendimiento de acceso a datos y los datos.

– Memoria, red y gestión de espacio en disco.

– Prácticas de codificación.

– El cumplimiento de las mejores prácticas de programación orientada a objetos y estructurado.

– El cumplimiento de la programación de buenas prácticas en las consultas SQL.



La mayoría de las vulnerabilidades de seguridad son el resultado de una pobre codificación y prácticas arquitectónicas erróneas tales como la inyección SQL o cross-site scripting.

La evaluación de la seguridad requiere al menos comprobar que durante el proceso de ingeniería del software se han llevado a cabo las mejores prácticas y para ello se han de revisar los siguientes atributos técnicos:

– Prácticas de Arquitectura de aplicaciones correctas.

– Realizar un correcto diseño Multi-capa en cada fase.

– Las buenas prácticas de seguridad (validación de entrada, inyección de SQL, Cross-Site Scripting, etc.

– Prácticas adecuadas de programación a nivel de código.

– Error y Control de excepciones.

– Buenas prácticas de seguridad en el acceso a las funciones del sistema, control de acceso a los programas.

La capacidad de mantenimiento incluye los conceptos de modularidad, la comprensiblidad, la mutabilidad, la capacidad de prueba, la reutilización, y la transferibilidad de un equipo de desarrollo a otro.

Estas prácticas no tienen la forma de elementos críticas a nivel de código.

Más bien, la mala capacidad de mantenimiento es normalmente el resultado de miles de violaciones de menor importancia de las buenas prácticas, como por ejemplo en la documentación, no usar estrategias para evitar la complejidad y las prácticas básicas de programación que marcan la diferencia el código limpio y una estilo desorganizado con código que sea difícil de leer.

La evaluación de capacidad de mantenimiento requiere comprobar la siguiente lista de buenas prácticas de ingeniería del software y los atributos técnicos que se nombran a continuación:

- Prácticas de Arquitectura de aplicaciones.
- Arquitectura, Programas y documentación Código incrustado en el código fuente.
- Legibilidad Código.
- Nivel de complejidad de las transacciones.
- La complejidad de los algoritmos.
- La complejidad de las prácticas de programación.
- El cumplimiento de las mejores prácticas de programación orientada a objetos y estructurada (cuando corresponda).
- De componentes o patrón de relación de re-utilización.
- Nivel controlado de codificación dinámica.
- Relación de acoplamiento.
- Documentación.
- Hardware, sistema operativo, middleware, componentes de software y base de datos de la independencia.
- Portabilidad.
- Prácticas de programación a nivel de código.
- Reducción de código y evitar la duplicación de funciones.
- Organización y limpieza en los archivos de código fuente.



La Mantenibilidad está estrechamente relacionado con el concepto de deuda técnica, que es una expresión de los costes derivados de la falta de mantenimiento.

Las razones de por qué la mantenibilidad es baja se puede clasificar como deliberada o inadvertida, y con frecuencia tienen su origen en la incapacidad de los desarrolladores, la falta de tiempo y metas, su descuido y discrepancias en el coste de creación y beneficios de documentación y, en particular, de mantener el código fuente .

La medición del tamaño de software requiere que todo el código fuente se reuna correctamente, incluyendo scripts estructura de base de datos, el código fuente de manipulación de datos, encabezados de componentes, archivos de configuración, etc.



Hay esencialmente dos tipos de tamaños de software a medir, el tamaño técnico y el tamaño funcional:

Hay varias técnicas de dimensionamiento de software a nivel técnico que han sido ampliamente descritos.

El método de dimensionamiento más común es el número de líneas de código LOC el cual vienen dado por la tecnología, el número de archivos, funciones, clases, etc, de la que se pueden calcular el tamaño y su complejidad.

La forma más común de medición de tamaño funcional es Análisis de Puntos de Función.

Análisis de Puntos de Función mide el tamaño de la entrega de software desde la perspectiva del usuario.

Las métricas basadas en el punto de función se realiza en base a los requisitos del usuario y proporciona una representación precisa de tanto en tamaño para el desarrollador / estimador y el valor (funcionalidad para ser entregado) y refleja la funcionalidad de negocio de ser entregado al cliente.

El método incluye la identificación y ponderación de entradas reconocibles por el usuario, los productos y los almacenes de datos.

El valor del tamaño está entonces disponible para su uso en conjunción con numerosas medidas para cuantificar y evaluar la entrega de software y rendimiento.

Los Errores de programación críticas son debidos malas prácticas arquitectónicas y/ o de codificación específicos que dan lugar a la más alta probabilidad, inmediatas o a largo plazo, que puede poner en grave riesgo de interrupción el negocio.

Esto esta muy a menudo relacionada con la tecnología, y dependen en gran medida en el contexto, los objetivos de negocio y los riesgos. Estos son fundamentales de detectar y eliminar antes de que el software entre en producción

5.2. Análisis de métricas y estándares de calidad

Un elemento clave de cualquier proceso de ingeniería es la medición.

Se emplean medidas para entender mejor los atributos de los modelos creados.

Pero, fundamentalmente, se emplean las medidas para valorar la calidad de los productos de ingeniería o de los sistemas construidos.



A diferencia de otras disciplinas, la ingeniería del software no está basada en las leyes cuantitativas básicas de la Física o de otras ciencias. Las medidas absolutas, como el voltaje, la masa, la velocidad o la temperatura no son comunes en el mundo del desarrollo software.

En lugar de esto, la ingeniería del software ha trabajado para obtener un conjunto de medidas indirectas que dan lugar a métricas que proporcionan cierta indicación de la calidad de algún tipo de representación del software.

Se han propuesto una gran cantidad de métricas para el software, pero no todas presentan un soporte práctico para el desarrollo de software.

Algunas requieren mediciones que son demasiado complejas. Un conjunto de atributos que deberían tener las métricas del software son los que se muestran a continuación:

– Simples y sencilla de calcular: debería ser relativamente fácil saber a obtener la métrica y su cálculo no debería requerir un esfuerzo o una cantidad de tiempo elevadas.

– Empírica e intuitivas: la métrica debe satisfacer las nociones intuitivas que el ingeniero tiene sobre el atributo del producto en que se está midiendo

  Por ejemplo, una métrica que mida la cohesión de un módulo debe de aumentar su valor absoluto a mientras que crezca el nivel de cohesión.

– Consistentes y objetivas: la métrica debe siempre producir un resultado no ambiguo. Un segundo equipo ha de ser capaz de obtener el mismo valor de métrica usando la misma información del sistema.

– Consistentes en el empleo de unidades y tamaños: el cálculo de la métrica debería emplear medidas que no conduzcan a combinaciones de unidades un tanto raras.

  Por ejemplo, multiplicando el número de personas de un equipo por las variables del lenguaje de programación en el programa resulta una rara mezcla de unidades que no son nada intuitivas en absoluto.

– Independientes del lenguaje de programación. Las métricas deberían basarse en el modelo de análisis, modelo de diseño o en la propia estructura interna del sistema.

  Las métricas no deberían depender de las particularidades de la sintaxis o estructura del lenguaje de programación que se uso.

– Un mecanismo eficaz para la realimentación de calidad: La métrica debe de proporcionar, al desarrollador de software, suficientes datos como para que le lleve a un producto final de mayor calidad. Métricas del software se han definido muchas, pero a continuación se van a exponer dos que tienen como objetivo medir la calidad del producto y del proceso de desarrollo del software:



Índice de madurez del software: indica como de asentado está un producto software.

El concepto de "asentado" puede ser un poco ambiguo pero se refiere a la cantidad de cambios que va a ser necesario ir incorporando al producto para satisfacer las necesidades que se espera que cubra. Realmente es una medida específicamente diseñada para el mantenimiento e indica si el producto se encuentra aún en evolución o no.

Se calcula según la siguiente ecuación:

IMS=(Mt-(Fa+Fc+Fd))/Mt

Siendo:

– Mt: Módulos totales de la versión actual.

– Fa: Módulos añadidos a la versión actual.

– Fc. Módulos modificados en la versión actual.

– Fd: Módulos borrados en la versión actual.

Como se indicó anteriormente uno de los objetivos del sistema de calidad es encontrar defectos cuanto antes, para evitar que el sobrecosto que supone corregirlos en fases posteriores del desarrollo. Por tanto la eficacia en la eliminación de defectos nos indica la adecuación de sistema de calidad en la localización de estos.

Una medida de la efectividad de este proceso es la relación entre el número de defectos corregidos antes de entregar el producto y los defectos corregidos tras la entrega. De esta manera se sabe como de correcto fue el software que se entregó frente a lo que realmente el usuario quería.

Esta relación además se puede generalizar para todas las fases del ciclo de vida para detectar en qué etapas debe refinarse más los procesos de calidad.

La Eficacia en la eliminación de defectos se calcula:

EED=E/(E+D)

Siendo:

– E: errores encontrados entes de entregar el producto.

– D: errores encontrados tras entregar el producto.

A continuación se exponen algunos de los estándares internacionalmente aceptados para el proceso de desarrollo de software.

La serie de normas ISO-9000 establece los requisitos básicos que debe de cumplir un sistema de control que garantice productos o servicios de calidad consistente.

Son un grupo de 5 estándares internacionales de administración de la calidad y aseguramiento de calidad.

Estas normas son genéricas, no específicas para cualquier producto y fueron desarrollados para documentar que elementos de los sistemas de calidad deben ser utilizados para mantener un sistema eficiente de calidad en la empresa.

ISO 9001 abarca diseño, manufactura, instalación y sistemas de servicio. ISO 9002 cubre producción e instalación, e ISO 9003 cubre inspección de productos finales.

ISO 9000-3 Esta parte de la ISO 9000 contiene orientaciones que facilitan la aplicación de la Norma ISO 9001 a las organizaciones dedicadas al desarrollo, suministro y mantenimiento del software.

Se pretende con ella dar orientaciones en relación con situaciones en las que un contrato entre dos partes exija la demostración de la capacidad de determinado proveedor para desarrollar, suministrar y mantener productos de software.

Tales orientaciones describen las clases de control y los métodos sugeridos para la producción del software, que satisfagan los requisitos establecidos. Los principales aspectos que establece la norma son los siguientes:

– Responsabilidades de la dirección: La dirección de la empresa debe definir y documentar su política y sus objetivos con respecto a la calidad.

  La empresa debe asegurarse que esta política es conocida, entendida e implementada en todos los niveles de la organización.

– Sistemas de calidad: La empresa debe establecer y mantener un sistema de calidad documentado, es decir, un manual como medio que permite asegurar que los productos van a cumplir con los requerimientos especificados.

  Este manual debe incluir los siguientes aspectos:

  · La preparación de procedimientos e instrucciones del sistema de calidad de acuerdo con los requerimientos de esta especificación.

  · La aplicación efectiva de los procedimientos y de las instrucciones documentadas del sistema de calidad.

– Revisión del contrato: la empresa debe establecer y mantener procedimientos para la revisión de los contratos y para mantener la coordinación de dichas actividades.

  Cada contrato debe ser revisado por la empresa para asegurar que:

  · Los requisitos están adecuadamente definidos y documentados.

  · La empresa tenga la capacidad de cumplir con todos los requerimientos contractuales.

  · Se actualiza adecuadamente la documentación de estos requisitos conforme evoluciona el sistema.

– Control de documentos y datos: la empresa debe establecer y mantener procedimientos para controlar todos los documentos y datos que se relacionen con esta norma.

– Productos suministrados: La empresa debe establecer y mantener procedimientos para la verificación, almacén y mantenimiento de productos suministrados por empresas externas para ser incorporados al producto final.

– Identificación y trazabilidad del producto: donde sea necesario la empresa debe establecer y mantener una serie de procedimientos para identificar el producto desde la etapa de diseño hasta la entrega e instalación, pasando por todas las etapas de producción.

  Cuando la trazabilidad del producto sea un requisito especificado, cada uno de los productos individuales o los lotes deben tener una identificador único.

– Inspección y pruebas: La empresa debe asegurar que los productos adquiridos no se usen hasta que estos hayan sido inspeccionados y verificados que cumplen con los requerimientos necesarios.

  Las verificaciones deben estar de acuerdo con el plan de calidad y los procedimientos documentados.

– Equipos de Inspección, medición y pruebas: La empresa debe controlar un equipo de inspección, medición y pruebas (dentro de ésta o de manera externa) para verificar la conformidad del producto con los requisitos especificados.

  El equipo debe ser usado de una manera que asegure que la incertidumbre de medición sea conocida y que esté dentro de la capacidad de medición requerida.

– Estado de Inspección y pruebas: el estado de inspección y pruebas del producto debe ser identificado mediante un mecanismo adecuado.

  Estos elementos deben indicar la conformidad o no-conformidad del producto con respeto a las pruebas e inspecciones efectuadas.

  La identificación del estado debe ser mantenida en todo el proceso de producción e instalación del producto para asegurar que sólo los elementos que hayan pasado las pruebas e inspecciones que se requieren sean entregados al cliente.

– Control de producto no conforme: la empresa debe mantener y controlar los procedimientos que aseguren que los productos que no cumplan los requerimientos especificados, no sean usados o instalados inadvertidamente.

– Acciones correctivas y preventivas: La empresa debe establecer, documentar y mantener procedimientos para:

  · Investigar la causa de no conformidad y las acciones correctivas necesarias para prevenir la que vuelva a ocurrir

  · Analizar todos los procesos, operaciones de trabajo, registros de calidad y reclamaciones de clientes para determinar y eliminar causas potenciales de productos no conformes.

– Auditorías internas de calidad: la empresa debe llevar un sistema de auditorías internas de calidad, planificado y documentado, que determine la efectividad del sistema de calidad.

Las auditorias deben programarse de acuerdo con la importancia de la actividad. El resultado de las auditorias debe ser documentado y mostrado al personal que tenga responsabilidad en el área auditada.

El personal administrador responsable del área debe tomar acciones correctivas sobre las deficiencias encontradas por la auditoria.

– Capacitación: La empresa debe establecer y mantener procedimientos para identificar las necesidades de capacitación y proveer de formación a todo el personal que realice algunas de las tareas especificas. Se deben mantener registros apropiados de capacitación.

– Técnicas estadísticas: Cuando sea apropiado, la empresa debe establecer los procedimientos para identificar técnicas estadísticas adecuadas, requeridas para verificar la capacidad de proceso y características del producto.



En el Modelo de Madurez de la Capacidad del Software (Software Capability Maturity Model, SW-CMM) se detallan una serie de áreas clave del proceso, las cuales describen una serie de funciones de ingeniería del software que se han de llevar a cabo para el desarrollo de una buena práctica de desarrollo, Estas áreas están agrupadas en cinco niveles inclusivos: Inicial repetible, definido, gestionado y optimizado.

Estos niveles sirven de referencia para determinar el estado de la madurez del proceso de desarrollo software en una organización.

Mediante un conjunto de métricas se puede medir la calidad de cada una de las áreas clave, y de esta manera se obtiene una visión precisa del rigor, la eficacia y la eficiencia de la metodología de desarrollo que posee una determinada organización desarrolladora de software.

Tal como se ha comentado, los niveles en los que se agrupan las áreas claves de proceso son inclusivos: para alcanzar uno es necesario haber logrado y además mantener todos los niveles los anteriores.

Una breve descripción de cada uno de estos niveles es la que se detalla a continuación:

– Inicial: Está caracterizado por una aproximación de tipo intuitivo al proceso de desarrollo del software. El éxito depende del esfuerzo individual de los que participan.

  No se han definido procesos metodológicos, o se han definido pero no se utilizan. En este tipo de organizaciones lo primero que es necesario es realizar medidas que sirvan para estimar y planificar en el futuro.

– Repetible: La madurez metodológica de la organización permite estimar de manera fiable el tamaño funcional o físico del sistema, así como recursos, esfuerzo, costes y calendario necesarios para completar el proyecto.

  Se han puesto las bases para poder repetir éxitos anteriores en proyectos con características similares. Las áreas clave de proceso definidas en este nivel, cuyo estado se puede conocer mediante la utilización de diversas métricas, son las siguientes:

  · Gestión de requisitos.

  · Planificación del proyecto software.

  · Seguimiento y control del proyecto.

  · Gestión de la subcontratación del software.

  · Aseguramiento de la calidad del software.

  · Gestión de la configuración del software.

– Definido. Además de una buena gestión de proyectos, a este nivel las organizaciones disponen de procedimientos adecuados de coordinación entre grupos, una política formación del personal, técnicas de ingeniería específicas y un nivel más avanzado de métricas en los procesos.

  Las áreas clave que se han de desarrollar para alcanzar este nivel son:

  · Desarrollo y mejora de los procesos de la organización.

  · Definición de los procesos de la organización.

  · Programa de formación.

  · Gestión integrada del software.

  · Ingeniería de producto software.

- Coordinación entre grupos.
- Revisión conjunta.

– Gestionado: se añade un sistema adecuado de gestión a un proceso definido. Se utilizan técnicas de realimentación desde las el comienzo de las actividades del proyecto para determinar prioridades en las actividades y conocer cómo se emplean los recursos.

  Los efectos de cambios en una actividad se pueden determinar en otras. Se recopilan además medidas detalladas del proceso de desarrollo software y de la calidad del producto implementado.

  En definitiva, en el nivel gestionado se evalúa la efectividad de todas las actividades que se llevan a cabo en el proceso. Las áreas clave definidas en este nivel son estas:

  - Gestión cuantitativa del proyecto.
  - Gestión de calidad del software.

– Optimizado: Existe un proceso mejora continua de los procesos de desarrollo. Las mediciones de actividades se utilizan para optimizar el proceso, eliminando y añadiendo actividades y reorganizando su estructura interna como respuesta a los resultados de las métricas.

  Las áreas definidas para este nivel son:

  - Prevención de defectos.
  - Gestión de cambios tecnológicos.
  - Gestión de cambios en los procesos.

UD5
Lo más importante

- Software de calidad es aquel que está en consonancia con los requisitos sean estos tanto funcionales como de rendimiento que hayan sido explícitamente establecidos, y además con los estándares de desarrollo explícitamente documentados y con las características implícitas que se desean de todo software desarrollado profesionalmente.

- En el contexto de la ingeniería de software, calidad del software se refiere a dos conceptos relacionados pero distintos que existen allí donde la calidad se define en un contexto de negocios: calidad funcional y estructural

- Calidad consiste en la ausencia de deficiencias Sin embargo, en un manual como este, es conveniente estandarizar una breve definición de la palabra calidad como “la aptitud para el uso” “.

- El análisis estructural de calidad y la medición se realiza a través del análisis del código fuente, la arquitectura, plataforma de software, esquemas de bases de datos en relación a los principios y normas que en conjunto definen la arquitectura conceptual y lógico de un sistema.

- Los errores de programación críticos: Estas vulnerabilidades no pueden simplemente hacer fallar un caso de prueba, sino que son el resultado de malas prácticas que en determinadas circunstancias pueden conducir a cortes catastróficos, degradaciones de rendimiento, brechas de seguridad, datos dañados, y una miríada de otros problemas

- La Mantenibilidad está estrechamente relacionado con el concepto de deuda técnica, que es una expresión de los costes derivados de la falta de mantenimiento.

- Al igual que con la fiabilidad, las causas de la ineficiencia de rendimiento se encuentran a menudo en violaciones de la buena práctica de la arquitectura y la codificación que puede ser detectado midiendo los atributos estáticos de calidad de una aplicación.

- En la ingeniería del software ha trabajado para obtener un conjunto de medidas indirectas que dan lugar a métricas que proporcionan cierta indicación de la calidad de algún tipo de representación del software.

- Medida de la eficacia en la eliminación de defecto uno de los objetivos del sistema de calidad es encontrar defectos cuanto antes, para evitar que el sobrecosto que supone corregirlos en fases posteriores del desarrollo.

- La serie de normas ISO-9000 establece los requisitos básicos que debe de cumplir un sistema de control que garantice productos o servicios de calidad consistente.

- En el Modelo de Madurez de la Capacidad del Software (Software Capability Maturity Model, SW-CMM) se detallan una serie de áreas clave del proceso, las cuales describen una serie de funciones de ingeniería del software que se han de llevar a cabo para el desarrollo de una buena práctica de desarrollo, Estas áreas están agrupadas en cinco niveles inclusivos.

UD5
Autoevaluación

1. Los factores directos de calidad…

    a. Los que el cliente aprecia de un vistazo.
    b. Uno de ellos es la facilidad de uso.
    c. Son los que se pueden medir directamente sobre el producto.
    d. Ninguna de las anteriores es correcta.

2. La calidad estructural del software…

    a. Se refiere a la infraestructura que lo soporta.
    b. Se refiere a la forma en que cumple los requisitos funcionales.
    c. Se refiere al modo en que este cumple con los requisitos no funcionales.
    d. Sólo se mide a través de las pruebas.

3. La mala calidad de una infraestructura de sistema:

    a. Esta correlacionada con el alto impacto que tienen las interrupciones debidas a datos dañados y otros factores.
    b. No afecta al coste de mantenimiento.
    c. Sólo se aplica a sistemas embebidos.
    d. No se aplica en sistemas ERP y CRM.

4. La fiabilidad:

   a. Sólo se aplica en entornos empresariales.
   b. Es esencial en sistemas de alta velocidad de ejecución.
   c. Es un atributo de resistencia y solidez estructural.
   d. Todas las respuestas son incorrectas..

5. La medición de la calidad del software:

   a. Consiste en cuantificar en qué medida un sistema posee características deseables.
   b. Es realizada por los programadores.
   c. Solamente se aplica en sistemas o proyectos de gran envergadura.
   d. Es una medida de los atributos de código adecuados del sistema.

6. El análisis estructural de calidad y la medición…

   a. Ser realizan a través del análisis del código, la arquitectura, plataformas de software y esquemas de bases de datos.
   b. Miden el esquema de la base de datos.
   c. Se realizan siguiendo los principios de fiabilidad y exhaustividad.
   d. Ninguna de las respuestas es correcta.

7. La mantenibilidad:

   a. Está relacionada con el concepto de deuda técnica.
   b. Ha de medirse en los sistemas empotrados.
   c. Se debe cuidar solo si se prevé que el sistema va a fallar a menudo.
   d. Una baja mantenibilidad siempre tiene su origen en la falta de tiempo.

8. Que atributo NO es de los que deberían tener las métricas de software:

   a. Empíricas e intuitivas.
   b. Simples y sencillas de calcular.
   c. Subjetivas y objetivas.
   d. Consistentes en el empleo de unidades y tamaños.

9. Las normas ISO 9000:

    a. Determinan como se deben formar a los programadores en un entorno de calidad.
    b. Indican los documentos de calidad que debe generar una organización.
    c. Establecen los requisitos de un sistema de control de calidad.
    d. Indican como deben hacerse las mediciones de calidad del software.

10. El modelo de madurez del software CMM:

    a. Indica como cumple la normativa ISO una organización.
    b. Detalla las tareas que se tienen que realizar conforme a la calidad.
    c. Sirve de referencia para indicar el estado de I+D de una empresa.
    d. Detalla cinco niveles inclusivos para determinar la madurez del proceso de desarrollo e una organización.

Área: informática y comunicaciones

UD6

Identificación de los tipos de pruebas

6.1. Identificación de los tipos de pruebas

6.2. Análisis de pruebas de defectos. Pruebas de caja negra. Pruebas estructurales. Pruebas de trayectoria. Pruebas de integración. Pruebas de interfaces

- 6.2.1. Preparación de los casos de prueba
- 6.2.2. Casos de prueba
- 6.2.3. Codificar las pruebas
- 6.2.4. Definir procesos de prueba
- 6.2.5. Ejecución de pruebas
- 6.2.6. Generación de informes de pruebas

6.1. Identificación de los tipos de pruebas

Las pruebas del software son un elemento crítico para la garantía de calidad del software y representa una revisión final de las especificaciones, del diseño y de la codificación.

Es un elemento que tiene como objetivo la verificación y validación del producto. La verificación se refiere al conjunto de actividades que aseguran que el software implementa correctamente una función específica.

La validación se refiere a un conjunto diferente de actividades que aseguran que el software construido se ajusta a los requisitos del cliente.

El proceso de ingeniería del software se puede ver como una espiral. Inicialmente, la ingeniería de sistemas define el papel del software y conduce al análisis de los requisitos del software, donde se establece el dominio de información, la función, el comportamiento, el rendimiento, las restricciones y los criterios de validación del software.

En el interior de la espiral, llegamos al diseño y, por último, a la codificación. Para desarrollar software se dan vueltas en espiral a través de una serie de flujos o líneas que disminuyen el nivel de abstracción cada vez.También se puede ver la estrategia para la prueba del software en el contexto de la espiral.

La prueba de unidad comienza en el vértice de la espiral y se centra en cada unidad del software, tal como está implementada en código fuente.

La prueba avanza, hacia fuera de la espiral, hasta llegar a la prueba de integración, donde el foco de atención es el diseño y la construcción de la arquitectura del software.

Dando otra vuelta por la espiral hacia fuera, encontramos la prueba de validación, donde se validan los requisitos establecidos como parte del análisis de requisitos del software, comparándolos con el sistema que ha sido construido.

Finalmente, llegamos a la prueba del sistema, en la que se prueban como un todo el software y otros elementos del sistema.

Probar software de computadora es como moverse hacia fuera por una espiral que, a cada vuelta, aumenta el alcance de la prueba y valida cada una de las fases anteriormente realizadas.

A continuación pasamos a comentar cada una de estas fases de prueba por separado:

– Prueba unidad: la prueba de unidad centra el proceso de verificación en la menor unidad del diseño del software: el componente software o módulo.

  Las pruebas que se dan como parte de la prueba de unidad son las siguientes:

  · Se prueba la interfaz del módulo para asegurar que la información fluye de forma adecuada hacia y desde la unidad de programa que está siendo probada.

  · Se examinan las estructuras de datos locales para asegurar que los datos que se mantienen temporalmente conservan su integridad durante todos los pasos de ejecución del algoritmo.

  · Se prueban las condiciones límite para asegurar que el módulo funciona correctamente en los límites establecidos como restricciones de procesamiento.

  · Se ejercitan todos los caminos independientes de flujo de la estructura de control con el fin de asegurar que todas las sentencias del módulo se ejecutan por lo menos una vez.

  · Se prueban todos los caminos de manejo de errores.

  Debido a que un componente o módulo no es un programa independiente, se debe desarrollar para cada prueba de unidad un software para probarlo, el controlador y el resguardo.

  Estos dos elementos son muy importantes para todas las pruebas no solamente para la prueba unidad.

  En la mayoría de las aplicaciones, un controlador es un programa que acepta los datos del caso de prueba, pasa estos datos al módulo a ser probado e imprime los resultados importantes.

Los resguardos sirven para reemplazar módulos que están subordinados, es decir, son llamados, al componente que hay que probar.

Un resguardo o subprograma simulado usa la interfaz del módulo subordinado, lleva a cabo una mínima manipulación de datos, imprime una verificación de entrada y devuelve control al módulo de prueba que lo invocó.

– Prueba de integración: La prueba de integración es una técnica sistemática para construir la estructura del programa mientras que, al mismo tiempo, se llevan a cabo pruebas para detectar errores asociados con la interacción entre módulos.

El objetivo es coger los módulos probados mediante la prueba de unidad y construir una estructura de programa que esté de acuerdo con lo que dicta el diseño.

A menudo se suele intentar una integración no incremental (bigbang). En ella se combinan todos los módulos por anticipado.

A menudo se prueba todo el programa en conjunto. En esta situación se encuentran un gran conjunto de errores.

La corrección se hace difícil, puesto que es complicado aislar las causas al tener delante el programa entero en toda su extensión.La integración incremental es la antítesis del enfoque del «bigbang».

En la integración incremental el programa se construye y se prueba en pequeños segmentos en los que los errores son más fáciles de aislar y de corregir.

De esta forma es más probable que se puedan probar completamente las interfaces y se puede aplicar un enfoque de prueba sistemática.A la hora de realizar una integración incremental se puede seguir dos enfoques:

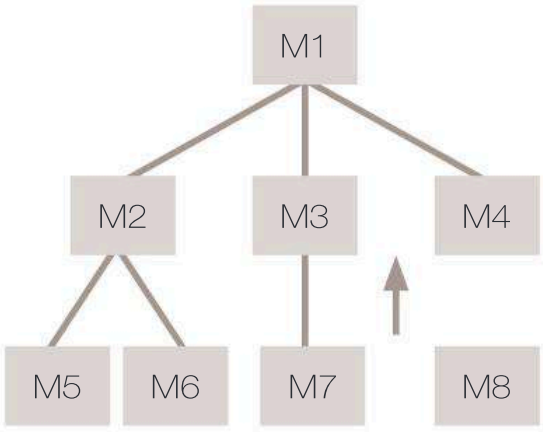
Se integran los módulos moviéndose hacia abajo por la jerarquía de control, comenzando por el módulo de control principal (programa principal).

Los módulos subordinados al módulo de control principal se van incorporando en la estructura, bien de forma primero en profundidad o primero en anchura.

En el modo primero en profundidad se integran todos los módulos subordinados a uno antes de pasar a integrar otro módulo.

En forma primero en anchura(se integran todos los módulos subordinados antes de pasar al siguiente nivel. Mientras se desarrolla la prueba es necesario contar con respaldos de todos los módulos a integrar.

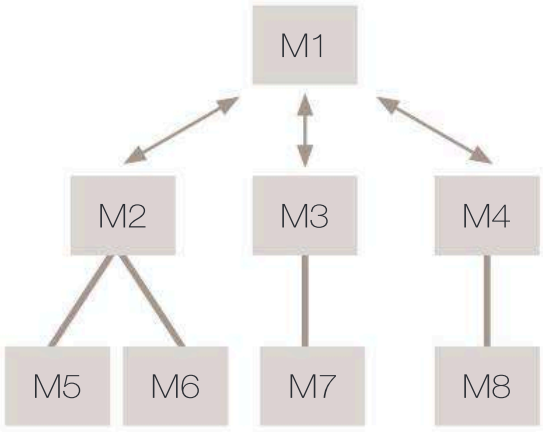
Un esquema de su desarrollo es el siguiente:



La prueba de la integración ascendente, como su nombre indica, empieza la construcción y la prueba con los módulos de los niveles más bajos de la estructura del programa) y los va integrado sucesivamente con los módulos que los utilizan hasta llegar a integrar toda la aplicación.

Se combinan los módulos de bajo nivel en grupos (denominados también construcciones) que realicen una subfunción específica del software para los cuales se implementa un controlador que los ejecute. Una vez comprobada su validad los grupos se van integrando con otros grupos.

Dado que los módulos se integran de abajo hacia arriba, los módulos subordinados siempre están disponibles y se elimina la necesidad de resguardos.



– Pruebas de validación: la validación se consigue cuando el software funciona de acuerdo con las expectativas razonables del cliente.

  Cuando se construye software a medida para un cliente, se llevan a cabo una serie de pruebas de aceptación para permitir que el cliente valide todos los requisitos. A este conjunto de pruebas para validar requisitos es a lo que se llaman pruebas de validación. Se han clasificado generalmente en dos tipos, pruebas alfa y beta.

  Las pruebas de validación o generalmente las realiza el usuario final en lugar del responsable del desarrollo del sistema. Si el software se desarrolla como un producto que va a ser usado por muchos clientes, no es práctico realizar pruebas de aceptación formales para cada uno de ellos.

  La mayoría de los desarrolladores de productos de software llevan a cabo un proceso denominado prueba alfa y beta para descubrir errores.

  · La prueba alfa se lleva a cabo, por un cliente, en el lugar de desarrollo. Se usa el software de forma natural con el desarrollador como observador del usuario y registrando los errores y los problemas de uso. Las pruebas alfa se llevan a cabo en un entorno controlado.

  · La prueba beta se lleva a cabo por los usuarios finales del software en los lugares de trabajo de los clientes.

  A diferencia de la prueba alfa, el desarrollador no está presente normalmente. Así, la prueba beta es una aplicación «en vivo» del software en un entorno que no puede ser controlado por el desarrollador.

  El cliente registra todos los problemas que encuentra durante la prueba beta e informa a intervalos regulares al desarrollador.

  Como resultado de los problemas informados durante la prueba beta, el desarrollador del software lleva a cabo modificaciones y así prepara una versión del producto de software para toda la clase de clientes.

– Prueba del sistema: la prueba del sistema, realmente, está constituida por una serie de pruebas diferentes cuyo propósito primordial es ejercitar profundamente el sistema basado en computadora.

Aunque cada prueba tiene un propósito diferente, todas trabajan para verificar que se han integrado adecuadamente todos los elementos del sistema y que realizan las funciones apropiadas. Dentro de la prueba del sistema se realizan las siguientes pruebas:

– La prueba de recuperación es una prueba del sistema que fuerza el fallo del software de muchas formas y verifica que la recuperación se lleva a cabo apropiadamente.

– La prueba de seguridad intenta verificar que los mecanismos de protección incorporados en el sistema lo protegerán, de hecho, de accesos no autorizados.

– La prueba de resistencia ejecuta un sistema de forma que demande recursos en cantidad, frecuencia o volúmenes anormales.

6.2. Análisis de pruebas de defectos. Pruebas de caja negra. Pruebas estructurales. Pruebas de trayectoria. Pruebas de integración. Pruebas de interfaces

Las técnicas de prueba del software básicamente se pueden clasificar dentro de dos tipos, las que examinan la estructura del programa, denominadas pruebas de la caja blanca, y las que no, que toman el software como una caja negrea a la que se el suministran datos y solamente comprueba la corrección de los resultados que devuelve con respecto a los datos administrados.

A continuación vamos a examinar cada uno de estos tipos así como los ejemplos concretos de pruebas que se incluyen en cada uno de ellos.

Pruebas de caja blanca

La prueba de caja blanca, denominada a veces prueba de caja de cristal es un método de diseño de casos de prueba que examina la estructura de control interna del programa para obtener los casos de prueba.

Mediante los métodos de prueba de caja blanca, el programador puede obtener casos de prueba que:

– Garanticen que se ejercita por lo menos una vez todos los caminos independientes del programa.

– Ejerciten todas las decisiones lógicas en sus vertientes verdadera y falsa.

– Ejecuten todos los bucles en sus límites.

– Ejerciten las estructuras internas de datos para asegurar su validez.

Se puede plantear el hecho de que las pruebas de caja blanca son muy exhaustivas y costosas, y si no sería más indicado dedicar ese esfuerzo a realizar pruebas de caja negra, que al fin y al cabo se centran en la validez de las salidas frente a las entradas, que es lo que nos interesa de un programa.

Hay varias razones que aconsejan no actuar así:

– Los errores lógicos y las suposiciones incorrectas son inversamente proporcionales a la probabilidad de que se ejecute un camino del programa.

– Los errores tienden a introducirse en el trabajo cuando se implementan funciones, condiciones o controles que se encuentran fuera de lo normal.

A menudo se considera que un camino lógico tiene pocas posibilidades de ejecutarse cuando, de hecho, se puede ejecutar de forma normal.

El flujo lógico de un programa a veces no es nada intuitivo, lo que significa que las suposiciones intuitivas sobre el flujo de control y los datos nos pueden llevar a tener errores de diseño que sólo se descubren con las pruebas de caja blanca.

Los errores tipográficos son aleatorios. Cuando se traduce un programa a código fuente en un lenguaje de programación, es muy probable que se den algunos errores de escritura.

Muchos serán descubiertos por los mecanismos de comprobación de sintaxis, pero otros permanecerán sin detectar hasta que comience la prueba.

Es igual de probable que haya un error tipográfico en un oscuro camino lógico que en un camino principal. Por esta razón en esta prueba se considera la prueba exhaustiva de todos los caminos que forman el flujo del programa, independientemente de lo “importantes “ que se consideren.

Prueba del camino básico

La prueba del camino básico es una técnica de prueba de caja blanca propuesta por Tom McCabe.

El método del camino básico permite obtener una medida de la complejidad lógica de un programa y usar esa medida como guía para la definición de un conjunto básico de caminos de ejecución.

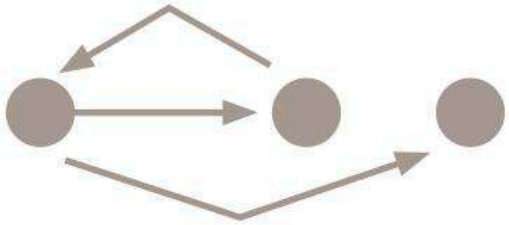
Los casos de prueba obtenidos del conjunto básico garantizan que durante la prueba se ejecuta por lo menos una vez cada sentencia del programa. Antes de hablar del método del camino básico se debe conocer una sencilla notación para la representación del flujo de control, denominada grafo de flujo (o grafo del programa).

El grafo de flujo representa el flujo de control lógico mediante la siguiente notación, en la que cada construcción estructurada tiene su correspondiente símbolo en el grafo del flujo.

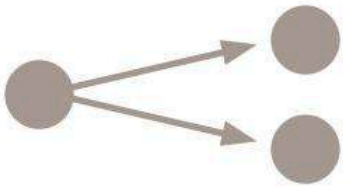
Estructura secuencial:



Estructura repetir… hasta que:



Estructura condicional:



La complejidad ciclomática es una medida del software que proporciona una medición cuantitativa de la complejidad lógica de un programa.

Cuando se usa en el contexto del método de prueba del camino básico, el valor calculado como complejidad ciclomática define el número de caminos independientes del conjunto básico de un programa.

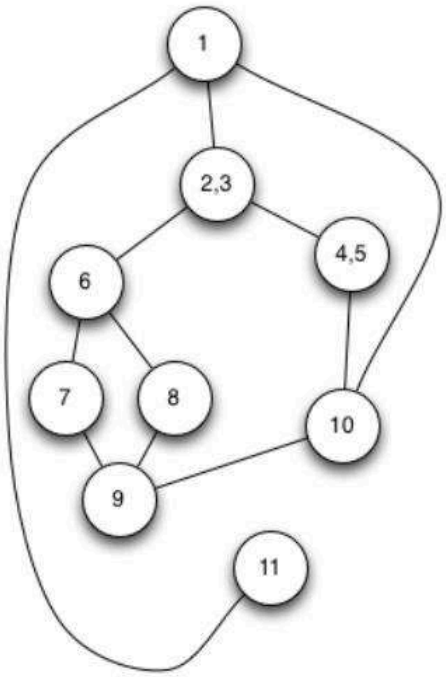
Esto nos da un límite superior para el número de pruebas que se deben realizar para asegurar que se ejecuta cada sentencia al menos una vez.

Un camino independiente es cualquier camino del programa que introduce, por lo menos, un nuevo conjunto de sentencias de proceso o una nueva condición.

En términos del grafo de flujo, un camino independiente está constituido por lo menos por una arista que no haya sido recorrida anteriormente a la definición del camino.

Ejemplo

Por ejemplo, para el grafo de flujo siguiente:



Estructuras y su grafo asociado

Un conjunto de caminos independientes sería:

- camino 1: 1-11
- camino 2: 1-2-3-4-5-10-1-11
- camino 3: 1-2-3-6-8-9-10-1-11
- camino 4: 1-2-3-6-7-9-10-1-11

Para saber cuántos caminos hemos de buscar, la complejidad ciclomática nos da la respuesta. La complejidad se puede calcular de tres formas, según el número de regiones del grafo de flujo, según el número de nodos o según el número de aristas.

La complejidad ciclomática, V(G), de un grafo de flujo G se define como V(G) = A - N + 2 donde A es el número de aristas del grafo de flujo y N es el número de nodos del mismo.



Definición

La **complejidad ciclomática,** V(G), de un grafo de flujo G también se define como V(G)=P+ 1 donde P es el número de nodos predicado contenidos en el grafo de flujo G.

Cada nodo que contiene una condición se denomina nodo predicado y está caracterizado porque dos o más aristas del grafo salen de él.

Una vez calculada la complejidad ciclomática debemos escoger los datos de forma que las condiciones de los nodos predicado estén adecuadamente establecidas, con el fin de comprobar cada camino.

Prueba de condiciones

La prueba de condiciones lo que considera es que los errores son mucho más comunes en el entorno de los condiciones lógicas que en los sentencias de proceso secuencial. Por esta razón se centran en probar estas situaciones.

La prueba de condición es un método de diseño de casos de prueba que ejercita las condiciones lógicas contenidas en el módulo de un programa.

Una condición simple es una variable lógica o una expresión relacional, una expresión relacional relaciona entre sí dos variables o constantes numéricas con los operadores de igualdad, mayor que y menor que.

Una condición compuesta está formada por dos o más condiciones simples, operadores lógicos y paréntesis.

Los operadores lógicos permitidos en una condición compuesta generalmente son OR, AND y NOT. A una condición sin expresiones relacionales se la denomina Expresión lógica.

Por tanto, los tipos posibles de componentes en una condición pueden ser: un operador lógico, una variable lógica, un par de paréntesis lógicos (que rodean a una condición simple o compuesta), un operador relacional o una expresión aritmética.

Si una condición es incorrecta, entonces es incorrecto al menos un componente de la condición. Así, los tipos de errores de una condición pueden ser los siguientes:

– Error en operador lógico.

– Error en variable lógica.

– Error en paréntesis lógico.

– Error en operador relacional.

– Error en expresión aritmética.

El método de la prueba de condiciones se centra en la prueba de cada una de las condiciones del programa.

El propósito de la prueba de condiciones es detectar, no sólo los errores en las condiciones de un programa, sino también otros errores en dicho programa.

La base de la prueba de condiciones es considerar que si un conjunto de pruebas de un programa es efectivo para detectar errores en las condiciones que se encuentran en su interior, es probable que el conjunto de pruebas también sea efectivo para detectar otros errores en dicho programa.

Hay dos tipos de pruebas de condiciones, la prueba de ramificaciones y la prueba de dominio.

– La prueba de ramificaciones es la estrategia de prueba de condiciones más sencilla. Para una condición compuesta, es necesario ejecutar al menos una vez las ramas verdadera y falsa de dicha condición y así como cada condición simple que constituye la compuesta.

  Para una expresión lógica con n variables, habrá que realizar por tanto 2n pruebas. Esta técnica puede detectar errores de un operador, de una variable y de un paréntesis lógico, pero sólo es práctica cuando el valor de n es pequeño.

– La prueba del dominio requiere la realización de tres o cuatro pruebas para una expresión relacional.

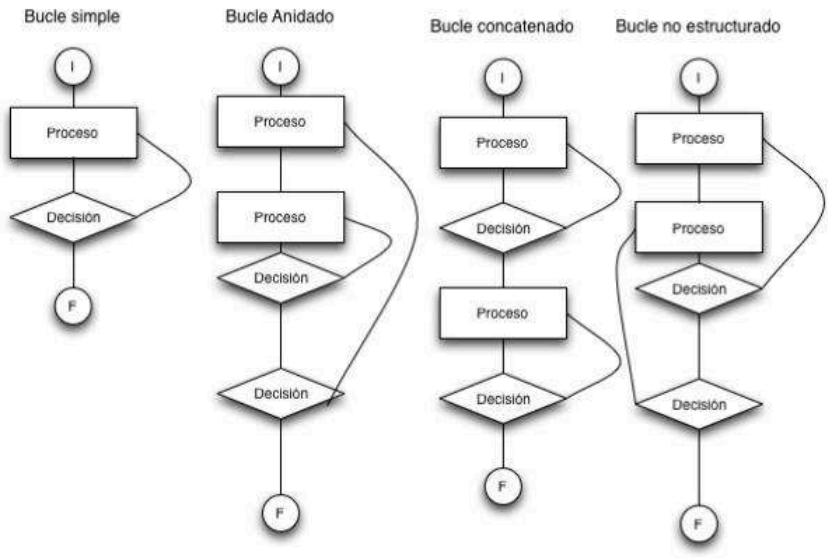
  Para una expresión relacional de la forma E1, <operador-relacional> E2 se requieren tres pruebas con valores de E1 mayor, igual o menor que el valor de E2 respectivamente.

Si el <operador-relacional> es incorrecto y E, y E, son correctos, entonces estas tres pruebas garantizan la detección de un error del operador relacional.

Prueba de bucles

La prueba de bucles es una técnica de prueba de caja blanca que se centra exclusivamente en la validez de las construcciones de bucles.

Se pueden definir cuatro clases diferentes de bucles: bucles simples, bucles concatenados, bucles anidados y bucles no estructurados:



Tipos de bucles

Para cada uno de los tipos de bucles se usa una determinada técnica.

– Bucles simples: a los bucles simples se les debe aplicar el siguiente conjunto de pruebas, donde n es el número máximo de pasos permitidos por el bucle:

- Pasar por alto totalmente el bucle.
- Pasar una sola vez por el bucle.
- Pasar dos veces por el bucle.
- Hacer m pasos por el bucle con m < n.
- Hacer n - 1, n y n+1 pasos por el bucle.

– Bucles anidados: si extendiéramos el enfoque de prueba de los bucles simples a los bucles anidados, el número de posibles pruebas aumentaría geométricamente a medida que aumenta el nivel de anidamiento.

  Esto lleva un número impracticable de pruebas. Un enfoque que ayuda a reducir el número de pruebas es el siguiente:

  - Comenzar por el bucle más interior. Establecer o configurar los demás bucles con sus valores mínimos.
  - Llevar a cabo las pruebas de bucles simples para el bucle más interior, mientras se mantienen los parámetros de iteración (por ejemplo, contador del bucle) de los bucles externos en sus valores mínimos.
  - Realizar otras pruebas para valores fuera de rango o excluidos.
  - Progresar hacia fuera, llevando a cabo pruebas para el siguiente bucle, pero manteniendo todos los bucles externos en sus valores mínimos y los demás bucles anidados en sus valores “típicos”.
  - Continuar hasta que se hayan probado todos los bucles.

– Bucles concatenados. Los bucles concatenados se pueden probar mediante el enfoque anteriormente definido para los bucles simples, mientras cada uno de los bucles sea independiente del resto.

  Sin embargo, si hay dos bucles concatenados y se usa el controlador del bucle 1 como valor inicial del bucle 2, entonces los bucles no son independientes.

  Cuando los bucles no son independientes, se recomienda usar el enfoque aplicado para los bucles anidados.

– Bucles no estructurados. Siempre que sea posible, esta clase de bucles se deben reescribir para que se ajusten a las construcciones de programación estructurada.

Pruebas de caja negra

Las pruebas de caja negra, también denominada prueba de comportamiento, se centran en los requisitos funcionales del software independientemente de cómo estos se hayan implementado.

O sea, la prueba de caja negra permite obtener conjuntos de condiciones de entrada que ejerciten completamente todos los requisitos funcionales de un programa. La prueba de caja negra no es una alternativa a las técnicas de prueba de caja blanca.

Más bien se trata de un enfoque complementario que intenta descubrir diferentes tipos de errores que los métodos de caja blanca. Las técnicas más utilizadas son las que a continuación se describen.

Pruebas de comparación

Hay situaciones, generalmente con sistemas en tiempo real de alta fiabilidad (por ejemplo, aviónica de aeronaves, control de planta nuclear) en las que la fiabilidad del software es algo absolutamente crítico.

En ese tipo de aplicaciones, a menudo se utiliza hardware y software redundante para minimizar la posibilidad de error.

Cuando se desarrolla software redundante, varios equipos separados desarrollan versiones independientes de una aplicación, usando las mismas especificaciones. En esas situaciones, se deben probar todas las versiones con los mismos datos de prueba, para asegurar que todas proporcionan una salida idéntica.

Esas versiones independientes son la base de una técnica de prueba de caja negra denominada prueba de comparación: Cuando se han producido múltiples implementaciones de la misma especificación, a cada versión del software se le proporciona como entrada los casos de prueba diseñados mediante alguna otra técnica de caja negra (por ejemplo, la partición equivalente).

Si las salidas producidas por las distintas versiones son idénticas, se asume que todas las implementaciones son correctas. Si la salida es diferente, se investigan todas las aplicaciones para determinar el defecto responsable de la diferencia en una o más versiones.

6.2.1. Preparación de los casos de prueba

Es cierto que cualquier código que escribamos debería de ser probado antes de dar por finalizada la implementación de una clase, ya que si no, no estaríamos totalmente seguros de su correcto funcionamiento.

Para verificar que el código no contiene errores de programación y que además realiza adecuadamente lo que nosotros esperábamos de él, es realizando una serie de test que lo corroboren.



Sabías que

Todo el mundo alguna vez ha ido escribiendo mensajes de salida de texto a lo largo de su código. Esta es una técnica muy utilizada para controlar el valor de ciertas variables y para la detección de posibles errores, lo que confirma que todos realizamos pruebas a nuestros programas de alguna u otra manera.

Pero existen formas más avanzadas, eficientes y seguras de probar el código.

Podemos utilizar para ello JUnit. A través de JUnit tendremos la posibilidad de realizar test a nuestras clases de forma sencilla, rápida y elegante, así como validar los resultados que nos ofrecen.

JUnit es un conjunto de bibliotecas que se utilizan en programación para hacer Pruebas Unitarias a aplicaciones Java.

El conjunto de clases (framework) incluidas en JUnit permite controlar la ejecución de las clases Java, de tal forma que podremos evaluar si el funcionamiento de cada uno de los métodos de la clase realiza su trabajo como debería.

JUnit se emplea para indicar si la clase en cuestión ha pasado los test de prueba, en otro caso se devolverá una notificación de fallo en el método correspondiente.

Actualmente, el entorno de desarrollo NetBeans cuanta con plugins que permiten la generación automática de plantillas de pruebas de clases Java.

A continuación se enumeran algunas razones de peso por la que este framework está muy extendido entre la comunidad de programadores Java:

– A través de los test de JUnit se incrementa la calidad y velocidad de generación de código.

– La escritura de test es sencilla.

– Los test JUnit chequean sus propios resultados y proporcionan información de retorno al instante.

– Los testsJUnit incrementan la estabilidad del software.

– Los test JUnit se escriben en Java.

– JUnit es gratuito.

El IDE NetBeans ya incorpora JUnit desde la instalación, por lo que podemos beneficiarnos de esta herramienta de testeo sólo con utilizarla desde el IDE.



La Depuración de programas es el proceso por el cual se identifican y corrigen errores de programación.

Generalmente, en el argot de programación se utiliza la palabra debugging, que significa localización y eliminación de bichos (bugs) o errores de programa.

A través de este proceso se descubren los errores y se identifica qué zonas del programa los producen. Hay tres etapas por las que un programa pasa cuando éste es desarrollado y que pueden generar errores:

– Compilación: una vez que hemos terminado de afinar un programa, solemos pasar generalmente cierto tiempo eliminando errores de compilación. El compilador mostrará una serie de chequeos en el código, sacando a la luz errores que pueden no apreciarse a simple vista. Una vez que el programa es liberado de los errores de compilación, obtendremos de él algunos resultados visibles pero quizá no haga aún lo que se desea que se haga.

– Enlazado: todos los programas hacen uso de librerías de métodos y otros utilizan métodos generados por los propios programadores. Un método es enlazado en los lenguajes de programación modernos (linked) sólo cuando éste es llamado, durante el proceso de ejecución. Pero cuando el programa es compilado, se realizan comprobaciones para saber si los métodos llamados existen y sus parámetros son correctos en número y tipo. Así que los errores de enlazado son detectados durante la etapa de compilación.

– Ejecución: cuando el programa entra en ejecución, es muy frecuente que éste no funcione como se esperaba. de hecho, es normal que el programa falle.

Algunos errores serán detectados automáticamente y el programador será informado, como acceder a una parte de un array que no existe (error de índices) por ejemplo.

Otros son más sutiles y dan lugar simplemente a comportamientos no esperados, debido a la existencia de errores ocultos (bugs) en el programa.

De ahí los términos bug y debugging. El problema de la depuración es que los síntomas de un posible error son generalmente poco claros, hay que recurrir a una labor de investigación para encontrar la causa.

La depuración de programas es algo así como ser doctor: existe un síntoma, hemos de encontrar la causa y entonces determinar el problema.

Para realizar el proceso de depuración se utiliza una herramienta específica, el depurador, que normalmente viene junto a un entorno de desarrollo integrado. El depurador permite ir ejecutando un programa línea a línea o paso a paso, siendo por tanto una potente herramienta para el programador, ya que este puede ir viendo como se comporta su programa internamente en tiempo de ejecución.

Además el depurador permite definir lugares donde se ha de detener el programa o puntos de ruptura, y también a va a permitir examinar el valor de las variables durante la ejecución de un programa, utilizando lo que se suele llamar un inspector. A continuación detallaremos como se realiza este proceso



Sabías que

La depuración y la prueba suelen requerir una cantidad de tiempo considerable en comparación con el tiempo dedicado a la primera codificación del programa. Pero no te preocupes, es normal emplear más tiempo en este proceso.

La depuración de programas se lleva a cabo a través del debugger o depurador del sistema de desarrollo que estemos utilizando.

Este depurador será una herramienta que nos ayudará a eliminar posibles errores de nuestro programa. Podremos utilizar depuradores simples, como el jdb propio de Java basado en línea de comandos. O bien, utilizar el depurador existente en nuestro IDE.

Este último tipo de depuradores muestra los siguientes elementos en pantalla: El programa en funcionamiento, el código fuente del programa y los nombres y valores actuales de las variables que se seleccionen.

Existen al menos dos elementos fundamentales que podemos utilizar en nuestro debugger o depurador, son los siguientes:

Breakpoints o puntos de ruptura: Estos puntos pueden ser determinados por el propio programador a lo largo del código fuente de su aplicación.



Un **Breakpoint** es un lugar en el programa en el que la ejecución se detiene.

Estos puntos se insertan en una determinada línea del código, entonces el programa se pone en funcionamiento y cuando el flujo de ejecución llega hasta él, la ejecución queda congelada y un puntero indica el lugar en el que la ejecución se ha detenido.

El depurador muestra los valores de las variables tal y como están en ese momento de la ejecución. Cualquier discrepancia entre el valor actual y el valor que deberían tener supone una importante información para el proceso de depuración.



Ejecución paso a paso: el depurador también nos permite ejecutar un programa paso a paso, es decir, línea por línea.

A través de esta herramienta podremos seguir el progreso de ejecución de nuestra aplicación y supervisar su funcionamiento.

Cuando la ejecución no es la esperada quizá estemos cerca de localizar un error o bug. EN este caso hay que examinar detalladamente el entorno en el que se ha producido el comportamiento inesperado con el fin de tratar de localizar el fallo de programación. Es buena idea ir remontando mentalmente el valor que han tenido las variables utilizando el inspector, comparándolo con aquel que desearíamos que hubieran tenido, retrocediendo mentalmente cada paso de ejecución del programa.

En ocasiones, si utilizamos métodos procedentes de librerías estándar no necesitaremos hacer un recorrido paso a paso por el interior de éstos métodos, ya que es seguro que no contendrán errores internos y podremos ahorrar tiempo no entrando en su interior paso a paso. El debugger ofrece la posibilidad de entrar o no en dicho métodos.

6.2.2. Casos de prueba

A veces, los programas son complejos y es difícil hacer que éstos funcionen correctamente.

Las pruebas del software son un conjunto de técnicas utilizadas para verificar que un programa lleva a cabo su tarea correctamente.

Viéndolo desde otro punto de vista, podríamos decir que la realización de pruebas en nuestros programas intentan revelar la existencia de errores.Cuando detectamos que existe un error, necesitamos localizarlo llevando a cabo técnicas de depuración de código, para luego acometer las modificaciones necesarias que eliminen dicho error

No obstante, las técnicas de prueba del software no garantizan que todos los errores vayan a salir a la luz y aún así, habrá programas de gran tamaño que sigan conteniendo errores ocultos en ellos.

De todas maneras, la realización de pruebas de software está considerada como una etapa de gran importancia en el desarrollo de software. Casi la mitad del tiempo dedicado al desarrollo de un programa, se emplea en la realización de pruebas al mismo. Por esta razón conviene planificar las pruebas con exhaustividad, ya que si no se realizan correctamente, se puede haber desperdiciado un valioso tiempo de desarrollo en ellas.



Sabías que

En algunas organizaciones, la realización de pruebas se considera tan importante que existen equipos de desarrollo de software en los que hay tantos probadores de software como programadores.

Los procesos de prueba y depuración requieren una considerable cantidad de tiempo y esfuerzo, haciéndose difícil tomar una decisión acertada entre continuar la prueba del programa o entregarlo en su estado actual al cliente.



Definición

Un concepto muy importante es el de **verificación de programas.** La verificación de programas es un proceso por el que se intenta corroborar que un programa hace lo que se espera de él.

Se trata de confirmar que el programa cumple con sus especificaciones. En esta parte de la unidad nos centraremos en las técnicas de prueba sistemática de programas, verificación y análisis de deficiencias del software.

Un pequeño programa que consiste sólo en una única clase, normalmente puede ser probado de una sola vez. En cambio, un programa de mayor tamaño que incluye más cantidad de clases, es posible que por su complejidad, deba ser probado por partes.

En Java, el tamaño natural de cada una de esas partes será la clase y será conveniente probar nuestros programas clase por clase. Es decir, cada vez que se desarrolle una clase, esta se va a probar para comprobar que contempla la funcionalidad que realmente se le había asignado en el diseño Post.

A este tipo de pruebas detalladas es llamado Pruebas de Unidad. También existen otras, como cuando se realizan pruebas de funcionamiento reuniendo todas las partes del programa completo, a estas pruebas reciben el nombre

de Pruebas de Integración o Pruebas de Sistema. En ella se prueban las interfaces entre módulos.

El punto de partida para cualquier prueba es la especificación. No podemos considerar una pérdida de tiempo el análisis claro de dicha información.

En ocasiones, podría ser necesario volver a entrevistar a un cliente o al futuro usuario del programa.

Para entender la importancia de este elemento, supongamos la siguiente especificación:

Ejemplo

Es necesario escribir un programa que solicite la entrada al usuario de números a través de cajas de texto. El programa debe calcular y mostrar la suma de los números.

A priori, esta especificación puede parecer simple y clara. Nada más lejos de la realidad, ya que existen lagunas como:

– ¿Los números son enteros o coma flotante?

– ¿Cuál es el rango y precisión de dichos números?

– ¿Pueden incluirse números negativos?

Si estas preguntas no son aclaradas antes de que el programador comience su trabajo, pueden producirse dificultades en el desarrollo del programa.

Por tanto, será parte del proceso de programación el estudio de la especificación para descubrir cualesquiera omisiones o confusión y para conseguir una especificación totalmente clara.

Ejemplo

A continuación se muestra una versión mucho más clara de la especificación anterior:

Es necesario escribir un programa que solicite la entrada al usuario de una serie de números enteros a través de una caja de texto. Los enteros están en el rango 0 a 10000. El programa calcula y muestra la suma de los números.

Como puedes apreciar, esta especificación es más precisa al establecer el rango permitido de entrada de valores, por ejemplo.

Una vez valorada la importancia de una correcta especificación, el programador puede comenzar el proceso de pruebas del software.

¿Se han de probar todas las posibilidades? Es una pregunta que puede surgir cuando comenzamos a programar.

Pensemos un momento en el amplio rango de posibles valores que pueden tomar los números int de Java. ¿Te imaginas lo que puede suponer realizar una prueba de este tipo? ¡podríamos estar hablando de años!.

Podríamos pensar en que la realización de una prueba requeriría probar nuestro software con todos los posibles valores de entrada

Este tipo de prueba se denomina prueba exhaustiva y significa que seleccionamos todos los posibles valores de entrada, así como todas sus posibles combinaciones y hacemos que nuestro programa opere con ellos.

Partición equivalente

La partición equivalente es un método de prueba de caja negra que divide el campo de entrada de un programa en clases de datos de los que se pueden derivar casos de prueba.

Un caso de prueba ideal descubre de forma inmediata una clase de errores que, de otro modo, requerirían la ejecución de muchos casos antes de detectar el error genérico.

La partición equivalente se dirige a la definición de casos de prueba que descubran clases de errores, reduciendo así el número total de casos de prueba que hay que desarrollar.

Una clase de equivalencia representa un conjunto de estados válidos o no válidos para condiciones de entrada.

Típicamente, una condición de entrada es un valor numérico específico, un rango de valores, un conjunto de valores relacionados o una condición lógica.

Las clases de equivalencia se pueden definir de acuerdo con las siguientes reglas:

– Si una condición de entrada especifica un rango, se define una clase de equivalencia válida y dos no válidas. Por ejemplo el rango [1-99] permite definir tres clases de equivalencia: dentro del rango(ej: 32), inferior al rango(ej -4) y superior al rango(ej 109)

- Si una condición de entrada requiere un valor específico, se define una clase de equivalencia válida y dos no válidas, de la misma manera que en caso de un rango.

- Si una condición de entrada especifica un miembro de un conjunto, se define una clase de equivalencia válida y una no válida, una que pertenezca al conjunto y otra que no.

- Si una condición de entrada es lógica, se define una clase de equivalencia válida y una no válida, para los dos posibles valores lógicos



Una vez determinadas las clases de equivalencia para un programa se debe de desarrollar casos de prueba que contemplen cada una de ellas, teniendo en cuenta que en un caso de prueba se pueden probar varias clases de equivalencia correctas a la vez, pero solamente una clase de equivalencia incorrecta.

Análisis de valores límite

El análisis de valores límite es una técnica de diseño de casos de prueba que complementa a la partición equivalente.

En lugar de seleccionar cualquier elemento de una clase de equivalencia, el AVL elige los casos de prueba en los "extremos" de la clase, ya que los errores tienden a darse más en los límites del campo de entrada que en el "centro"

Las directrices de AVL son similares a las que proporciona la partición equivalente:

- Si una condición de entrada especifica un rango delimitado por los valores a y b, se deben diseñar casos de prueba para los valores a y b, y para los valores justo por debajo y justo por encima de a y b, respectivamente.

- Si una condición de entrada especifica un número de valores, se deben desarrollar casos de prueba que ejerciten los valores máximo y mínimo.

– También se deben probar los valores justo por encima y justo por debajo del máximo y del mínimo.

– Se deben aplicar también las directrices anteriores a las condiciones de salida.

– Si una salida tiene un límite, han de proporcionarse entradas que lo prueben de la misma manera que las entradas.

Si las estructuras de datos internas tienen límites preestablecidos (por ejemplo, una matriz que tenga un límite definido de 100 entradas), hay que asegurarse de diseñar un caso de prueba que ejercite la estructura de datos en sus límites.

6.2.3. Codificar las pruebas

Un script de prueba en las pruebas de software es un conjunto de instrucciones que se van a ejecutar en el sistema sometido a prueba para comprobar que el sistema funciona como se esperaba.

Hay varios medios para ejecutar scripts de prueba.

– Las pruebas manuales . Estos son los más comúnmente llamados casos de prueba .

– Las pruebas automatizadas, utilizando guiones de prueba.

Un guión de pruebas es un corto programa escrito en un lenguaje de programación utilizado para probar parte de la funcionalidad de un sistema de software.

Los guiones de prueba pueden ser escritos como un programa corto puede ser hecho utilizando una herramienta de prueba funcional especial o en un lenguaje de programación conocido.

Los script de prueba pueden tomar la forma de programas ampliamente parametrizados, también llamados prueba controlada por datos

También se usan elementos reutilizables creados en una tabla también llamada pruebas generadas por palabra clave o prueba basada en tablas .Estos dos últimos tipos también se aplican en las pruebas manuales.



Las pruebas automatizadas presentan un buen número de ventajas: pruebas pueden ser ejecutadas continuamente sin la necesidad de intervención humana, son fácilmente repetible, y a menudo más rápidas.

Las pruebas automatizadas son útiles en situaciones en las que la prueba es para ser ejecutado varias veces, por ejemplo como parte de las pruebas de regresión.

Las pruebas automatizadas pueden ser desventajoso cuando el código está mal escrito, dando lugar a pruebas incorrecta o fallos en las pruebas que se llevan a cabo.

Las desventajas de las pruebas automatizadas son que las pruebas automatizadas pueden, como cualquier pieza de software, estar mal escritos o simplemente se fallar durante la reproducción.

También sólo pueden examinar lo que han sido programados para examinar. Como la mayoría de los sistemas están diseñados con la interacción humana en mente, es una buena práctica que un humano comprueba el sistema en algún momento.

Un probador manual entrenado puede notar que el sistema bajo prueba no se está portando como es debido sin que nadie se lo indique sin embargo las pruebas automatizadas sólo pueden examinar lo que han sido programados para examinar.

Cuando se utiliza en las pruebas de regresión, los probadores manuales pueden encontrar nuevos errores al tiempo que garantiza que los errores antiguos no vuelven a aparecer, mientras que un test automatizado sólo se puede garantizar esta último.

La **Prueba mixta,** que combina eltesting automatizado y manual se utiliza con mucha frecuencia.

Consiste en la automatización de lo que necesita ser examinada con frecuencia y puede ser fácilmente comprobado por una máquina, y el uso de pruebas manuales para hacer el diseño de pruebas y pruebas exploratorias generales.

No hay que caer en la trampa de gastar más tiempo automatizando una prueba de lo que se necesitaría para simplemente ejecutarlo de forma manual, a menos que esté previsto para ser ejecutado varias veces.

El siguiente código muestra un método de prueba JUnit.

Ejemplo

```
publicvoidmultiplicationOfZeroIntegersShouldReturnZero () {

 // MyClass de prueba

MyClass probador = new MyClass();

 // pruebas

assertEquals (“10 x 0 deben ser 0”, 0, tester.multiply (10,
0));

assertEquals (“0 x 10 debe ser 0”, 0, tester.multiply (0,
10));

assertEquals (“0 x 0 deben ser 0”, 0, tester.multiply (0,
0));

 }
```



Convenciones de nombres JUnit: hay varias convenciones de nombres potenciales para pruebas JUnit. En uso generalizado es utilizar el nombre de la clase bajo prueba y para agregar el sufijo “Test” para la clase de prueba.

Para los nombres de los métodos de prueba que se recomienda utilizar con frecuencia la palabra “debería” en el nombre del método de prueba.

Ejemplo

Por ejemplo los métodos “ordersShouldBeCreated” o “menuShouldGetActive” ya que esto da una buena pista de lo que debería ocurrir si se ejecuta el método de prueba.

Como regla general, un nombre de la prueba debe explicar lo que hace la prueba de modo que se puede evitar para leer la implementación real.



Conjuntos de pruebas JUnit: si se tienen varias clases de prueba, puede combinarse en un conjunto de pruebas. La ejecución de un conjunto de pruebas ejecutará todas las clases de prueba en esa suite en el orden especificado.

El siguiente código de ejemplo muestra un conjunto de pruebas que define que dos clases de prueba (MYClassTest y MySecondClassTest) deben ser ejecutados.

Ejemplo

```
com.vogella.junit.first paquete;

importorg.junit.runner.RunWith;

importorg.junit.runners.Suite;

importorg.junit.runners.Suite.SuiteClasses;

RunWith (Suite.class)

SuiteClasses ({MyClassTest.class, MySecondClassTest.class})

alltestspublicclass {

}
```

También puede ejecutar las pruebas JUnit fuera de un entorno integrado a través de código Java estándar. Marcos de construcción como Apache Ant o Apache Mavensesuelen utilizar para ejecutar pruebas de forma automática en una base común.

La clase org.junit.runner.JUnitCore ofrece los métodos que le permite ejecutar una o varias pruebas de clases runClasses().



Como parámetro de retorno que recibe un objeto del tipo org.junit.runner.Result. Este objeto se puede utilizar para recuperar información sobre las pruebas.

La clase siguiente muestra cómo ejecutar el MyClassTest. Esta clase se ejecutará su clase de prueba y escribir potenciales fallas en la consola.

Ejemplo

```
Importorg.junit.runner.JUnitCore;

Importorg.junit.runner.Result;

Importorg.junit.runner.notification.Failure;

publicclassMyTestRunner {

 (String [] args) {publicvoid

  Resultado Resultado = JUnitCore.runClasses (MyClassTest.
class);

 para (insuficiencia Fracaso: result.getFailures ()) {

System.out.println (failure.toString ());

 }

 }

}
```

Para ejecutar las pruebas JUnit fuera de l entorno es necesario agregar el archivo jar de la biblioteca JUnit a la ruta de clase del programa.

JUnit utiliza anotaciones para marcar y métodos para configurar la prueba de funcionamiento.La anotaciónTest identifica un método como un método de ensayo:

– Test (Esperado = Exception.class) falla si el método no lanza la excepción nombrada.

- Test (Timeout = 100) falla si el método toma más tiempo de 100 milisegundos.

- Before: método publicvoid () Este método se ejecuta antes de cada prueba. Se utiliza para preparar el entorno de prueba (por ejemplo, leer los datos de entrada, inicializar la clase).

- After: método publicvoid () Este método se ejecuta después de cada prueba.

Se utiliza para la limpieza del entorno de prueba (por ejemplo, eliminar los datos temporales, restaurar los valores predeterminados). También puede guardar la memoria mediante la limpieza de las estructuras de memoria costosas.

- BeforeClass: método publicstaticvoid () Este método se ejecuta una vez, antes del comienzo de todas las pruebas. Se utiliza para llevar a cabo actividades intensivas de tiempo, por ejemplo, para conectarse a una base de datos. Métodos marcados con esta anotación es necesario definir como estática para trabajar con JUnit.

- AfterClass: método publicstaticvoid () Este método se ejecuta una vez, después de haber terminado todas las pruebas. Se utiliza para realizar las actividades de limpieza, por ejemplo, para desconectarse de una base de datos.

- Ignore: Ignora el método de ensayo. Esto es útil cuando el código subyacente ha sido cambiado y el caso de prueba todavía no se ha adaptado.

Por ejemplo si el tiempo de ejecución de esta prueba es demasiado larga para ser incluidos.



Definición

Declaracionesasset:JUnit proporciona métodos estáticos en la clase Assert para detectar ciertas condiciones.

Estos métodos de aseveración normalmente comienzan con assert y le permiten especificar el mensaje de error, el esperado y el resultado real.

Un método afirmación compara el valor real devuelto por una prueba para el valor esperado, y lanza una AssertionException si la prueba de comparación falla.

La siguiente tabla ofrece una visión general de estos métodos. Los parámetros en corchetes [] son opcionales.

– fail (String): Deja que el método falle. Podría ser utilizados para comprobar que no se llega a una cierta parte del código o para tener una prueba de no haber antes de implementar el código de prueba. El parámetro de cadena es opcional.

– assertTrue ([mensaje], condición booleana) Comprueba que la condición booleana es verdadera.

– assertFalse ([mensaje], condición booleana) Comprueba que la condición booleana es falsa.

– assertEquals ([stringmessage], esperado, real) las pruebas de que dos valores son los mismos. Nota: para las matrices no se comprueba la referencia al contenido de las matrices.

– assertEquals ([stringmessage], esperado, real, la tolerancia) de prueba flotante o valores dobles partido. La tolerancia es el número de decimales que deben ser el mismo.

– assertNull ([mensaje], objeto) Comprueba que el objeto es nulo.

– assertNotNull ([mensaje], objeto) Comprueba que el objeto no es nulo.

– assertSame ([cadena], que se espera, real) Comprueba que ambas variables se refieren al mismo objeto.

– assertNotSame ([cadena], que se espera, real) Comprueba que ambas variables se refieren a diferentes objetos.

Se debe proporcionar mensajes significativos en las afirmaciones de modo que sea más fácil para los desarrolladores para identificar el problema.

Esto ayuda en la fijación de la cuestión, sobre todo si alguien mira el problema, que no escribió el código bajo prueba o el código de prueba.

JUnit asume que todos los métodos de prueba se pueden ejecutar en un orden arbitrario.



El código de ensayo bien escrito no debe asumir ninguna orden, es decir, las pruebas no deben depender de otras pruebas.

A partir de JUnit 4.11 se puede utilizar una anotación para definir los métodos de ensayo que están ordenados por nombre del método, en orden lexicográfico.

6.2.4. Definir procesos de prueba

Una estrategia de prueba es un esquema que describe el enfoque de prueba del ciclo de desarrollo de software .

Está se crea para informar a los administradores de proyectos, probadores y desarrolladores acerca de algunas cuestiones clave del proceso de pruebas.

Esto incluye el objetivo de pruebas, métodos de ensayo de nuevas funciones, tiempo total y los recursos necesarios para el proyecto, y el entorno de prueba.

Las **estrategias de prueba** describen cómo se mitigan los riesgos de los productos de las partes interesadas a nivel de prueba, que se van a realizar tipos de prueba, y se aplican criterios de entrada y salida.

Estos documentos se crean sobre la base de los documentos de diseño de desarrollo. Para ello los documentos de diseño del sistema son principalmente usados y los documentos de diseño conceptuales pueden ser también necesarios.

Los documentos de diseño describen la funcionalidad del software esté habilitado en la próxima versión. Para cada etapa del diseño de desarrollo, se debe crear una estrategia de prueba correspondiente para probar los nuevos conjuntos de características.

La estrategia de ensayo describe el nivel de prueba a realizar. Existen principalmente tres niveles de pruebas: las pruebas unitarias, pruebas de integración y pruebas del sistema.

En la mayoría de las organizaciones de desarrollo de software, los desarrolladores son los responsables de la pruebas unitarias.

Los Probadores individuales o equipos de ensayo han de ser los responsables de la integración y las pruebas del sistema. De este modo la responsabilidad de que desarrolla la prueba incluye la integración ya que es durante este proceso cuando más errores se detectan.

Las funciones y responsabilidades del encargado de la prueba, probadores individuales y director del proyecto deben estar claramente definidos a nivel de proyecto en esta sección. Puede no haber nombres asociados, pero el papel de cada uno tiene que estar muy claramente definido

Las Estrategias de ensayo deben ser revisados por los desarrolladores. También deben ser revisados por las puntas de prueba para todos los niveles de prueba para asegurarse de que la cobertura es completa aún no superposición.

Tanto el director de la prueba y los gerentes de desarrollo deben aprobar la estrategia de prueba antes de comenzar la prueba. Esto es bastante importante ya que las personas que realmente van a llevar a cabo las pruebas deben se ser las que estén de acuerdo en el protocolo que se va a seguir para probar el sistema.

Requisitos del entorno son también una parte importante de la estrategia de prueba. De debe describir en detalle qué sistemas operativos se utilizan para la prueba.

También informa claramente los niveles del Sistema operativo, parches necesarios y actualizaciones de seguridad necesarias.

Por ejemplo, un determinado plan de pruebas pueden requerir Windows XP Service Pack 3 para ser instalado como un requisito previo para la prueba.



Hay dos métodos utilizados en la ejecución de casos de prueba: manuales y automatizados. Dependiendo de la naturaleza de la prueba, es por lo general el caso de que una combinación de pruebas manual y automatizado es el mejor método de prueba.

Cualquier riesgo que afectarán el proceso de prueba deben ser listadas junto con una estrategia de mitigación. Al documentar un riesgo, su aparición se puede anticipar con bastante antelación.

Una acción proactiva se puede tomar para evitar que se produzca, o para mitigar sus daños. Riesgos típicos son la dependencia de la finalización de la codificación realizada por subcontratistas, o la capacidad de las herramientas de prueba.

Un plan de pruebas debe hacer una estimación de cuánto tiempo se tardará en completar la fase de prueba. Esta estimación es una de las mas importantes del proyecto ya que como se ha comentado, las pruebas requieren de bastante esfuerzo comparado don el esfuerzo total del proyecto. También hay que prever en esta estimación el número de cabios que será necesario añadir en el proyecto, ya que los errores que sean detectado en la fase de pruebas tendrán que ser corregidos en esta.

Hay muchos requisitos para completar las fases de prueba. En primer lugar, los probadores tienen que ejecutar todos los casos de prueba al menos una vez. Por otra parte, si se encuentra un defecto, los desarrolladores tendrán que solucionar el problema.

Los probadores luego deben volver a probar el caso de prueba fallido hasta que está funcionando correctamente. La prueba nunca se puede dar por válida hasta que todos los casos de prueba se hayan probado con un resultado correcto.

Por último, pero no menos importante, el probador necesita para llevar a cabo las pruebas de regresión hacia el final del ciclo para asegurarse de que los desarrolladores no se rompen accidentalmente partes del software, mientras que están resolviendo un problema de otra parte. Esto puede ocurrir en casos de prueba que funcionaban previamente correctamente.

El calendario de pruebas también debe documentar el número de probadores disponibles para pruebas. Si es posible, asignar casos de prueba para cada probador.

A menudo es difícil hacer una estimación precisa de la programación de la prueba ya que la fase de prueba involucra muchas incertidumbres.

Los planificadores deben tener en cuenta el tiempo adicional necesario para dar cabida a cuestiones contingentes.

Una forma de hacer esta aproximación es mirar el tiempo necesario para que las versiones anteriores del software.



Si el software es nuevo, multiplicando la aproximación calendario de pruebas inicial por dos es una buena manera de empezar.

Cuando se identifica un problema en particular, los programas se pueden depurar y la corrección se realiza para el programa. Para asegurarse de que la solución funciona, el programa se pondrá a prueba de nuevo por ese criterio.

Las Pruebas de regresión se asegurarán de que una solución no crea otros problemas en ese programa o en cualquier otra interfaz

Por lo tanto, un conjunto de casos de prueba relacionadas puede tener que repetirse de nuevo, para asegurarse de que nada más se ve afectada por una solución particular. Como hacer esto debe ser descrito en esta sección.

En algunas empresas, siempre que haya un cambio en una unidad, todos los casos de prueba sobre esa unidad se repetirán, para lograr un mayor nivel de calidad.

En la lista de requisitos, podemos identificar áreas relacionadas, cuya funcionalidad es similar. Estas áreas son los grupos de prueba.

Ejemplo

Por ejemplo, en un sistema de reservas de tren, todo lo relacionado con la reserva de entradas es un grupo funcional; todo lo relacionado con la generación de informes es un grupo funcional. De la misma forma, tenemos que identificar los grupos de prueba basados en el aspecto funcional.

Entre los casos de prueba, tenemos que establecer prioridades. Al probar los proyectos de software, ciertos casos de prueba serán tratados como los más importantes, y si no, el producto no puede ser liberado.

Algunos otros casos de prueba pueden ser tratadas como estéticos y si no se cumplen, se puede liberar el producto sin mucho compromiso en la funcionalidad. Este niveles de prioridad deben ser claramente establecidos. Estos pueden ser mapeados a los grupos de prueba también.

Cuando se ejecutan los casos de prueba, el líder de la prueba y el gerente de proyecto debe saber, dónde se encuentra el proyecto exactamente en términos de actividades de prueba.

Para saber dónde se encuentra el proyecto, las aportaciones de los probadores individuales deben llegar a la persona que lidera el proceso de prueba. Por esto es necesario que se escriban y se transmitan los informes adecuados cobre el desarrollo e las pruebas, como se verá más adelante en esta misma unidad.

Esto incluirá, qué prueba se ejecutan los casos, el tiempo que tomó, cuántos casos de prueba pasó, cuántas no, y cuántos no son aplicables. Además, la frecuencia con el proyecto recoge el estado debe ser claramente establecida.

Algunos proyectos tendrán como práctica el recoger el estado sobre una base diaria o semanalmente. Esto es lo ideal para que se pueda ir haciendo un seguimiento exhaustivo.



Cuando se ejecutan los casos de prueba, tenemos que hacer un seguimiento de los detalles de la ejecución, como cuando se ejecuta, quién lo hizo, cuánto tiempo le llevó, cuál es el resultado, etc.

Estos datos deben estar disponibles para el líder de prueba y el director del proyecto, junto con todos los miembros del equipo, en una ubicación conocida.

Esto puede ser almacenado en un directorio específico en un servidor central y el documento debe decir claramente sobre las direcciones y los directorios. La convención de nomenclatura para los documentos y los archivos también debe ser especificada.

Idealmente, el software debe satisfacer por completo el conjunto de requisitos. Desde el diseño, cada requisito debe abordarse en cada documento único en el proceso de software.

Los documentos incluyen el diseño a alto y bajo nivel, códigos fuente, los casos de prueba unidad, los casos de prueba de integración y los casos de prueba del sistema.

En una matriz de trazabilidad de requisitos, las filas tendrán los requisitos. Las columnas representan cada documento. La intersección de las celdas se marcan cuando un documento se dirige a un requisito particular con información relacionada con el requisito en el documento.

Lo ideal sería que, si todos los requisitos se aborda en cada documento único, todas las células individuales tienen identificadores de sección válidos o nombres introducidos. De esta manera sabemos que cada requisito se contempla. Si hay celdas que están vacías, esto representa que un requisito no se ha abordado correctamente.

La alta dirección normalmente le gustaría tener resumen de la prueba sobre una base semanal o mensual. Si el proyecto es muy crítico, es posible que lo necesitan incluso a diario.

En esta sección se debe abordar qué tipo de informes de resumen de prueba será producida por la alta dirección junto con la frecuencia.

La estrategia de la prueba deberá indicar una visión clara de lo que el equipo de pruebas va a hacer en todo el proyecto para toda la duración de este. Este documento puede ser presentado al cliente, si es necesario.

La persona, que prepara este documento, debe tener conocimientos amplios en el dominio del producto, y una muy buena experiencia, ya que es el documento que se va a manejar todo el equipo para las actividades de prueba.

La estrategia de prueba debe ser claramente explicado a los miembros del equipo de pruebas desde el principio del proyecto.



Un plan de pruebas es un documento que detalla un enfoque sistemático para probar un sistema, como una máquina o software . El plan contiene típicamente un conocimiento detallado del flujo de trabajo que tenga asociado este proceso.

Un plan de pruebas documenta la estrategia que se utilizará para verificar y asegurarse de que un producto o sistema cumple con las especificaciones de diseño y otros requisitos.

Un plan de prueba se prepara habitualmente por de los ingenieros de prueba.

Dependiendo del producto y la responsabilidad de la organización a la que se aplica el plan de pruebas, un plan de prueba puede incluir una estrategia para uno o más de los siguientes:

- Verificación del diseño o prueba de cumplimiento, que se realizarán durante las etapas de desarrollo o aprobación del producto, por lo general en una pequeña muestra de las unidades.

- Fabricación o prueba de producción que se deben realizar durante la preparación o montaje del producto de manera permanente a los efectos de la verificación del desempeño y control de calidad.

- La aceptación o el test Puesta en marcha - que se realizarán en el momento de la entrega o instalación del producto.

- Servicio **técnico** y pruebas tras reparación - a realizar según sea necesario durante la vida útil del producto.

- Prueba de Regresión, para llevar a cabo en un producto operacional existente, para verificar que la funcionalidad existente no se rompen cuando se modifican otros aspectos del entorno (por ejemplo, la actualización de la plataforma en la que se ejecuta una aplicación existente).

- Un sistema complejo puede tener un plan de pruebas de alto nivel para hacer frente a las necesidades generales y que sirva de apoyo y como criterio para abordar los detalles de diseño de los planes de subsistemas y componentes.



Los formatos de documentos del plan de prueba pueden ser tan variadas como los productos y organizaciones a las que se aplican.

Existen tres elementos principales que se deben describir en el plan de pruebas: La cobertura de pruebas, métodos de prueba, y las responsabilidades de prueba. Estos también se utilizan en una estrategia de prueba formal.

La cobertura de la prueba en el plan de pruebas establece qué requisitos serán verificados durante qué etapas de la vida del producto.

La cobertura de prueba se deriva de las especificaciones de diseño y otros requisitos, tales como las normas de seguridad o códigos de regulación, donde cada requisito o especificación del diseño idealmente tendrán uno o más medios de verificación correspondientes.

La cobertura de la prueba para las diferentes etapas de la vida del producto pueden superponerse, pero no necesariamente ser exactamente el mismo para todas las etapas.

Por ejemplo, algunos requisitos pueden ser verificados durante la prueba de verificación del diseño, pero no repitieron durante la prueba de aceptación. Esto es importante ya que se debe determinar en qué fase de las pruebas se verifican cada requisito por si en algún momento hay que trazar las pruebas realizadas.

Esta cobertura de la prueba también se alimenta de nuevo en el proceso de diseño, ya que el producto puede tener que ser diseñado para permitir que sea accesible a la prueba.

Los **métodos de ensayo** deben ser incluidos en el estado plan de pruebas y además tienen que indicar como se implementará cobertura de la prueba.

Los métodos de prueba pueden ser determinados por las normas, los organismos reguladores, o acuerdo contractual, o pueden tener que crear nuevos para el proyecto. Los métodos de ensayo también incluye los equipos de prueba a utilizar en la realización de las pruebas y establecer criterios de rechazo.

Los métodos de prueba utilizados para verificar los requisitos de diseño de hardware pueden ir desde medidas muy simples, como la inspección visual, para elaborar los procedimientos de prueba que se documentan por separado.

Las responsabilidades de prueba incluyen lo que las organizaciones llevarán a cabo los métodos de prueba y en cada etapa de la vida del producto. Esto permite a las organizaciones de prueba para planificar, adquirir o desarrollar equipos de prueba y otros recursos necesarios para poner en práctica los métodos de prueba de los que son responsables.

Las responsabilidades de las pruebas también incluye, lo que se recogerán los datos, y cómo se almacena y se informó de que los datos (a menudo denominados “entregas”).

Uno de los resultados de un plan de pruebas de éxito debe ser un registro o informe de la verificación de todas las especificaciones y requisitos acordados

por todas las partes del diseño. Este documento es fundamental ya que representa digamos que la certificación que el sistema cumple con los requisitos que se indicaron el diseño y la codificación.



El IEEE 829-2008, también conocida como el Estándar para la Documentación del software de pruebas, es un documento IEEE estándar que especifica la forma de un conjunto de documentos para su uso en etapas definidas de pruebas de software, cada etapa potencialmente producir su propio tipo separado del documento.

Este documento IEEE Establece una estructura determinada para la documentación de pruebas que satisface las características anteriormente mencionadas. Esta estructura es la siguiente:

- Código identificador de plan de pruebas.
- Introducción.
- Elementos de prueba.
- Características a ensayar.
- Características que no es necesario ensayar.
- Enfoque.
- Criterios de aceptación y rechazo.
- Criterios de suspensión y requisitos de reanudación.
- Entregables de prueba.
- Tareas de pruebas.
- Necesidades del entorno.
- Responsabilidades.

– Las necesidades de personal y de formación.

– Calendario.

– Riesgos y contingencias.

– Aprobaciones.

Los documentos de IEEE que se sugieren lo que debe estar contenido en un plan de pruebas son:

– 829-2008 IEEE estándar para software y prueba del sistema de documentación.

– 1008-1987 IEEE estándar para el software de pruebas unitarias.

– 1012-2004 IEEE estándar para el software de verificación y validación.

6.2.5. Ejecución de pruebas

Un caso de prueba, en la ingeniería de software, es un conjunto de condiciones o variables bajo las cuales un probador determinar si una aplicación, sistema de software o una de sus características está funcionando como se estableció originalmente para que lo haga.

El mecanismo para determinar si un programa de software o sistema ha pasado o ha fallado una prueba de este tipo se conoce como un oráculo de pruebas.

En algunas configuraciones, un oráculo podría ser un requisito o caso de uso, mientras que en otros podría ser una heurística.

Pueden ser necesarios preparar muchos casos de prueba para determinar que un programa de software o el sistema se considera suficientemente examinadas para ser liberado.

Los casos de prueba se refieren a menudo como scripts de prueba, sobre todo cuando se escribe n conjuntos de pruebas.



Casos de prueba formales: con el fin de probar plenamente que se cumplan todos los requisitos de una aplicación, debe haber por lo menos dos casos de prueba para cada requisito: una prueba positiva y una prueba negativa.

Si un requisito tiene sub-requisitos, cada sub-requisito debe tener al menos dos casos de prueba. Hacer un seguimiento de la relación entre la exigencia y la prueba se realiza con frecuencia usando una matriz de trazabilidad .

Los casos de prueba por escrito deben incluir una descripción de la funcionalidad que va a ser probada, y la preparación necesaria para asegurar que la prueba se puede realizar.

Un caso de prueba formal por escrito se caracteriza por una entrada conocida y por una salida esperada, que se trabajó a cabo antes de ejecutar la prueba. La entrada conocida debe probar una precondición y la salida esperada debe probar una condición posterior.



Casos de prueba informales: para aplicaciones o sistemas sin requisitos formales, los casos de prueba pueden escribirse en base a la operación normal aceptada de los programas de un tipo similar.

En algunas técnicas de pruebas, los casos de prueba no se escriben en absoluto, pero las actividades y los resultados se informan después han llevado a cabo las pruebas.

En las pruebas de escenario, se utilizan historias hipotéticas para ayudar el probador pensar a través de un problema o sistema complejo.

Estos escenarios por lo general no están escritas en ningún detalle. Pueden ser tan simples como un diagrama para un entorno de prueba o podrían ser una descripción escrita en prosa.

La prueba de escenario ideal es una historia que está motivada, es creíble, compleja, y fácil de evaluar. Por lo general son diferentes de los casos de prueba en el que los casos de prueba son pasos individuales, mientras que los escenarios abarcan una serie de medidas de la llave.

Un caso de prueba es por lo general un solo paso, o a veces consta de una secuencia de pasos, para probar el comportamiento y la correcta funcionalidad así como las características de una aplicación.

Un resultado esperado o se espera que pase por lo general se tiene que especificar siempre.

Información adicional que se puede incluir en un caso de prueba podía ser la detallada en el siguiente esquema:

– Prueba de identificación de caso.

– Descripción del caso de prueba.

– Paso de prueba o número orden de ejecución.

– Requisito o requisitos relacionados.

– Profundidad de la prueba.

– Categoría de prueba.

– Autor.

– Casillas de verificación para especificar si la prueba puede ser o ha sido automatizado.

– Una casilla para indicar si se pasa o falla el caso de prueba.

– Observaciones.

Casos de prueba más elaborador también pueden contener los estados de requisitos previos o pasos, y descripciones.



Un caso de prueba por escrito debe contener también un lugar para expresar el resultado real de la prueba.

Estos pasos se pueden almacenar en un documento de procesador de texto, hoja de cálculo, base de datos u otro repositorio común.

En un sistema de base de datos, también puede ser capaz de ver los resultados de pruebas anteriores y que genera los resultados y la configuración del sistema utilizado para generar esos resultados.

Estos resultados pasados generalmente se almacenan en una tabla separada.

Conjuntos de pruebas a menudo también contienen un Resumen de la prueba y la configuración detallada de la prueba.



Además de una descripción de la funcionalidad a ser probada, y la preparación necesaria para asegurar que la prueba se puede realizar, la parte que más tiempo lleva en la elaboración de un caso de prueba es la creación de las pruebas en sí y su modificación cuando se realiza algún cambio en el sistema.

En circunstancias especiales, podría haber una necesidad de ejecutar la prueba, producir resultados, y luego un equipo de expertos evaluaría si los resultados se pueden considerar como un pase.

Esto sucede a menudo en la determinación del número de prestaciones de nuevos productos. La primera prueba se toma como línea de base para los ciclos de ensayo de liberación / producto posteriores.

Las p**ruebas de aceptación** son las que utilizan una variación de un caso de prueba por escrito, se realizan comúnmente por un grupo de usuarios finales o clientes del sistema para asegurar que el sistema desarrollado cumple con los requisitos especificados o el contrato.

Las pruebas de aceptación del usuario se diferencian por la inclusión de un camino "fácil" o casos positivos de la prueba y la exclusión casi total de casos de pruebas negativos.

6.2.6. Generación de informes de pruebas

Como ya se ha comentado, las pruebas son una fase extremadamente importante en el ciclo de vida de desarrollo de software.

Por tanto, el trabajo de los probadores y del personal relacionado en el entorno de prueba, y en general todo lo que los miembros de la organización están haciendo en esta fase debe estar debidamente documentado.



Una tarea del personal que se dedica a la prueba ha de ser proporcionar la información necesaria para la organización en los informes adecuados.

En la siguiente tabla algunos documentos importantes para la fase de pruebas, su frecuencia y el contenido y motivo por el que se generan esos documentos se detalla:

Documentación de prueba	Frecuencia	Contenido
Alto Nivel de plan de pruebas	Cada proyecto	Un plan de prueba se produce para cada proyecto. Define qué funciones se pondrán a prueba y qué funciones no se pondrá a prueba. A este nivel de detalle se documentará los riesgos y contingencias, así como indicando las hipótesis y definir los recursos necesarios.
Especificación de pruebas	Cada etapa	Este documento detalla las condiciones de prueba y casos de prueba para cada etapa de la prueba. Esto será utilizado por los testers en el día a día de sus pruebas.
Registros de prueba	Cada etapa	Registros de los informesque debe ser producido por cada probador. Esto permitirá que el progreso sea monitorizado y controlado. También proporcionará una auditoría de pruebas adecuado al final del proyecto.
Informe de Progreso de las pruebas	Semanal	Este informe proporcionará a los administradores con un avance semanal en la prueba se lleva a cabo. Se utilizarán curvas “S” para mostrar el progreso de ensayo, así como los porcentjes entre fallos encontrados y fallos solucionados.
Resumen del informe de prueba	Cuando se requiera	Se producirá Este informe resumido cuando sea solicitado por la Dirección en cualquier etapa del ciclo de vida de desarrollo. Esta será una versión resumida del Informe Progreso de la prueba dirigida específicamente a la Alta Dirección.
Mensaje Informe del Proyecto	Cada proyecto	Al final de cada proyecto se produce un “reunión de Revisión de Proyectos” . Esta actividad incluirá un análisis de lo que salió bien y lo que no va bien en términos del proyecto. Un informe será producido detallando los cambios para la mejora de procesos.

Algunas o todas de las siguientes tablas se deben utilizar para monitorear el progreso.

Se recomienda que los gráficos se muestran claramente en algún lugar o documento público para que todo el mundo está al tanto de la situación actual.

La utilización de gráficas, en lugar de las tablas, también se recomiendan, ya que son más fáciles de leer y tienen más probabilidades de atraer la atención y ser tomado en consideración.

– Fallos encontrados: Esta gráfica simplemente registra el número de fallos encontrados durante cada día del calendario de pruebas.

  La gravedad de los fallos se podría indicar mediante el uso de diferentes columnas en la tabla.

– Fallos encontrados frente a fallos resueltos:

  Mostrar y detallar el número de fallos encontrados junto con el número de fallos solucionados es una manera útil de detectar posibles problemas de programación de forma temprana.

  Por ejemplo si se encuentra que hay un brecha creciente entre el número de fallos encontrados y el número solucionado sugiere que se necesita más esfuerzo de desarrollo para solucionar varios de los fallos pendientes.

  Dejar la solución de estos hasta el último momento, invariablemente termina en desastre (por ejemplo proyectos que acaban en liberación atrasada, sistema de mala calidad, o ambos).

– Pruebas ejecutadas y Pruebas superadas frente a lo Planificado: al inicio de un esfuerzo de la prueba la cantidad de pruebas que nos proponemos ejecutar (definido por el alcance) es posible planificar el número de pruebas que se deben completar cada día del período de pruebas.

  Esto normalmente tiene el aspecto de una curva en S, (esto es un polinomio de tercer orden. Se puede usar esta aproximación matemática para predecir con cierta fiabilidad como se van a comportar el proceso de pruebas en el futuro.

  Si se desvían en exceso el trazado tanto el número de pruebas en realidad se ejecutan y el número de pruebas que han planificado se podrá pondrá de relieve rápidamente cualquiera de una serie de problemas.

Por ejemplo, si el número de pruebas realizadas cae por debajo del número previsto a continuación, se necesitarán más bien probadores o más tiempo. De esta manera es una herramienta muy útil para los gestores porra saber como deben de ir distribuyendo los recursos disponibles en la organización que está desarrollando el software.

Si el número de pruebas pasadas cae muy por debajo del número de ejecuciones (es decir, si un gran número de pruebas fallan), entonces más fallos de los esperados se están encontrando y esto va a tener un impacto tanto el equipo de pruebas como en los desarrolladores.

– Fallos por testeador: Es importante saber cuántos fallos se está trabajando en un momento dado y por quién. Esta tabla muestra si hay un equilibrio de los fallos que se le asigne al equipo.

  Cada "columna" en el gráfico de barras puede mostrar prioridad o la gravedad para su posterior análisis.

– Número de defectos para cada ciclo: Este gráfico muestra el número de errores, san estos bugs, bugs nuevos, bugs asignados, bugs abiertos, errores corregidos, Bugs de no-conformidad, bugs cerrados, o errores diferidos, en cada ciclo para cada lanzamiento.

  El estado de los errores puede ser diferente de una compañía a otra. Así que según la empresa los probadores tienen que cambiar el estado en el Gráfico.

– Número de diferentes errores de gravedad en cada ciclo: Este gráfico muestra el número de errores de diferente gravedad (bugs críticos, errores importantes, errores menores, Errores cosméticos) en cada ciclo para cada lanzamiento.

  Esta severidad también todos y cada organización mantiene sus propios estándares de lo que puede ser diferente de una compañía a otra. Sobre la base de sus propias normas que tienen que cambiar el gráfico.

– Número de errores de diferentes gravedad en cada versión: Este gráfico muestra el número de errores de diferente gravedad (bugs críticos, importantes errores, bugs menores, insectos cosméticos) en cada versión.

  Esta severidad también todos y cada organización mantiene sus propios estándares de lo que puede ser diferente de una compañía a otra. Sobre la base de sus propias normas que tienen que cambiar el gráfico.

JUnit es uno de los muchos marcos unitarios,que han sido utilizados inicialmente por muchas aplicaciones Java como marco de prueba Unidad, y en la actualidad es de hecho un estándar para probar programas escritos en Java.

Por defecto, las pruebas JUnit generan unos simples archivos XML a modo de su informe de ejecución de la prueba.

Estos archivos XML se pueden utilizar para generar los informes personalizados si así lo exige la política de pruebas de la organización en la que se está trabajando.

Ejemplo

También se pueden generar informes HTML usando los archivos XML. ANt tiene una tarea de mucha utilidad que toma estos archivos JUnit XML como entrada y genera un informe HTML a partir de de ella.

TestNG es el programa que gpor defecto genera los informes JUnit XML para cualquier ejecución de la prueba (en la carpeta de test-output).

Podemos usar estos archivos de informe XML como entrada para la generación de un informe JUnit HTML. Veamos un ejemplo a continuación:

EN primer lugar se va a crear una clase que nos sirva como caso de pruebas.

Para ello creamos la siguente clase java denominada SampleTest.java en el directorio TestNG_WORKSPACE

```
importorg.testng.Assert;

importorg.testng.annotations.Test;

publicclassSampleTest {

 @Test

publicvoidtestMethodOne(){

Assert.assertTrue(true);

 }

 @Test

publicvoidtestMethodTwo(){
```

```
      Assert.assertTrue(false);
 }
 @Test(dependsOnMethods={“testMethodTwo”})
publicvoidtestMethodThree(){
Assert.assertTrue(true);
 }
}
```

La clase de prueba anterior contiene tres métodos de prueba de los cuales testMethodOne y testMethodThree pasarán las pruebas cuando se ejecuta.

Sin embargo que testMethodTwo se hace fallar premeditadamente al pasar un valor booleano false para el método Assert.assertTrue, que se utiliza para probar las condiciones de verdad en los casos de prueba.

Posteriormente es necesario crear el fichero testng.xml. Para ello editar un testng.xml en el directorio TestNG_WORKSPACE que nos permitirá ejecutar los casos de prueba.

```
<?xmlversion=”1.0” encoding=”UTF-8”?>
<suite name=”Simple Suite”>
<test name=”Simple test”>
<classes>
<classname=”SampleTest” />
</classes>
</test>
</suite>
```

Ejemplo

Hay que compilar la clase Sampletest usando javac. Para ello se introduce el siguiente comando

```
javac SampleTest.java
```

Posteriormente se ejecuta el testng.xml con la siguente instrucción.

```
java -cp “TestNG_WORKSPACE” org.testng.TestNG testng.xml
```

Ahora que yahemos obtenido los informes JUnit XML resultado de la ejecución anterior, vamos a crear un archivo XML de configuración de construcción Ant sencillo para generar un informe HTML para la ejecución de la prueba.

Crear un nuevo archivo llamado build.xml en el directorioTestNG_WORKSPACE. Con el siguente contenido:

```
<projectname=”TestNG_WORKSPACE” default=”junit-report” basedir=”.”>

<propertyname=”junit-xml-dir” value=”${basedir}/test-output/junitreports”/>

<propertyname=”report-dir” value=”${basedir}/html-report” />

<target name=”junit-report”>

<deletedir=”${report-dir}” failonerror=”false”/>

<mkdirdir=”${report-dir}” />

<mkdirdir=”${report-dir}/Junit” />

<junitreporttodir=”${report-dir}/Junit”>

<filesetdir=”${junit-xml-dir}”>

<includename=”**/*.xml” />

</fileset>

<reportformat=”noframes” todir=”${report-dir}/Junit” />

</junitreport>

</target>

</project>
```

El XML anterior define un archivo build.xml de Ant sencillo que tiene un objectAnt llamado junit-report específico que genera un informe JUnit cuando se ejecuta.

Este epígrafe busca los archivos XML de informe JUnit en el directorio indicado, junitreports.

Para el archivo de configuración Ant el destino predeterminado para ejecutar se configura como junit-report.

Abra la ventana del símbolo del sistema en la carpeta directorio TestNG_WORKSPACE en el símbolo del sistema y

Hay que ejecutar el siguentecomando:

```
C:\TestNG_WORKSPACE>ant
```

Una vez ejecutado, un informe JUnit HTML se generará en el directorio / html-report / Junit configurado.

Para ver el resultado es necesario abrir el archivo con el nombreunit-noframes.html con el navegador web por defecto se puede ver el informe de pruebas.

Hasta ahora se ha indicado **cómo utilizar el informe** JUnit XML generado por TestNG y generar informes HTML utilizando Ant.

Hay dos tipos de informes que se pueden generar con este método: con marcos y sin marcos. Si la generación de informes se configura con los marcos, habrá varios archivos generados para cada clase y el informe principal se conectará a ellos a través de enlaces.

Un informe sin marcos marcos consiste en un único archivo con todos los resultados de la ejecución de la prueba. Esto se puede configurar, proporcionando el valor correspondiente al atributo de formato de la tarea de informe en Ant.

UD6
Lo más importante

- Las pruebas del software son un elemento crítico para la garantía de calidad del software y representa una revisión final de las especificaciones, del diseño y de la codificación.

- La prueba de unidad comienza en el vértice de la espiral y se centra en cada unidad del software, tal como está implementada en código fuente.

- Los resguardos sirven para reemplazar módulos que están subordinados, es decir, son llamados, al componente que hay que probar.

- La validación se consigue cuando el software funciona de acuerdo con las expectativas razonables del cliente.

- La prueba beta se lleva a cabo por los usuarios finales del software en los lugares de trabajo de los clientes.

- La prueba del sistema, realmente, está constituida por una serie de pruebas diferentes cuyo propósito primordial es ejercitar profundamente el sistema basado en computadora.

- El flujo lógico de un programa a veces no es nada intuitivo, lo que significa que las suposiciones intuitivas sobre el flujo de control y los datos nos pueden llevar a tener errores de diseño que sólo se descubren con las pruebas de caja blanca.

– En términos del grafo de flujo, un camino independiente está constituido por lo menos por una arista que no haya sido recorrida anteriormente a la definición del camino.

– La prueba de condición es un método de diseño de casos de prueba que ejercita las condiciones lógicas contenidas en el módulo de un programa.

– Cuando los bucles no son independientes, se recomienda usar el enfoque aplicado para los bucles anidados.

– Un Breakpoint es un lugar en el programa en el que la ejecución se detiene.

– Las pruebas del software son un conjunto de técnicas utilizadas para verificar que un programa lleva a cabo su tarea correctamente.

– La partición equivalente es un método de prueba de caja negra que divide el campo de entrada de un programa en clases de datos de los que se pueden derivar casos de prueba.

– Un script de prueba en las pruebas de software es un conjunto de instrucciones que se van a ejecutar en el sistema sometido a prueba para comprobar que el sistema funciona como se esperaba.

– Las estrategias de prueba describen cómo se mitigan los riesgos de los productos de las partes interesadas a nivel de prueba, que se van a realizar tipos de prueba, y se aplican criterios de entrada y salida.

– Las Pruebas de regresión se asegurarán de que una solución no crea otros problemas en ese programa o en cualquier otra interfaz.

– La utilización de gráficas, en lugar de las tablas, también se recomiendan, ya que son más fáciles de leer y tienen más probabilidades de atraer la atención y ser tomado en consideración.

UD6
Autoevaluación

1. Sobre las pruebas de integración:

    a. Sólo se usan pruebas de caja blanca.
    b. Un resguardo simula un subprograma subordinado.
    c. Un controlador simula un subprograma subordinado.
    d. Sólo se usan pruebas de caja negra.

2. Sobre las pruebas de validación:

    a. Sirven para detectar errores de compilación.
    b. La prueba beta se lleva a cabo por clientes en el lugar de desarrollo.
    c. La prueba alfa se lleva a cabo por un cliente en el lugar de desarrollo.
    d. Se prueban exhaustivamente cada clase antes de integrarla.

3. Sobre la prueba del camino básico:

    a. Utiliza la complejidad ciclomática.
    b. Es un método de caja negra.
    c. Se han de ejecutar cada ruta del programa dos veces.
    d. Se han de probar la mitad de los nodos predicado al menos.

4. Sobre las pruebas de bucles:

    a. Se prueba el limite superior del bucle.
    b. Los bucles simples se prueban una vez.
    c. Con los bucles anidados se usa una estrategia diferente que con los bucles simples.
    d. Se prueba el limite inferior del bucle.

5. La depuración de programas es el proceso por el que:

    a. Se identifican y corrigen errores de programación en dicho programa.
    b. Se identifican fallos en dicho programa.
    c. Se identifican fallos de compilación en dicho programa.
    d. Ninguna respuesta es válida.

6. Si se tienen varias pruebas JUnit:

    a. Se pueden combinar en un conjunto de pruebas.
    b. Siempre han de ejecutarse manualmente una a una.
    c. JUNIT no permite más de una prueba a la vez.
    d. Siempre se ejecutan en el mismo orden.

7. En la ejecución de un caso de pruebas:

    a. Nunca se deben de automatizar su ejecución.
    b. Hay dos métodos, manual y automatizado.
    c. Se puede obviar el formato de JUnit.
    d. Se tienen que indicar si es una prueba de integración.

8. En la ejecución de un caso de pruebas:

    a. Nunca se deben de automatizar su ejecución.
    b. Hay dos métodos, manual y automatizado.
    c. Se puede obviar el formato de JUnit.
    d. Se tienen que indicar si es una prueba de integración.

9. Idealmente el software:

    a. Debe pasar al menos el 50% de las pruebas.
    b. Debe de satisfacer la mitad de los requisitos.
    c. Debe satisfacer todos los requisitos.
    d. Debe de liberarse cuanto antes para que el usuario detecte los errores lo antes posible.

10. El estándar IEEE 829-2008:

    a. Especifica como realizar las pruebas de software.
    b. Especifica como realizar pruebas de caja blanca.
    c. Especifica como realizar pruebas de caja negra.
    d. Especifica como debe ser la documentación de pruebas.

Área: informática y comunicaciones

UD7

Excepciones

7.1. Definición. Fuente de excepciones. Tratamiento de excepciones. Prevención de fallos. Excepciones definidas y lanzadas por el programador

7.2. Uso de las excepciones tratadas como objetos

7.1. Definición. Fuente de excepciones. Tratamiento de excepciones. Prevención de fallos. Excepciones definidas y lanzadas por el programador

Hay un gran número ocasiones en las que programas que aparentemente son muy buenos, por su interfaz, facilidad de uso, etc., pero comienzan a generar errores en tiempo de ejecución que hacen que el usuario desconfíe de ellos en el día a día.

Para evitar estas situaciones, Hay que tratar de fomentar la cultura de la detección, control y solución de errores a través de las herramientas que Java ofrece a los programadores.

A lo largo de l periodo de aprendizaje en un lenguaje de programación nos todo el mundo en alguna ocasión ha encontrado errores, pero éstos suelen ser los que ha indicado el compilador.

En general un punto y coma por aquí, un nombre de variable incorrecto por allá, pueden hacer que el compilador avise de que se ha cometido un descuido. Cuando se detectan, se corrigen y se obtiene una clase compilada correctamente.

Pero no sólo existen este tipo de errores ya que pueden existir errores no sintácticos en los programas que se desarrollan

Un programa perfectamente compilado en el que no existen errores de sintaxis, puede generar otros tipos de errores que puede que aparezcan en tiempo de ejecución. A estos errores se les conoce como excepciones.

A continuación se verá cómo gestionar de manera adecuada estas excepciones y la oportunidad que presenta el utilizar el potente sistema de manejo de errores que Java incorpora. La potencia de este sistema de manejo de errores radica en que:

El código que se encarga de manejar los errores, esta perfectamente identificado en los programas. Este código va a estar separado del código que maneja la aplicación.

Java tiene una gran cantidad de errores estándar los cuales van a estar asociados a multitud de fallos comunes, como por ejemplo divisiones por cero, fallos de entrada de datos, etc.

Al tener muchas excepciones localizadas, se va a poder gestionar de manera específica cada uno de los errores que se produzcan.

En Java se pueden detallar los fragmentos de código que pueden provocar errores de ejecución para si se produce una excepción esta se detecte correctamente.

En este caso el flujo del programa es lanzado (throw) hacia ciertas zonas o subrutinas que han sido creadas previamente por el programador y cuya finalidad va a ser el tratamiento adecuado de dichas excepciones. Hay que tener en cuenta que si no se captura una excepción, el programa se va a detener con toda probabilidad.



Definición

Una **excepción** es un evento que se produce durante la ejecución de un programa, que interrumpe el flujo normal de las instrucciones del programa. El término excepción es la abreviatura de la frase “caso excepcional”.

Cuando se produce un error en el interior de un método, el método crea un objeto y lo entrega al sistema de tiempo de ejecución. Este objeto, denominado un objeto de excepción, va a contener información sobre el error, incluyendo su tipo y el estado del programa cuando se produjo el error.

EL hecho de que se produzca la creación de un objeto de excepción y entregarla al sistema de tiempo de ejecución se llama lanzar una excepción.

En Java, las excepciones están representadas por clases y sus clases hijas. El paquete java.lang.Exception y sus subpaquetes contienen todos los tipos de excepciones que se pueden general en un programa, auqne un programador avanzado puede también crear sus propias clases de excepciones.

Todas las excepciones derivarán de la clase Throwable, de la que existen clases más específicas. Por debajo de la clase Throwable existen las clases Error y Exception, que definen los errores de la **máquina virtual de Java y lo que son las excepciones propiamente dichas.**

Error es una clase que se va a encargar de los errores que se produzcan en la máquina virtual de java, no en los programas de usuario que generalmente se programan. Un programador de aplicaciones no tienen generalmente que preocuparse por los errores derivados de "error" ya que poco puede hacer para evitarlos. Y por tanto la clase Exception será la que a un programador le interese conocer, pues gestiona los errores provocados en los programas.



Java lanzará una excepción en respuesta a una situación poco usual o de fallo. Cuando se produce un error se genera un objeto asociado a dicha excepción.

Este objeto es de la clase Exception o de alguna de sus clases hijas. Este objeto se pasa al código que se ha definido para manejar la excepción. Este código puede manipular las propiedades del objeto Exception que se ha generado.

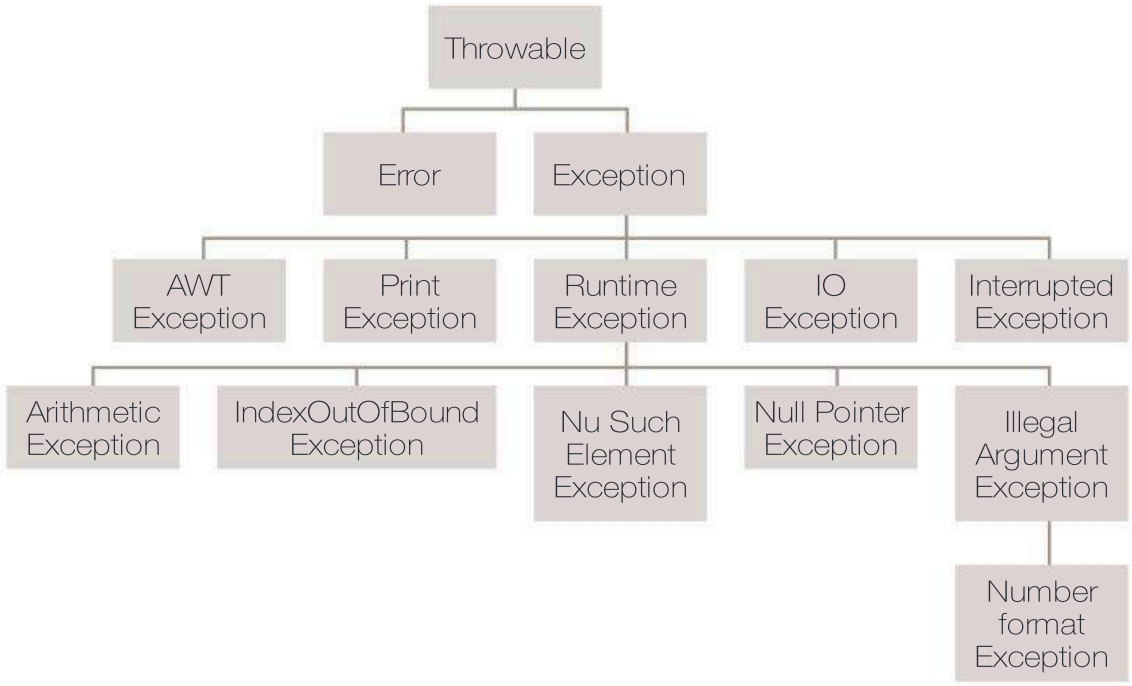
El programador también puede lanzar sus propias excepciones que el haya creado. Las excepciones en Java serán objetos de clases derivadas de la clase Exception que actúa como clase base.

Existe toda una gran jerarquía de clases heredadas de la clase base Exception. Estas clases derivadas se ubican en dos conjuntos principales:

- Las excepciones en tiempo de ejecución, que ocurren cuando el programador no ha tenido cuidado al escribir su código. Estas excepciones se suelen englobar dentro de la subclase "RuntimeException".

- Las excepciones que indican que ha sucedido algo inesperado o fuera de control del programador. Para ellas existen el resto de clases que heredan de la clase excpetion, excepto la ya anteriormente mencionada "RuntimeException.

En la siguiente imagen se ofrece una aproximación a la jerarquía de las excepciones en Java:



Importante

Después de un método lanza una excepción, el sistema de ejecución intenta encontrar "algo" para manejar la excepción.

El conjunto de posibles "algo" para controlar una excepción es la lista ordenada de los métodos que habían sido llamados para llegar al método en el que se produjo el error. A esta lista de métodos se conoce como la pila de llamadas.

El sistema de ejecución busca en la pila de llamadas de un método que contiene un bloque de código que pueda manejar adecuadamente la excepción.

Este bloque de código se denomina un controlador de excepciones. La búsqueda va a comenzar con el método en el que se produjo el error y Java procede a través de la pila de llamadas en el orden inverso en el que fueron llamados todos los métodos.

Cuando se encuentra un controlador adecuado para la excepción, el sistema pasa la excepción al manejador de excepciones. Un controlador de excepciones se va a considerar adecuado si el tipo del objeto de excepción lanzada coincide con el tipo que puede ser manejado por el manejador.

El manejador de excepción que se escoge se dice que lo que va a hacer es capturar la excepción.



Si el sistema de ejecución busca detalladamente todos los métodos en la pila de llamadas sin encontrar un manejador de excepción apropiado el sistema de ejecución y, por tanto, el programa, termina.

El uso de excepciones para manejar errores tiene algunas ventajas sobre las técnicas tradicionales de manejo de error. El **código válido** en lenguaje de programación Java ha de respetar la captura de excepciones o bien especificar los manejadores de excepciones.

Esto significa que el código que podría lanzar ciertas excepciones deben ser encerrado por cualquiera de las siguientes estructuras de control:

– Un declaración **try** las cual captura la excepción. La try debe proporcionar un manejador para la excepción, utilizando la sentencia catch.

– Un método que especifique que se puede lanzar la excepción.

En este caso el método debe proporcionar una cláusula throws con la lista de excepciones que lanza.

No todas las excepciones están sujetas a la captura o especificar los manejadores.

Para entender por qué, tenemos que mirar a las tres categorías básicas de excepciones, sólo uno de los cuales está sujeta al requisito.

Las tres clases de excepciones que hay en Java son excepciones comprobadas, errores y excepciones en tiempo de ejecución.

– Una **excepción comprobada** es un error que se produce en e programa durante su ejecución y que el programador debe verificar si se produce, y que hacer en caso de que se produzca.

  Ejemplo

  Por ejemplo, supongamos que una aplicación solicita al usuario un nombre de archivo de entrada, a continuación, abre el archivo pasando el nombre al constructor para java.io.FileReader.

  Normalmente, el usuario proporciona el nombre de un, archivo legible existente, por lo que la construcción de la FileReader objeto tiene éxito, y la ejecución de la aplicación continúa normalmente.

  Pero a veces el usuario proporciona el nombre de un archivo que no existe, y el constructor lanza java.io.FileNotFoundException .

  Un programa bien escrito detectar esta excepción y notificar al usuario el error, posiblemente solicite un nombre de archivo corregido.

  Excepciones comprobadas van a estar sujetas a la captura o a una especificación de los manejadores. Todas las excepciones son excepciones comprobadas, a salvo las que son indicadas por Error, RuntimeException, y sus subclases.

– El segundo tipo de excepción es el error. Estas son condiciones excepcionales que son externas a la aplicación, y que la aplicación por lo general no pueden anticipar o recuperarse.

  Por ejemplo, imaginemos que una aplicación abre correctamente un archivo de entrada, pero no es capaz de leer el archivo debido a un mal funcionamiento del hardware o del sistema.

  La lectura fallida de un archivo lanzará java.io.IOError. Una aplicación puede optar por capturar esta excepción, con el objetivo de notificar al usuario del problema,pero también podría tener en el programa que parar, imprimir un seguimiento de pila y salir.

  Los errores no están sujetos a la captura o especificar los manejadores. Los errores son las excepciones indicadas por Error y sus subclases.

– El tercer tipo de excepción es la excepción en tiempo de ejecución: estas son condiciones excepcionales que son internos al programa, y que la aplicación por lo general no pueden anticipar o recuperarse.

  Estos por lo general suelen indicar errores de programación, tales como errores lógicos o el uso incorrecto de una API.

Ejemplo

Por ejemplo, considere el uso descrito anteriormente que pasa un nombre de archivo al constructor para FileReader .

Si un error de lógica provoca que un a referencia a nullsea pasado al constructor, el constructor lanzará NullPointerException .

La aplicación puede detectar esta excepción, pero probablemente tiene más sentido para eliminar el error que causó la excepción que se produzca.

Las excepciones de tiempo de ejecución no están sujetas a la captura o a especificar los manejadores.

Excepciones en tiempo de ejecución son los indicados por RuntimeException y sus subclases.

Errores y excepciones de tiempo de ejecución se conocen colectivamente como excepciones sin control.

Algunos programadores, para su comodidad, en lugar de capturar o lanzar la excepción y majearla como es debido, la re-lanzan como excepción no comprobada.



Excepción mostrada en entorno Netbeans

Esto es mucho más cómodo a la hora de programar, pero no es en absoluto recomendable ya que impide al código llamador que pueda capturar la excepción.

A continuación se describe cómo utilizar los componentes de tres manejadores de excepciones- los bloques - try, catch, y finally para escribir un controlador de excepciones.

El siguiente ejemplo define e implementa una clase llamada ListOfNumbers. Esta clase va a servir para guardar una lista de números enteros.

Cuando esté construida, ListOfNumberscrea un objeto de la calse ArrayList que va a almacenardiezelementos enteros con valores secuenciales del 0 al 9.

La claseListOfNumbers también define un método denominado writeList, que escribe la lista de números en un archivo de texto llamadoOutFile.txt . Este ejemplo utiliza clases definidas en java.io, que cubren sus necesidades de E/S básicas, esto es, el hecho de guardar en fichero y poder imprimir datos por la consola.

Ejemplo

```
 // Nota: Esta clase no compilará por su diseño!
 import java.io. *;
 java.util.List import;
 java.util.ArrayList import;
 ListOfNumbers public class {
List privado <Integer> lista;
 private static int TAMAnO final = 10;
 ListOfNumbers public () {
 list = new ArrayList <Integer> (SIZE);
 for (int i = 0; i <TAMAnO; i ++) {
 list.add (new Integer (i));
 }
 }
 pública writeList void () {
 PrintWriter fuera = new PrintWriter (new FileWriter (“Out-
File.txt”));
 for (int i = 0; i <TAMAÑO; i ++) {
 out.println (“Value at:” + i + “=” + list.get (i));
 }
 out.close ();
 }
 }
```

La primera línea de código es una llamada a un constructor.El constructor inicializa un flujo de salida en un archivo. Si el archivo no se puede abrir, el constructor lanza una IOException .

La segunda línea en negrita es un llamado a laArrayList de clase es un método get, que lanza una IndexOutOfBoundsException si el valor de su argumento es demasiado pequeño (menos de 0) o demasiado grande (más que el número de elementos actualmente contenida por la clase ArrayList).

Si se intenta compilar la clase, el compilador muestra un mensaje de error sobre la excepción lanzada por el constructor FileWriter .

Sin embargo, no se muestra un mensaje de error sobre la excepción lanzada por get .

La razón es que la excepción lanzada por el constructor, IOException, es una excepción comprobada, y la lanzada por el getmétodo, IndexOutOfBoundsException, es una excepción no comprobada.

Ahora que hemos visto el funcionamiento de la clase ListOfNumbers **y donde** pueden ser lanzadas las excepciones dentro de ella, se va a proceder a escribirlos manejadores de excepciones para capturar y manejar esas excepciones



Los controladores de excepciones están asociados con un bloque try, el cual es manejado por uno o más bloques **catch** que deben aparecer directamente después del try de bloque.

Ningún código puede estar entre el final de la try de bloque y el comienzo de la primera catch bloque.

```
try {
 } Catch (ExceptionType nombre) {
 } Catch (ExceptionType nombre) {
 }
```

Cada bloque catch es un manejador de excepciones y maneja el tipo de excepción indicada por su argumento.

El tipo de argumento, ExceptionType, declara el tipo de excepción que el manejador puede manipular y debe ser el nombre de una clase que hereda de la clase Throwable. El manejador puede referirse al objeto que guarda la excepción con el nombre que se indique.



El bloque catch contiene código que se ejecuta siempre y cuando se invoca el controlador de excepciones.

El sistema de ejecución invoca el manejador de excepciones cuando el manejador es el primero en la pila de llamadas cuyaExceptionType coincide con el tipo de la excepción lanzada.

El sistema considera que coincide si el objeto lanzado puede ser correctamente asignado al argumento del manejador de excepciones.

Ejemplo

Los siguientes son dos manejadores de excepciones para el método anterior para cada uno de los dos tipos de excepciones comprobadas que pueden lanzarse dentro de la declaración try :

```
try {

 } Catch (FileNotFoundException e) {

 System.err.println («FileNotFoundException:» + e.getMessage
());

 throw new SampleException (e);

 } Catch (IOException e) {

  System.err.println («Caught IOException:» + e.getMessage
());

 }
```

Ambos manejadores se encargan de imprimir un mensaje de error por la consola. El segundo controlador no hace nada más. Al detectar cualquier IOException que no es capturada por el primer controlador, permite que el programa continúe con la ejecución.

El primer manejador, además de la impresión de un mensaje, emite una excepción definida por el usuario.

En este ejemplo, cuando el FileNotFoundException es capturado, provoca que una excepción definida por el usuario llamada SampleException sea lanzada.

Es posible que el programador hacer esto si quiere que su programa puede manejar una excepción en este caso de una manera más específica. Los controladores de excepciones pueden hacer algo más aparte de imprimir mensajes de error o detener el programa.



Los manejadores pueden hacer la recuperación de errores, pedir al usuario que tome una decisión, o propagar el error hasta un controlador de nivel superior utilizando excepciones encadenadas.

La captura de más de un tipo de excepción con un controlador de excepciones es posible en Java SE 7 y versiones posteriores, una solo bloque catch puede manejar más de un tipo de excepción.

Esta característica puede reducir la duplicación de código y reducir la tendencia acapturar una excepción demasiado amplia. De esta manera se puede usar un solo bloque try para capturar varios tipos de excepciones simultáneamente, colocando cada una de ellas en su propio catch.

En la cláusula catch, se especifican los tipos de excepciones que el bloque puede manejar y se separa cada tipo de excepción con una barra vertical (|).

Ejemplo

```
 catch (IOException | SQLException ex) {
 logger.log (ex);
throw ex;
 }
```



Si un bloque catch maneja más de un tipo de excepción, entonces los parámetros son implícitamente definido como final.

En este ejemplo, el parámetro ex de catch es final y por lo tanto no se puede asignar un valor a la misma dentro del bloque del manejador.

El finally del bloque siempre se ejecuta cuando se sale del bloque try. Esto asegura que el finally bloque se ejecuta incluso si se produce una excepción inesperada.

La parte finally es útil para algo más que el manejo de excepciones ya que permite al programador poner código de limpieza sin que sea pasado por alto accidentalmente por un return, continue o break.

Poner el código de limpieza en un bloque finally es siempre una buena práctica, incluso cuando no se prevén excepciones.



Si la JVM sale mientras que el try o catch se ejecuta el código, entonces el bloque finally no se ejecutará. Del mismo modo, si el hilo que ejecuta el try o catch se interrumpe o muere, el bloque finally no se podrá ejecutar aunque la aplicación en su conjunto sigua.

El try del writeList, el método que hemos estado trabajando, ahora va a abrir un PrintWriter .El programa debe cerrar ese PrintWriter antes de salir del método writeList.

Esto plantea un problema un tanto complicado bloque try de porque writeList puede salir en una de tres maneras:

– La declaración new FileWriter falla y lanza una IOException.

– El vector.elementAt(i) falla y lanza una ArrayIndexOutOfBoundsException .

– Todo tiene éxito y el bloque try sale normalmente.

El sistema de ejecución ejecuta siempre las declaraciones dentro de la finally bloquear independientemente de lo que sucede dentro del try de bloque.

Así que es el lugar perfecto para llevar a cabo la limpieza con todo aquello que se quiera ejecutar antes de la finalización

Ejemplo

El siguiente bloque finally añadido al método limpia y luego se cierra el Print-Writer .

```
finally {
 if (! fuera = null) {
 System.out.println («Cierre PrintWriter»);
 out.close ();
 } Else {
 System.out.println («PrintWriter no abrir»);
 }
 }
```

En elejemplo el writeList, puede proporcionar para la limpieza sin el uso de un finally.

Ejemplo

Por ejemplo, se podría poner el código para cerrar el PrintWriter **al final de la try y luego** de nuevo en el controlador de excepciones para ArrayIndexOutO-fBoundsException como se muestra en el ejemplo.

```
try {
 // No hagas esto; se duplica el código.
 out.close ();
```

```
 } Catch (FileNotFoundException e) {

 // No hagas esto; se duplica el código.

 out.close ();

  System.err.println («FileNotFoundException Atrapados:» +
e.getMessage ());

 throw new RuntimeException (e);

 } Catch (IOException e) {

  System.err.println («Caught IOException:» + e.getMessage
());

 }
```

Sin embargo, esto duplica el código, con lo que el código sea difícil de leer y propenso a errores en caso de que haya que modificarlo posteriormente.

Por ejemplo, si se agrega el código que puede lanzar un nuevo tipo de excepción a la try de bloque, usted tiene que recordar para cerrar el PrintWriter **dentro del nuevo controlador de excepciones.**



El bloque finally es una herramienta clave para la prevención de pérdidas de recursos. Siempre hay que cerrar un archivo o de lo contrario se pueden ocupar recursos y bloquear estos. Siempre conviene colocar el código en un finally para asegurar que los recursos siempre se recupera.

Si está utilizando Java SE 7 o posterior, puede utilizar la declaración try con recursos en estas situaciones, lo que automáticamente libera recursos del sistema cuando ya no sean necesarios.

7.2. Uso de las excepciones tratadas como objetos

Como ya se ha comentado, para poder capturar excepciones, emplearemos la estructura de captura de excepciones try-catch-finally.

Básicamente, para capturar cualquier excepción lo que haremos será declarar bloques de código donde es posible que ocurra una excepción.

Esto se va a hacer mediante un bloque try o "intentar". Si ocurre una excepción dentro de estos bloques, se lanza una excepción.

Estas excepciones lanzadas se pueden capturar utilizando para ello una serie de bloques catch. Va a ser dentro de este tipo de bloques donde se hará el manejo de las excepciones.

Su sintaxis es:

```
try {

código que puede generar excepciones;

} catch (Tipo_excepcion_1 objeto_excepcion) {

Manejo de excepción de Tipo_excepcion_1;

} catch (Tipo_excepcion_2 objeto_excepcion) {

Manejo de excepción de Tipo_excepcion_2;

}

...

finally {

instrucciones que se ejecutan siempre

}
```

En esta estructura, la parte del catch puede repetirse tantas veces como excepciones diferentes se vayan a capturar. La parte finally es opcional y, si aparece, solo podrá hacerlo una sola vez.

Cada catch maneja un solo tipo de excepción. Cuando se produce una excepción, se busca el catch que posea el manejador de excepción adecuado, y va a ser el que utilice el mismo tipo de excepción que se ha producido. Esto

puede causar algún problema si no se obra con cuidado, ya que la clase Exception es la supercase.

Por eso el último catch que se indique debe ser el que capture excepciones más genéricas y los primeros deben ser los más específicos.

Por tanto si vamos a tratar a todas las excepciones (sean del tipo que sean) de la misma manera, entonces basta con un solo catch que capture objetos Exception.



Sabías que

Debido a que el lenguaje de programación Java no requiere métodos para coger o para especificar excepciones sin capturar (RuntimeException, Error y sus subclases), los programadores pueden tener la tentación de escribir código que lanza excepciones sólo sin marcar, o para hacer todas sus subclases heredan de RuntimeException.

Ambos atajos permiten a los programadores escribir código sin molestarse con los errores del compilador y sin molestarse en especificar o para coger alguna excepción.

Aunque esto puede parecer conveniente para el programador, que deja de lado la intención de la catch y de sus requisitos y puede causar problemas a los demás que utilicen sus clases.

Los diseñadores de java decidieron forzar a que un método deba especificar todas las excepciones controladas que no captura que se pueden lanzar dentro de el ya que cualquier Exception que puede ser lanzada por un método es parte de la interfaz de programación pública del método.

Los que van a llamar a un método debe saber las excepciones que un método puede lanzar de manera que puedan decidir qué hacer con ellas.

Estas excepciones son tan parte de la interfaz de programación de ese método como sus parámetros o su valor de retorno.

La siguiente pregunta podría ser Si es tan buena documentar la API de un método, incluidas las excepciones que puede lanzar, por qué razón no se han de especifican excepciones de tiempo de ejecución también.



Las excepciones de tiempo de ejecución representan problemas que son el resultado de un problema de programación, y como tal, no es razonable esperar que el código de cliente API pueda recuperarse de ellos o de manejarlos de manera alguna.

Tales problemas incluyen por ejemplo excepciones aritméticas, tales como la división por cero; excepciones de puntero, como intentar acceder a un objeto a través de una referencia nula; y excepciones de indexación, como el intento de acceder a un elemento de matriz a través de un índice que es demasiado grande o demasiado pequeño.

Excepciones de tiempo de ejecución pueden ocurrir en cualquier parte de un programa, y en un típico pueden ser muy numerosas. El tener que agregar excepciones de tiempo de ejecución en cada declaración de método reduciría la claridad de un programa.

De este modo, el compilador no requiere que el programador capture o especificar excepciones de tiempo de ejecución, aunque se puede hacer sin problema, solo que no es obligatorio.

Un caso donde es práctica común lanzar una RuntimeException es cuando el usuario llama a un método con unos parámetros incorrectos.

Por ejemplo, un método puede comprobar si uno de sus argumentos es un null incorrecto. Si un argumento es null, el método puede lanzar una NullPointerException, que es una excepción no comprobada.



Si razonablemente se puede esperar un llamador pueda recuperarse de una excepción, conviene convertirla en una excepción comprobada. Si un método llamador no puede hacer nada para recuperarse de la excepción, debe ser definida como una excepción no comprobada.

Ahora que se conoce lo que son excepciones y cómo usarlas, es el momento adecuado de mostrar las ventajas de utilizar las excepciones en sus programas.

Principal ventaja de las excepciones: Separar el código de manejo de errores del Código “normal”.

Las excepciones proporcionan medios para separar los detalles de qué se debe hacer cuando ocurre algo fuera de lo normal de lo que es la lógica principal de un programa.

En programación tradicional, la detección de errores, la presentación de la información, y el proceso de datos a menudo conducen a un denominado “código espagueti” que puede ser muy confuso.

Por ejemplo, considere el método en pseudocódigo que lee un archivo en la memoria que se muestra a continuación.

```
readFile {
abrir el archivo;
determinar su tamaño;
asignar suficiente memoria;
leer el archivo en la memoria;
cierre el archivo;
}
```

A primera vista, esta función parece simple, pero no tiene en cuenta todos los errores posibles que puede ocurrir como por ejemplo.

– ¿Qué pasa si el archivo no se puede abrir?

– ¿Qué ocurre si la longitud del archivo no se puede determinar?

– ¿Qué sucede si no hay suficiente memoria o esta no se puede asignar?

– ¿Qué pasa si la lectura del fichero falla?

– ¿Qué sucede si el fichero no se puede cerrar antes de acabar?

Para tratar estos casos, la función readFile debe tener mucho más código para realizar la detección de errores, además de la presentación de informes, y la manipulación.

A continuación se muestra un ejemplo de lo que la función tendría que hacer:

Ejemplo...

```
errorCodeType readFile {
inicializar errorCode = 0;
abrir el archivo;
if (theFileIsOpen) {
determinar la longitud del archivo;
if (gotTheFileLength) {
asignar suficiente memoria;
if (gotEnoughMemory) {
leer el archivo en la memoria;
if (readFailed) {
errorCode = -1;
}
} Else {
errorCode = -2;
}
} Else {
errorCode = -3;
}
cierre el archivo;
if (theFileDidntClose && errorCode == 0) {
errorCode = -4;
```

```
} Else {
errorCode = errorCode y -4;
}
} Else {
errorCode = -5;
}
volver errorCode;
}
```

El elevado código de detección de errores hace que tanto, la presentación de datos como el proceso queden ocultos, y hacen que las siete líneas de código originales se pierden en un desorden.

Además el mayor inconveniente es que el flujo lógico del código también se ha perdido, por lo que es difícil saber si el código está haciendo lo correcto

Por ejemplo, se puede intentar contestar a esta pregunta ¿realmente el archivo se ha cerrado si la función no puede asignar suficiente memoria?

Además es más difícil asegurar que el código continúa haciendo lo correcto si se modifica el método un período de tiempo después de codificarlo.



Muchos programadores solucionan este tipo de problemas simplemente ignorándolo, se informa de errores solo cuando el programa falla.

Las excepciones permiten al escribir el flujo principal de su código y para trabajar con los casos excepcionales en otras partes.

Si el readFile función utiliza excepciones en lugar de las técnicas tradicionales de manejo de error, que se vería más como la siguiente.

Ejemplo

```
readFile {
 try {
 abrir el archivo;
 determinar su tamaño;
 asignar suficiente memoria;
 leer el archivo en la memoria;
 cierre el archivo;
 } Catch (fileOpenFailed) {
 doSomething;
 } Catch (sizeDeterminationFailed) {
 doSomething;
 } Catch (memoryAllocationFailed) {
 doSomething;
 } Catch (readFailed) {
 doSomething;
 } Catch (fileCloseFailed) {
 doSomething;
 }
 }
```

Hay que tener en cuenta que en las excepciones no sobra el esfuerzo de hacer el trabajo de detección, comunicación y manejo de errores, pero ayudan a organizar el trabajo de manera más eficaz.

Una segunda ventaja de excepciones es la capacidad de propagar la presentación de informes de error hasta la pila de llamadas de métodos.

Supongamos que el método readFile es el cuarto método en una serie de llamadas a métodos anidadas realizadas por el programa principal: method1 llama method2, que llama method3, que finalmente llamara a readFile .

```
method1 {
 llamar method2;
 }
 method2 {
 llamar metodo3;
 }
 metodo3 {
 llamar readFile;
 }
```

Supongamos también que el método method1 es el único método interesado en los errores que puedan producirse dentro readFile .

La técnica tradicional de error y notificación obligan a que method2 y method3 propaguen los códigos de error devueltos por readFile a a pila de llamadas hasta que los códigos de error finalmente llegan method1 -el único método que está interesada en ellos.

Ejemplo:

```
method1 {
 error errorCodeType;
 error = method2 llamada;
 if (error)
 doErrorProcessing;
 demás
 proceder;
 }
 errorCodeType method2 {
```

```
error errorCodeType;
error = metodo3 llamada;
if (error)
error regresar;
demás
proceder;
}
errorCodeType metodo3 {
error errorCodeType;
error = llamada readFile;
if (error)
error regresar;
demás
proceder;
}
```



Conviene recordar que el entorno de ejecución Java busca hacia atrás a través de la pila de llamadas para encontrar cualquier método que esté interesado en el manejo de una excepción particular.

Un método puede re-lanzar una excepción lanzada dentro de el, lo que permite un método que esté más arriba en la pila de llamadas pueda atraparlo.

Por lo tanto, utilizando excepciones, sólo los métodos que se interesen por los errores tienen que preocuparse acerca de la detección de errores.

Ejemplo:

```
method1 {
 try {
 llamar method2;
 } Catch (Exception e) {
 doErrorProcessing;
 }
 }
 method2 lanza la excepción {
 llamar metodo3;
 }
 metodo3 lanza la excepción {
 llamar readFile;
 }
```

Sin embargo, como muestra el programa anterior, esquivar una excepción requiere un poco de esfuerzo por parte de los métodos de intermediarios.

Cualquier excepciones comprobadas que pueden lanzarse dentro de un método deben ser especificados en su cláusula throws Debido a que todas las excepciones lanzadas dentro de un programa son objetos, la agrupación o categorización de las excepciones es un resultado natural del uso de la jerarquía de clases.

Un ejemplo de un grupo de clases de excepciones relacionadas en la plataforma Java son los definidos en java.io.IOException . y sus descendientes. IOException es el más general y representa cualquier tipo de error que puede producirse al realizar E / S.

Sus descendientes representan errores más específicos. Por ejemplo, FileNotFoundException significa que un archivo no se encuentra en el disco.

Un método puede escribir controladores específicos que pueden manejar una excepción muy específica.

La clase FileNotFoundException no tiene descendientes, por lo que el siguiente controlador puede manejar solamente un tipo de excepción.

```
catch (FileNotFoundException e) {
 ...
 }
```

Un método puede capturar una excepción basada en su grupo o tipo general especificando cualquiera de las superclases de la excepción en la catch comunicado.

Por ejemplo, para capturar todas las excepciones de E / S, independientemente de su tipo específico, un manejador de excepción especifica un argumento como IOException .

```
catch (IOException e) {
 ...
 }
```

Este controlador será capaz de capturar todas las excepciones de E / S, incluyendo FileNotFoundException, EOFException, y así sucesivamente.

Puede encontrar detalles sobre lo que ha ocurrido en el error mediante la consulta del argumento pasado al controlador de excepciones. Por ejemplo, se puede utilizar el siguiente **código** para imprimir el segmento de la pila.

Ejemplo

```
catch (IOException e) {
 // Salida va a System.err.
 e.printStackTrace ();
 // Enviar rastro a la salida estándar.
 e.printStackTrace (System.out);
 }
```

Incluso se puede configurar un controlador de excepciones que maneja cualquier Exception usando el siguiente manejador.

```
// A (también) manejador de excepción general
catch (Exception e) {
...
}
```



La clase Exception está muy cerca de la parte superior de la jerarquía de clases Throwable . Por lo tanto, este controlador se encargará de capturarmuchas otras excepciones, además de las que el controlador tiene la intención de atrapar.

Es posible que desee controlar las excepciones de esta manera si todo lo que se quiere que el programa haga, por ejemplo, es imprimir un mensaje de error para el usuario y luego salir.

En la mayoría de situaciones, sin embargo, se desea que los controladores de excepciones estén diseñador para ser lo más específico posible. La razón es que la primera cosa que un controlador debe hacer es determinar qué tipo de excepción se produjo antes de que pueda decidir cual es la mejor estrategia de recuperación.



En efecto, al no detectar errores específicos, el código debe adaptarse a cualquier posibilidad. Los manejadores de excepciones que son demasiado generales pueden hacer que el código sea más propenso a errores por la captura y manejo de excepciones que no fueron previstas por el programador y que no se desean manipular.

Como se ha señalado antes, puede crear grupos de excepciones y manejarlas de manera general, o puede utilizar el tipo de excepción específico para diferenciar excepciones y manejarlas de una manera exacta.

Dentro de una excepción también se puede incluir información relevante sobre el error.

Una excepción que se lanza no solamente se dedica a transferir el control desde una posición de un programa otra como si fuera un goto muy elaborado sino que también puede enviar información.

Debido a que una excepción es realmente un objeto con todas sus características, el programador puede incluir toda la información que desee en ese objeto, como si se tratara de cualquier otro objeto de java.

Se debe utilizar esta característica para incluir información sobre el error que se ha producido y por el que se ha generado la excepción.

En el catch se puede obtener dicha información, simplemente haciendo una solicitud al objeto correspondiente.

La clase Exception permite al programador incluir un mensaje que se puede obtener del objeto excepción llamando al método getMessage, como se indica en el código que se especifica a continuación:

Ejemplo

Esta es la clase que define la excepción:

```
class UnusualTasteException extends Exception {
 public UnusualTasteException() {
 }
 public UnusualTasteException(String msg) {
 super(msg);
 } }
```

El programador puede crear un objeto excepción de dos maneras distintas, o bien usando:

```
new UnusualTasteException();
```

o

```
new UnusualTasteException (“Este café sabe como un te”);
```

En el catch se le puede preguntar al objeto por el mensaje de error, por ejemplo utilizando este código que se muestra:

```
class VirtualCafe {
public static void serveCustomer(
 VirtualPerson cust, CoffeeCup cup) {
 try {
 cust.drinkCoffee(cup);
 System.out.println(
“El café sabe bien.”);
}
catch (UnusualTasteException e) {
System.out.println(
“El cliente se queja de un sabor raro en el café.”);
String s = e.getMessage();
if (s != null) {
System.out.println(s);
}
// Aquí se incluye como tratar con el cliente insatisfecho
}
}
}
```

Se puede añadir más información que simplemente una cadena, para ello basta con agregar lo métodos necesarios a la clase que implementa la excepción.

Ejemplo

```
abstract class TemperatureException
 extends Exception {
 private int temperature; // in Celsius
 public TemperatureException(int temperature) {
 this.temperature = temperature;
 }
public int getTemperature() {
return temperature;
}
}
class TooColdException
extends TemperatureException {
public TooColdException(int temperature) {
super(temperature);
}
}
class TooHotException
extends TemperatureException {
public TooHotException(int temperature) {
super(temperature);
}
}
```

El campo temperatura debe ser almacenado el valor cuando se crea el objeto de excepción de temperatura, de la siguiente manera:

```
 class VirtualPerson {
 private static final int tooCold = 65;
private static final int tooHot = 85;
 public void drinkCoffee(CoffeeCup cup) throws
 TooColdException, TooHotException {
int temperature = cup.getTemperature();
if (temperature <= tooCold) {
throw new TooColdException(temperature);
}
else if (temperature >= tooHot) {
throw new TooHotException(temperature);
}
//...
}
//...
}
```

Entonces ya en un catch se puede fácilmente determinar que excepción ha generado el error por temperatura, si lo que ocurre es que el café estaba muy caliente o muy frío, de la siguiente manera:

Ejemplo

```
class VirtualCafe {
 public static void serveCustomer(
 VirtualPerson cust, CoffeeCup cup) {
 try {
```

```
 cust.drinkCoffee(cup);
 System.out.println(“Coffee is just right.”);
}
catch (TooColdException e) {
12
int temperature = e.getTemperature();
System.out.println(“Coffee temperature is “
+ temperature + “ degrees Celsius.”);
16
if (temperature > 55 && temperature <= 65) {
18
System.out.println(
“EL café esta helado
// añadir mas café caliente
}
else if (temperature > 0
&& temperature <= 55) {
25
System.out.println(“El café esta frio.”);
// Dar un café al cliente
// conla temperatura decuada
}
else if (temperature <= 0) {
31
System.out.println(“El café esta helado
// Gestionar cliente iracundo
}
```

```
}
catch (TooHotException e) {
37
int temperature = e.getTemperature();
System.out.println("Coffee temperature is "
+ temperature + " degrees Celsius.");
if (temperature >= 85 && temperature < 100) {
System.out.println("El café esta muy caliente.");
// Preguntar al cliente si quiere esperar a que se enfrie
// Esperar unos minutos...
}
else if (temperature >= 100
&& temperature < 2000) {
48
System.out.println(
"El café y el cliente están hirviendo.");
// Gestionar cliente iracundo
}
else if (temperature >= 2000) {
54
System.out.println(
"El café es plasma");
// Gestionar cliente iracundo
}
}
}
}
```

UD7
Lo más importante

- Una excepción es un evento, que se produce durante la ejecución de un programa, que interrumpe el flujo normal de las instrucciones del programa. Un programa perfectamente compilado en el que no existen errores de sintaxis, puede generar otros tipos de errores que puede que aparezcan en tiempo de ejecución. A estos errores se les conoce como excepciones.

- Las tres clases de excepciones que hay en java son excepciones comprobadas, errores y excepciones en tiempo de ejecución.

- Una excepción comprobada es un error que se produce en e programa durante su ejecución y que el programador debe verificar si se produce, y que hacer en caso de que se produzca.

- El segundo tipo de excepción es el error. Estas son condiciones excepcionales que son externas a la aplicación, y que la aplicación por lo general no pueden anticipar o recuperarse.

- El tercer tipo de excepción es la excepción en tiempo de ejecución: Estas son condiciones excepcionales que son internos a la aplicación, y que la aplicación por lo general no pueden anticipar o recuperarse.

- Los controladores de excepciones están asociados con un bloque try, el cual es manejado por uno o más bloques catch que deben aparecer directamente después del try de bloque.

– Los controladores de excepciones pueden hacer la recuperación de errores, pedir al usuario que tome una decisión, o propagar el error hasta un controlador de nivel superior utilizando excepciones encadenadas.

– Las excepciones de tiempo de ejecución representan problemas que son el resultado de un problema de programación, y como tal, no es razonable esperar que el código de cliente API puede recuperarse de ellos o de manejarlos de manera alguna.

– Si razonablemente se puede esperar un llamador pueda recuperarse de una excepción, conviene convertirla en una excepción comprobada. Si un método llamador no puede hacer nada para recuperarse de la excepción, debe ser definida como una excepción no comprobada.

– Los controladores de excepciones están asociados con un bloque try, el cual es manejado por uno o más bloques catch que deben aparecer directamente después del try de bloque.

– El sistema considera que coincide si el objeto lanzado puede ser correctamente asignado al argumento del manejador de excepciones

– El finally del bloque siempre se ejecuta cuando se sale del bloque try.

– El finally de bloque es una herramienta clave para la prevención de pérdidas de recursos. Siempre hay que cerrar un archivo o de lo contrario se pueden ocupar recursos y bloquear estos. Siempre conviene colocar el código en un finally para asegurar que los recursos siempre se recupera.

– Dentro de una excepción también se puede incluir información relevante sobre el error.

UD7
Autoevaluación

1. Un programa perfectamente compilado:

    a. Generalmente no va a ocasionar ningún problema.
    b. NUNCA puede tener un error ya que se ha compilado perfectamente sin errores.
    c. Puede generar otros errores cuando se ejecuta.
    d. Ninguna de las anteriores es correcta.

2. Una excepción es:

    a. Una instrucción de Java.
    b. Una clase de Java que hereda de Error.
    c. Un evento que se produce durante la ejecución de un programa que interrumpe su flujo normal de instrucciones.
    d. Una clase de Java incorporada al paquete IOException.

3. Las tres clases de excepciones que hay son:

    a. Las excepciones comprobadas, las no comprobadas y los errores.
    b. Los errores y las excepciones propiamente dichas.
    c. Las Throwable y las IOException.
    d. Solo hay una clase de excepciones en Java.

4. Las excepciones se capturan…

   a. Con la sentencia throw.
   b. Mediante un bloque finally{…}.
   c. Mediante un bloque try{…} catch{…}.
   d. Con la sentencia throws.

5. Las excepciones se lanzan…

   a. Con la sentencia throw.
   b. Mediante un bloque finally{…}.
   c. Mediante un bloque try{…} catch{…}.
   d. Con la sentencia throws.

6. El bloque finally...

   a. Es una herramienta clave para la prevención de pérdida de recursos.
   b. Es obligatorio incluirlo.
   c. Se coloca antes del try.
   d. Ninguna de las respuestas es correcta.

7. La principal ventaja de las excepciones es:

   a. Separa el código normal del código de detección y recuperación de errores.
   b. Que el código es código espagueti.
   c. UN método debe recuperarse de los errores que genere.
   d. Solo se usa en lenguaje C.

8. Un método puede re-lanzar una excepción…

   a. No nunca.
   b. De hecho siempre deben re-lanzarse las excepciones.
   c. Y así la puede capturar otro método que esté en la parte superior de la pila.
   d. Solamente las que están por debajo en la jerarquía de IOException.

9. ¿Se puede incluir información relevante en un objeto de excepción?

    a. No es recomendable trabajar así.
    b. Solamente las excepciones que se van a re-lanzar.
    c. Si, una excepción puede incluir información en el objeto asociado.
    d. Las excepciones solo se pueden capturar, y no se usan para transmitir información.

10. La información que se incluya en un objeto excepción puede ser:

    a. No se puede incluir información en un objeto excepción.
    b. Solamente cadenas de texto.
    c. Solamente un código de error numérico.
    d. No solamente cadenas, sino de cualquier tipo válido.

Área: informática y comunicaciones

UD8

Documentación

8.1. Cómo producir un documento

8.2. Estructura de un documento

8.3. Generación automática de documentación

8.1. Cómo producir un documento

“La documentación es una parte vital del desarrollo y uso de un sistema informático. En algunas organizaciones, del 20 al 40% del esfuerzo total de desarrollo entra en la documentación del nuevo sistema, grabando cómo el nuevo sistema trabajar y cómo fue desarrollado“.



La documentación del software es el texto que acompaña a los programas informáticos. Esta documentación debe explicar bien cómo funciona o cómo usarlo, y puede significar cosas diferentes para personas con diferentes roles en la organización.

La documentación es una parte muy importante del proceso de ingeniería del software. Tipos de documentación que se generan en un proyecto incluyen:

– Requisitos: declaraciones que identifican atributos, capacidades, características o cualidades de un sistema. Esta es la base para lo que será o ha sido implementado.

– Arquitectura, Diseño y Vista general del software. Incluye las relaciones a un medio ambiente y los principios de construcción para ser utilizados en el diseño de componentes de software.

– Documentación de carácter técnico como documentación de código, algoritmos, interfaces y APIs documenntadas.

– Manuales para el usuario final, los administradores de sistemas y el personal de apoyo.

– Marketing: indica cómo comercializar el producto y el análisis de la demanda del mercado.

La documentación de requisitos es la descripción de lo que un determinado software hará o no lo hará. Se utiliza en todo el desarrollo para comunicar lo que hace el software o hará cuando se desarrolle.

También es, en definitiva, como un acuerdo o base para un acuerdo sobre lo que el software debe hacer.

Requisitos son producidos y generados por todos los involucrados en la producción de software como usuarios finales, clientes, gerentes de producto, gerentes de proyectos, ventas, comercialización, Arquitectos del, ingenieros de usabilidad, diseñadores de interacción, desarrolladores y probadores, por nombrar algunos.

Por lo tanto, la documentación requisitos tiene muchos propósitos diferentes. Además los requisitos vienen en una variedad de estilos, de notaciones y formalizaciones



Los requisitos pueden ser definidos una meta, como por ejemplo, entorno de trabajo distribuido, o bien una descripción de la manera de diseñar, por ejemplo, el sistema se debe de poder “construir” haciendo clic derecho en un archivo de configuración y seleccionando la función ‘build’, o en general cualquier otra cosa.

Los requisitos se pueden especificar como las declaraciones en lenguaje natural, dibujado figuras, o utilizando fórmulas matemáticas, y como una combinación de todos ellos.

La variación y la complejidad de la documentación de requisitos hace que sea todo un reto probarla.

Los requisitos pueden ser implícitos y difíciles de descubrir. Es difícil saber exactamente cuánto y qué tipo de documentación se necesita y cuánto puede dejarse a la documentación de la arquitectura y el diseño, y es difícil saber cómo documentar los requisitos teniendo en cuenta la variedad de personas que deberán leer y utilizar la documentación.

Por lo tanto, los requisitos de la documentación suelen estar a menudo incompletos o ser inexistentes.

Sin la documentación apropiada de requisitos, los cambios de software se vuelven más difíciles y por lo tanto más propensa a errores, lo que implica una disminución de la calidad del software y consume mucho más tiempo, lo que lo hace más caro.



La necesidad de documentación de los requisitos está típicamente relacionado con la complejidad del producto, el impacto del producto, y la esperanza de vida del software.

Si el software es muy complejo o desarrollado por muchas personas (por ejemplo, software de teléfono móvil), los requisitos pueden ayudar a comunicar mejor lo que debe lograr.

Si el software es crítico para la seguridad y puede tener un impacto negativo en la vida humana como por ejemplo, los sistemas de energía nuclear o equipos médicos, a menudo se requiere documentación de los requisitos más formales.

Si se espera que el software vaya a tener una vida de sólo por un mes o dos como por ejemplo, pequeñas aplicaciones para teléfonos móviles desarrolladas específicamente para una determinada campaña es suficiente con muy poca documentación de requisitos.

Si el software es una primera versión sobre la que más tarde se va a seguir trabajando, la documentación de los requisitos es muy útil en la gestión del cambio de software y verificar que no se ha deteriorado en el software cuando se modifica.

Tradicionalmente, los requisitos se especifican en documentos de requisitos como por ejemplo, el uso de aplicaciones de procesamiento de textos y hojas de cálculo.

Para gestionar la creciente complejidad y la naturaleza cambiante de la documentación de los requisitos y documentación del software en general, los sistemas de bases de datos de uso especial como herramientas para la gestión de requisitos se suelen utilizar en muchas organizaciones.

La documentación de Arquitectura,también conocida como de descripción de la arquitectura software es un tipo especial de documento de diseño. La descripción dela arquitectura proporciona un contexto en el que se va a desarrollar el sistema software que de está diseñando.

En cierto modo, los documentos de la arquitectura son una tercera derivada a partir del código (documento de diseño es la segunda derivada, y documentos de codificación la primera).

En los documentos de la arquitectura una pequeña parte es específico sobre el propio código.

Estos documentos no describen cómo programar una rutina particular, o incluso por qué existe esa rutina particular, en la forma que lo hace, sino que se limita a establecer los requisitos generales que motiven la existencia de tal rutina.



Un buen documento de arquitectura es parco en detalles, pero detallado en la explicación. Se puede sugerir enfoques para el diseño de nivel inferior, pero dejan los estudios comerciales de exploración a otros documentos.

Otro tipo de documentos de diseño es el documento de comparación, o el estudio comercial.

Esto a menudo toma la forma de un libro blanco. Se centra en un aspecto específico del sistema y se sugiere enfoques alternativos.

Podría ser en la interfaz de usuario, código, diseño, o incluso a nivel arquitectónico. Tratará de delinear cuál es la situación, y describe una o más alternativas, enumerando los pros y los contras de cada una.



Un buen documento de estudio comercial está basado en una investigación profunda, en el que el autor expresa sus ideas con claridad sin depender en gran medida de jerga obtusa para deslumbrar al lector, y lo más importante, es imparcial.

Debe explicar con honestidad y claridad los costos de cualquier solución que ofrece lo mejor.

El objetivo de un estudio comercial es idear la mejor solución, en lugar de apoyar un punto de vista particular.

Es perfectamente aceptable para indicar a ninguna conclusión, o llegar a la conclusión de que ninguna de las alternativas son bastante mejores que la línea de base para justificar un cambio.

Se debe abordarse como un esfuerzo científico, no como una técnica de marketing.



Una parte muy importante del documento de diseño en el desarrollo de software empresarial Documento de Diseño de base de datos (DDD).

Contiene el diseño conceptual, lógico y físico y los elementos de diseño. El DDD incluye la información formal de que necesitan las personas que interactúan con la base de datos.

El propósito de la preparación que es crear una fuente común para ser utilizado por todos los participantes dentro del escenario. Los usuarios potenciales de este documento son:

– Diseñador de bases de datos.

– Desarrollador de bases de datos.

– Administrador de base de datos.

– Diseñador de la aplicación.

– Desarrollador de aplicaciones.



Al crear software, el código por sí solo es insuficiente. Debe haber texto junto con él para describir diversos aspectos de su operación prevista.

Es importante que los documentos de código que sean específicos, pero no sean tan detallado que se haga difícil mantenerlas.

Esta documentación podrá ser utilizada por los desarrolladores, probadores y también los clientes finales o clientes que utilizan esta aplicación de software.

Hoy, vemos gran cantidad de aplicaciones de gama alta en el campo de la energía, la energía, el transporte, las redes, la industria aeroespacial, seguridad, automatización de la industria y una variedad de otros dominios.

La documentación técnica se ha convertido en importante dentro de este tipo de organizaciones tanto el nivel básico y avanzado ya que la información puede cambiar en un período de tiempo con los cambios de arquitectura.

Por lo tanto, la documentación técnica ha ganado mucha importancia en los últimos tiempos, especialmente en el campo del software.

A menudo, las herramientas como Doxygen, NDoc, javadoc, EiffelStudio se pueden utilizar para generar automáticamente los documentos a partir de código, que extraen los comentarios y especificaciones de software, cuando esté disponible, a partir de el código fuente y crear manuales de referencia en formas tales como texto o HTML archivos.



Los documentos de código se organizan a menudo al estilo de una guía de referencia, lo que permite a un programador para buscar rápidamente una función arbitraria o clase.

La idea de la documentación automática de generación es atractivo para los programadores por varias razones.

Ejemplo

Por ejemplo, debido a que se extrae de la propia código fuente (por ejemplo, a través de los comentarios), el programador puede escribir mientras se hace referencia al código, y utilizar las mismas herramientas utilizadas para crear el código fuente para hacer la documentación. Esto hace que sea mucho más fácil mantener la documentación actualizada.

Por supuesto, una desventaja es que sólo los programadores pueden editar este tipo de documentación, y que depende de ellos para actualizar la documentación obtenida.



Donald Knuth ha insistido en el hecho de que la documentación puede ser un proceso de idea de última hora" y ha abogado por la "programación literaria", es decir, ir escribiendo al mismo tiempo y en la misma ubicación que el código fuente y para que posteriormente extraiga por medios automáticos.

La **Programación explicativa** es el resultado de la aplicación práctica de la escritura de programación auto explicativa en contextos de programación real.

El paradigma explicativo propone que el código fuente y la documentación se almacenarán por separado.

A menudo, los desarrolladores de software tienen que ser capaces de crear y acceder a información que no va a ser parte del propia archivo de código fuente.

Estas anotaciones son generalmente parte de varias actividades de desarrollo de software, tales como diseños de código y de transferencia, donde se analiza el código fuente del partido de una manera funcional.

Las anotaciones, por lo tanto, pueden ayudar al desarrollador en cualquier etapa del desarrollo de software, donde un sistema de documentación formal podría obstaculizar el progreso.

Kelp almacena anotaciones en archivos separados, vinculando la información al código fuente de forma dinámica.



A diferencia de los documentos de código, los documentos de usuario simplemente describen cómo se utiliza un programa.

En el caso de una librería de software, la documentación de código y los documentos del usuario podría ser equivalente pero para una aplicación general, esto no suele ser cierto.

Por lo general, la documentación del usuario se describe cada función del programa y ayuda al usuario en la realización de estas funciones.

Un buen documento usuario puede ir tan lejos como para proporcionar la solución a fondo de problemas que requieran asistencia técnica.

Es muy importante que los documentos de usuario no sean confusos, y que estén actualizados a la fecha. Los documentos de los usuarios no necesitan estar organizados de una manera determinada, pero es muy importante para los usuarios el hecho de tener un índice exhaustivo.

La coherencia y la simplicidad también son muy valiosas. De esta materna se va simplificar la lectura del documento y se va a mejorar la comprensión por parte de la persona que tienen que asimilar esta.

La documentación del usuario se considera que constituye un contrato que especifica lo que el software hará. Esta es la razón por la que tienen tanta importancia, ya que realmente representa el compromiso entre la organización cliente y de desarrollo.



Los escritores API deben también escribir los documentos de usuario, ya que sería muy estos son conscientes de las técnicas de arquitectura y programación de software utilizados.

Existen tres formas generales en que la documentación del usuario se puede organizar.

– Tutorial: un tutorial enfoque se considera el más útil para un usuario nuevo, en el que se guían a través de cada paso de la realización de tareas específicas.

– Temática: un enfoque de temática, donde los capítulos o secciones se concentran en un área particular de interés, es de uso más general a un usuario intermedio.

  Algunos autores prefieren expresar sus ideas a través de un artículo basado en el conocimiento para facilitar las necesidades de los usuarios.

  Este enfoque se practica generalmente en las industrias dinámicas, tales como tecnología de la información, donde el grupo de usuarios es en gran medida correlacionada con las solución de problemas.

– Lista o de referencia: el último tipo de principio organizador es aquella en la que los comandos o tareas son simplemente aparecen en orden alfabético o lógicamente agrupados, a menudo a través de los índices de referencia cruzada.

Este último enfoque es más útil para los usuarios avanzados que saben exactamente qué tipo de información que están buscando.

Una queja común entre los usuarios con respecto a la documentación del software es que sólo uno de estos tres enfoques se toma y lleva a la casi exclusión de los otros dos.

Es común la documentación del software límite previsto ordenadores personales a la ayuda en línea que le dan sólo información de referencia sobre los comandos o elementos de menú.

El trabajo de tutorización a nuevos usuarios o ayudar a los usuarios más experimentados obtener el máximo rendimiento de un programa se deja a los editores privados, que a menudo obtienen una ayuda significativa por el desarrollador de software.

A continuación se enumeran una serie de principios importantes a la hora de generar cualquier documento.

– Documentar con imágenes si es posible:

  El viejo adagio de que una imagen vale más que mil palabras significa que mediante el uso de imágenes para aumentar su texto, se puede minimizar la longitud y la complejidad de la documentación.

  Los usuarios del sistema les interesa tener imágenes, diagramas, tablas y listas con viñetas para una referencia rápida.

– Proporcionar ejemplos que siempre son muy importantes tanto para los usuarios técnicos como para los inexpertos:

  Los ejemplos son una excelente manera para que los usuarios finales puedan comprender rápidamente los conceptos que pueden no entender completamente.

  También es una buena manera para que un usuario final aprender un nuevo software para sentarse y hacer frente a un nuevo desafío con mayor facilidad.

– No asumir nada que el usuario deba saber:

  Incluso si se conoce a la base de usuarios objetivo, la documentación debe ser escrita de manera que cualquier persona con sólo conocimientos básicos de informática puede leerlo y aprender a utilizar correctamente el sistema.

Instrucciones paso a paso deben de darse cuando sea posible, pero tenga en cuenta de colocarlos en un apéndice, un capítulo aparte, o su puesta a disposición a través de un hipervínculo para evitar el desorden.

– Si se está haciendo la documentación, conviene cambiar la forma de pensar para colocarse en la piel de un nuevo usuario del sistema.

  Eso puede ser difícil de hacer al principio, pero si se presta atención a los detalles y se trata de documentar todas las características y funciones, puede crear documentos que no asuman que el usuario puede encontrar la información y los procedimientos que no se han incluido.

  No asuma que su usuario final entiende todas esas siglas que ensucian el entorno de TI. La primera vez que se presenta un nuevo acrónimo, detalle qué significa el acrónimo.

– Anticipar los problemas:

  Al probar un sistema, se debe haber intentado todo lo posible para probar el software de cualquier forma que pudiera.

  Si el software tiene problemas conocidos (a los desarrolladores gustaría llamarlos cuestiones; usuarios finales llamarlos errores), documentar una solución y ofrecer a sus usuarios ayuda.

  No sólo ahorrará un montón de frustración para los usuarios finales, sino también un montón de llamadas adicionales al servicio técnico de ayuda.

– Documentar los eventos que son inevitables durante la vida útil de cualquier sistema:

  · ¿Qué soluciones están disponibles, mientras que el sistema o la red esta colapsada?

  · ¿Cómo recuperarse de una interrupción del servidor, un fallo del disco duro, o la corrupción de bases de datos?

  · ¿Cómo alguien que no sabe absolutamente nada acerca de su sistema puede poner el sistema en marcha y funcionando de nuevo?

  La documentación debe anticiparse a estos problemas y proporcionar un plan detallado y las instrucciones para la recuperación del sistema.

– Probar la documentación

  Esto se puede resumir e n llevar a cabo los siguientes pasos:

  · Sentarse y seguir las instrucciones dadas en la documentación.

  · Por ejemplo si se está documentando la construcción de un servidor, una red, o cualquier otro sistema informático, comenzar con una partición limpia y construir todo desde cero.

  · Sin duda, se descubrirá que se ha dejado algo o que algunas de sus instrucciones no son claras.

  · También conviene trabajar con un alguien desinformado pero comprometido para conseguir la prueba antes de publicar. Se puede hacer que pongan a prueba sus documentos.

– Una gran cantidad de características del software que son obvias para que no sea tan obvio para alguien que es honesto y dispuesto a trabajar en su desarrollo.

  Preste especial atención a lo que hace su conejillo de indias, mientras que la navegación de su software. Pida retroalimentación y tomar notas.

– Humanizar el trabajo:

  Muchas veces se lee un manual de usuario y el lector se pregunta si realmente había un ser humano al otro extremo de la creación del manual, o era un equipo el que hizo que el manual.

  Humanizar el documento sólo lo suficiente con un poco de su personalidad para que un lector se sentirá un poco más cómodo mientras lo leía.

– Conviene utilizar nuevas tecnologías:

  La documentación puede ser costosa de, incluso si se hace correctamente. Las nuevas tecnologías seguirán siendo creado para ayudar a crear documentación más eficaz que es menos costoso para desarrollar.

  Hay que ver estas nuevas herramientas como oportunidades para reducir el tiempo y costo del proceso de documentación.

  La documentación como parte de un equipo de proyecto puede ser especialmente difícil. Esto es porque constituye un elemento que debe de-

sarrollar el equipo completo y es muy difícil repartir satisfactoriamente las tareas entre todos los participantes.

La documentación debe ser compartida y se añade a la documentación de los otros miembros del equipo. Los cambios tienen que hacerse, a menudo sobre una base diaria.

Existe software que permitirá para esto y no sólo ayudará a asegurar un producto final estandarizado sino que también ayudará a fomentar el intercambio de ideas y conocimientos entre los miembros del equipo.

– Hacer la documentación uno mismo, si es posible:

La mejor persona para generar un documento es el constructor. Después de todo, ¿quién conoce el sistema mejor que el integrador de sistemas?

Aunque no es exactamente entretenido, la documentación puede ser gratificante si se hace correctamente. Muchas veces hay que llevar a cabo una serie de tareas de motivación entre el equipo para que se conciencien de la importancia de una buena documentación.

No sólo va a tener un mejor proyecto global para presentar a su cliente, también se reducirá en gran medida el futuro el tiempo de soporte, tendrá que proporcionar.

Coordinar la elaboración de la documentación de usuario final con el responsable de Documentación Interna del Sistema. Esta persona debe encargarse de que todos los aspectos de la documentación e lleven a cabo satisfactoriamente.

Se puede reducir el tiempo de documentación si se construye la documentación del usuario, al mismo tiempo que escribe la documentación del sistema. De esta manera se van desarrollando la visión del sistema a la misma vez que se hace la documentación que muestra como el usuario va a interactuar con el, en el fondo son como dos caras de la misma moneda.

Se trata de que se puede compartir parte de la información entre los dos documentos y reducir la información que falta, así como ampliar aquella que sea necesaria.

– Incluso si no se quiere no quiere o no se considera apropiado compartir información entre los documentos, se puede beneficiarse de temas en un documento que se pueden plantear si se desea incluir documentación adicional en el otro.

Conviene seguir las pautas de documentación corporativa, crear y seguir formatos y recomendaciones que se hayan normalizado en la organización.

Esto ayudará a asegurar que información importante no se excluye y permite una lectura más fácil por los usuarios del sistema. Generalmente los estándares de documentación sirven para el que desarrolla sepa que información debe incluir en ella, y para que la persona que la lee sepa donde debe de buscar la información relevante y no se pierda entre los datos.

8.2. Estructura de un documento

Documentar el código de un programa es añadir suficiente información como para explicar lo que hace, punto por punto, de forma que otros programadores puedan entender qué están haciendo y por qué.

Documentar un programa no es sólo un acto de buen hacer del programador para dejar la obra rematada.

Es además una necesidad que sólo se aprecia en su debida magnitud cuando hay errores que reparar o hay que extender el programa con nuevas capacidades o adaptarlo a un nuevo escenario. Hay dos reglas que no se deben olvidar nunca:

Todos los programas tienen errores y descubrirlos sólo es cuestión de tiempo y de que el programa se utilice frecuentemente.

Todos los programas sufren modificaciones a lo largo de su vida.

Por una u otra razón, todo programa será modificado en el futuro, bien por el programador original, bien por otro programador que le sustituya.

Pensando en esta revisión de código es por lo que es importante que el programa se entienda: para poder repararlo y modificarlo.

En los comentarios hay que añadir explicaciones a todo lo que no es evidente, es decir, no hay que repetir lo que se hace, sino explicar por qué se hace.

Por ejemplo ¿de qué se encarga una función o clase? ¿Un módulo o paquete? ¿Qué hace un método? ¿Para qué se usa una variable? ¿Qué algoritmo estamos usando? ¿Qué limitaciones tiene el algoritmo o la implementación?

Es conveniente poner un comentario en las siguientes situaciones:

– Al principio de cada fichero, función, clase o método

– Delante cada variable.

– Al principio de fragmento de código no evidente y en los bucles.

– Es conveniente que los comentarios más vale que sobren que falten.

Además, cuando un programa se modifica, los comentarios deben modificarse al mismo tiempo, para evitar que los comentarios acaben refiriéndose a un algoritmo o funcionalidad que ya no se utiliza.

Es conveniente seguir estándares de codificación, es decir, normas acordadas por un conjunto de programadores sobre como se va a organizar visualmente el código fuente de una aplicación.



Sabías que

Las organizaciones que se dediquen a desarrollar software deben implementar y fomentar el uso de estos estándares debido a las múltiples ventajas que conlleva su utilización.

El uso de estándares de codificación conlleva multitud de ventajas entre las que se pueden enumerar las siguientes:

– Reduce la probabilidad de introducir errores.

– Precisa algunos errores ocultados o inesperados.

– Hace que el código más uniforme y más fácil a leer.

– Hace que el software más fácil a mantener.

– Aumenta la robustez y la confiabilidad.

– Permitirá Revisar el código que ha sido entregado por los proveedores.

– Permite aprender a los programadores junior para que su desarrollo sea de software de calidad.

Ejemplo

Como ejemplo se muestra un estándar de codificación de una organización para codificación en C:

– Descripción de cada segmento del programa

```
Autor: Nombre del creado de la hoja

Fecha: Fecha de la última actualización o incremento

Versión : Debe anotarse en forma correlativa el número de
veces que se ha programado.
```

Ejemplo:

```
//----INFORMACIÓN DEL SEGMENTO DE PROGRAMA------//

  AUTOR: Juan Pérez

  FECHA: 28-06-2003

  VERSIÓN: 1.0

//-------------FIN DE LA INFORMACIÓN------------//
```

– Variables. Debe especificarse la siguiente información:

- Nombre: explicación del dato que se va a almacenar.
- Tipo Dato: especificación del tipo de datos al que va a corresponder la variable.
- Clasificación: especificación del tipo de variable de la que se trata.
- Objetivo: breve descripción para que sirve, como se maneja y que es en definitiva la variable.

Clasificación	Descripción	Ejemplo
G	Variable de tipo global	G_cuenta
L	Variable de tipo local	L_ cuenta
P	Variable de tipo puntero	P_ cuenta
A	Variable de tipo array o vector	A_ cuenta
GP	Variable de tipo global puntero	GP_ cuenta
GA	Variable de tipo global array o vector	GA_ cuenta
LP	Variable de tipo local puntero	LP_ cuenta
LA	Variable de tipo local array o vector	LA_ cuenta

Ejemplo:

Ejemplo de uso de variables :

```
Int contador //ALMACENA UNA VARIABLE CONTADOR
```

Los comentarios se especifican siguiente el formato y con la posición indicada posición.

Formato: especifica la forma en que va estar escrito el comentario, el cual deberá ser escrito con mayúsculas.

Posición: la situación del comentario deberá ser al final de la línea de código de la que trata.

Ejemplo:

```
Int iva //ALMACENA EL VALOR DEL IVA
```

De cada una de las funciones y procedimientos hay que especificar una serie de datos, como el autor, fecha, versión y objetivo:

Ejemplo:

Autor: nombre del creador de la función

Fecha: fecha de la última actualización de la función

Objetivo: breve descripción del problema que se resuelve; además variables de E/S que utilizad

Versión: debe indicarse en forma correlativa el número de veces que se ha modificado.

Ejemplo:

```
//------------- INICIO DE FUNCION-----------------//
   //------- INFORMACIÓN DE LA FUNCIÓN ---------//
     AUTOR: Juan Pérez
     FECHA: 28-06-2003
     OBJETIVO: Validar entradas del formulario
     VERSIÓN: 1.0
  //------------FIN DE LA INFORMACIÓN------------//
void valida(alumno as char)
{
…
}
```

Conviene Añadir comentarios descriptivos junto a cada declaración de variables, si es necesario.

La indentación y sangrado tendrá una longitud de tres espacios.

Hay que insertar una línea en blanco antes y después de una declaración de datos que aparezca entre instrucciones ejecutables.

Ejemplo:

```
a = b + c;
// línea en blanco
int f; // declaración entre instrucciones
// línea en blanco
f = a;
```

– Deben incluirse espacios a los dos lados de los operadores binarios.

  Ejemplo :

```
y = 50 + 15 - x;
```

– Los operadores unarios (++, --, etc.) deben ponerse junto a sus operandos, sin ningún espacio intermedios.

  Ejemplo

```
a++
```

– Para hacer más detallada una expresión, es aceptable y conveniente agregar paréntesis innecesarios. Dichos paréntesis se van a denominar paréntesis redundantes.

8.3. Generación automática de documentación

Hay que tener en cuenta que accederá a la documentación del código fuente los autores del propio código u otros desarrolladores.

Se ha de documentar el código porque facilitaremos su mantenimiento y reutilización.

Obligatoriamente se deben de documentar clases, paquetes, constructores, métodos y atributos.

Opcionalmente se puede bucles, partes de algoritmos que estimemos oportuno comentar…

A lo largo de nuestra vida como programadores es probable que nos veamos en la necesidad de reutilizar, modificar y mantener nuestro propio código o incluso, código de otros desarrolladores.

Sería muy útil que dicho código estuviera convenientemente documentado.

Todos os programadores se han encontrado una infinidad de veces en la situación en la que han leído código de otros programadores y posiblemente no hayan comprendido qué estaban haciendo en tal o cual método.

Por esta razón es de importancia crucial la auto-documentación del código ya que permite que otros programadores no se encuentren en esta situación con respecto a nuestro propio código.



La generación de una documentación adecuada de nuestros programas puede suponer una inestimable ayuda para realizar ciertos procesos en el software.

Si se analiza la documentación de las clases proporcionada en los paquetes que distribuye Sun y Oracle se puede ver de que dicha documentación ha sido generada con una herramienta llamada Javadoc.

Pues bien, también se puede generar la documentación de cualquier código a través de dicha herramienta.

Si desde el principio el programador se acostumbra a documentar el funcionamiento de sus clases desde el propio código fuente, se está facilitando la generación de la futura documentación de nuestras aplicaciones.

Esto se hace a través de una serie de comentarios especiales, llamados comentarios de documentación que serán tomados por Javadoc.

Posteriormente la herramienta los utilizará para generar una serie de archivos HTML que permitirán posteriormente, navegar por la documentación con cualquier navegador web.

Los comentarios de documentación tienen una marca de comienzo (/**) y una marca de fin (*/). En su interior podremos encontrar dos partes diferenciadas: una para realizar una descripción y otra en la que encontraremos más etiquetas de documentación.

Veamos un **ejemplo:**

```
/**
* Descripción principal (texto/HTML)
*
* Etiquetas (texto/HTML)
*/
```

Este es el formato general de un comentario de documentación. Comenzamos con la marca de comienzo en una línea.

Cada línea de comentario comienza con un asterisco.

El final del comentario de documentación debe incorporar la marca de fin. Las dos partes diferenciadas de este comentario son:

– Zona de descripción: es aquella en la que el programador escribe un comentario sobre la clase, atributo, constructor o método que se vaya a codificar bajo el comentario.

  Se puede incluir la cantidad de texto que se necesite, pudiendo añadir etiquetas HTML que formateen el texto escrito y así ofrecer una visualización mejorada al generar la documentación mediante Javadoc.

– Zona de etiquetas: en esta parte se colocará un conjunto de etiquetas de documentación a las que se asocian textos.

  Cada etiqueta tendrá un significado especial y va a aparecer en lugares determinados de la documentación, una vez esta haya sido generada.



Documentar el código de un programa es añadir suficiente información como para explicar lo que hace, punto por punto, de forma que no sólo los ordenadores sepan qué hacer, sino que además los humanos entiendan qué están haciendo y por qué.

Documentar un programa no es sólo un acto de buen hacer del programador por para tratar de dejar la obra rematada de manera estética.

Es además una necesidad que sólo se aprecia en su debida magnitud cuando hay errores que reparar o hay que extender el programa con nuevas capacidades o adaptarlo a una nueva situación.

Cuando se ha de incorporar determinadas etiquetas a nuestros comentarios de documentación es necesario conocer dónde y qué etiquetas colocar, según el tipo de código que estemos documentando en ese momento.

Exixten dos tipos generales de etiquetas:

– Etiquetas de bloque: son etiquetas que sólo pueden ser incluidas en un bloque de documentación, después de la descripción principal y comienzan con el símbolo @.

– Etiquetas en texto: son etiquetas que pueden ponerse en cualquier punto de la descripción o en cualquier punto de la documentación asociada a una etiqueta de bloque. Son etiquetas que se definen entre llaves, de la siguiente forma {@etiqueta}

A continuación se enumera y describe la función de las etiquetas más habituales a la hora de generar comentarios de documentación.

– @autor texto con el nombre del autor: esta etiqueta sólo se admite en clases e interfaces. El texto después de la etiqueta no necesitará un formato especial. Podremos incluir tantas etiquetas de este tipo como necesitemos.

– @version seguido del texto de la versión: el texto de la versión no necesitará un formato especial. Es conveniente incluir el número de la versión y la fecha de ésta. Podremos incluir varias etiquetas de este tipo una detrás de otra.

– @deprecated texto: indica que no debe utilizarse, indicando en el texto las causas de ello. Se puede utilizar en todos los apartados de la documentación. Si se ha realizado una sustitución debería indicarse qué de debe utilizar en su lugar.

– @deprecated indica que el método no funciona correctamente. Se recomienda el uso de {@link metodoCorrecto}.

– @exception nombre-excepción texto: Esta etiqueta es equivalente a @ throws.

– @param nombre-atributo texto: esta etiqueta es aplicable a parámetros de constructores y métodos. Describe los parámetros del constructor o método. El Nombre-atributo es igual al nombre del parámetro. Cada etiqueta @param irá seguida del nombre del parámetro y después de ella una descripción de éste.

– @retum texto: esta etiqueta se puede omitir en los métodos que devuelven void. Debe aparecer en todos los métodos que devuelven algo, dejando explícito qué tipo o clase de valor devuelve y sus posibles rangos de valores.

– @throws nombre-excepción texto: En nombre-excepción se ha de indicar el nombre completo de la excepción que se lanza. Se puede añadir una etiqueta por cada excepción que se lance explícitamente con una cláusula throws, pero siempre siguiendo el orden alfabético. Esta etiqueta es aplicable a constructores y métodos, describiendo las posibles excepciones del constructor o del método.

Ejemplo

Un ejemplo sería el siguente:

```
br> /

br> * Chequea si un vector no contiene elemento

br>

br> * @return <code>verdadero</code>si solo si este vec-
tor no contiene componentes, esto es, su tamaño es cer

br> * <code>falso</code> en cualquier otro cas

br>

br> public boolean VectorVacio()

br> return elementCount ==

br>

br> @see referencia: Se aplica a clases, interfaces,
constructores, métodos, atributos y paquetes. Añade en-
laces de referencia a otras partes de la documentación.
Podremos añadir a la etiqueta: cadenas de caracteres,
enlaces HTML a páginas y a otras zonas del código. Por
ejempl

br>

@see “Diseño de patrones: La reusabilidad de los elemen-
tos de la programación orientada a objetos”

* @see <a href=”http://www.w3.org/WAI/”>Web Accessibili-
ty Initiative</a>

* @see String#equals(Object) equals

>
```

A continuación se especifican una serie de reglas útiles de estilo que conviene seguir cuando se crea la documentación con javadoc.

– La descripción escrita debe de ser independiente de la implementación, pero si es necesario especificando dichas dependencias donde haga falta.

  Esto ayuda a los ingenieros escriben código a "escribir una vez, ejecutar en cualquier lugar." Que es una máxima del lenguaje java

  En todos los caso que sea posible, se debe escribir comentarios de documentación como una especificación de API independiente de la implementación.

  También hay que definir claramente lo que se requiere y lo que se le permite variar a través de distintas plataformas o implementaciones.

– Lo ideal es que sea lo suficientemente completa para satisfacer a todos los que la utilicen.

  Siendo realistas, conviene incluir suficiente información para que alguien que lee el código fuente puede escribir una parte sustancial de las pruebas de conformidad.

  Básicamente, la especificación debe ser completa, incluyendo las condiciones de frontera o límite, los rangos de los parámetros y los casos extremos.

  En su caso, mencionar todo lo que se sale de la especificación no especificado o permite a variar entre implementaciones.

– Si se debe documentar el comportamiento específico de la implementación, es muy importante documentarlo en un párrafo aparte con una frase que deje claro que es específico de la implementación.

  Si la aplicación varía según la plataforma, a continuación, especifique "en <plataforma>" al principio del párrafo.

  En otros casos que puedan variar con implementaciones en una plataforma que podría utilizar el la frase "aplicación específica".

  A continuación se detalla un ejemplo de una parte dependiente de la implementación de la especificación para java.lang.Runtime:

En los sistemas Windows, el comportamiento de búsqueda de pasos del método loadLibrary es idéntica a la del procedimiento LoadLibrary de la API de Windows.

El uso de "En los sistemas Windows Windows" al principio de la frase deja claro desde el principio que esto es una nota de aplicación.

– Reutilización automática de los comentarios de método: se puede evitar volver a escribir comentarios de los documentos, al estar al saber cómo los duplica la herramienta Javadoc cuando se hereda un comentarios en el caso de métodos que anulen o implementan otros métodos.

  Esto ocurre en tres casos:

  · Cuando un método en una clase anula un método en una superclase.

  · Cuando un método en una interfaz anula un método en un superinterfaz.

  · Cuando un método en una clase implementa un método en una interfaz.

  En los dos primeros casos, si un método m () anula otro método, la herramienta Javadoc generará un subapartado "sobreescrituras" en la documentación para el método m (), con un enlace al método que está anulando.

  En el tercer caso, si un método m () en una clase dada implementa un método en una interfaz, la herramienta Javadoc generará una subsección "especificado por" en la documentación para el m (), con un enlace al método que está implementando.

  En estos tres casos, si el método m () no contiene comentarios de los documentos o las etiquetas, la herramienta Javadoc también copiará el texto del método que está anulando o la aplicación de la documentación generada por m ().

Así que si la documentación del método reemplazado o aplicado es suficiente, no es necesario añadir documentación para m ().

Si agrega cualquier comentario de documentación o etiqueta para m (), seguirán apareciendo los "anulaciones" o "especificado por" subapartado y y enlace, pero si no hay texto serán copiados.

Los siguientes son consejos útiles y convenciones para escribir descripciones en comentarios de los documentos.

– Usar la etiqueta <code> para palabras clave y nombres.

  Palabras clave y nombres se compensan con <code>... </ code> cuando se menciona en la descripción.

  Esto incluye los siguientes elementos:

  · Palabras clave de Java.

  · Nombres de paquetes.

  · Nombres de las clases.

  · Nombres de los métodos.

  · Nombres de interfaz.

  · Nombres de los campos.

  · Nombres de los argumentos.

  · Ejemplos de código.

  No se debe abusar de los enlaces en línea. Se le debe a añadir enlaces de los nombres de la API (listadas inmediatamente superior) utilizando la etiqueta {@ link}.

  No es necesario añadir enlaces para todos los nombres de la API en un comentario de documentación.

– Debido enlaces llaman la atención sobre sí mismos (por su color y destacan en HTML, y por su longitud, en los comentarios del código fuente doc), esto puede hacer que los comentarios sean más difícil de leer si se utiliza profusamente.

  Por lo tanto, se recomienda añadir un enlace a un nombre de API si:

  · El usuario podría realmente desee hacer clic en él para obtener más información (en su criterio), y sólo para la primera aparición de cada nombre de API en el comentario de documentación (no de debe de estar repitiendo un enlace).

- El público que lee la documentación de javadoc se supone que es usuario avanzado y no novatos programadores, por lo que generalmente no es necesario vincular a API del el paquete java.lang (como la clase por ejemplo String), u otra API que debe de ser muy conocida.

– Se debe omitir paréntesis para la forma general de los métodos y constructores Cuando se hace referencia a un método o constructor que tiene varias formas, y que significaría para referirse a una forma específica, utilice paréntesis y tipos de argumentos.

  Por ejemplo, ArrayList tiene dos métodos que suman: add (Object) y add (int, Object). El add (int, Object) método agrega un elemento en una posición especificada en este ArrayList.

  Sin embargo, si se hace referencia a ambas formas del método, omita los paréntesis por completo.

  Es engañoso incluir paréntesis vacíos, porque eso implicaría especificer una forma particular del método.

  La intención en la documentación es distinguir el método general de cualquiera de sus formas particulares.

  Para indicar que es un método se debe escribir expresamente la palabra “método” para distinguirlo como un método y no un campo.

  El método add permite insertar elementos (preferido).

  El método add () le permite insertar elementos. Por tanto se debe evitar cuando quiere decir “todas las formas” del método add.

  Se debe usar frases en lugar de oraciones completas, en aras de la brevedad. esto es válido especialmente en el resumen inicial y las descripciones de parámetros y métodos así como las descripciones de etiqueta.

  Conviene utilizar la tercera persona (descriptiva) no segunda persona (prescriptiva). La descripción se hace en tercera persona en la documentación de referencia mejor que en segunda persona.

  Por ejemplo:”Devuelve la etiqueta”. (mejor) que “Obtenga la etiqueta”. (Evitar).

  Las Descripciones de los métodos deben comenzar con una frase verbal. Un método implementa una operación, por lo que por lo general comienza con una frase verbal:

Por ejemplo: “Devuelve etiqueta de este botón”. (mejor) a “Este método obtiene la etiqueta de este botón”.

Descripciones de clase o interfaz o campo pueden omitir el tema y simplemente indicar el objeto.

Estas partes de la API menudo describen las cosas en lugar de acciones o comportamientos:

Una etiqueta del botón. (preferido) a Este campo es una etiqueta de botón. (Evitar)

Utilice “esto” en lugar de “la” al referirse a un objeto creado a partir de la clase actual.

Por ejemplo, la descripción del método getToolkit queda redactado como sigue:

- Obtiene el conjunto de herramientas para este componente. (preferido).
- Obtiene el conjunto de herramientas para el componente. (Evitar).
- Añadir una descripción además del nombre API:

Los mejores nombres de la API son “auto-documentado”, lo que significa que te dicen, básicamente, lo que hace el API.

– Si el comentario de documentación se limita a repetir el nombre de API en forma de frase, no está proporcionando más información.

Por ejemplo, si la descripción método utiliza sólo las palabras que aparecen en el nombre del método, entonces se está añadiendo nada a lo que se podría inferir.

El comentario ideales va más allá de esas palabras y debe siempre te recompensará con algunos bits de información que no era inmediatamente obvio por el nombre de API.

Por ejemplo: La siguiente descripción no dice nada más allá de lo que sabe de la lectura del nombre del método. La palabra “set”, “herramienta”, “consejo”, y “texto” simplemente se repiten en una oración.

Ejemplo

```
/ **
 * Establece el texto de información sobre herramientas.
 *
 * @ Param texto el texto de la punta de la herramienta
 * /
public void setToolTipText (String text) {
```

Sin embargo Esta descripción define más completamente lo que un extremo de la herramienta es, en el contexto más amplio de registrarse y que se muestra en respuesta al cursor.

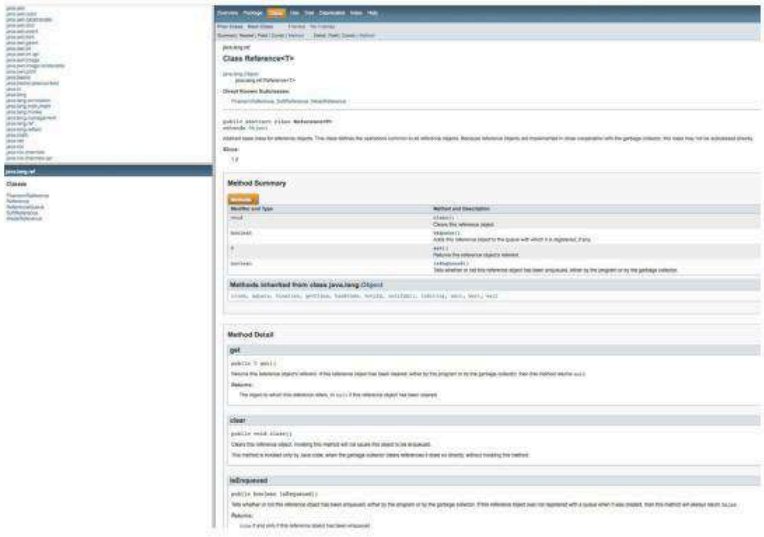
Ejemplo

```
/ **
 * Registra el texto que se mostrará en una punta de la herramienta. el texto
 * Aparece cuando el cursor se detiene en el componente.
 *
 * @ Param texto la cadena que se vea. Si el texto es nulo,
 * La punta de la herramienta se desactiva para este componente.
 * /
public void setToolTipText (String text) {
```

Conviene ser claro cuando se utiliza el término “campo”. Hay que tener en cuenta que la palabra “campo” tiene dos significados:

- Campo estático, que es otro término para la “variable de clase”.
- Campo de texto, como en la clase TextField.

Conviene considerar que este tipo de campo puede estar restringido a la asignación de fechas, números o cualquier texto.



Ejemplo de documentación generada con javadoc

Los nombres alternativos podrían ser “campo de fecha” o “campo de número”, según corresponda.

UD8
Lo más importante

- La documentación del software es el texto que acompaña a los programas informáticos. Esta documentación debe explicar bien cómo funciona o cómo usarlo, y puede significar cosas diferentes para personas con diferentes roles en la organización.

- Si el software es una primera versión sobre la que más tarde se va a seguir trabajando, la documentación de los requisitos es muy útil en la gestión del cambio de software y verificar que no se ha deteriorado en el software cuando se modifica.

- Al crear software, el código por sí solo es insuficiente. Debe haber texto junto con él para describir diversos aspectos de su operación prevista.

- Los documentos de código se organizan a menudo en un estilo de guía de referencia, lo que permite a un programador para buscar rápidamente una función o clase cualquiera.

- Es muy importante que los documentos de usuario no sean confusos, y que estén actualizados a la fecha. Los documentos de los usuarios no necesitan estar organizados de una manera determinada, pero es muy importante para los usuarios el hecho de tener un índice exhaustivo.

- Si el software tiene problemas conocidos, a los desarrolladores gustaría llamarlo cuestiones pendientes pero usuarios finales prefieren llamarlos errores conviene documentar una solución y ofrecer a sus usuarios ayuda. Documentar los eventos que son inevitables durante la vida útil de cualquier sistema.

– Documentar el código de un programa es añadir suficiente información como para explicar lo que hace, punto por punto, de forma que otros programadores puedan entender qué están haciendo y por qué.

– En los comentarios, hay que añadir explicaciones a todo lo que no es evidente, es decir, no hay que repetir lo que se hace, sino explicar por qué se hace.

– Los comentarios de documentación tienen una marca de comienzo (/**) y una marca de fin (*/). En su interior podremos encontrar dos partes diferenciadas: una para realizar una descripción y otra en la que encontraremos más etiquetas de documentación.

– Documentar el código de un programa es añadir suficiente información como para explicar lo que hace, de forma que otros programadores entiendan qué están haciendo y por qué.

– La descripción escrita debe de ser independiente de la implementación, pero si es necesario especificando dichas dependencias donde haga falta.

– Omitir paréntesis para la forma general de los métodos y constructores Cuando se hace referencia a un método o constructor que tiene varias formas, y que significaría para referirse a una forma específica, utilice paréntesis y tipos de argumentos.

– Conviene ser claro cuando se utiliza el término “campo”. Hay que tener en cuenta que la palabra “campo” tiene dos significados: campo estático, que es otro término para la “variable de clase” o campo de texto, como en la clase TextField.

UD8
Autoevaluación

1. Si el software es una primera versión…

    a. No hace falta documentación de requisitos.
    b. La documentación de diseño se puede eliminar.
    c. La documentación de requisitos es muy útil como guía.
    d. Ninguna de las anteriores es correcta.

2. El documento de diseño de la base de datos…

    a. Indica el diseño conceptual y físico de la base de datos.
    b. No se suele utilizar muy a menudo.
    c. Contiene el diseño conceptual, lógico y físico además de los elementos de diseño de la base de datos.
    d. Es el documento más importante de la fase de pruebas.

3. La programación explicativa…

    a. Es el resultado de la aplicación práctica de la escritura de programación auto explicativa en contextos de programación real.
    b. No existe tal concepto.
    c. Es un paradigma de programación estructurada.
    d. Solo se practica en el lenguaje Java.

4. Cuando se documenta

   a. Se puede asumir que el usuario tendrá unos conocimientos avanzados de informática.
   b. No hay que incluir imágenes en una documentación seria y formal.
   c. No se debe de asumir nada que el usuario deba saber.
   d. Todas las respuestas son incorrectas.

5. Las excepciones se lanzan…

   a. Con la sentencia throw.
   b. Mediante un bloque finally{…}.
   c. Mediante un bloque try{…} catch{…}.
   d. Con la sentencia throws.

6. Los estándares de documentación:

   a. Convienen utilizarlos siempre.
   b. No es necesario que se usen, son documentos orientativos.
   c. No existe ninguno para el lenguaje Java.
   d. Ninguna de las respuestas es correcta.

7. Se ha de documentar el código.

   a. Para facilitar su mantenimiento y reutilización.
   b. Principalmente por motivos estéticos.
   c. Porque se indica en los estándares ISO.
   d. Solamente en las organizaciones muy grandes como Oracle o Sun.

8. La documentación de Javadoc:

   a. Se basa en el uso de etiquetas que empiezan por #.
   b. Si no se incluye la clase no compila correctamente.
   c. En ella existen etiquetas de bloque y etiquetas de texto.
   d. Se usa para documentar las excepciones que lanza un método.

9. Para indicar el valor de retorno de un método en Javadoc…

    a. Se usa la etiqueta @param.
    b. Se usa la etiqueta @version.
    c. Se usa la etiqueta @return.
    d. Se usa la etiqueta @throws.

10. Las descripciones de los métodos deben empezar…

    a. Por una frase nominal.
    b. Con la etiqueta @methodDescription.
    c. Con la etiqueta @throw.
    d. Por una frase verbal.

Área: informática y comunicaciones

Glosario

- **Ciclo de vida:** etapas que se pueden identificar durante todo el tiempo que está un sistema software operativo.
- **Caso de uso:** forma de especificar los requisitos de un sistema como una interacción entre el sistema y un usuario externo a él.
- **Polimorfismo:** consiste en la característica de un lenguaje orientado a objetos en la que dependiendo de varios factores, se puede ejecutar un código u otro.
- **Diagrama de clases:** diagrama UML de estructura que muestra como es el sistema diseñado a nivel de clases, atributos, métodos y relaciones entre estas.
- **Herencia:** mecanismo por el cual las clases más específicas llamadas subclases o clases derivadas, incoporan la estructura y comportamiento de una clase más general, llamada superclse o clase base.
- **Swing:** conjunto de clases de Java que sirven para implementar la interfaz gráfica de usuario.
- **Clase interna:** en Java, es una clase definida en el mismo ámbito de visibilidad que otra clsse, y por tanto solo se pueden crear objetos de la clase interna en esta.

- **Trazabilidad:** la trazabilidad se refiere al hecho de conocer en todo momento por que fases y procesos ha ido pasando cada unidad de un producto, de tal forma que cualquier problema pueda ser reastreado hasta su origen.

- **Modulo subordinado:** es un módulo o componente de software que es llamado o utilizado por otro, al que se dice que estña subordinado.

- **Grafo:** un grafo es una estructura matemática que consta de una serie de nodos identificados conectados entre sí por una serie de aristas.Puede ser dirigido si las aristas tienen dirección o no dirigido en el caso contrario.

- **Regiones de un grafo:** es el número de secciones en las que dividen las aristas de un grafo al plano en el que está representado el grafo.

- **Repositorio:** cuando hablamos de repositorio en el contexto de desarrollo del software nos referimos a cualquier lugar donde se almacene información del proyecto que se está llevando a cabo.

- **Ant:** herramienta que automatiza todos los paso para la compilación y linkado de un programa o la generación de documentación. Sustituye a la clásica herrmienta make en entornos multiplataforma.

- **Pila:** zona de memoria en la que se almacenan todas las llamadas a métodos o funciones que se han ido realziando en un programa, así como otra información.

- **API:** abreviatura de Aplication Program Interface, es un conjunto de funciones o clase, una libraría, doucmentada, que una organización pone a disposición de otros programadores para que la usen en sus programas.

- **Entorno de ejecución:** funciones que porporciona el sistema operativo a un programa para que realice su función. EN el caso del lenguaje de programación java esto lo hace la Máquina Virtual de Java o JVM.

- **Lista o manual de referencia:**manual en el que las funciones del sistema aparecen ordenadas según algún criterio, por ejemplo el orden alfabético.

- **Indentación:** consiste en la técnica para organizar de manera visible el código fuente en la que las estructuras embebidas en otras se escriben con un na serie de espacio o tabuladores delante para poder distinguirlas con facilidad.

- **Independencia de la implementación:**este concepto hace referencia a diferenciar en un programa QUE hace de cómo lo HACE (implementación)

– **Notación:** una notación es una especificación detallada de cómo se debe representar mediante texto o de manera gráfica una serie de conceptos haciendo uso de unos determinados símbolos. La notación incluye los símbolos, el orden en el que se deben colocar y su significado.

Área: informática y comunicaciones